



上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

Transformer Models

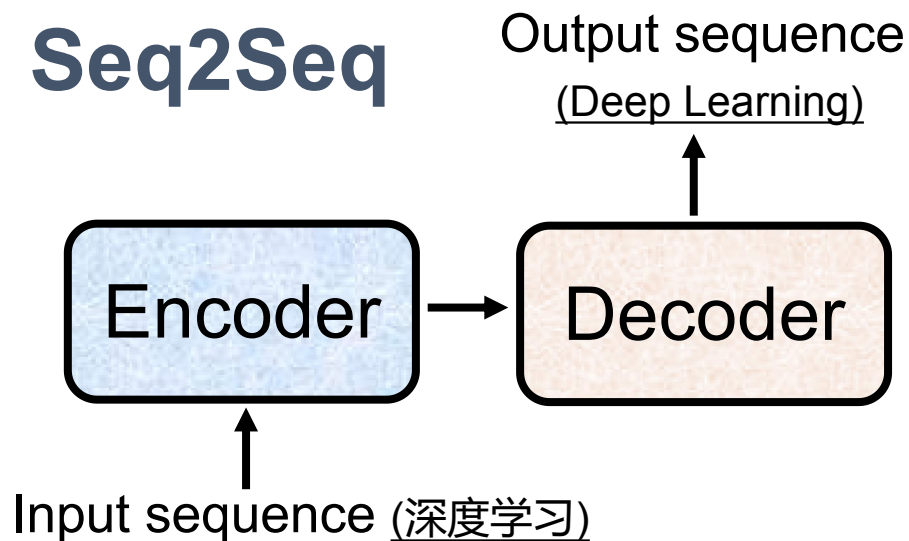
(2026)

Lecturer: Xue Yang

yangxue.site

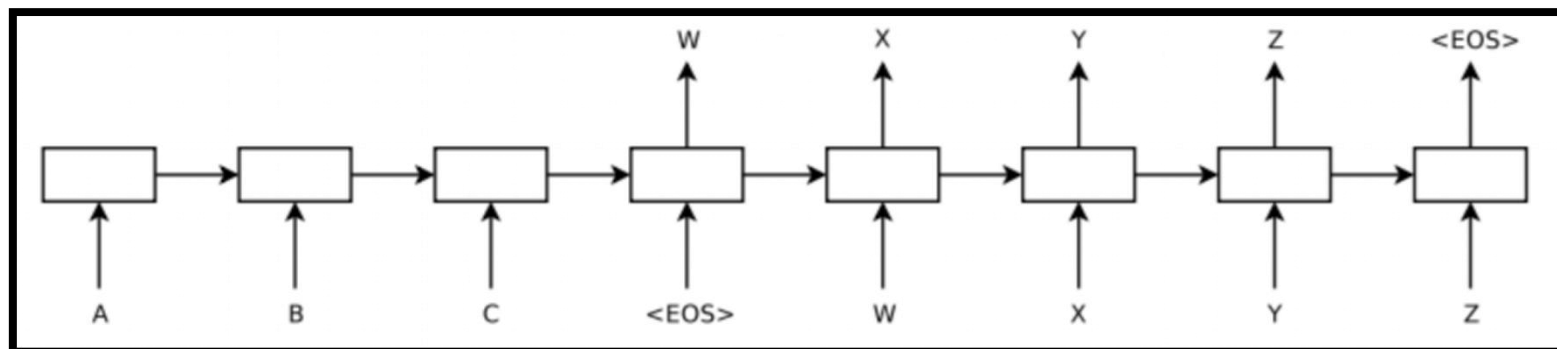


Seq2Seq

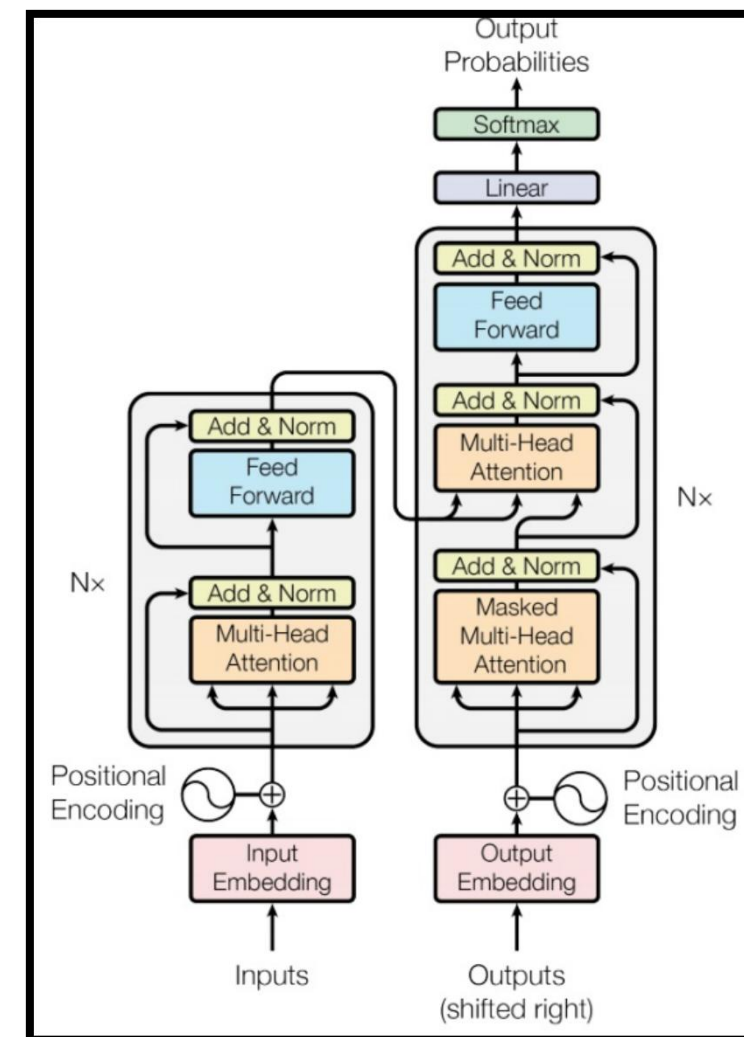


Transformer

<https://arxiv.org/abs/1706.03762>



Sequence to Sequence Learning with Neural Networks,
<https://arxiv.org/abs/1409.3215>



Overview: Transformer



Attention Is All You Need

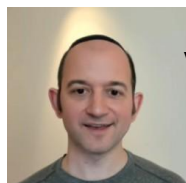
Equal contribution.
Listing order is random.



Co-Founder
and CEO
Essential AI

Ashish Vaswani*

Google Brain
avaswani@google.com



VP Engineering,
Gemini Co-lead
Google

Noam Shazeer*

Google Brain
noam@google.com



Member of
Technical Staff
Anthropic

Niki Parmar*

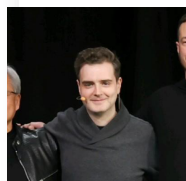
Google Research
nikip@google.com



Co-Founder
Inception

Jakob Uszkoreit*

Google Research
usz@google.com



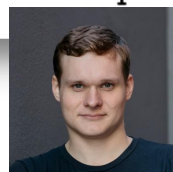
Llion Jones*
Google Research
llion@google.com

Co-Founder
Sakana AI



Aidan N. Gomez* †
University of Toronto
dan@cs.toronto.edu
Co-Founder and CEO
Cohere

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

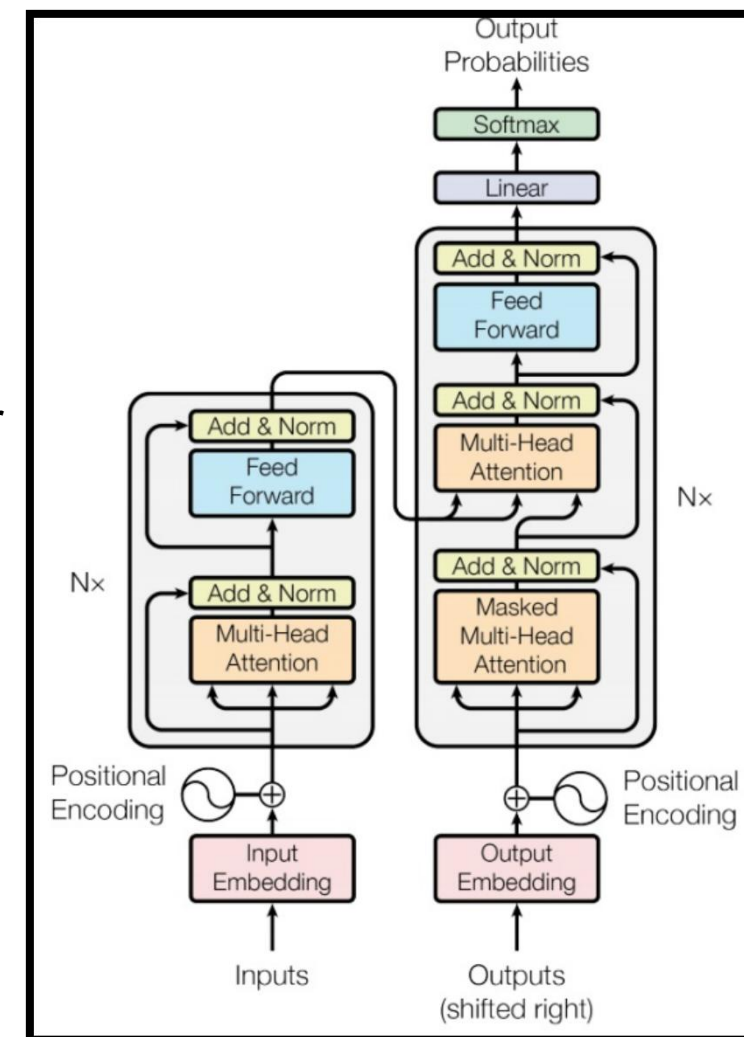


Co-Founder
NEAR Protocol

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com



Member of
Technical Staff
OpenAI



Transformer, Citation: 203461 (6/9/25)

<https://arxiv.org/abs/1706.03762>

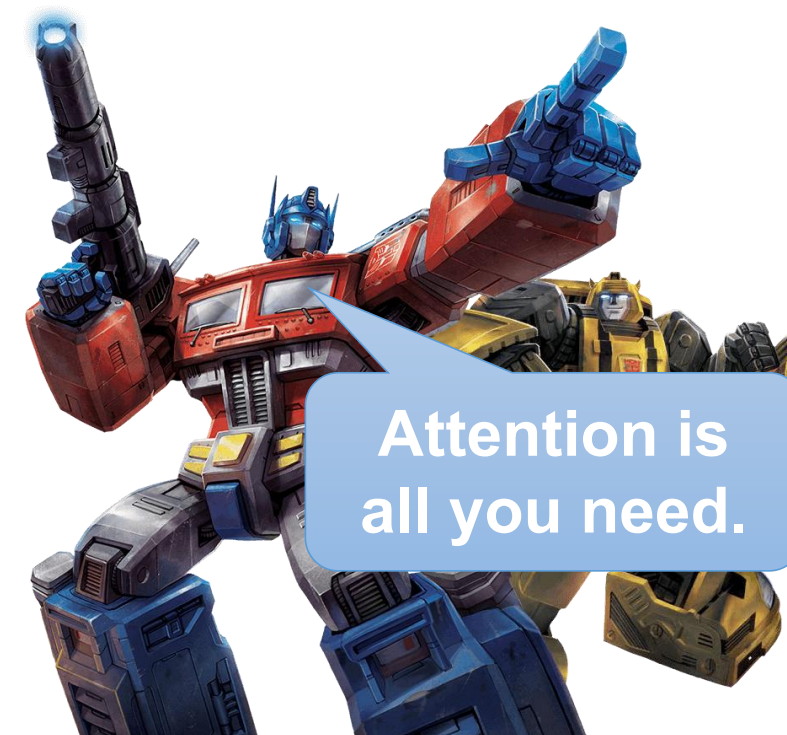
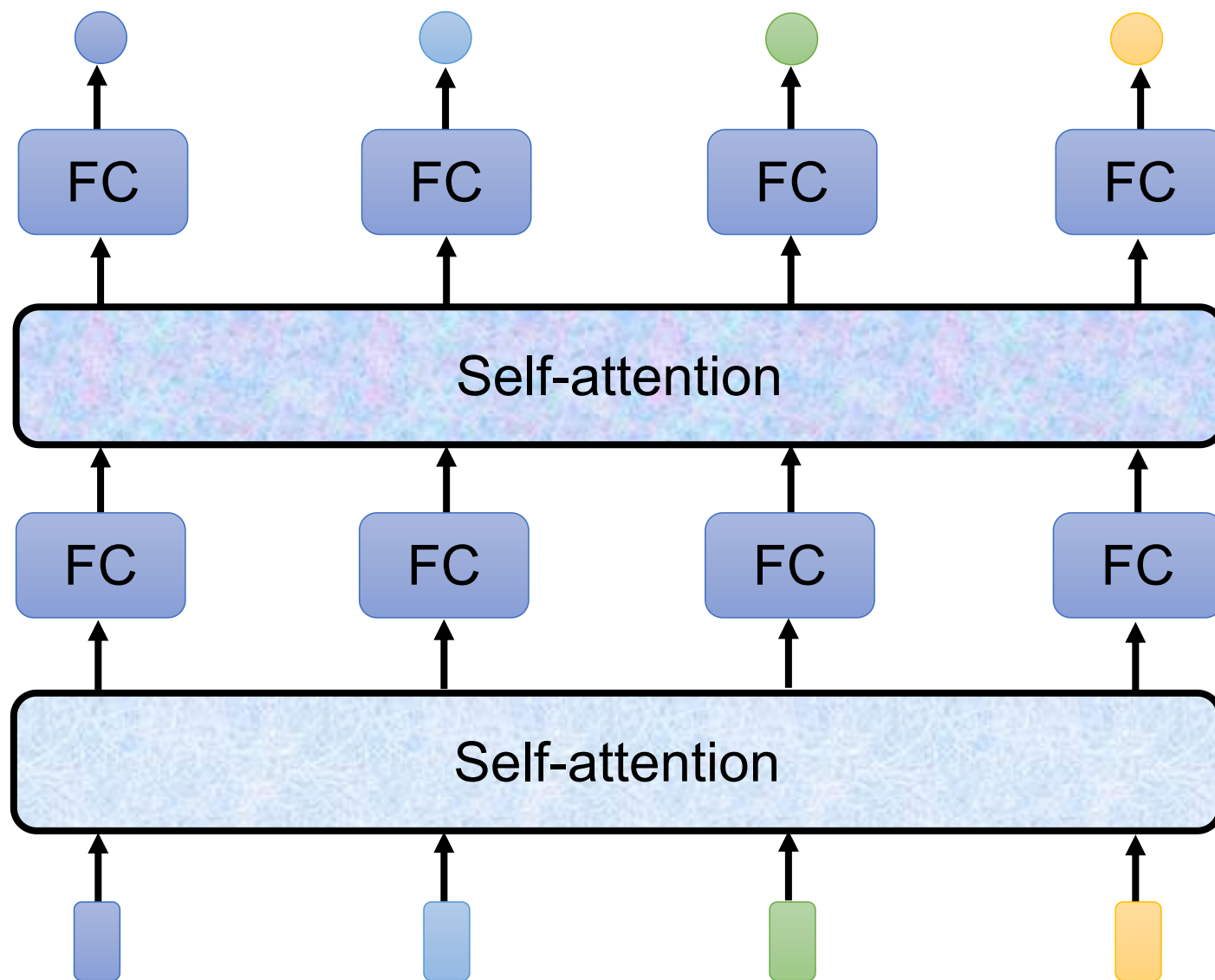


上海交通大学

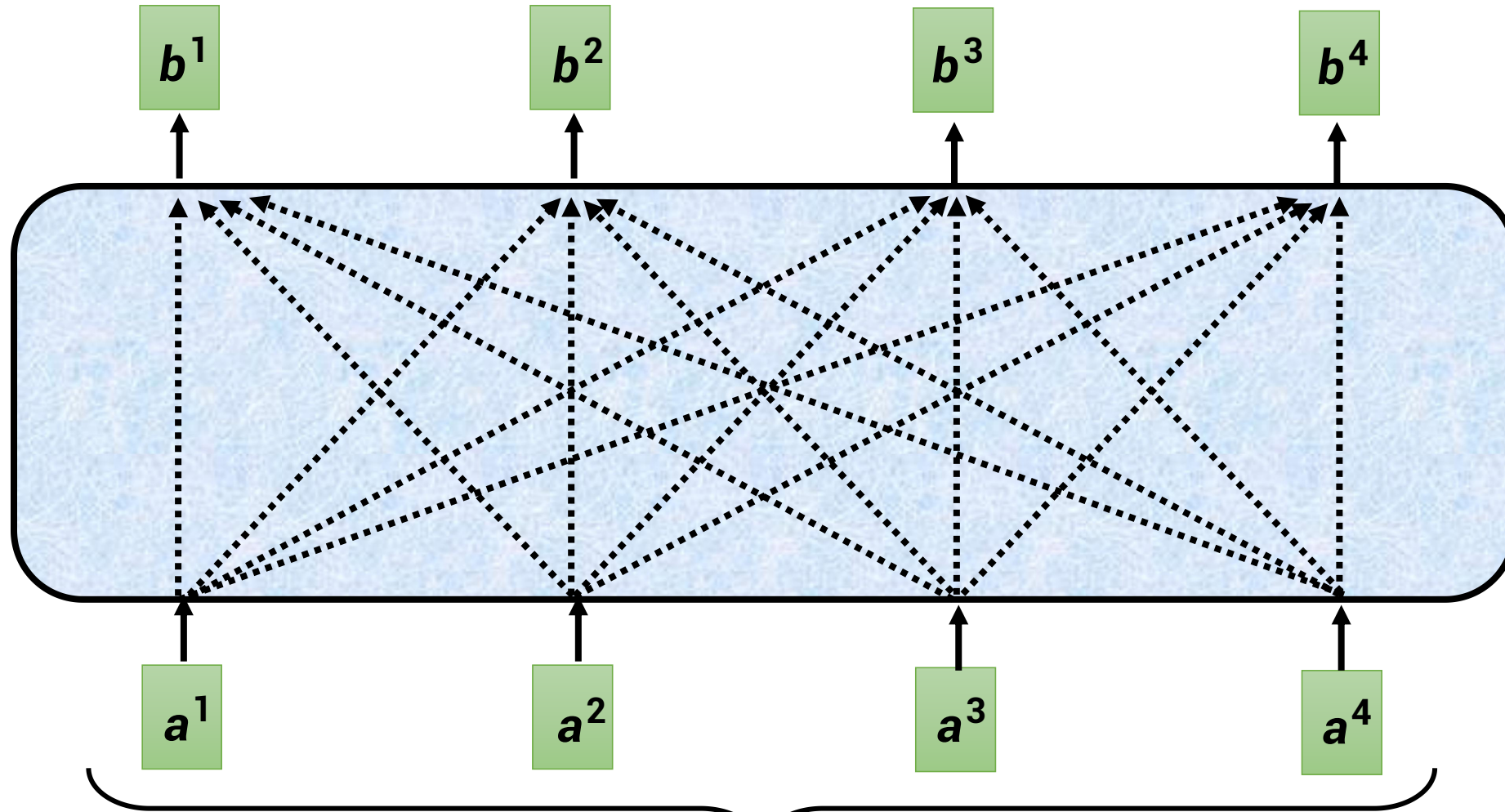
SHANGHAI JIAO TONG UNIVERSITY

Contents

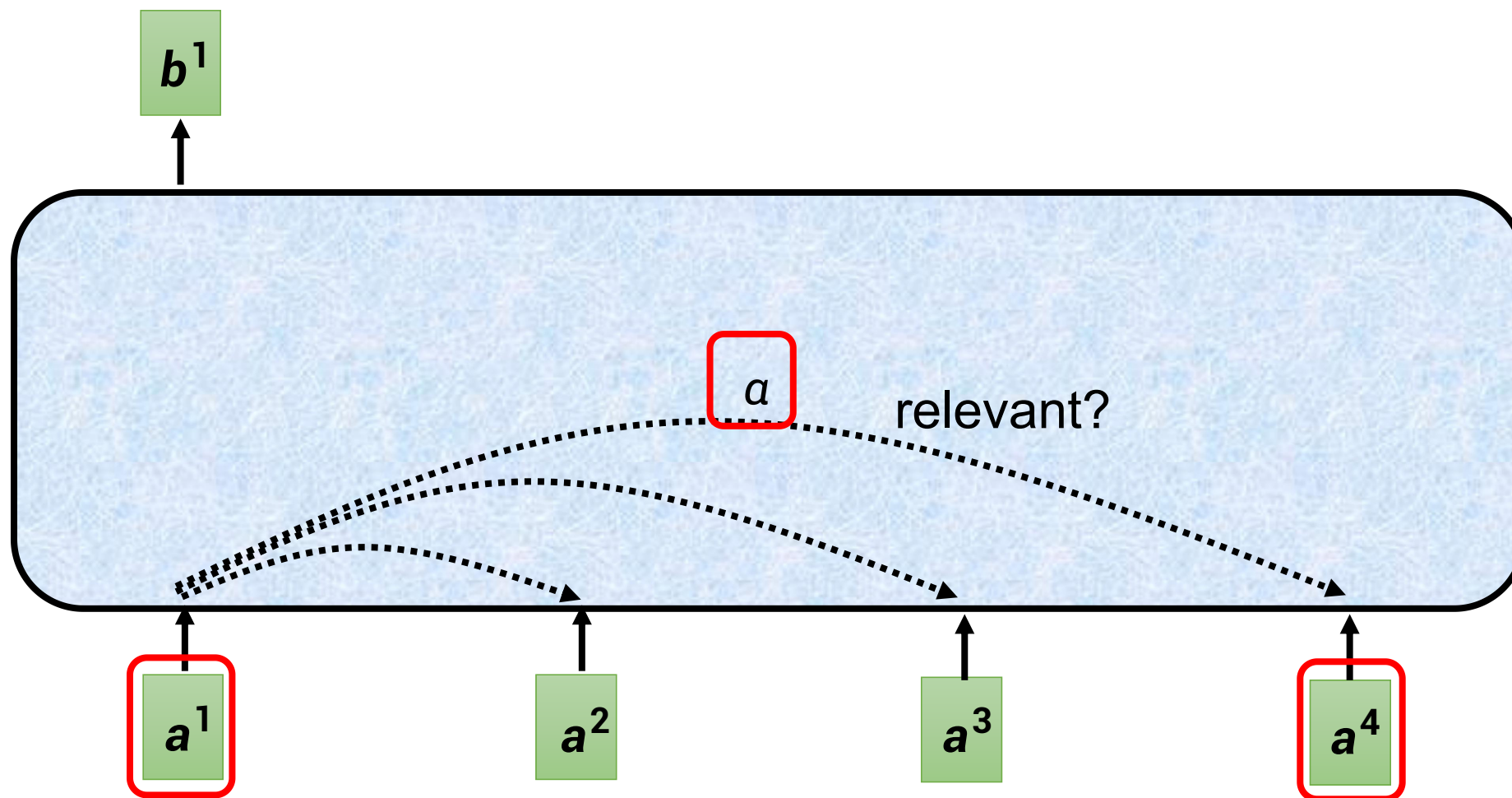
- **Self-attention**
- Encoder
- Decoder
- Encoder-Decoder
- NLP/Vision Transformer
- Mixture of Experts



<https://arxiv.org/abs/1706.03762>

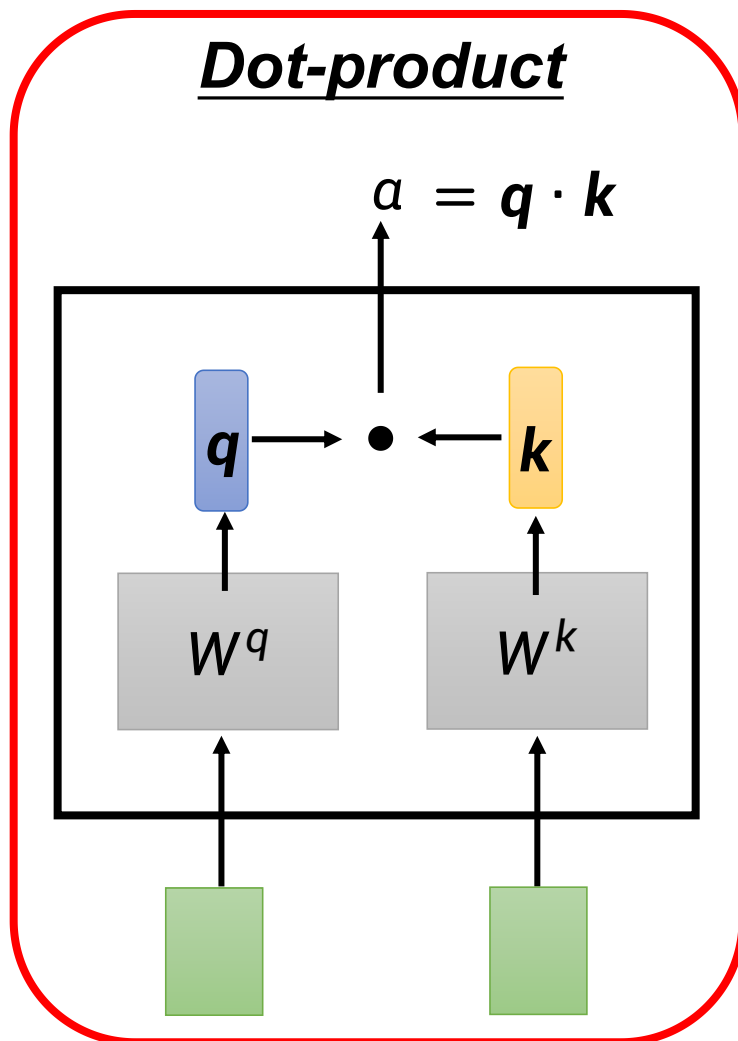


Can be either **input** or **a hidden layer**

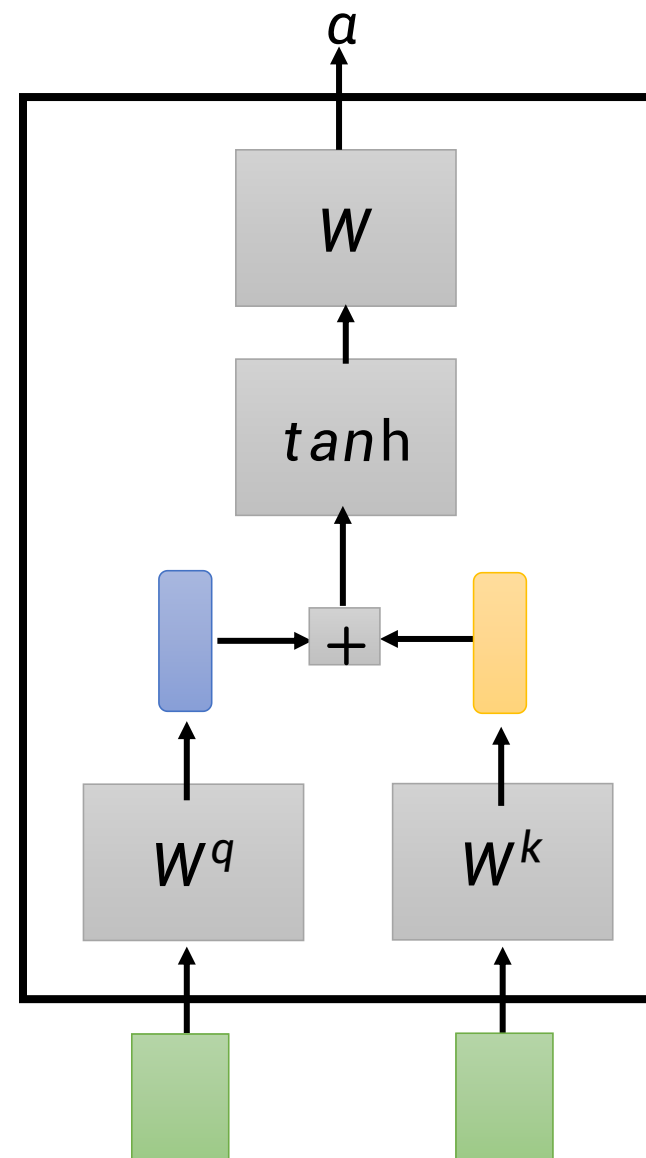


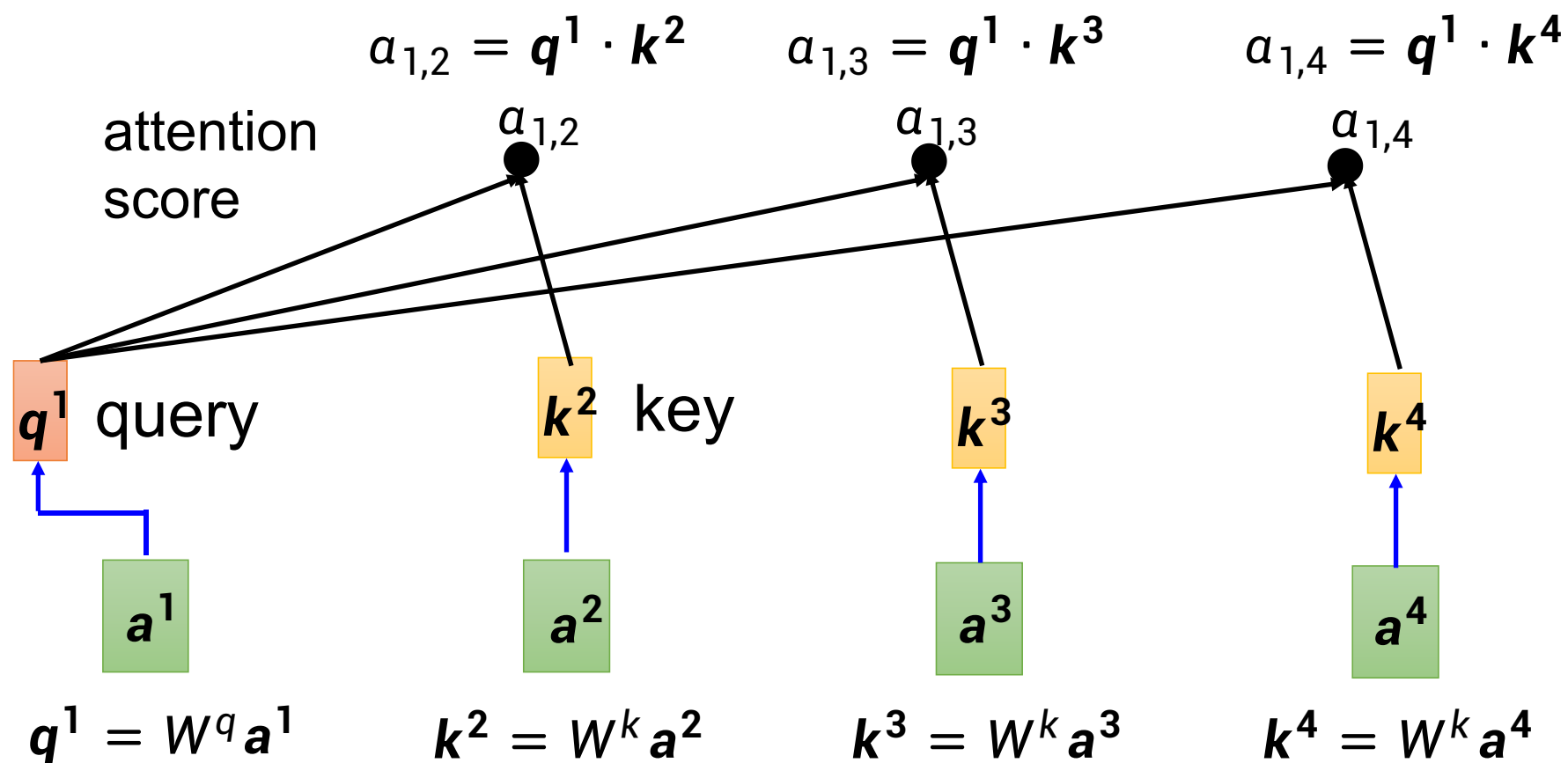
Find the relevant vectors in a sequence

Dot-product

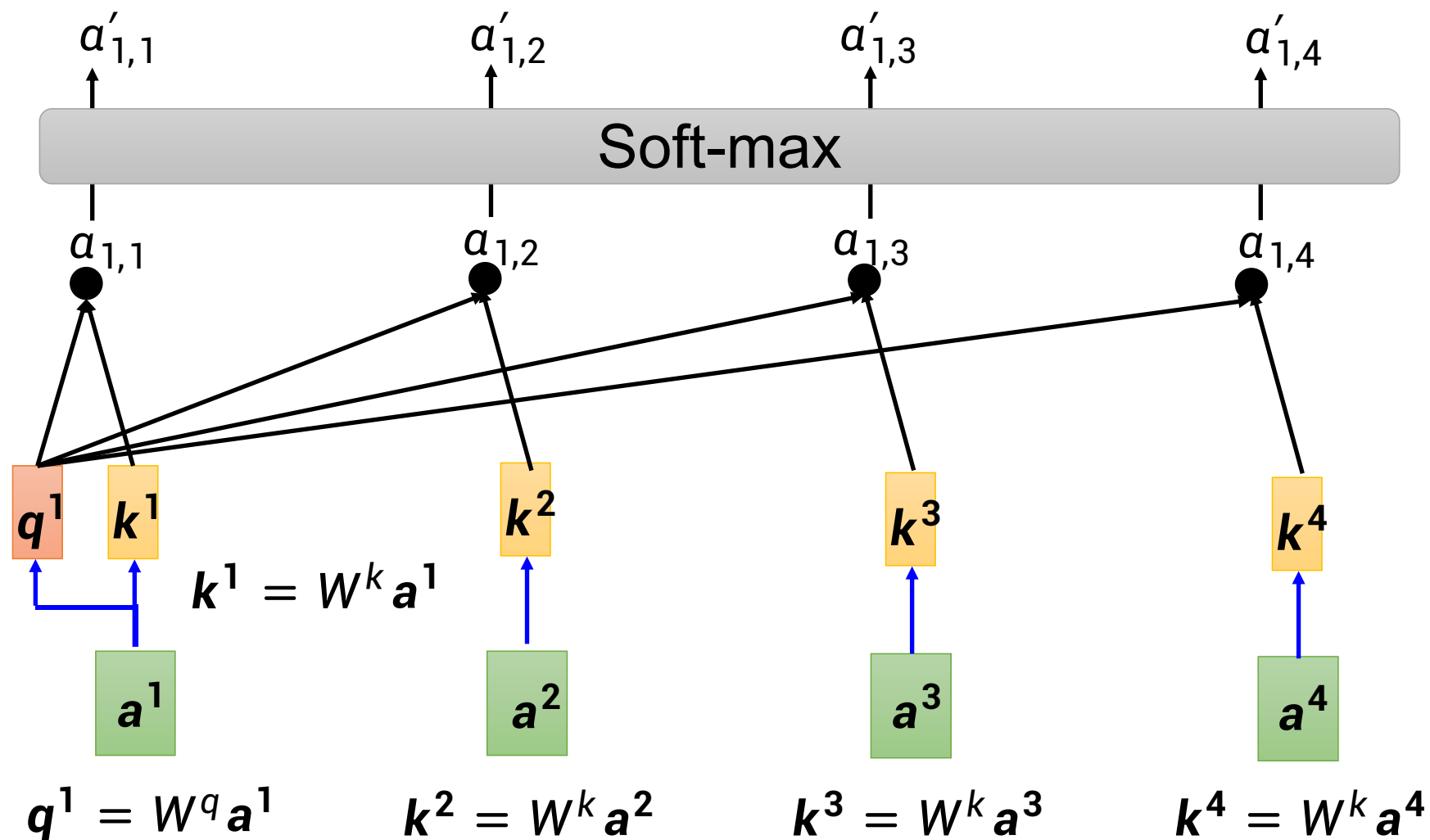


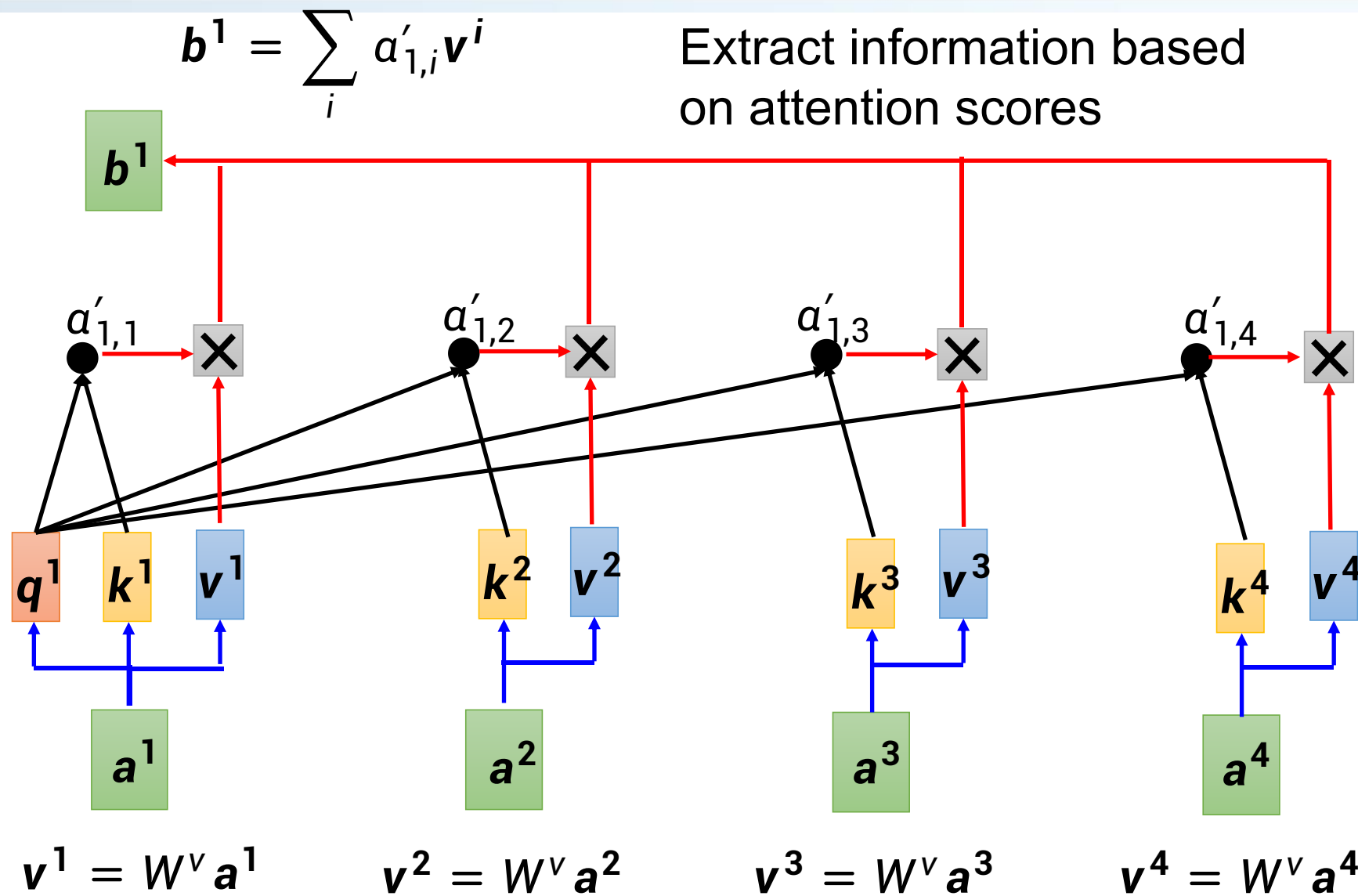
Additive

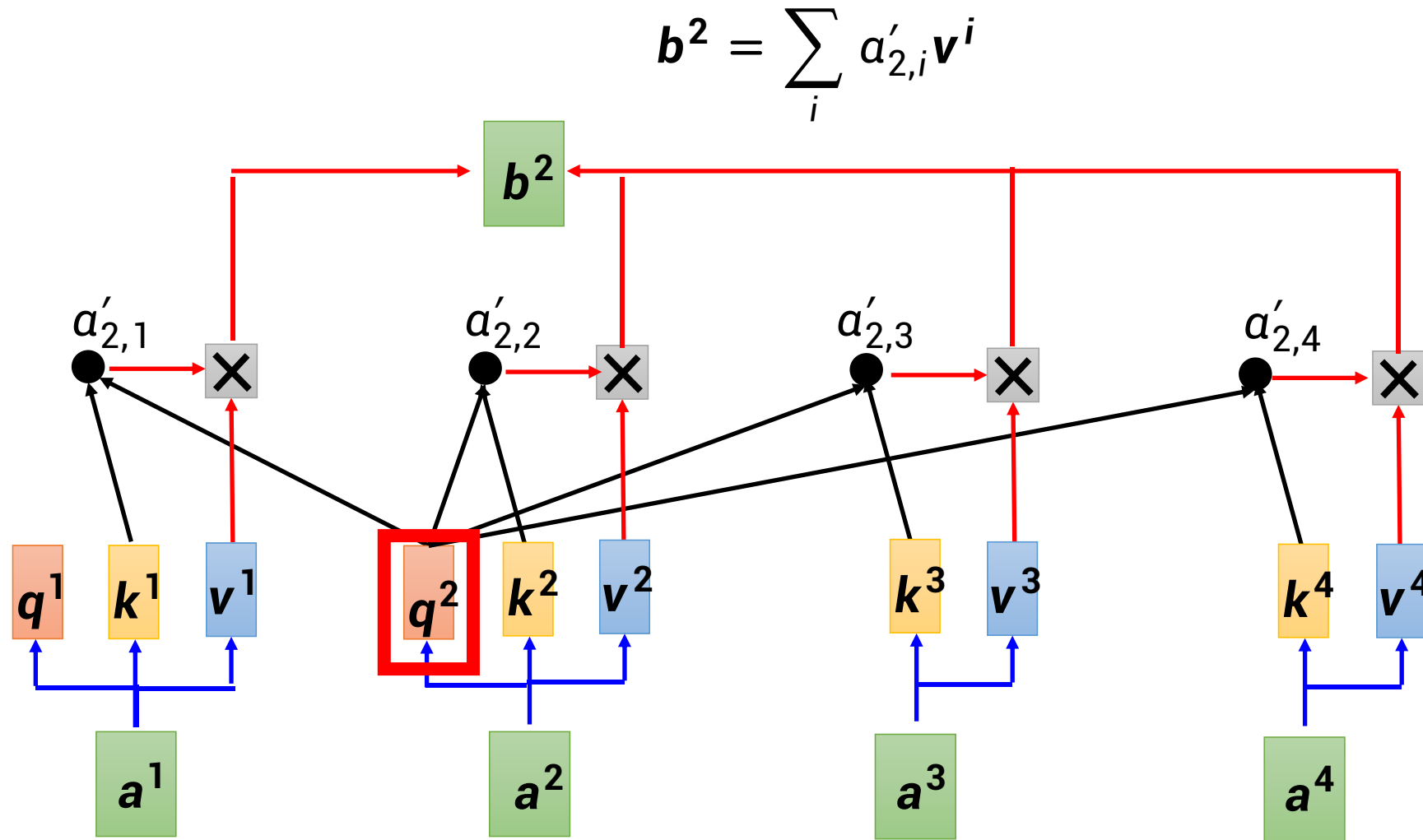


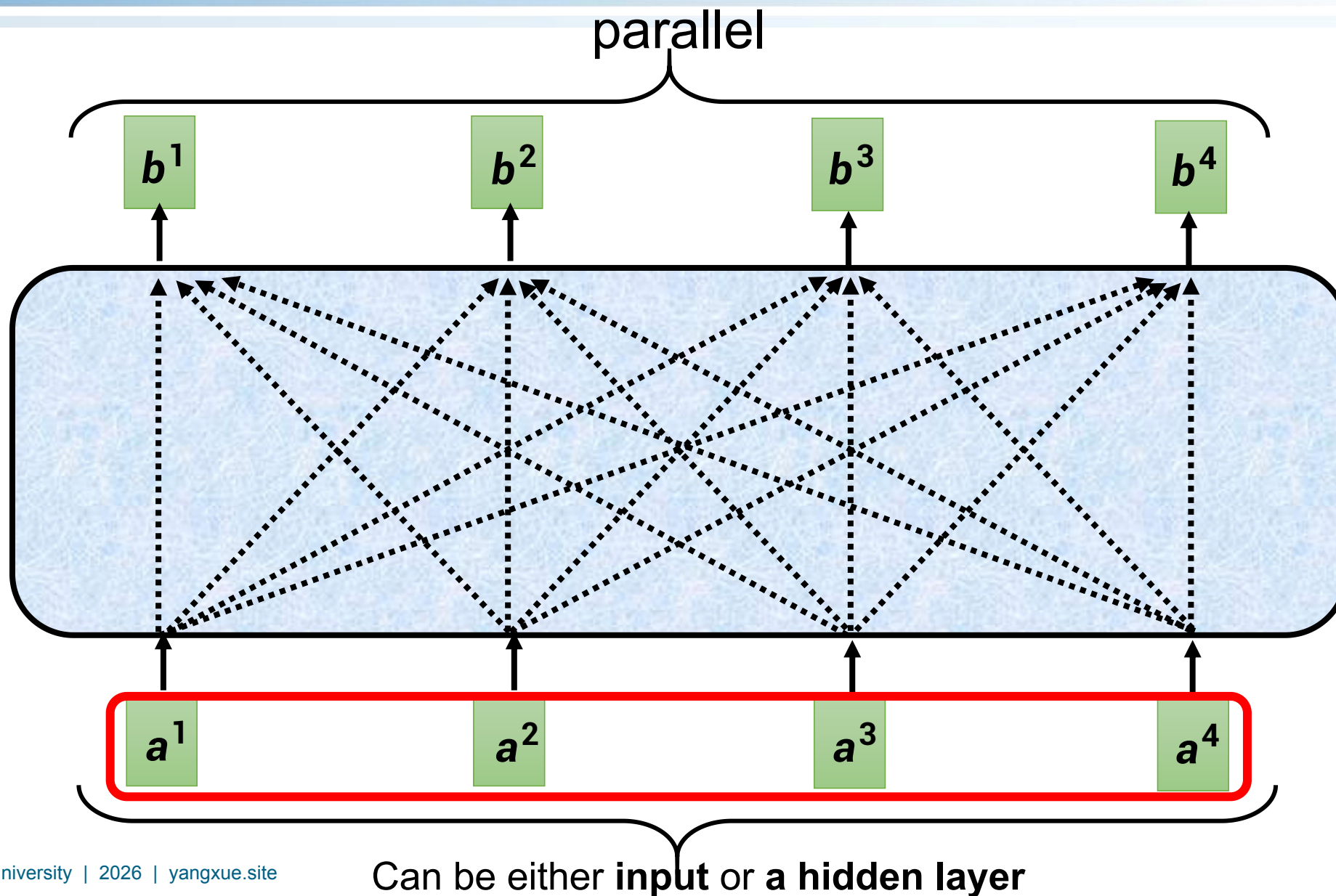


$$a'_{1,i} = \exp(a_{1,i}) / \sum_j \exp(a_{1,j})$$





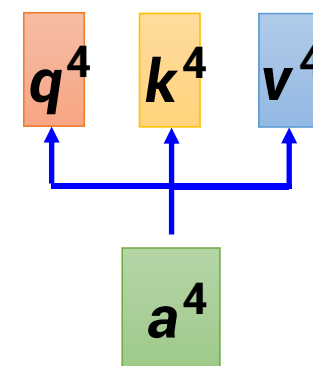
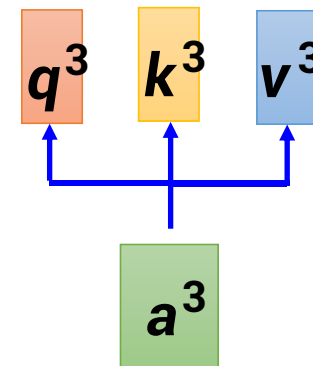
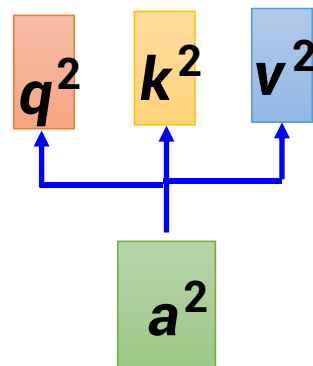


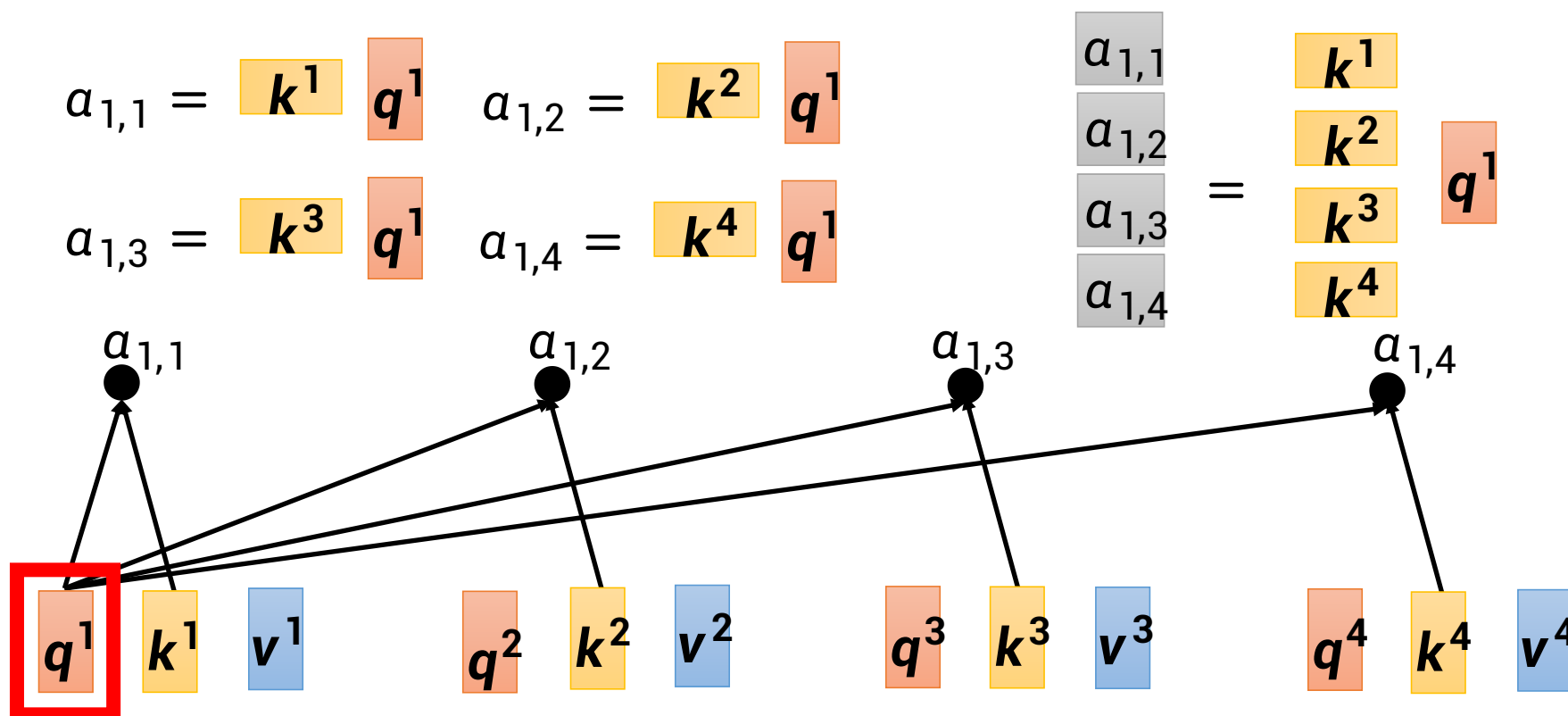


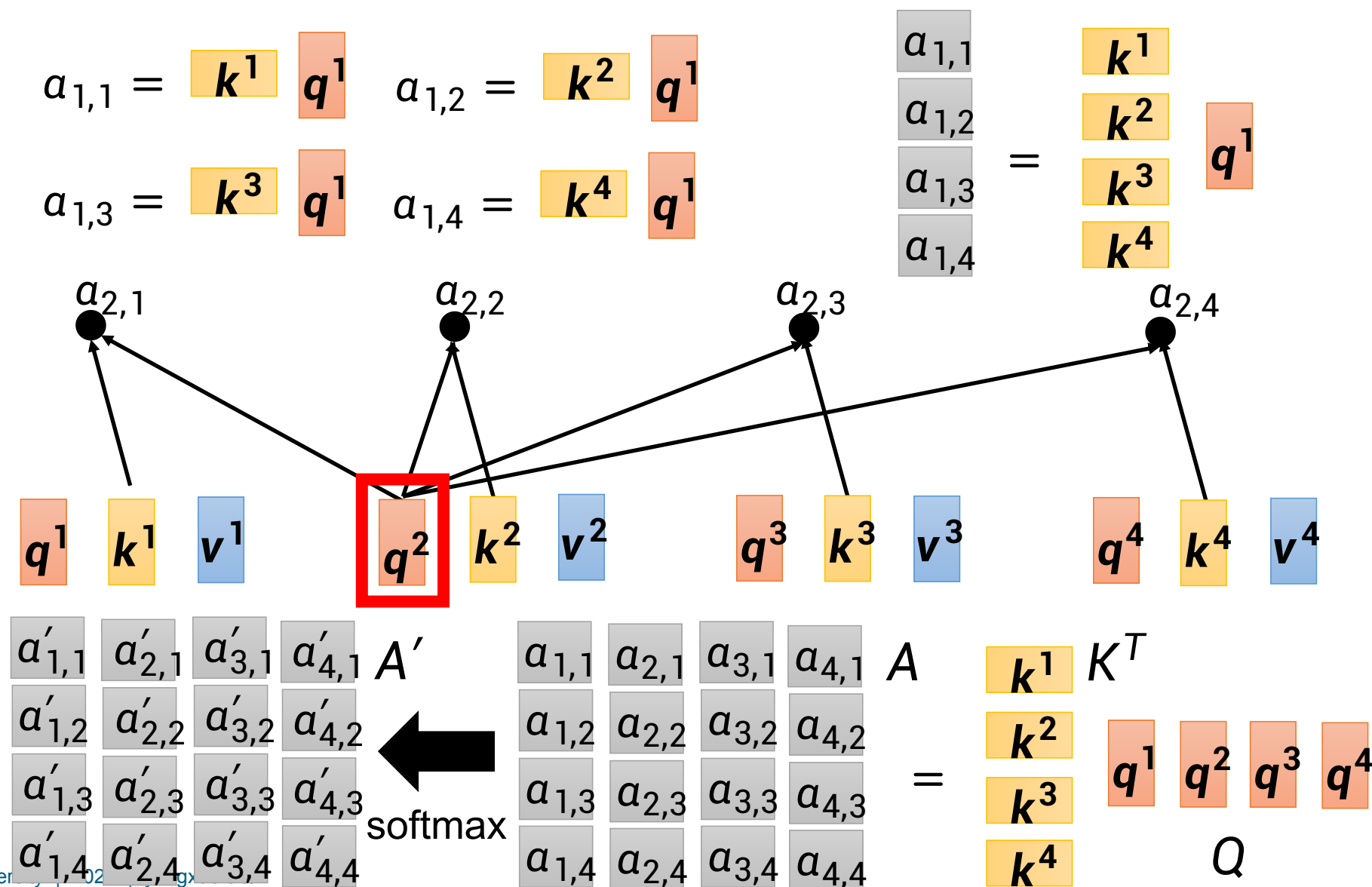
$$\mathbf{q}^i = \mathbf{W}^q \mathbf{a}^i$$

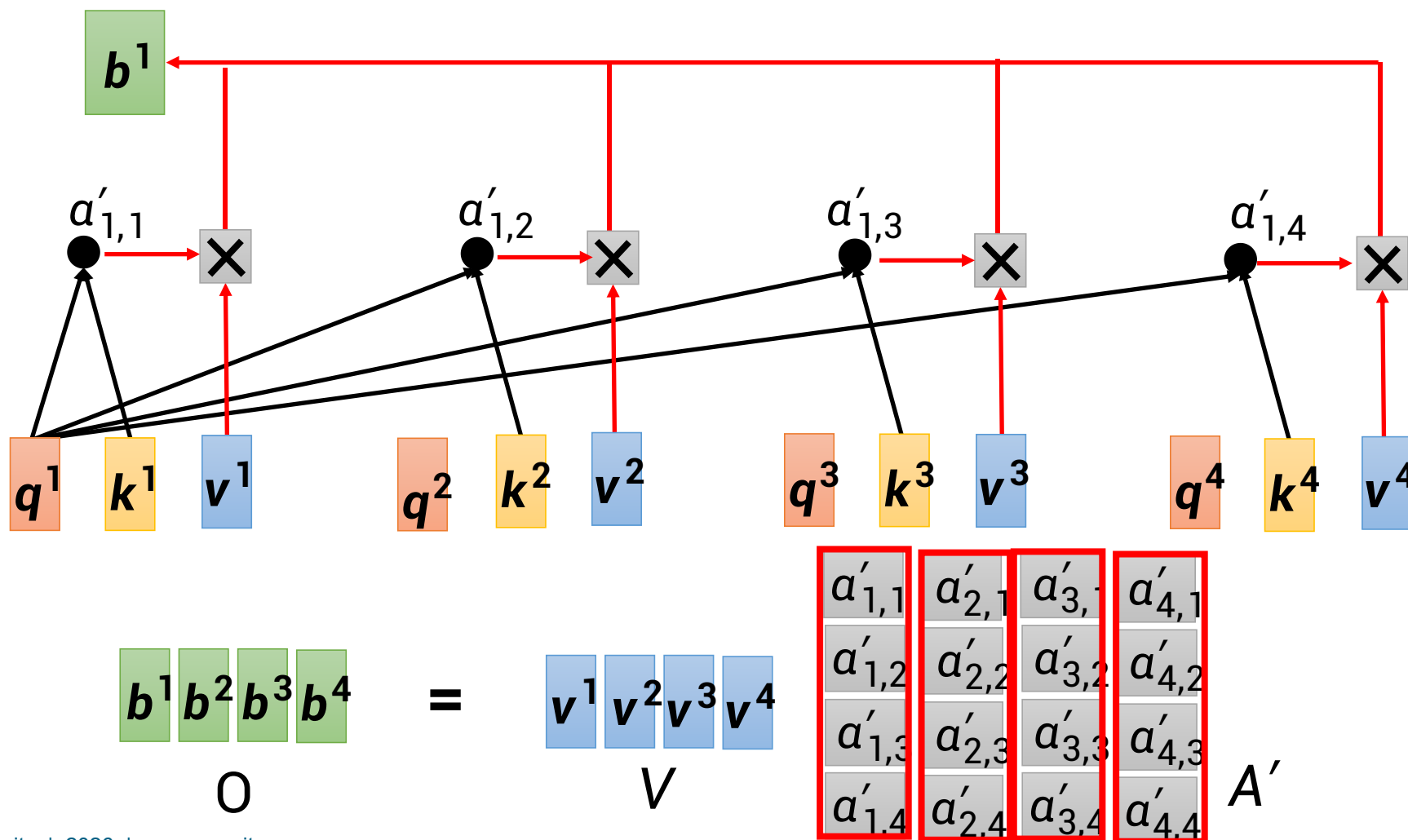
$$\mathbf{k}^i = \mathbf{w}^k \mathbf{a}^i$$

$$\mathbf{v}^i = \mathbf{W}^v \mathbf{a}^i$$





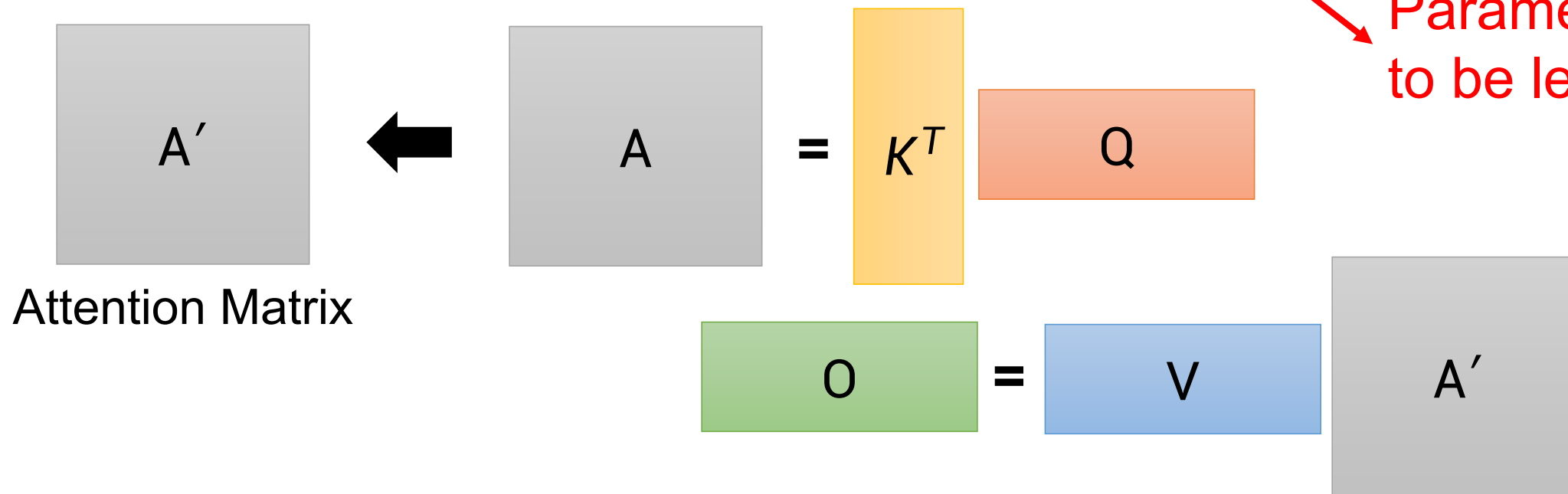


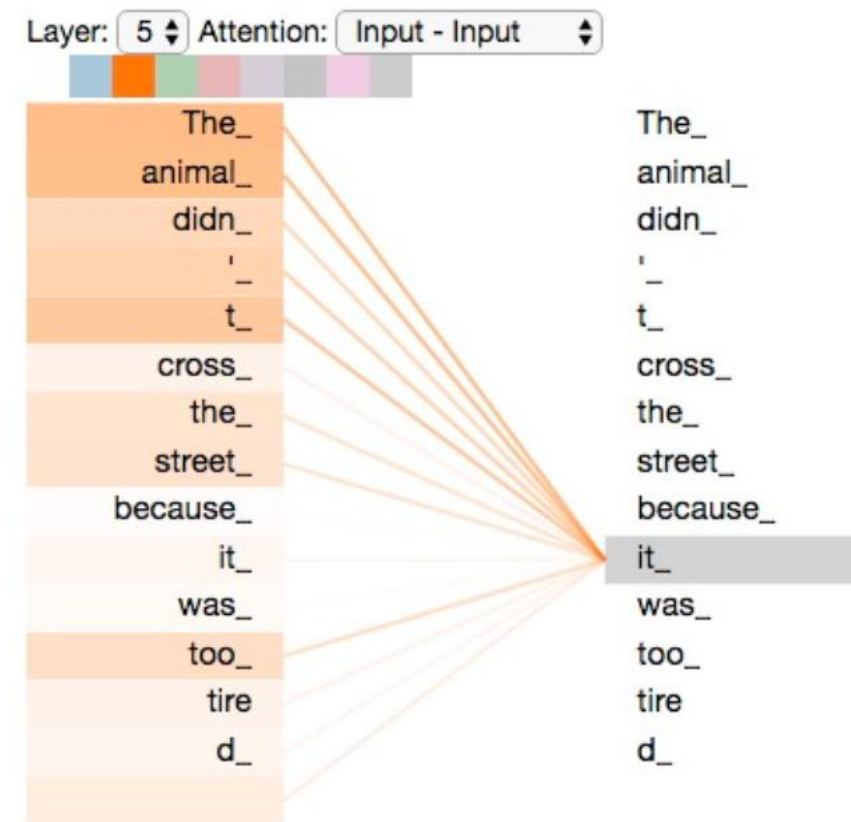
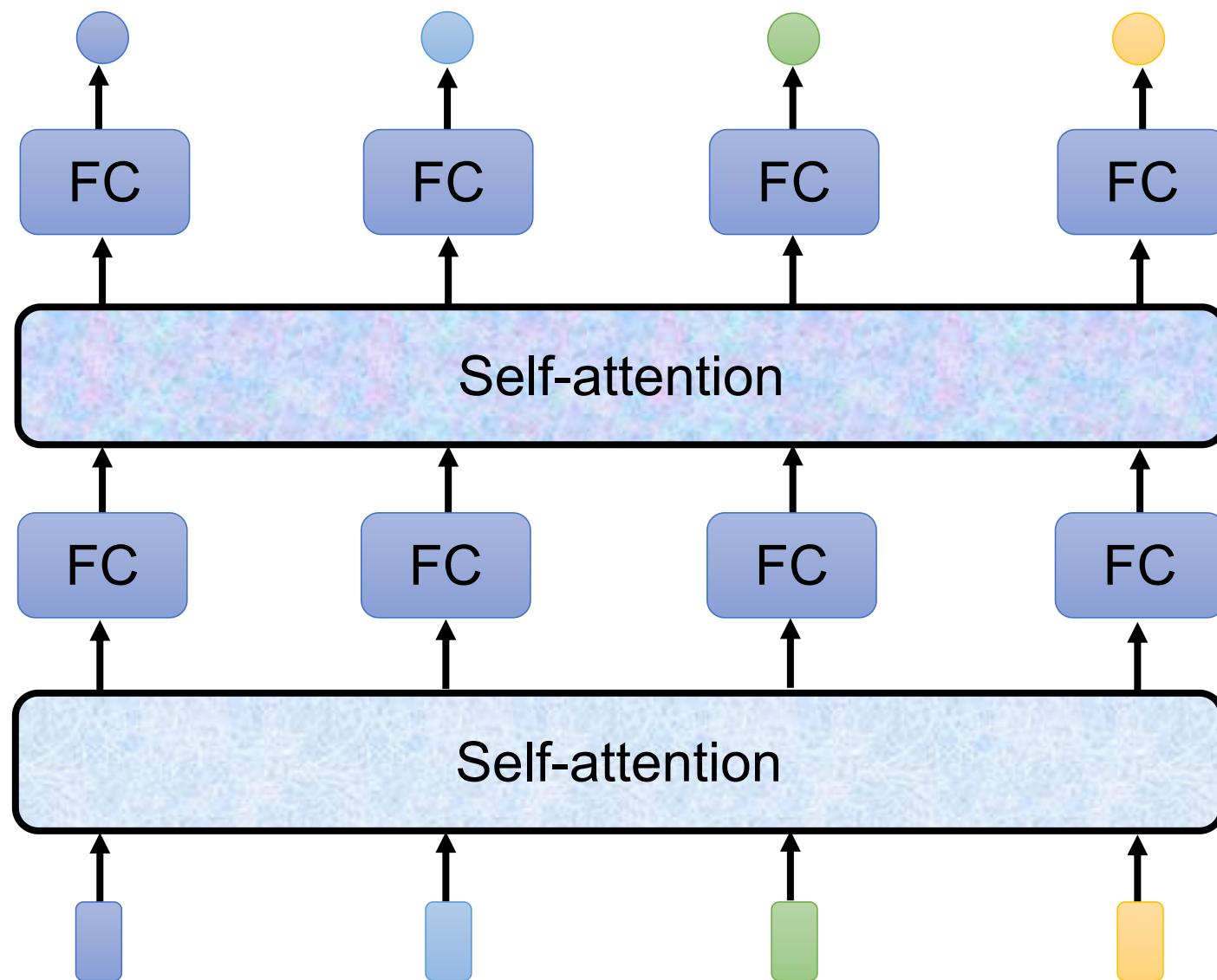


$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

$$\begin{aligned}\mathbf{Q} &= \mathbf{W}^q \mathbf{I} \\ \mathbf{K} &= \mathbf{W}^k \mathbf{I} \\ \mathbf{V} &= \mathbf{W}^v \mathbf{I}\end{aligned}$$

Parameters
to be learned



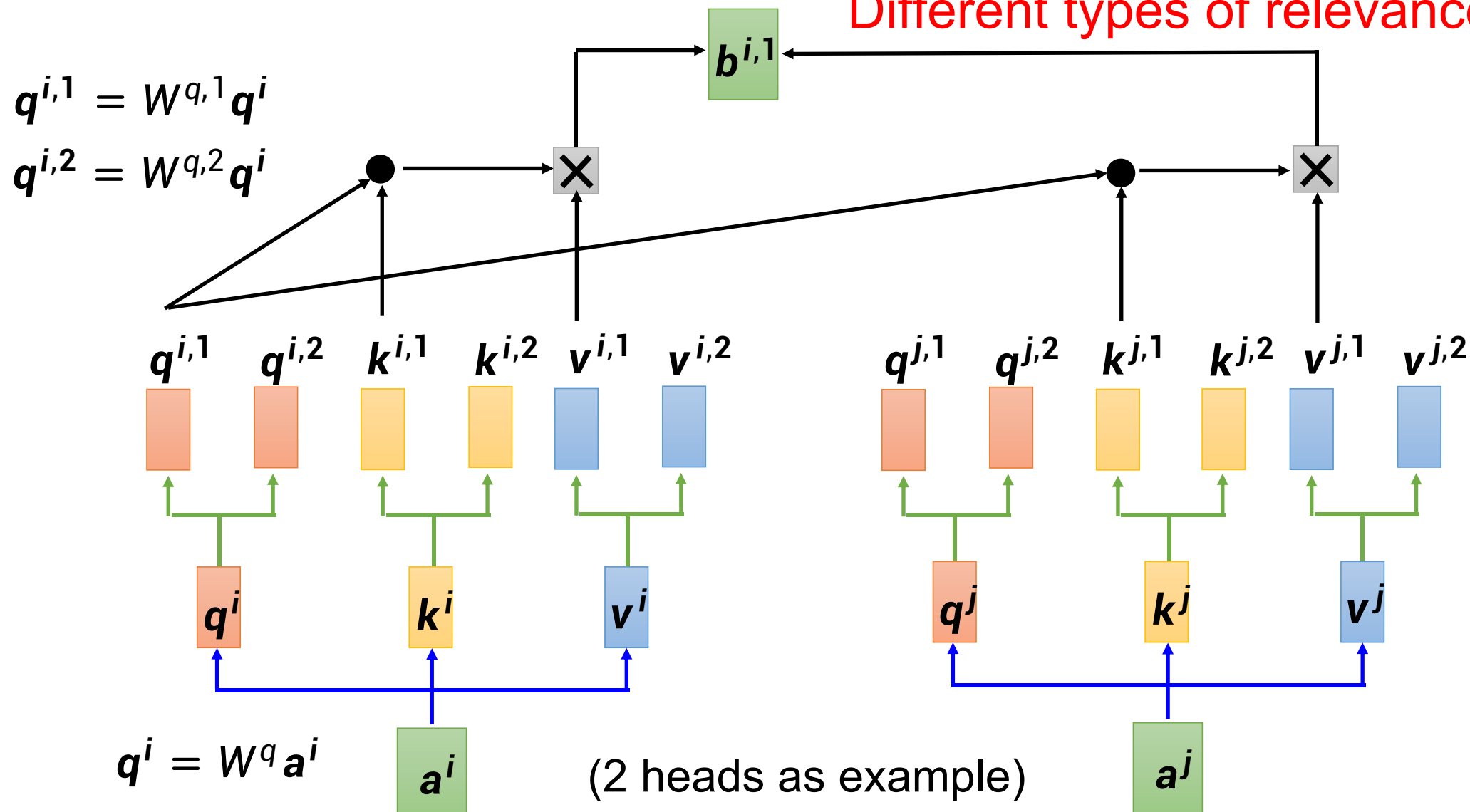


When the model is processing the word "it", self-attention allows it to associate "it" with "animal".

Multi-head Self-attention

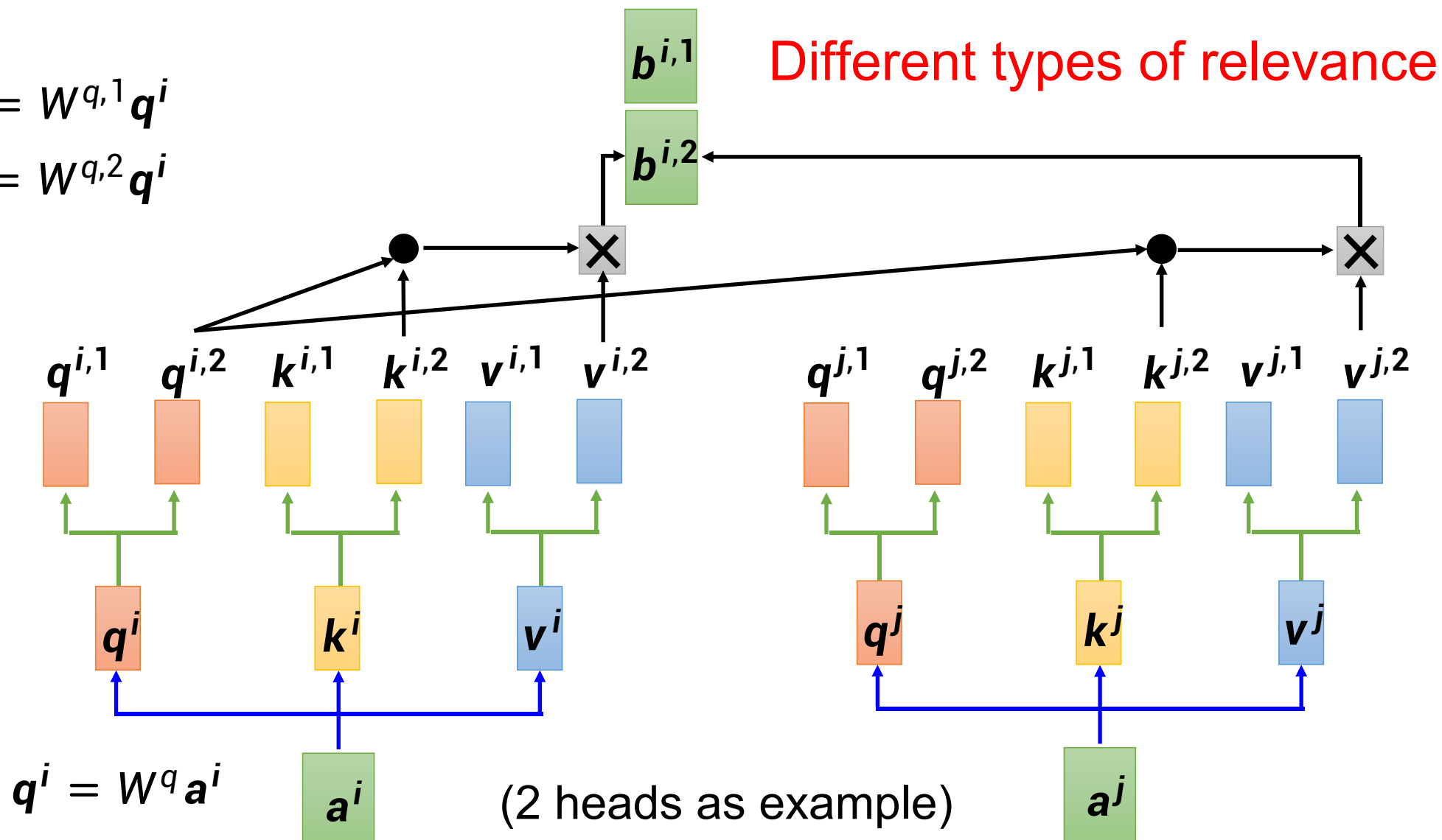


Different types of relevance



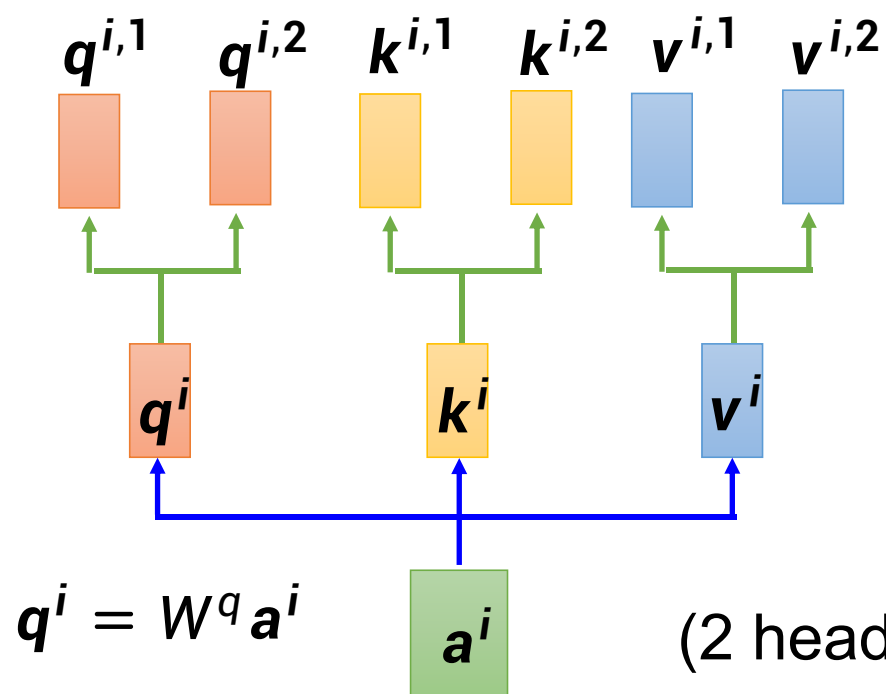
$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$

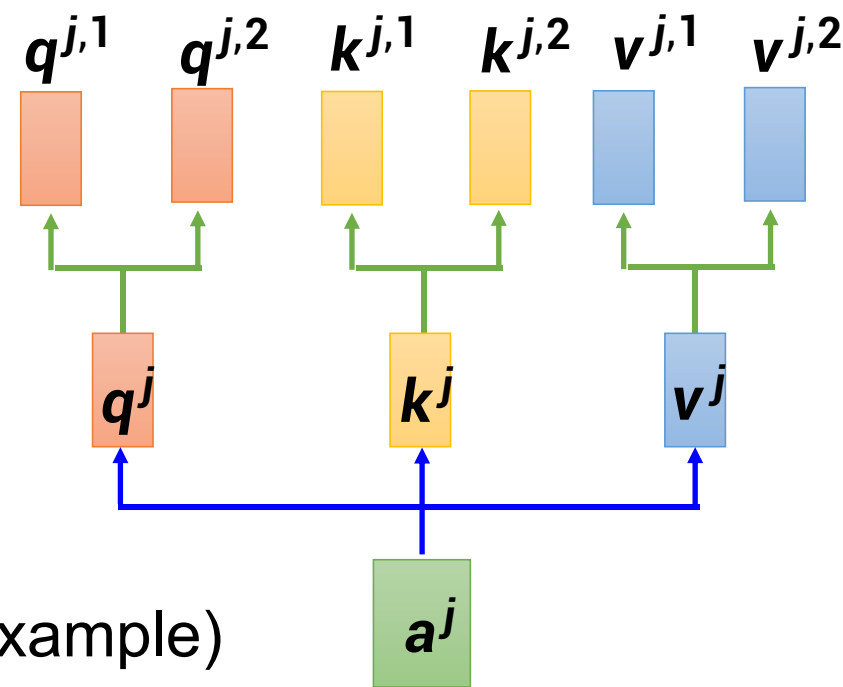


$$b^i = W^O \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$

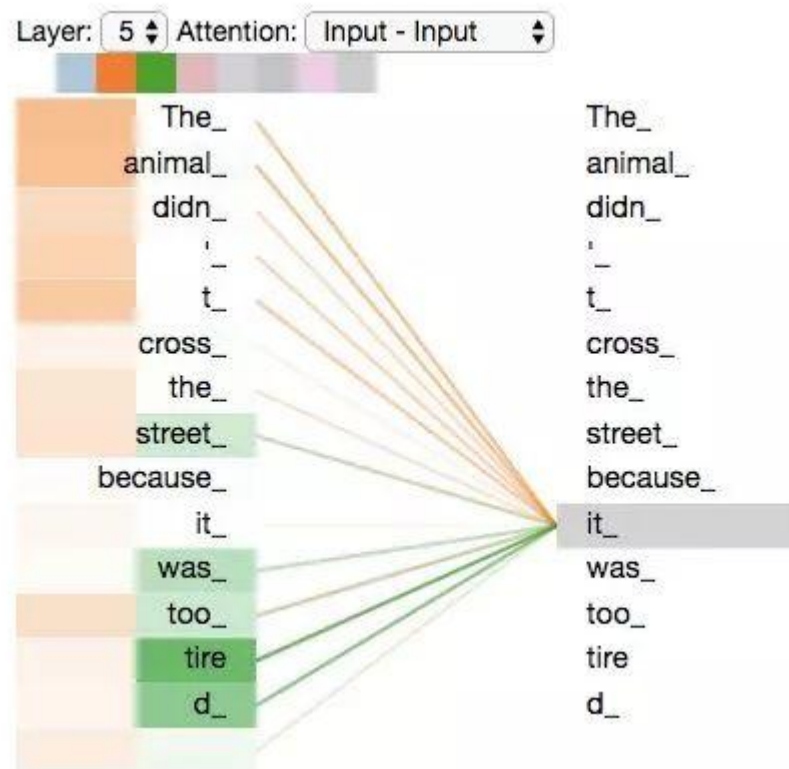
Different types of relevance



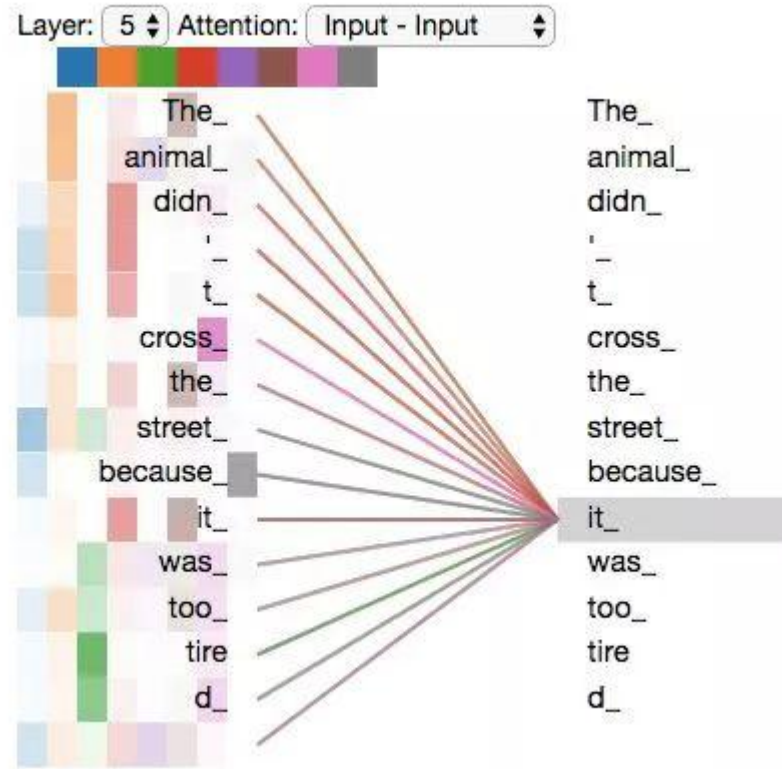
(2 heads as example)



- The model's representation of the word "it" bakes in some of the representation of both "animal" and "tired"

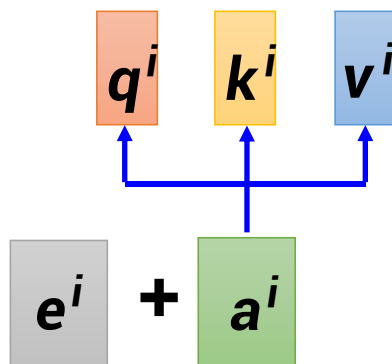


2 heads

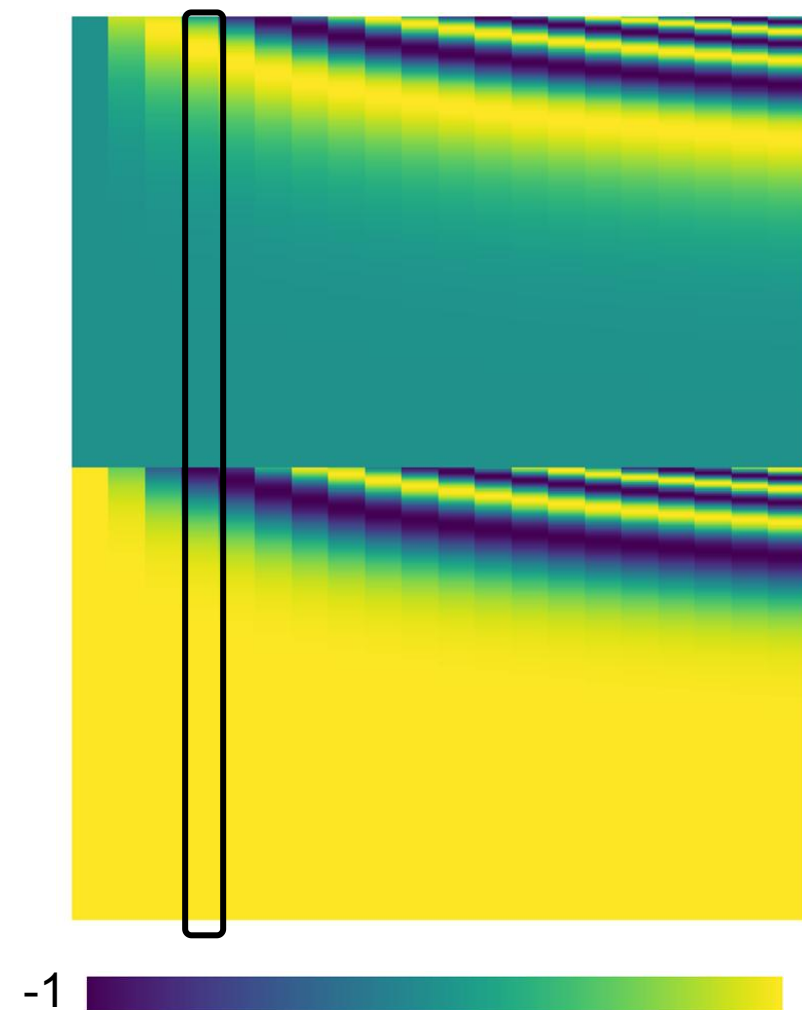


7 heads

- No position information in self-attention
- Each position has a unique positional vector e^i
- **hand-crafted**
- **learned from data**



Each column represents a positional vector e^i



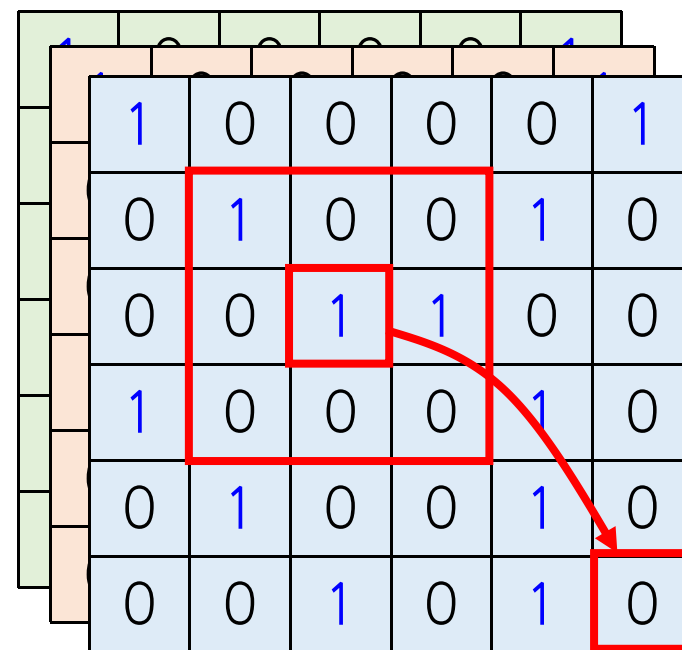
Self-attention v.s. CNN

CNN: self-attention that can only attends in a receptive field

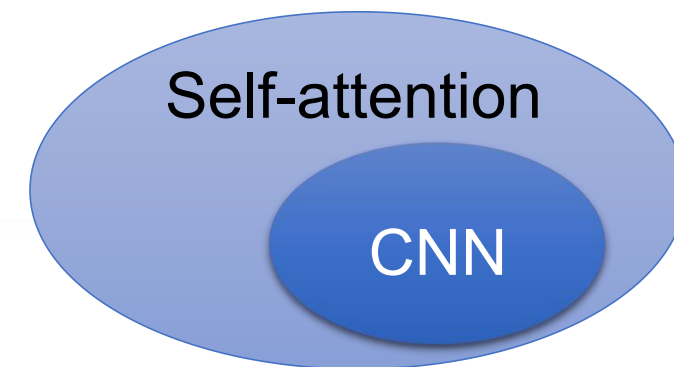
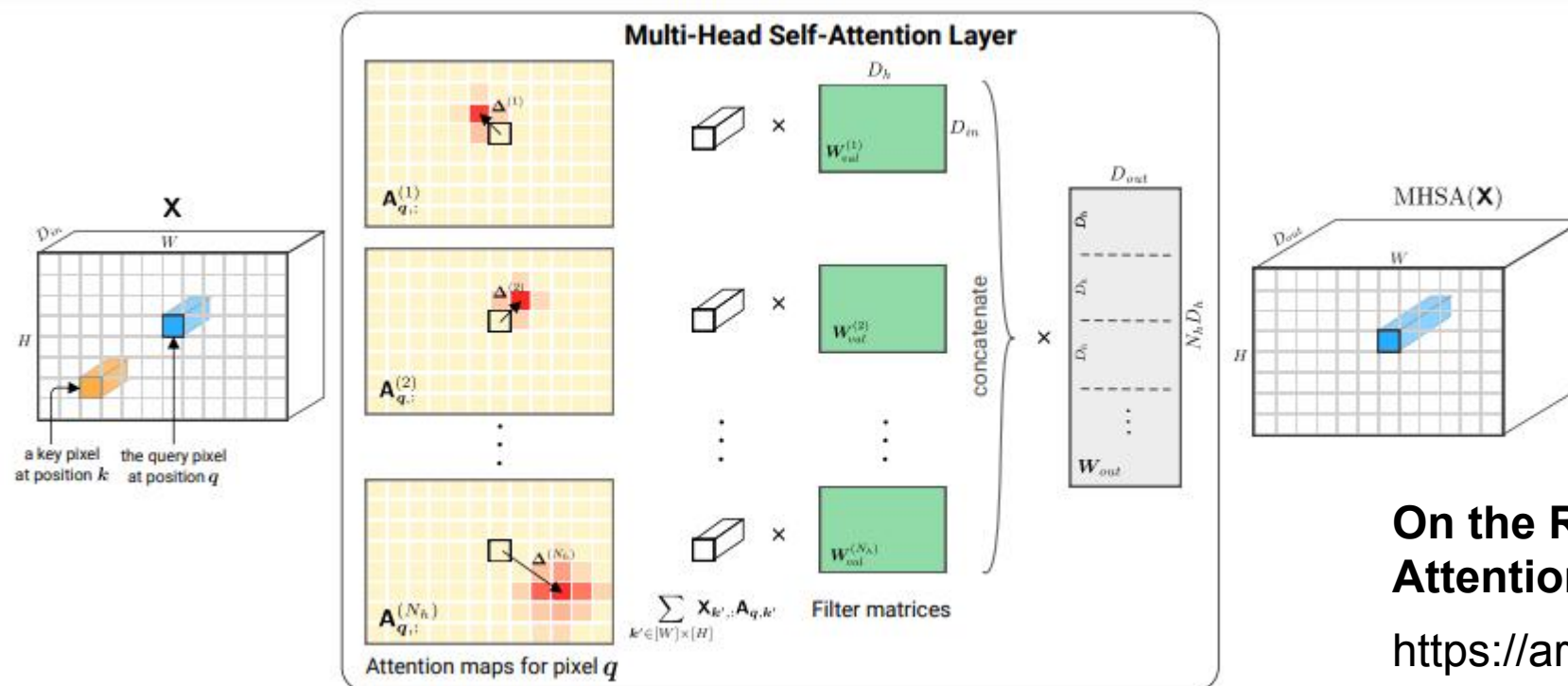
- CNN is simplified self-attention.

Self-attention: CNN with learnable receptive field

- Self-attention is the complex version of CNN.



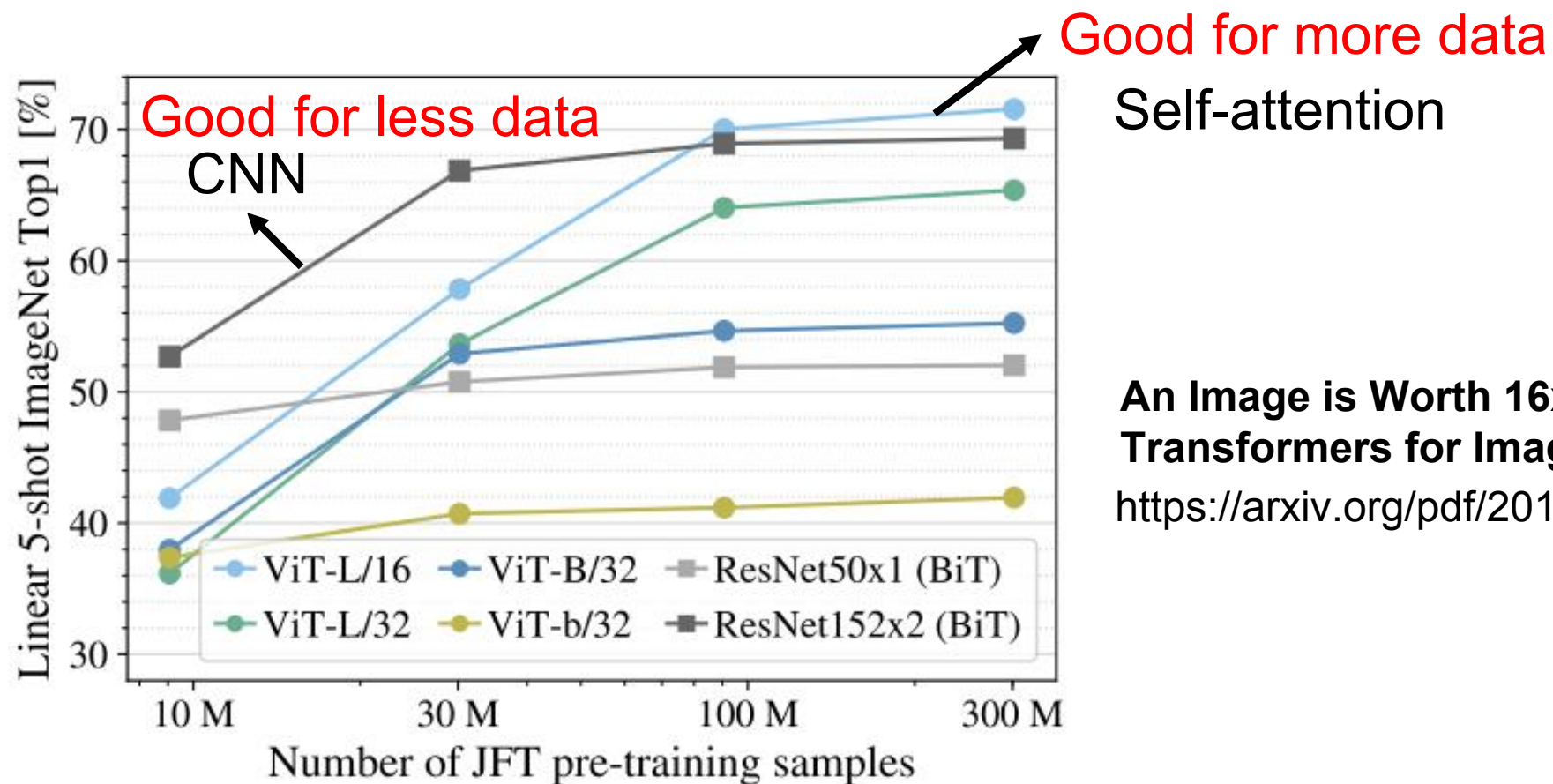
Self-attention v.s. CNN



On the Relationship between Self-Attention and Convolutional Layers

<https://arxiv.org/abs/1911.03584>

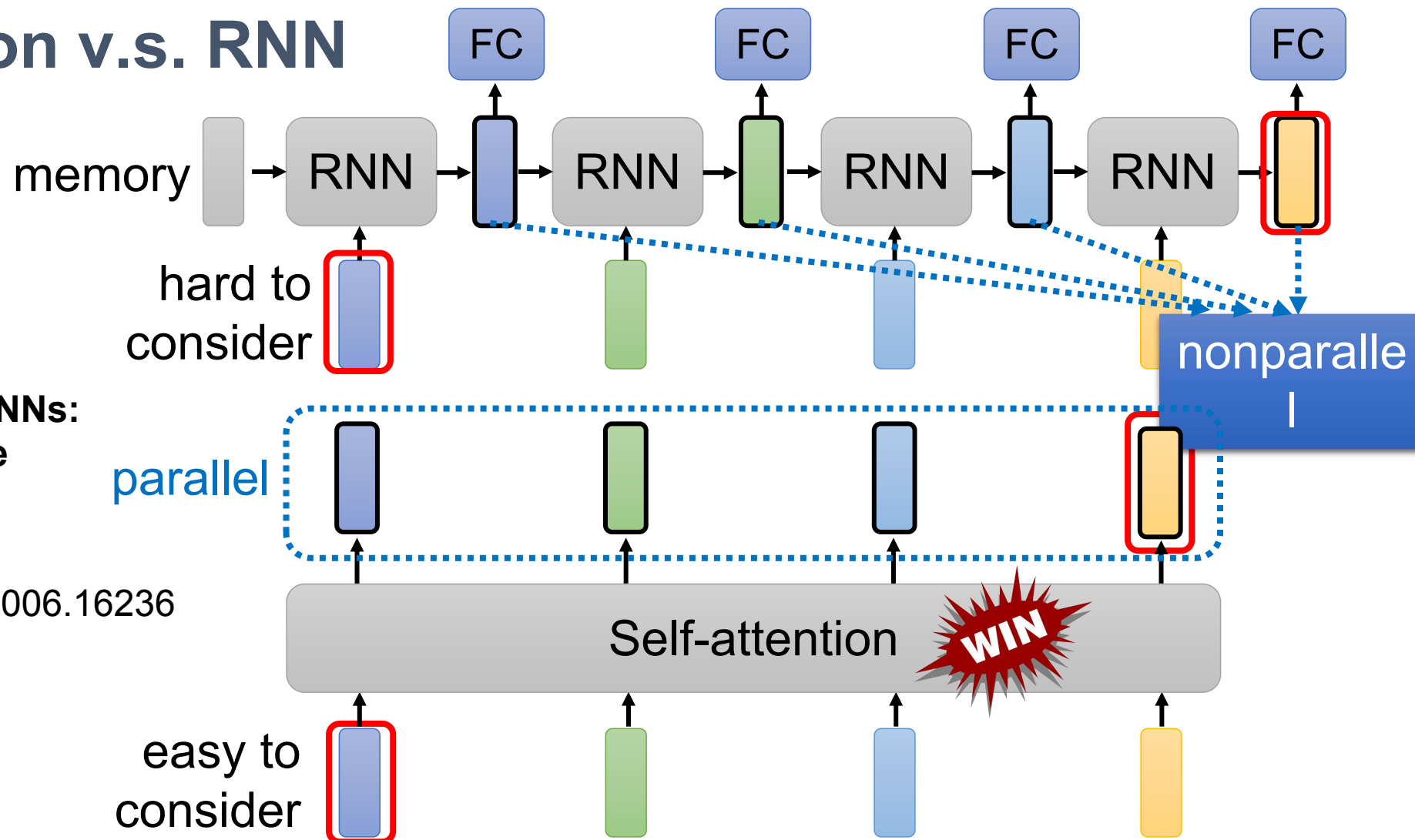
Self-attention v.s. CNN



**An Image is Worth 16x16 Words:
Transformers for Image Recognition at Scale**
<https://arxiv.org/pdf/2010.11929.pdf>

Self-attention v.s. RNN

Recurrent Neural Network (RNN)



Transformers are RNNs:
Fast Autoregressive
Transformers with
Linear Attention

<https://arxiv.org/abs/2006.16236>

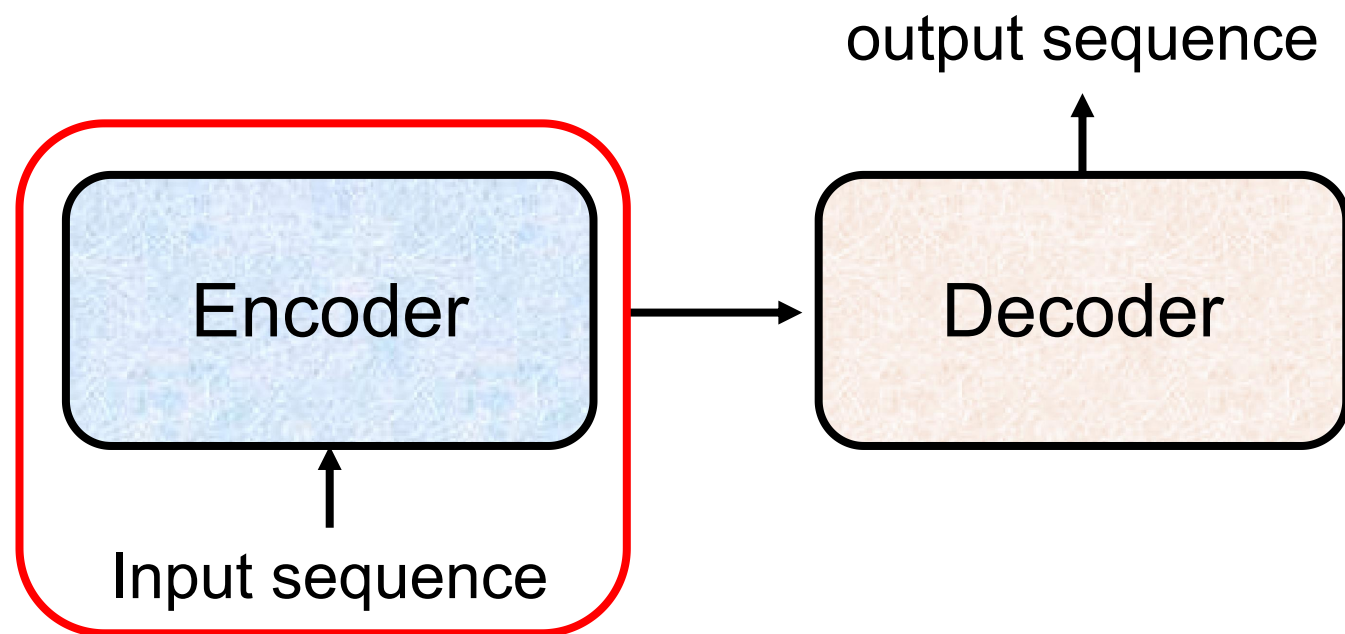


上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

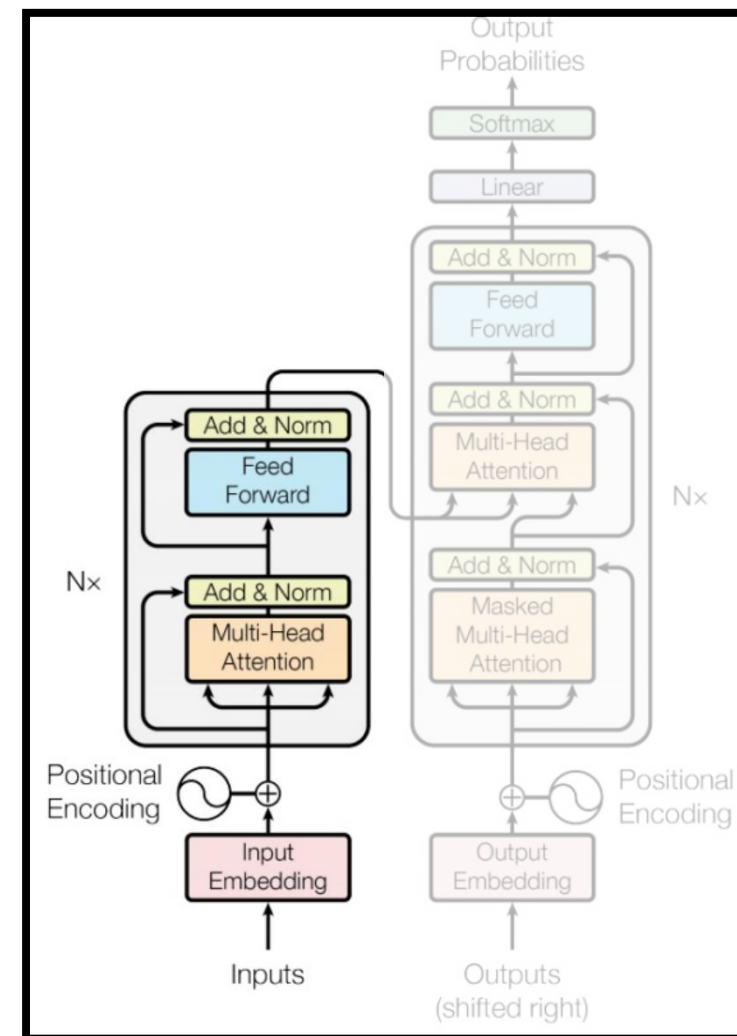
Contents

- Self-attention
- **Encoder**
- Decoder
- Encoder-Decoder
- NLP/Vision Transformer
- Mixture of Experts

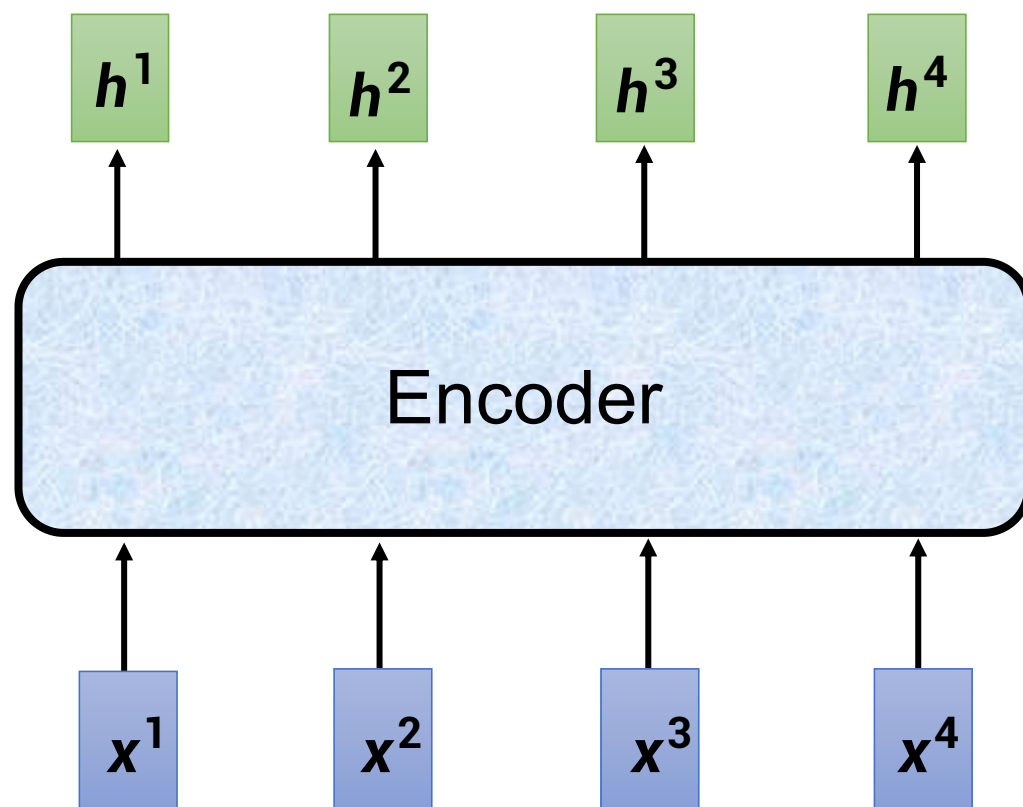




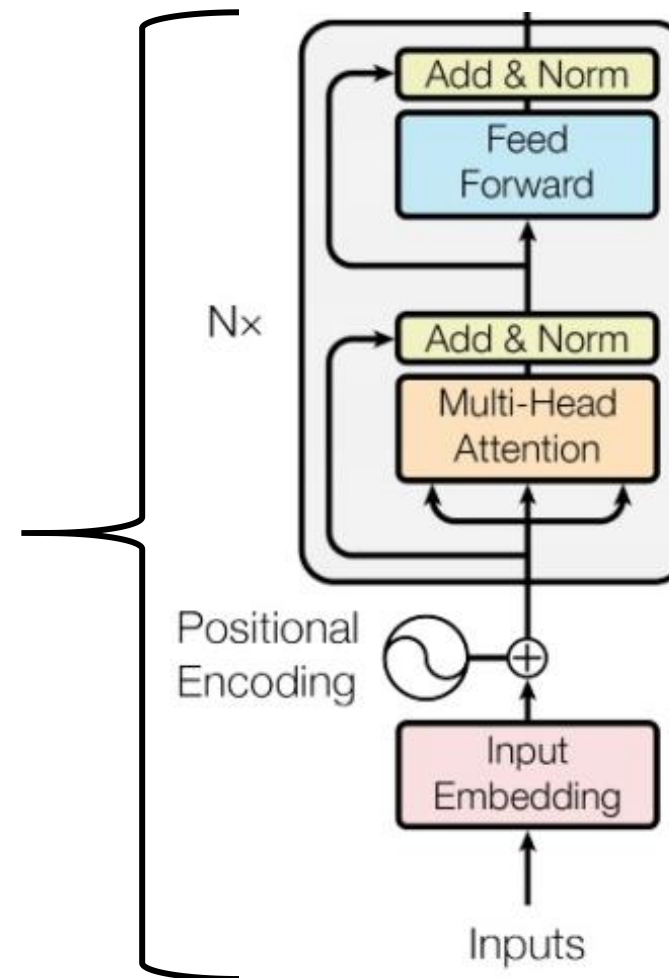
Transformer
<https://arxiv.org/abs/1706.03762>

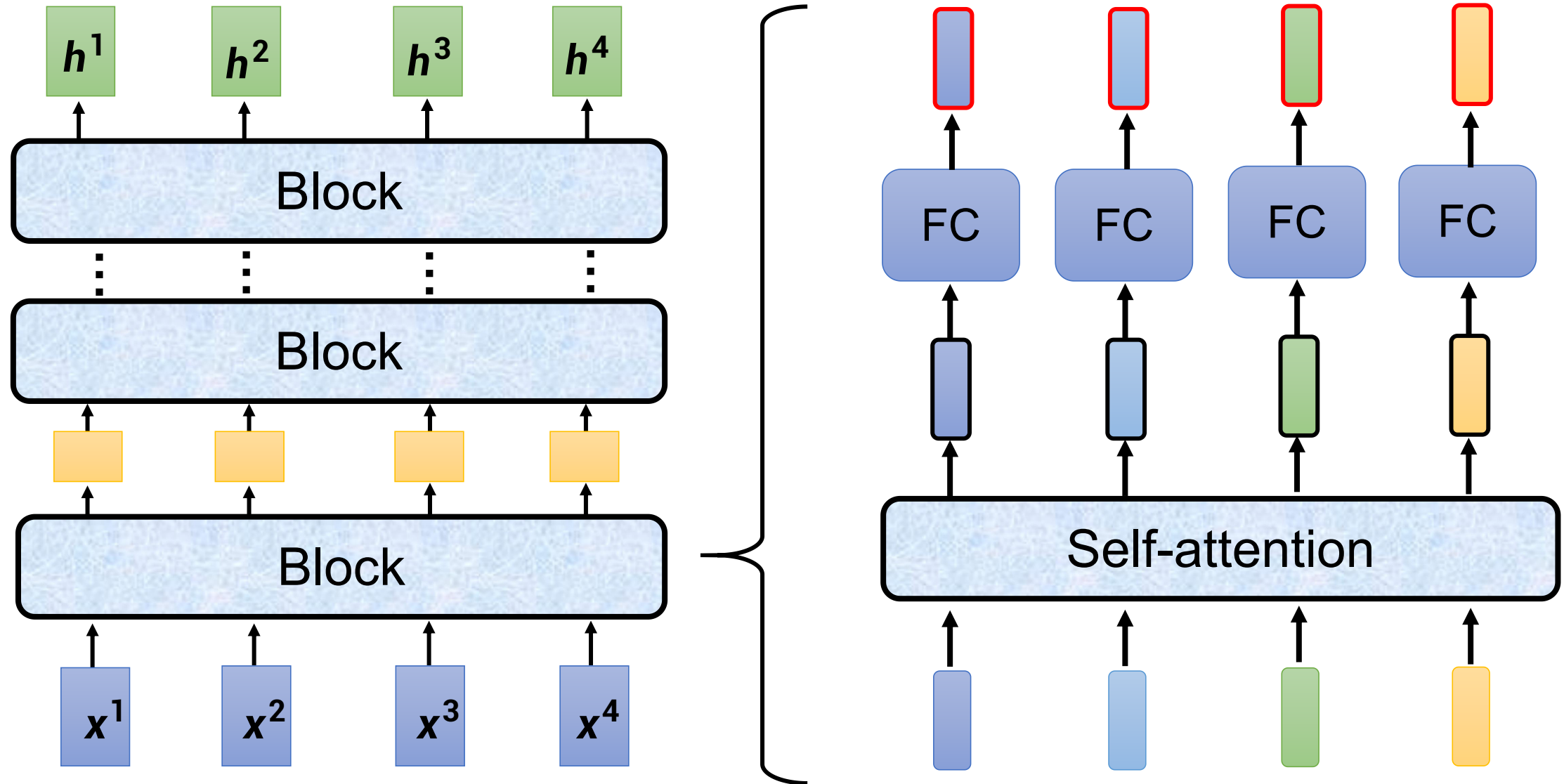


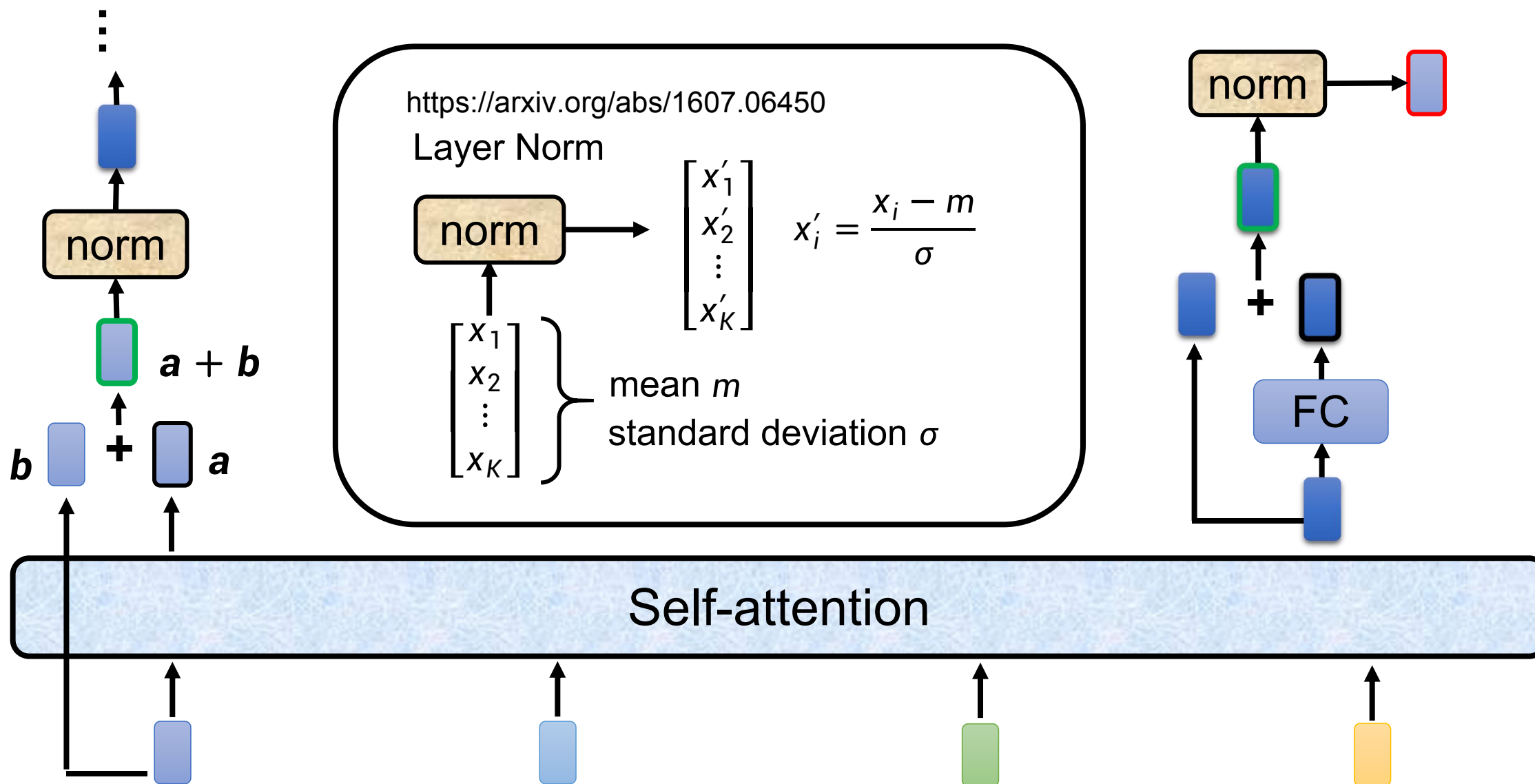
You can use **RNN** or **CNN**.



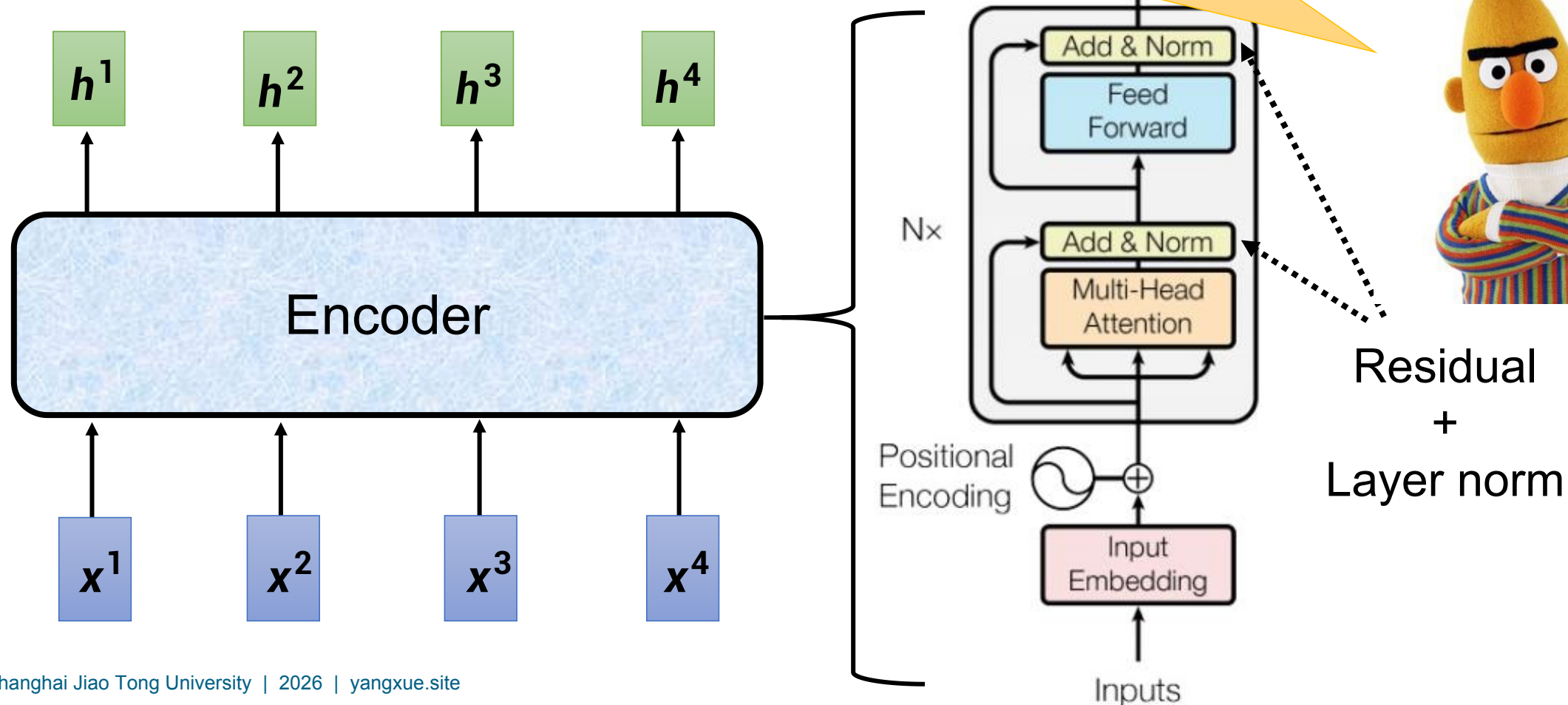
Transformer's Encoder







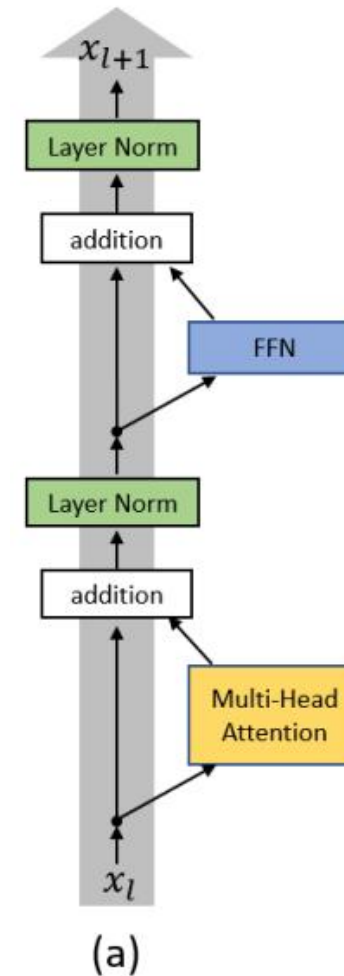
I use the **same** network architecture as **transformer encoder**.



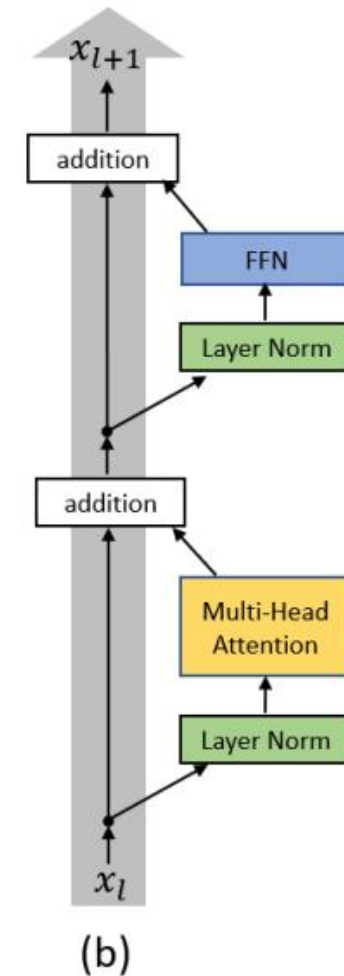
BERT

To learn more.....

- **On Layer Normalization in the Transformer Architecture**
<https://arxiv.org/abs/2002.04745>
- **PowerNorm: Rethinking Batch Normalization in Transformers**
<https://arxiv.org/abs/2003.07845>



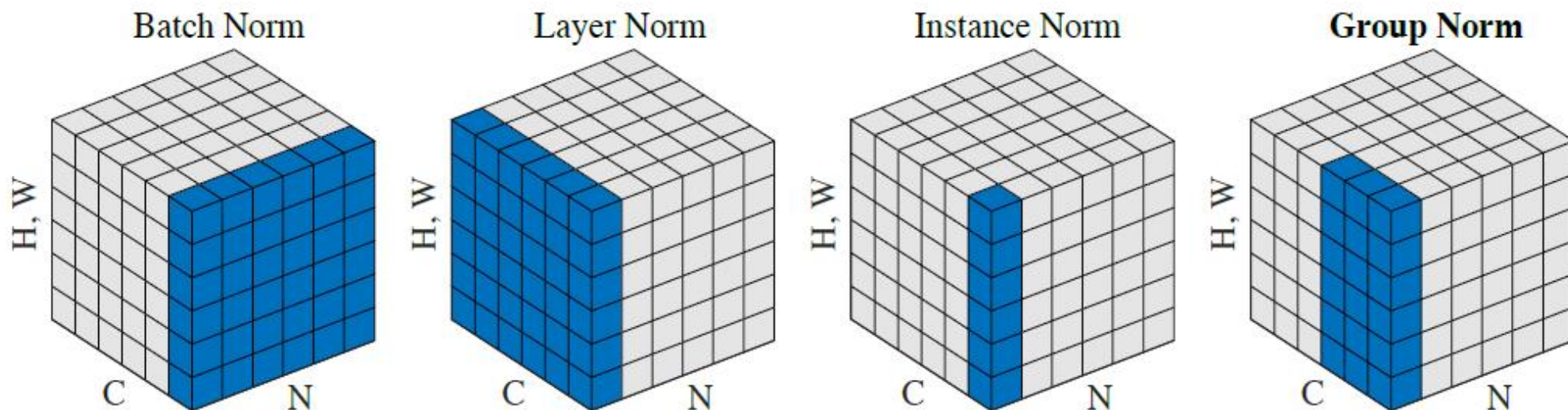
Post-LN



Pre-LN

	Post-LN $\text{LN}(x + \text{Module}(x))$	Pre-LN $x + \text{Module}(\text{Norm}(x))$
Definition	Normalization is applied after the self-attention mechanism or feed-forward network.	Normalization is applied before the self-attention mechanism or feed-forward network.
Training Stability	More stable in the early stages of training, but may become unstable later.	May be unstable in the early stages of training, but becomes stable later.
Gradient Behavior	Prone to gradient vanishing, especially in deep networks.	Alleviates gradient vanishing, making it suitable for deep networks.
Applicability	Suitable for relatively shallow Transformer architectures or tasks requiring initial training stability.	Suitable for deep networks and large-scale pretraining language models.

- **Batch Norm**: batch方向做归一化, 算 $N \times H \times W$ 的均值
- **Layer Norm**: channel方向做归一化, 算 $C \times H \times W$ 的均值
- **Instance Norm**: 一个channel内做归一化, 算 $H \times W$ 的均值
- **Group Norm**: 将channel方向分group, 然后每个group内做归一化, 算 $(C//G) \times H \times W$ 的均值





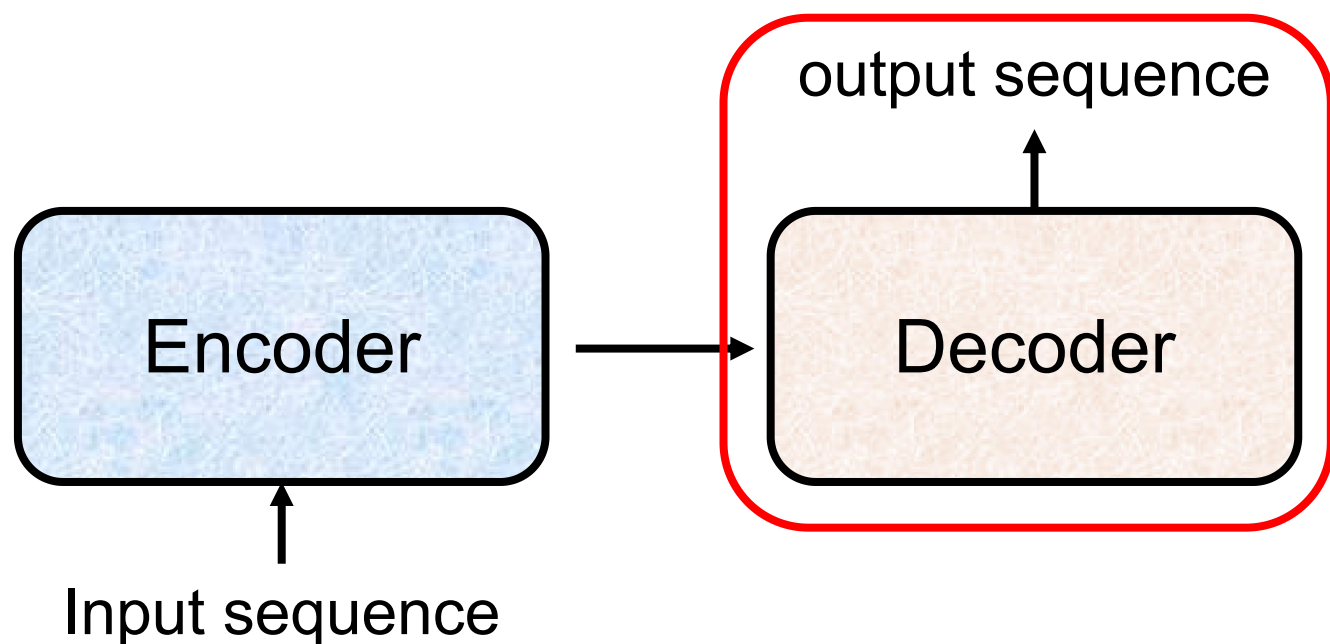
上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

Contents

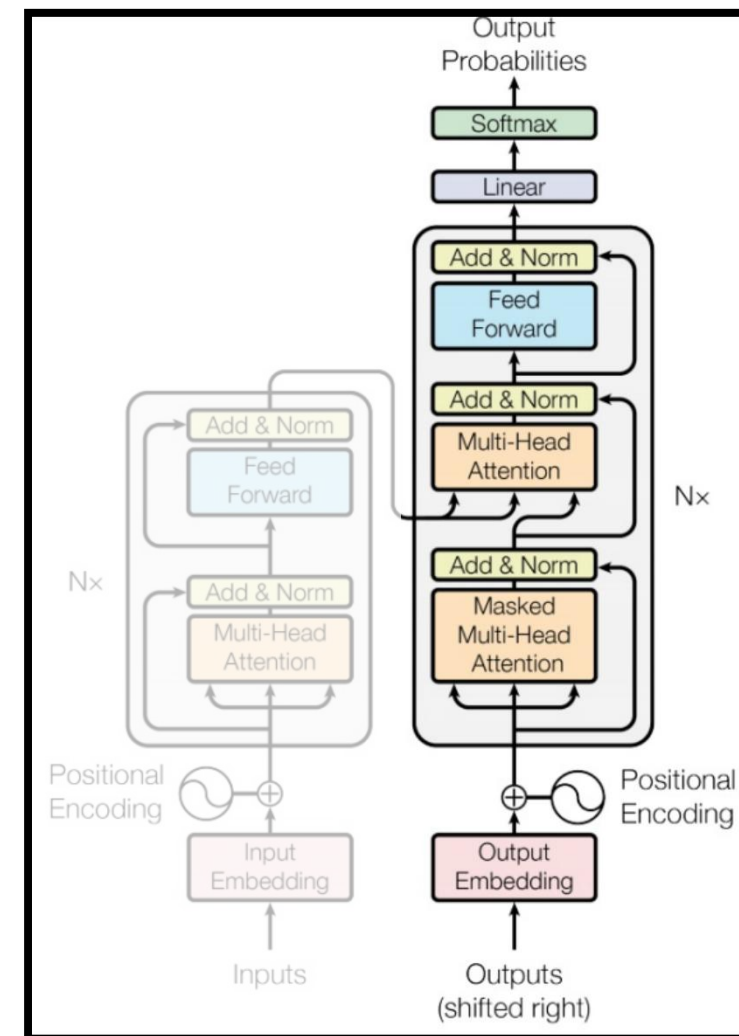
- Self-attention
- Encoder
- **Decoder**
- Encoder-Decoder
- NLP/Vision Transformer
- Mixture of Experts

Autoregressive (AR)

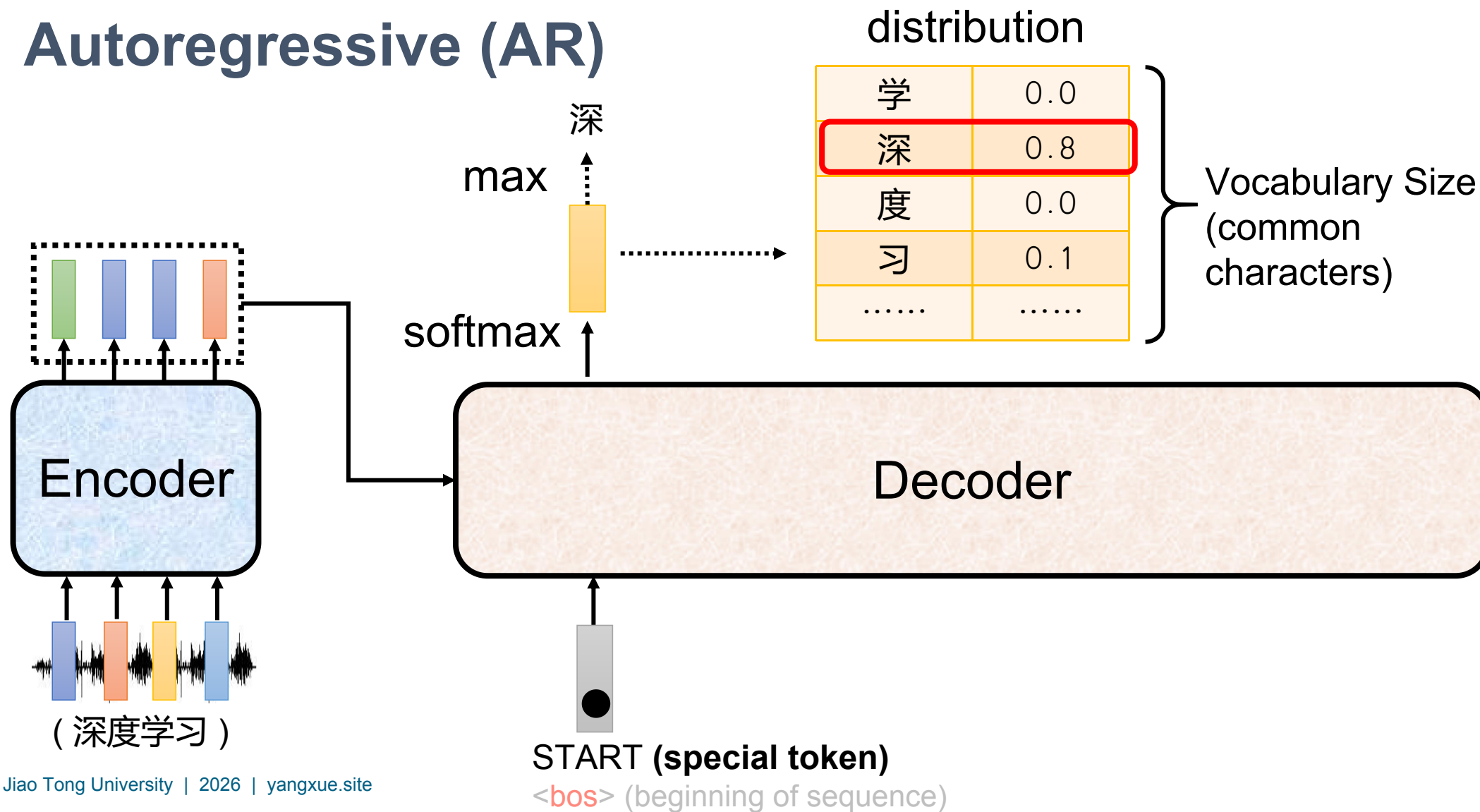


Transformer

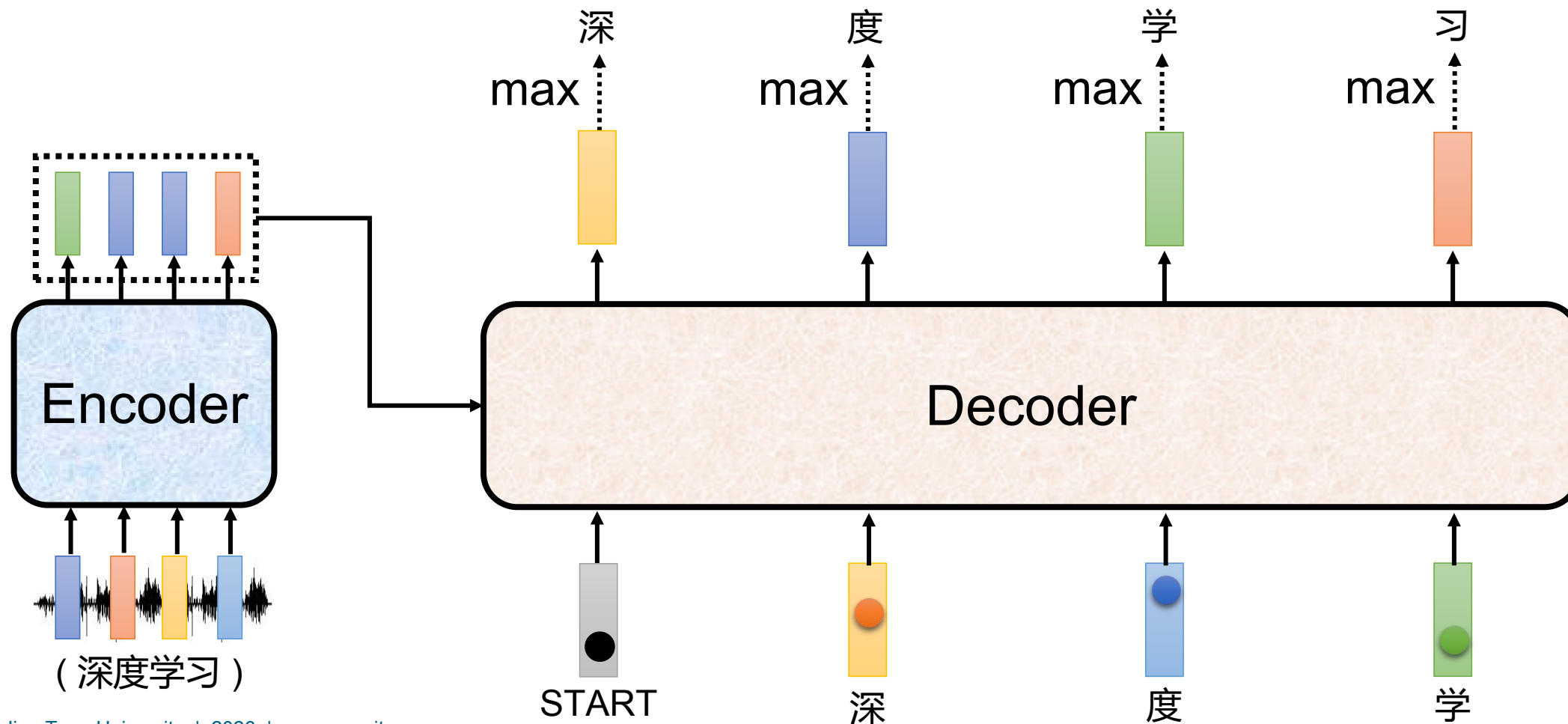
<https://arxiv.org/abs/1706.03762>



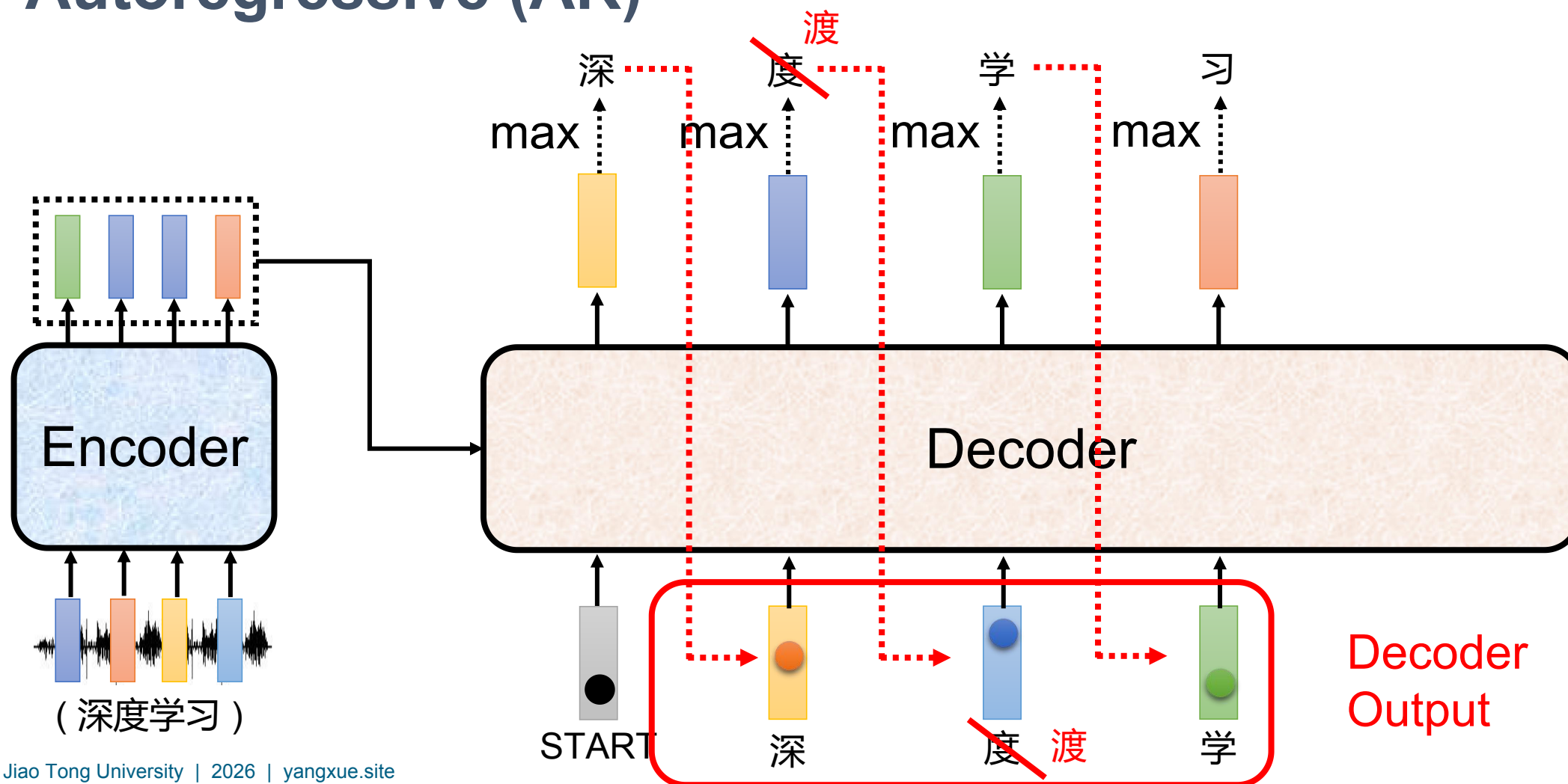
Autoregressive (AR)



Autoregressive (AR)

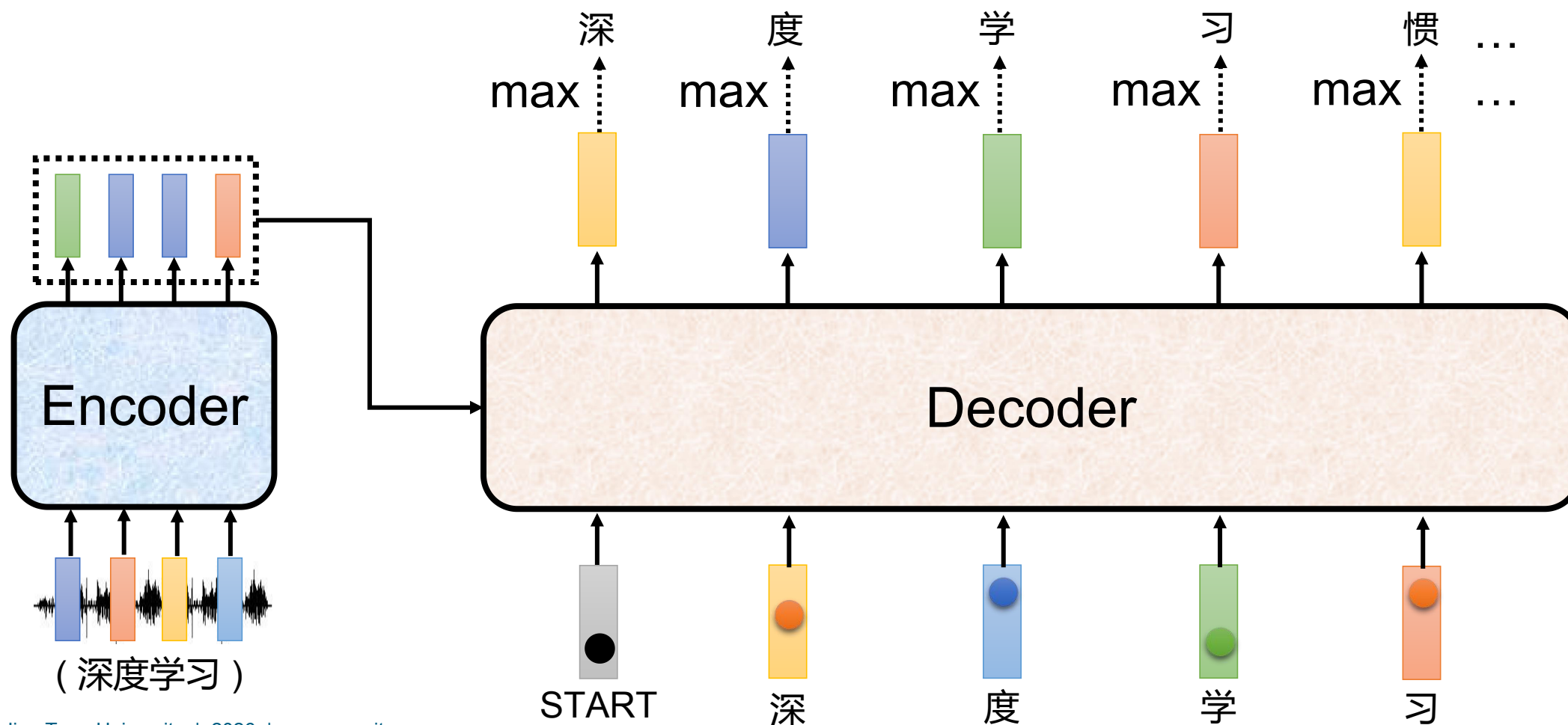


Autoregressive (AR)

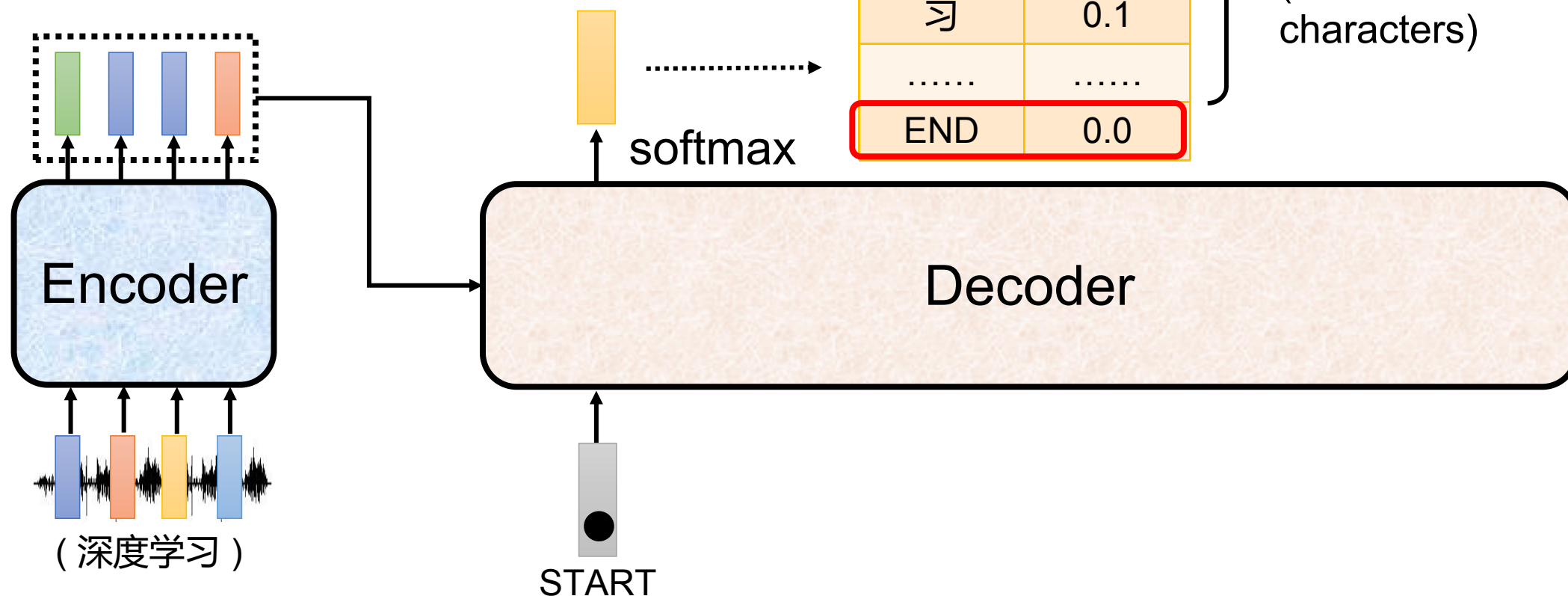


Autoregressive (AR)

We do not know the correct output length.

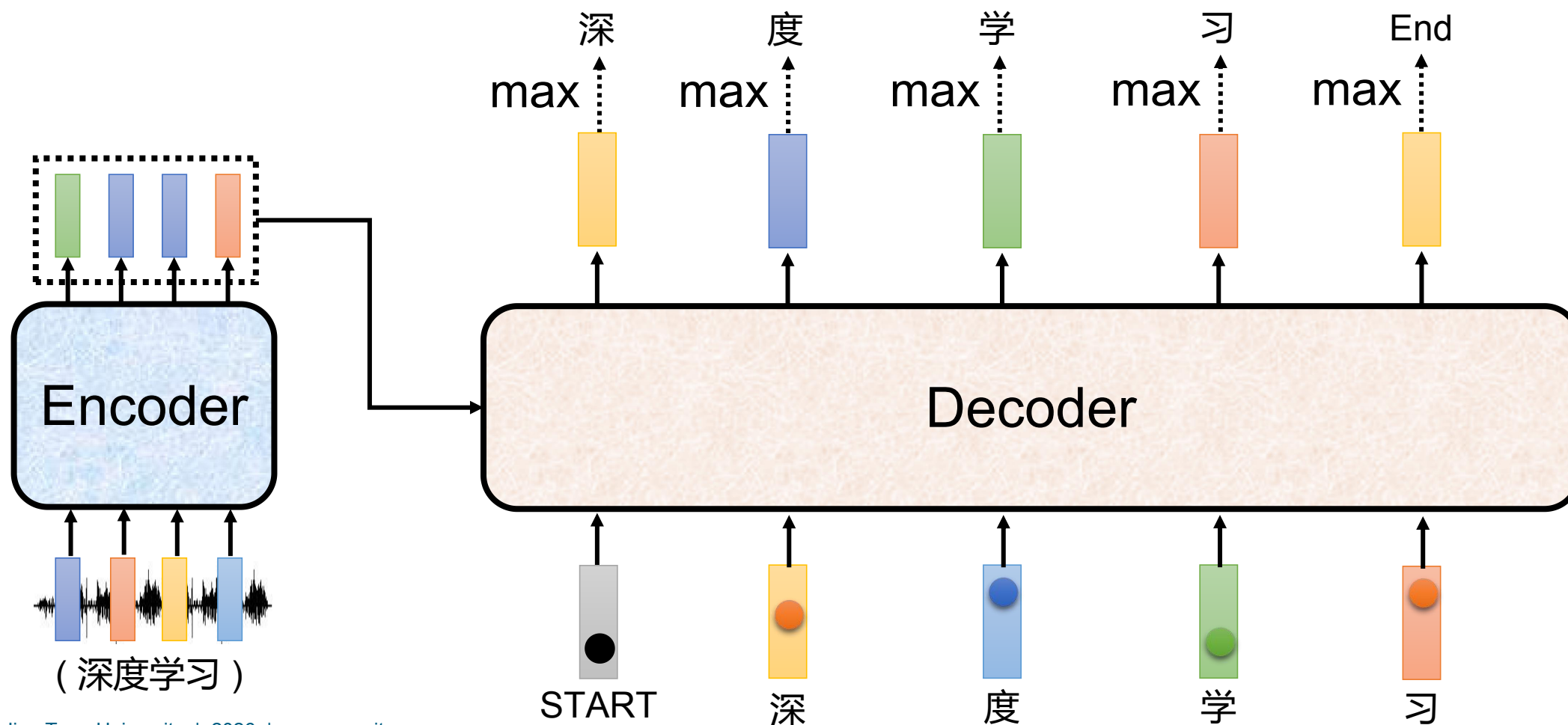


Adding “Stop Token”

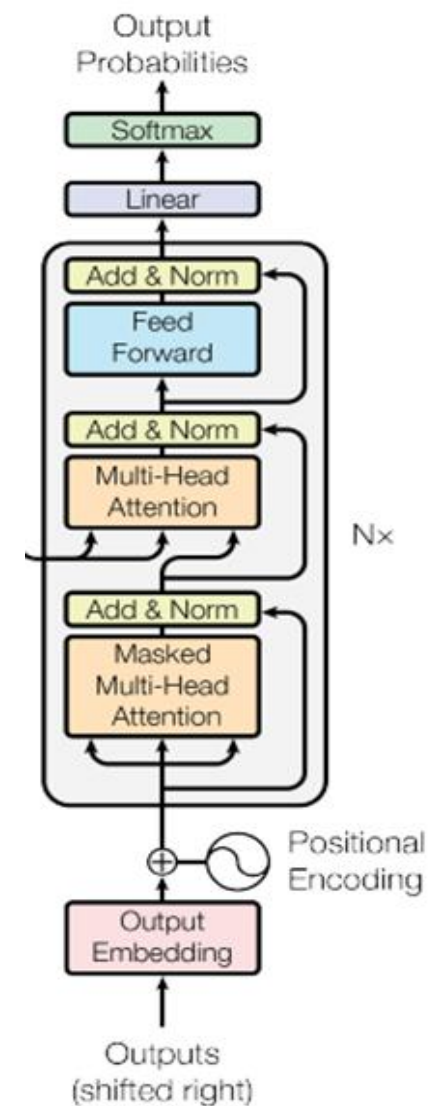
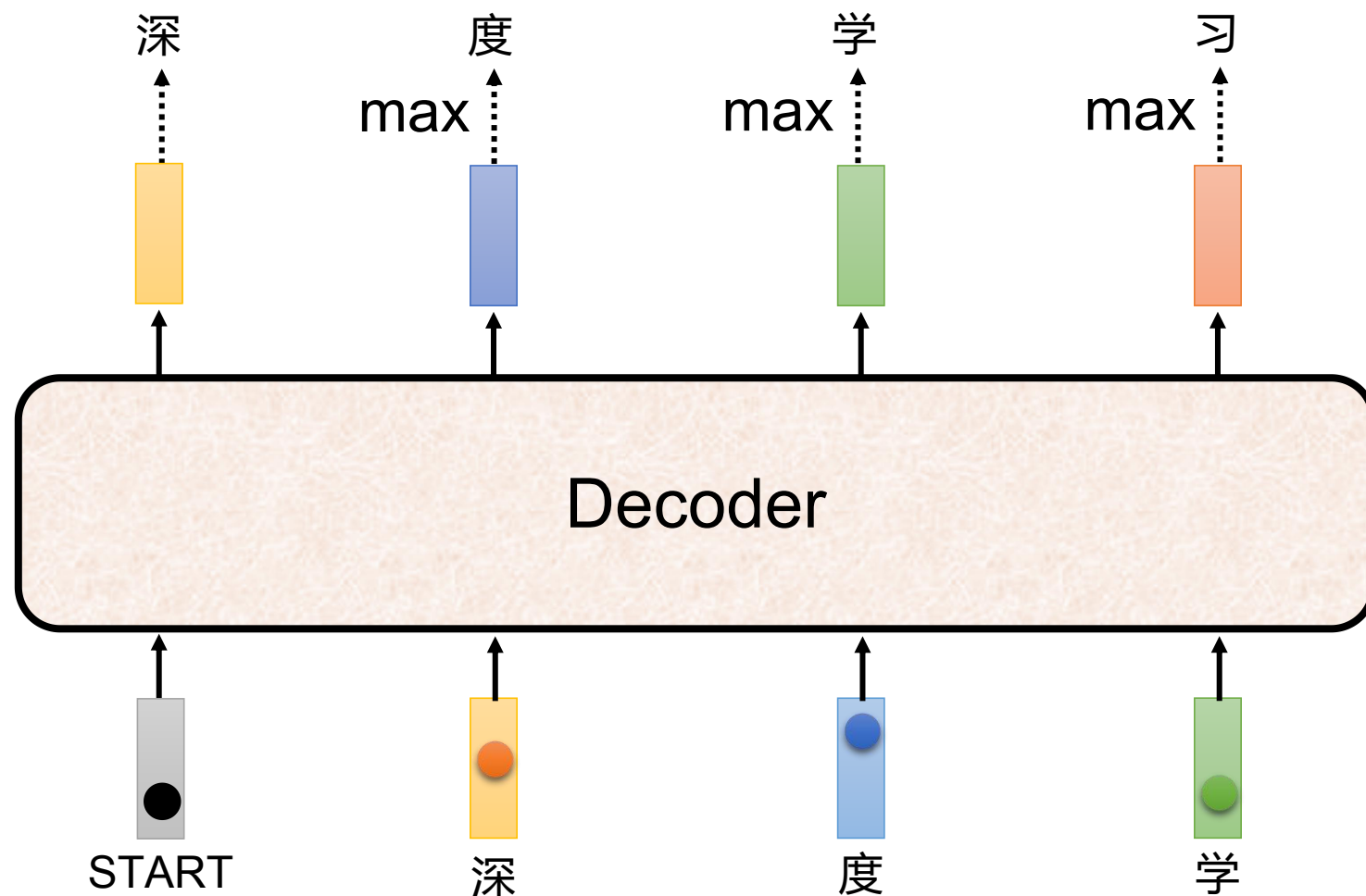


Autoregressive (AR)

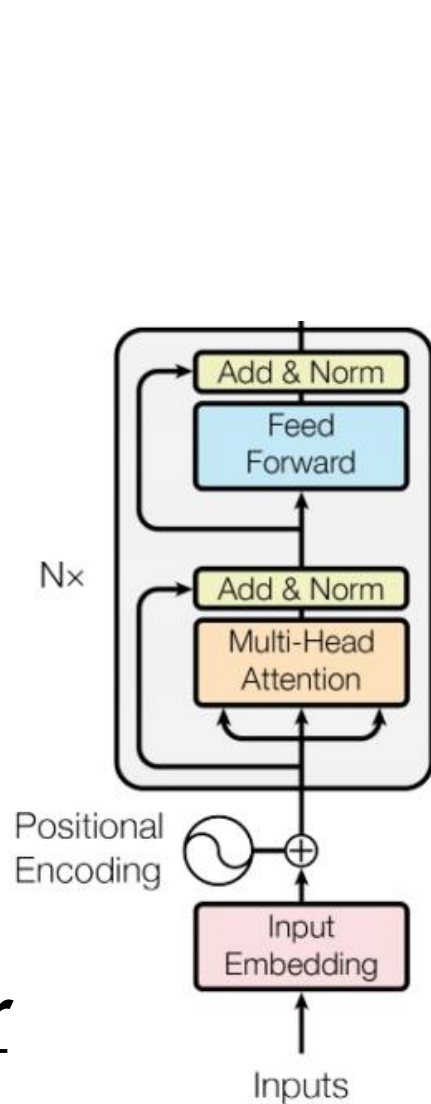
Stop at here!



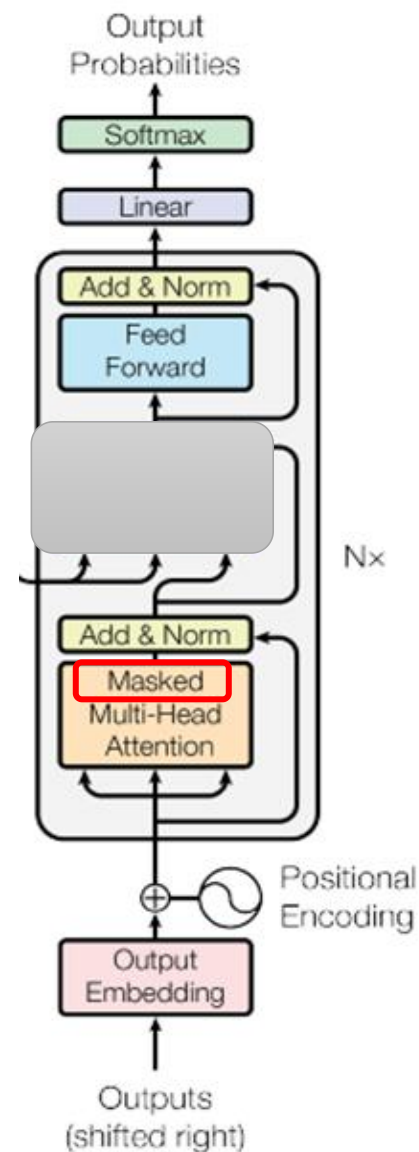
Ignore the input from the encoder here 😊



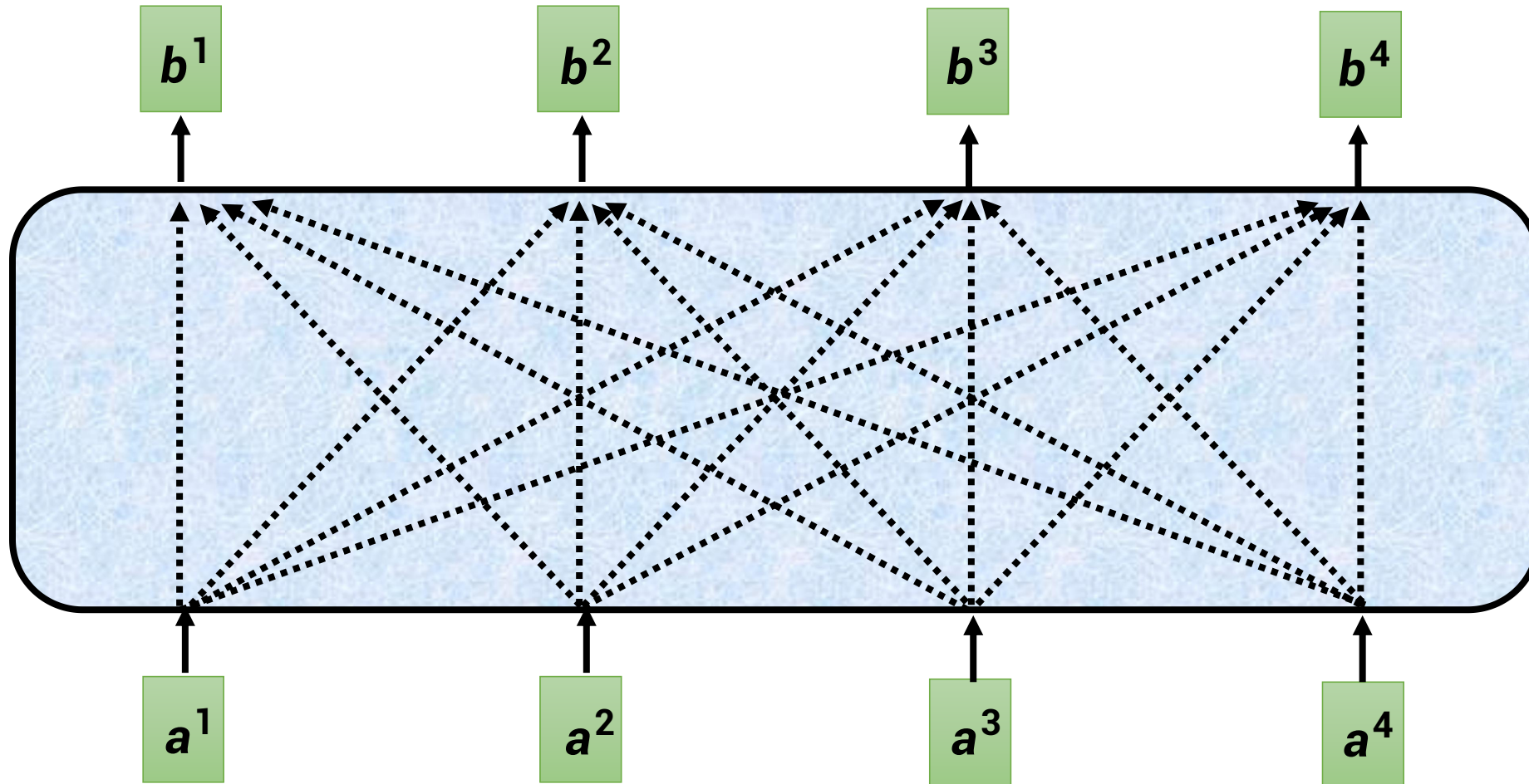
Encoder



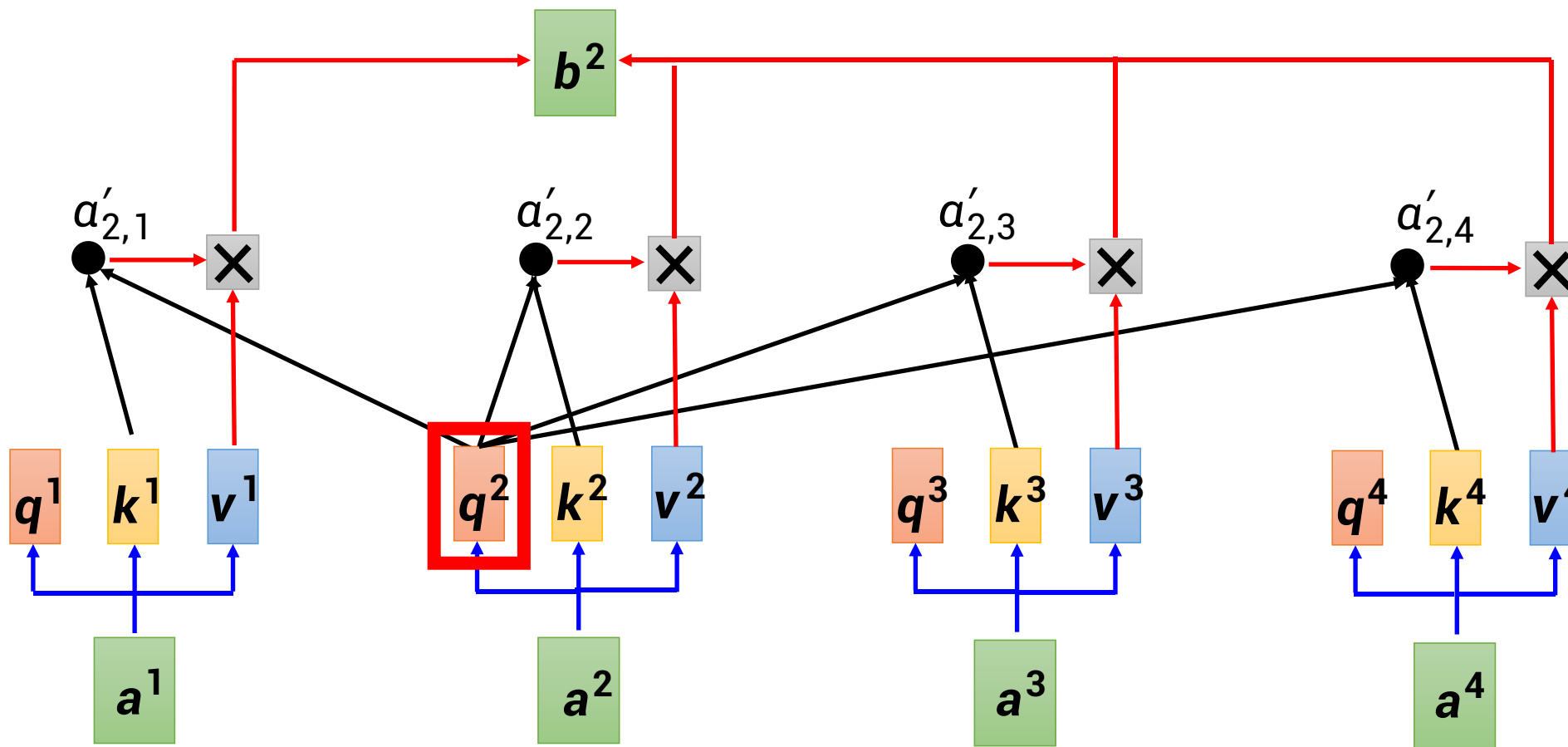
Decoder



Self-attention $\rightarrow$ Masked Self-attention

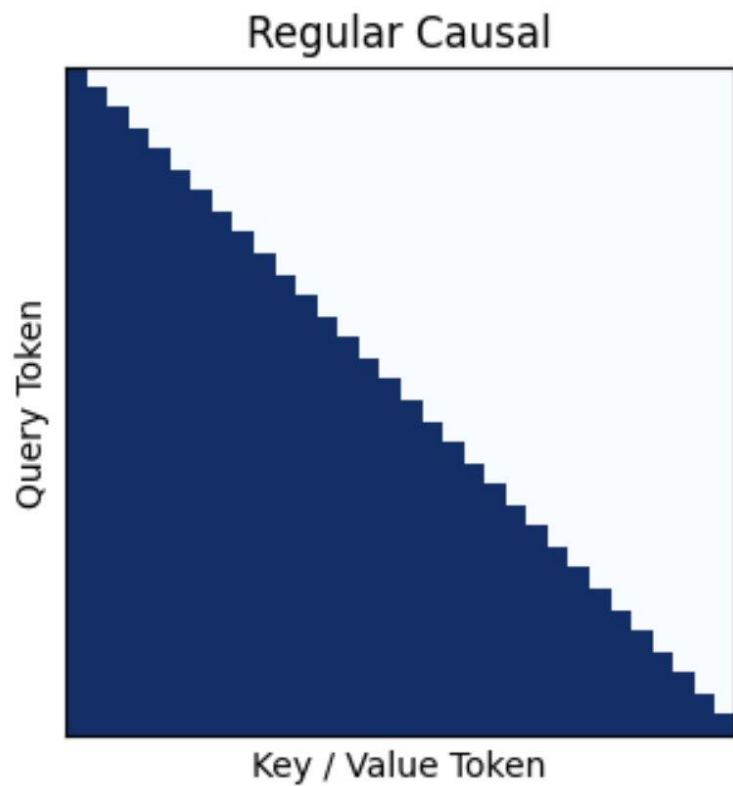


Self-attention $\rightarrow$ Masked Self-attention



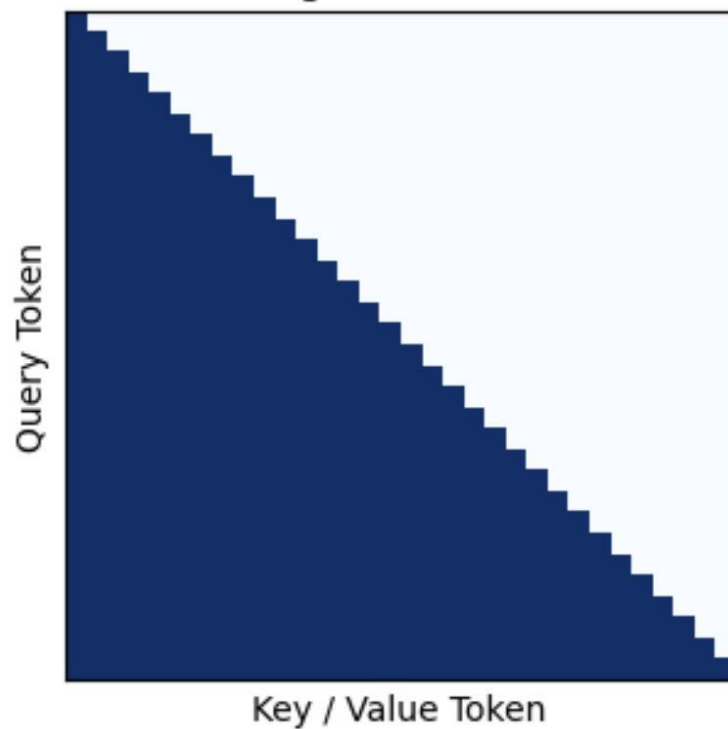
Why masked? Consider how does decoder work

Attention Mask

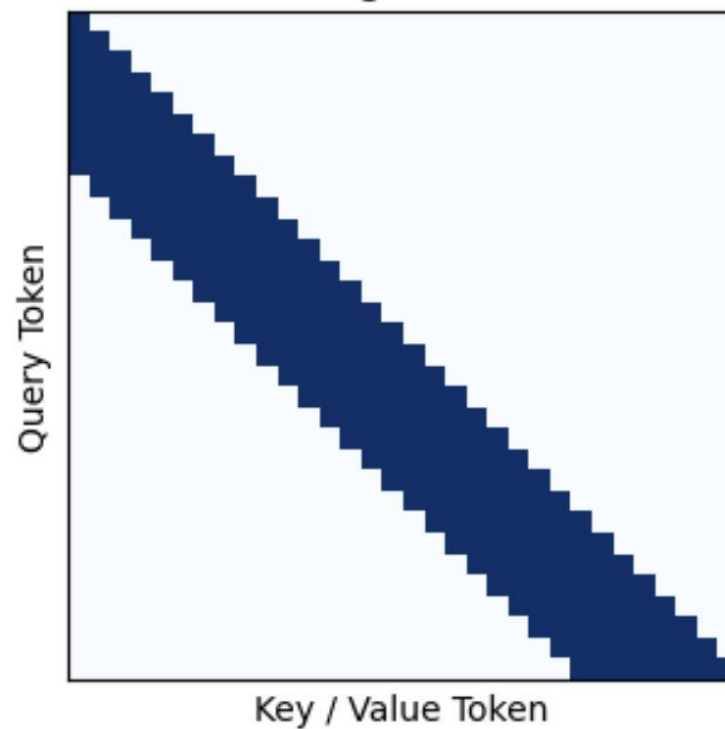


Attention Mask

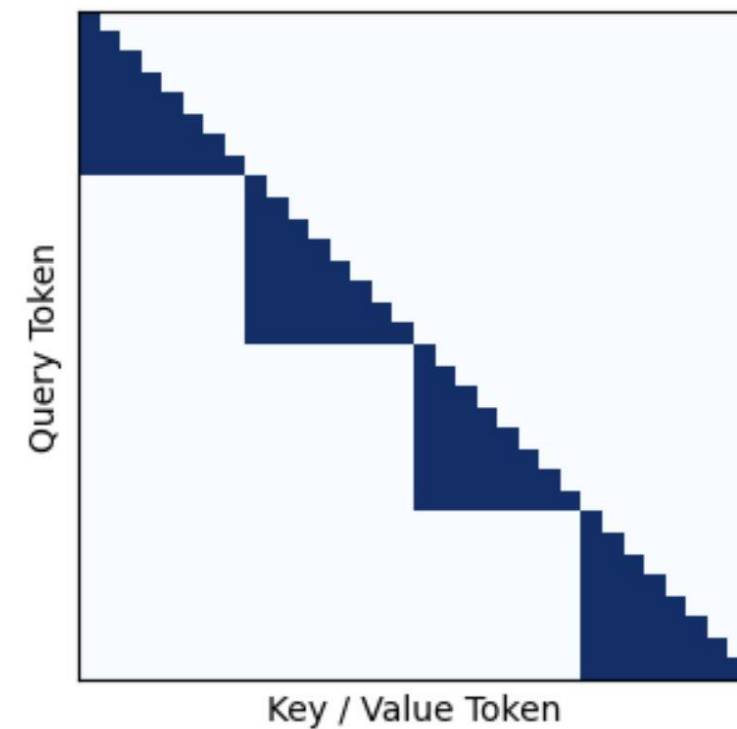
Regular Causal



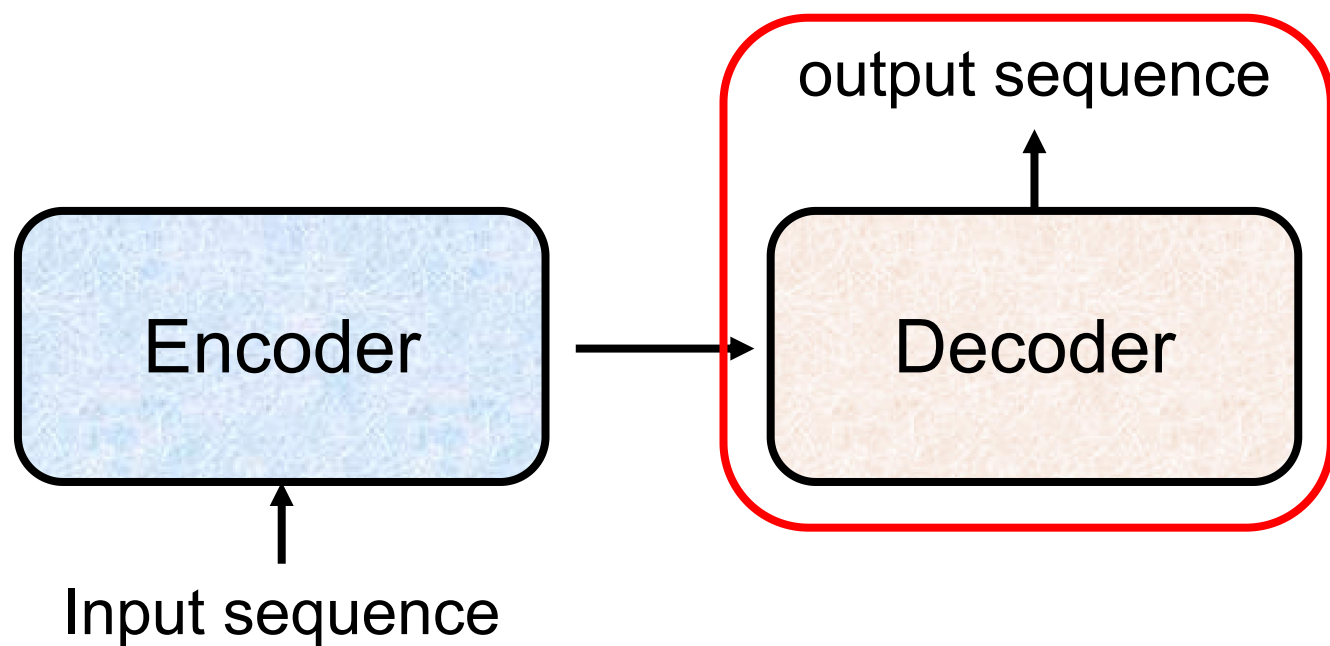
Sliding Window



Chunked

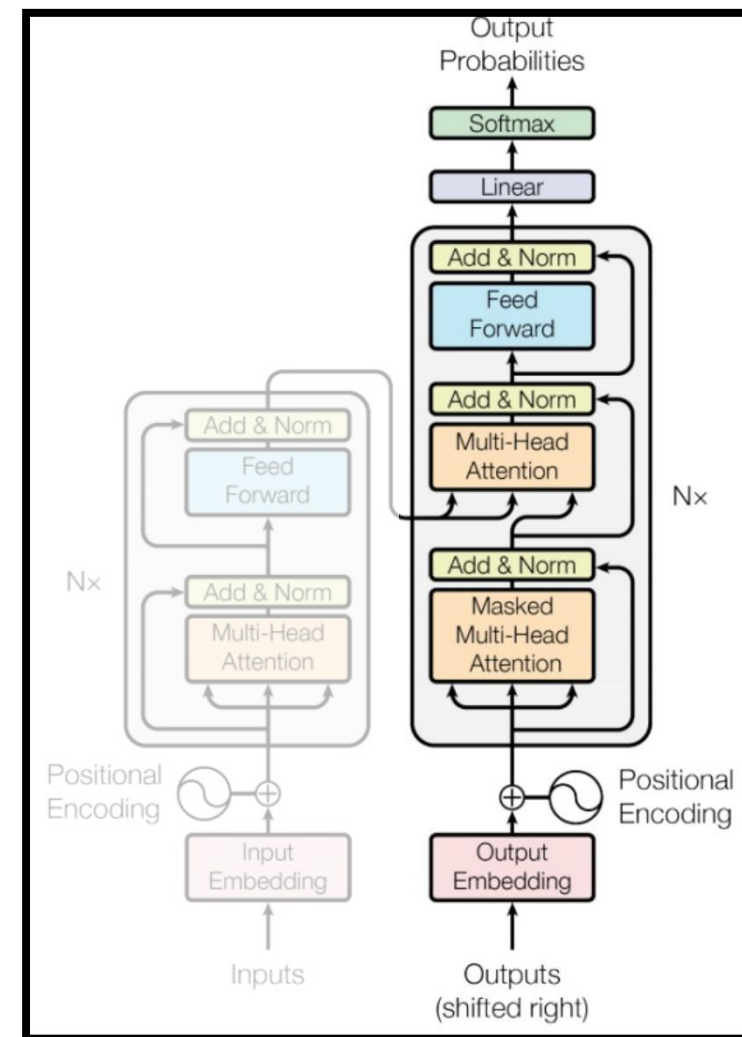


Non-Autoregressive (NAR)

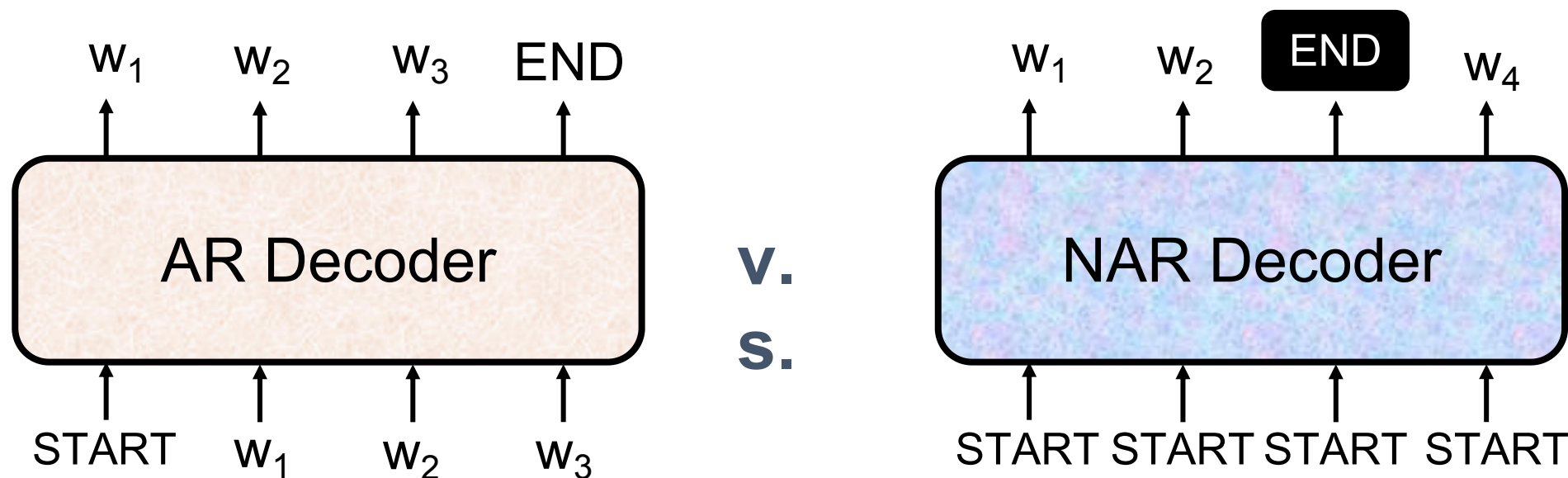


Transformer

<https://arxiv.org/abs/1706.03762>

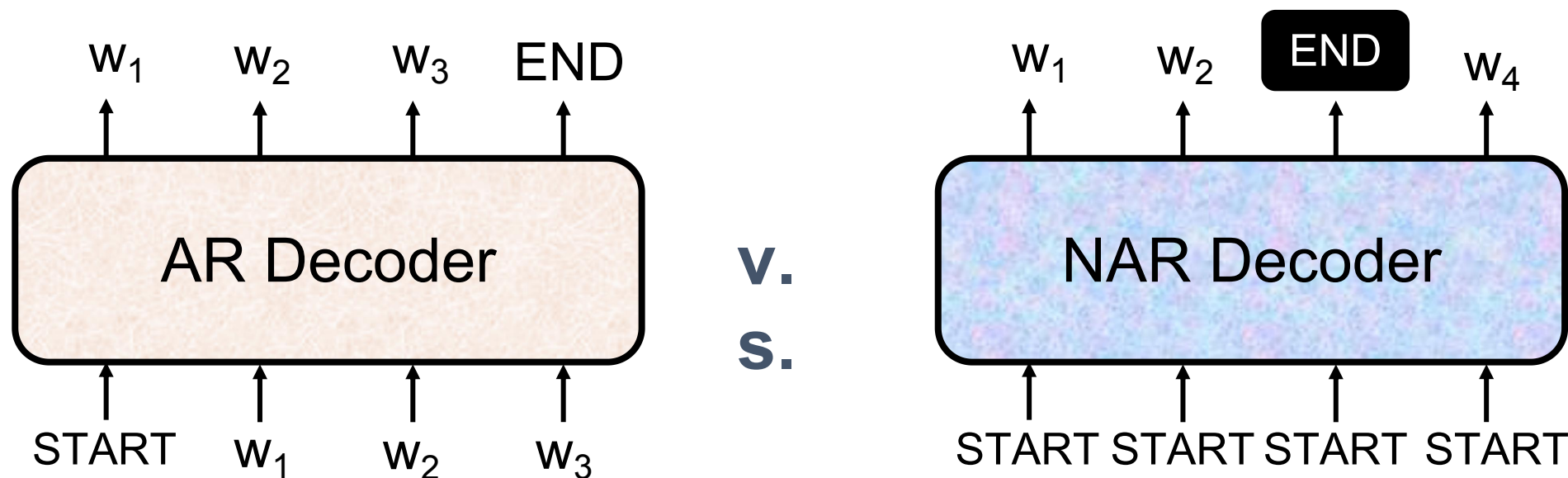


AR v.s. NAR



- How to decide the output length for NAR decoder?
 - Another predictor for output length
 - Output a very long sequence, ignore tokens after END

AR v.s. NAR



- Advantage: parallel, more stable generation (e.g., TTS)
- NAR is usually worse than AR (why? **Multi-modality**)



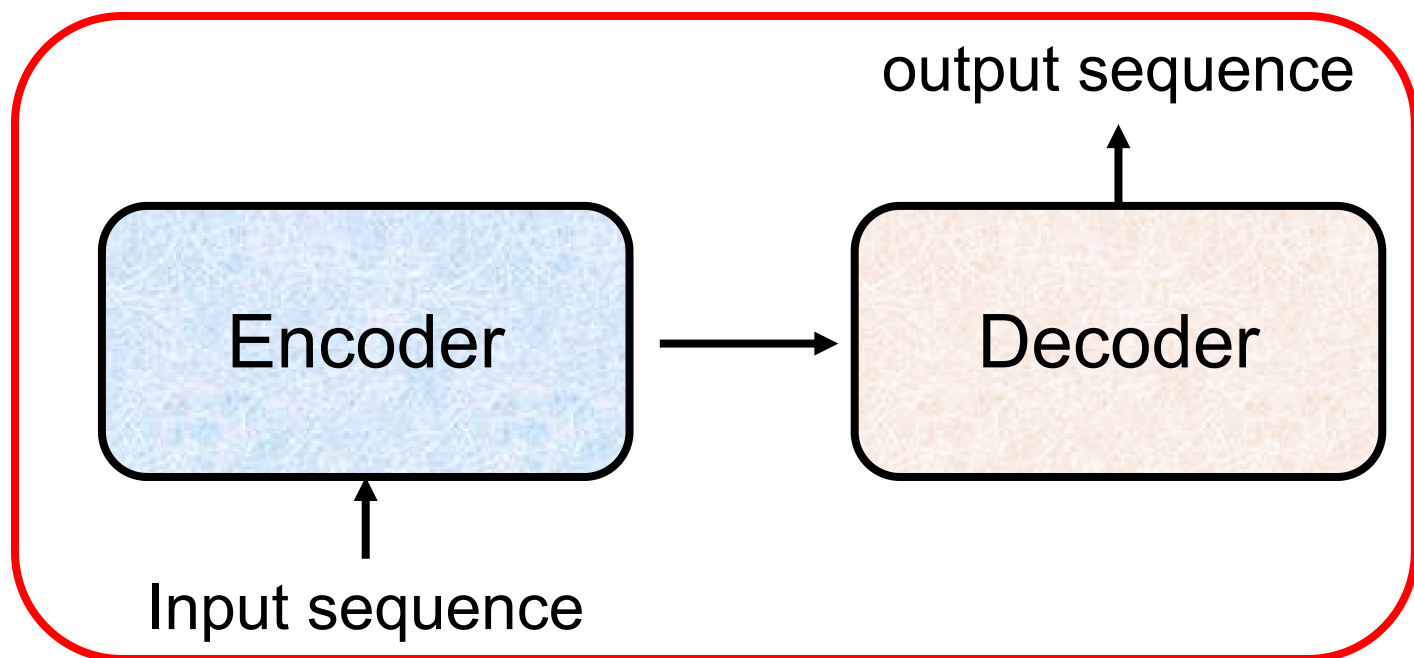
上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

Contents

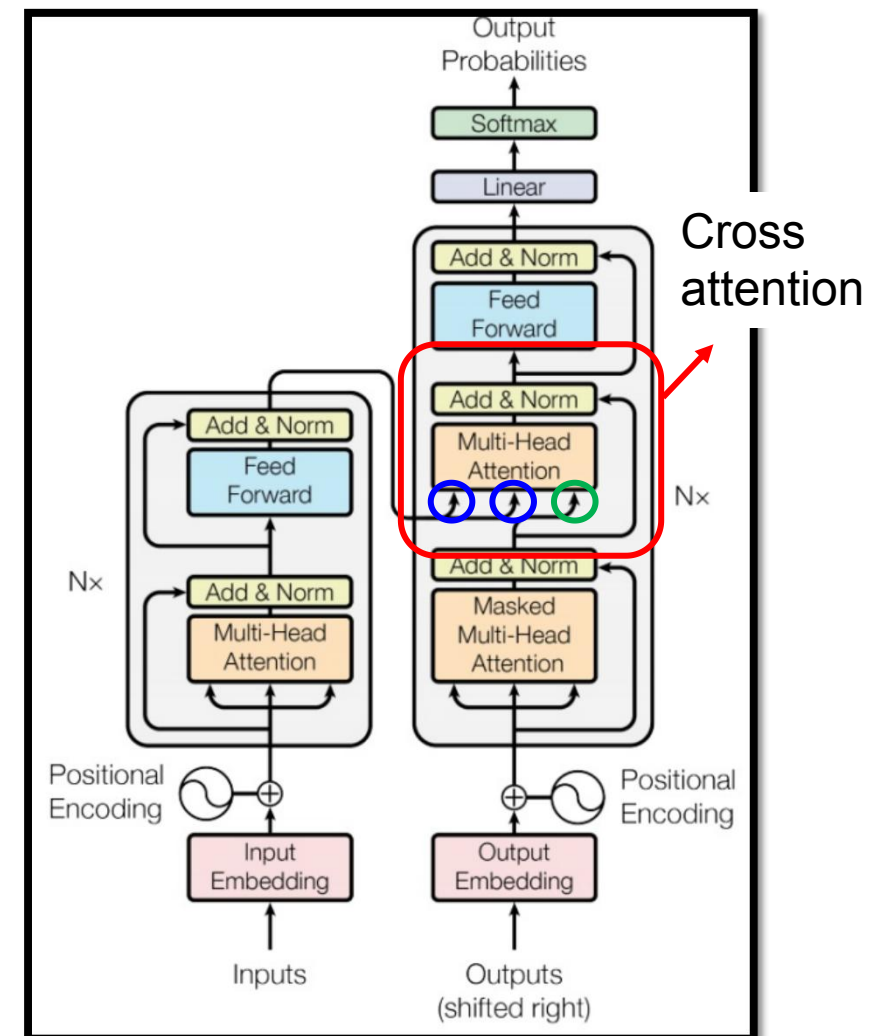
- Self-attention
- Encoder
- Decoder
- **Encoder-Decoder**
- NLP/Vision Transformer
- Mixture of Experts

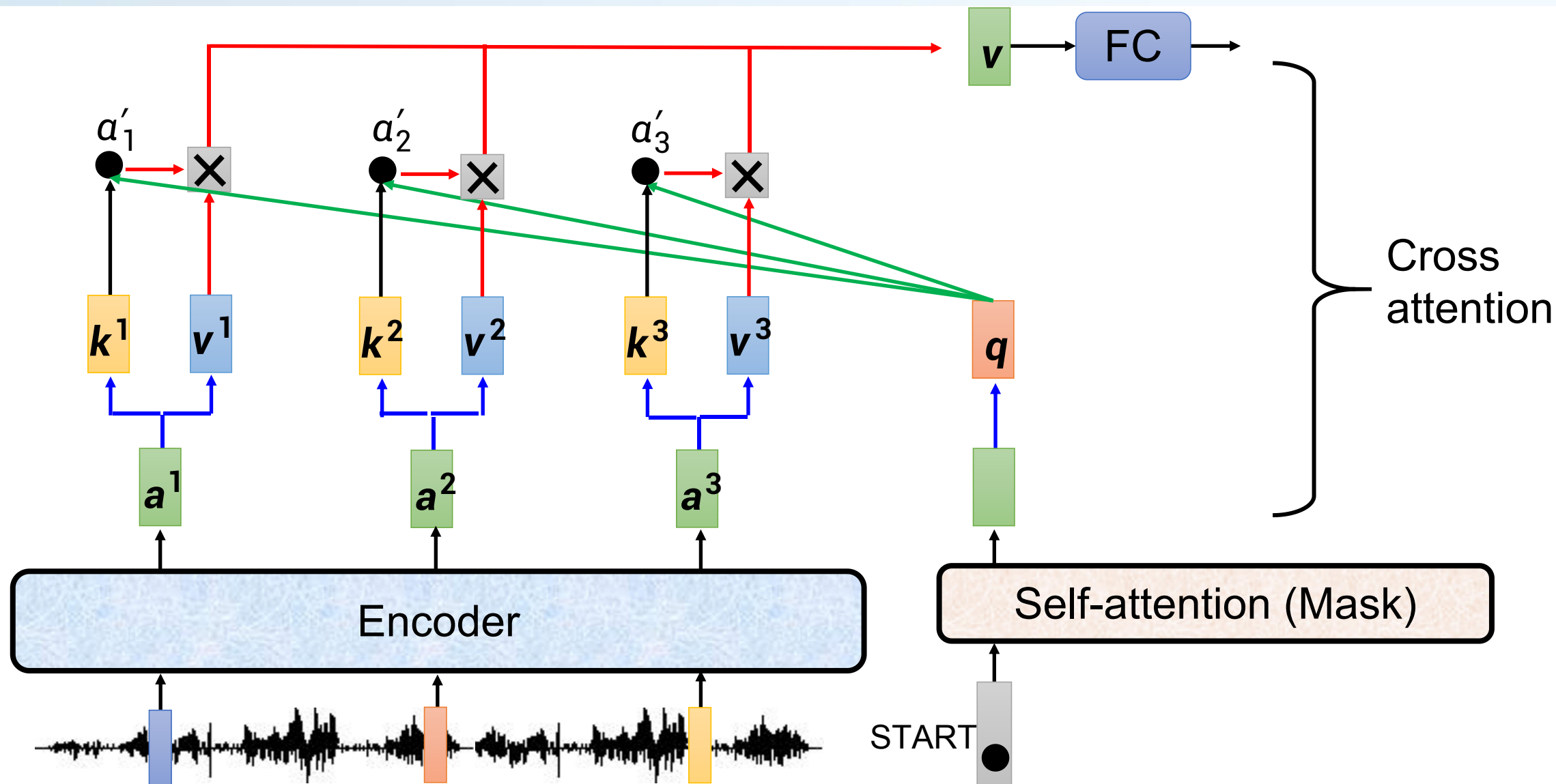


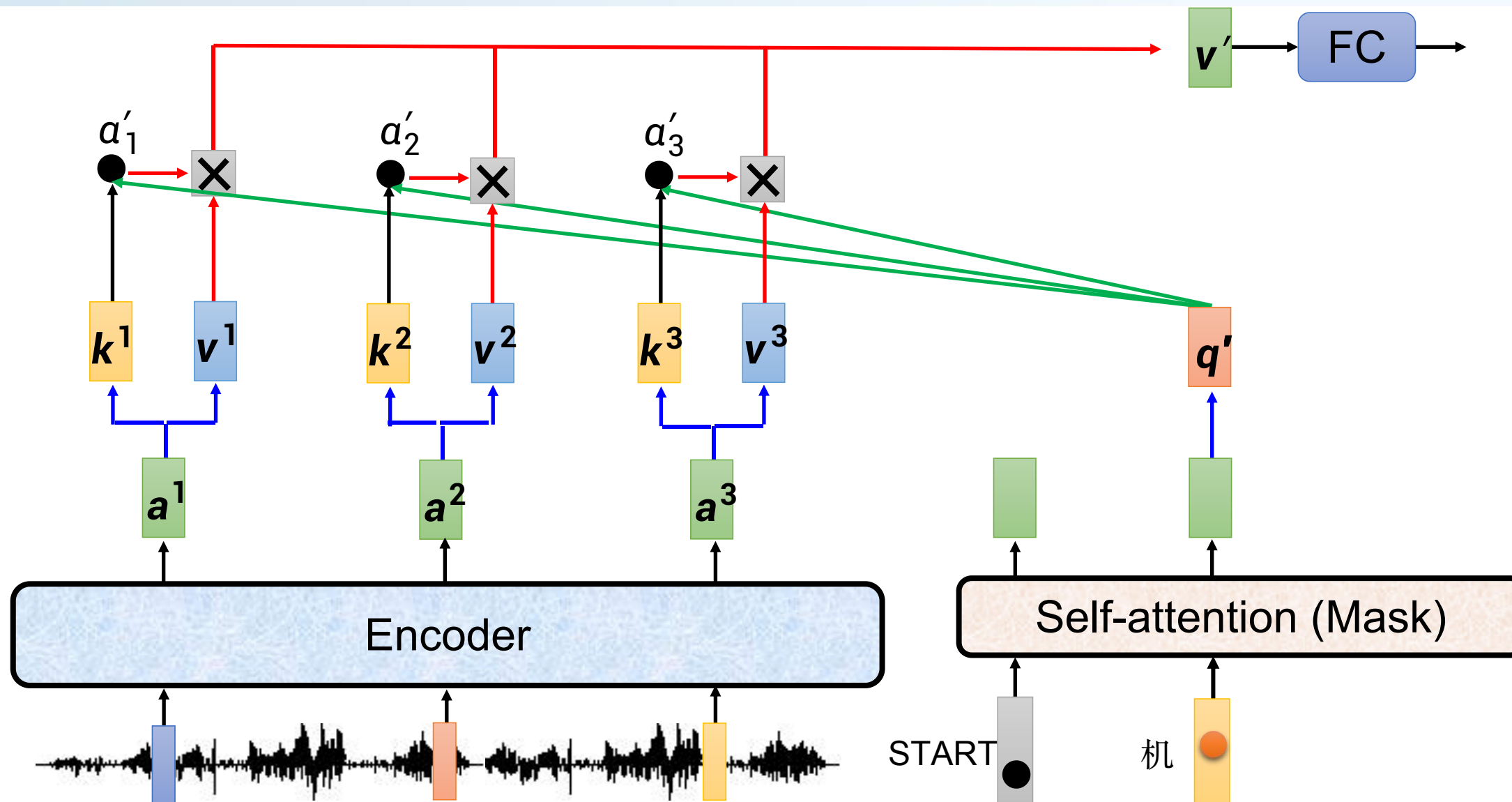


Transformer

<https://arxiv.org/abs/1706.03762>



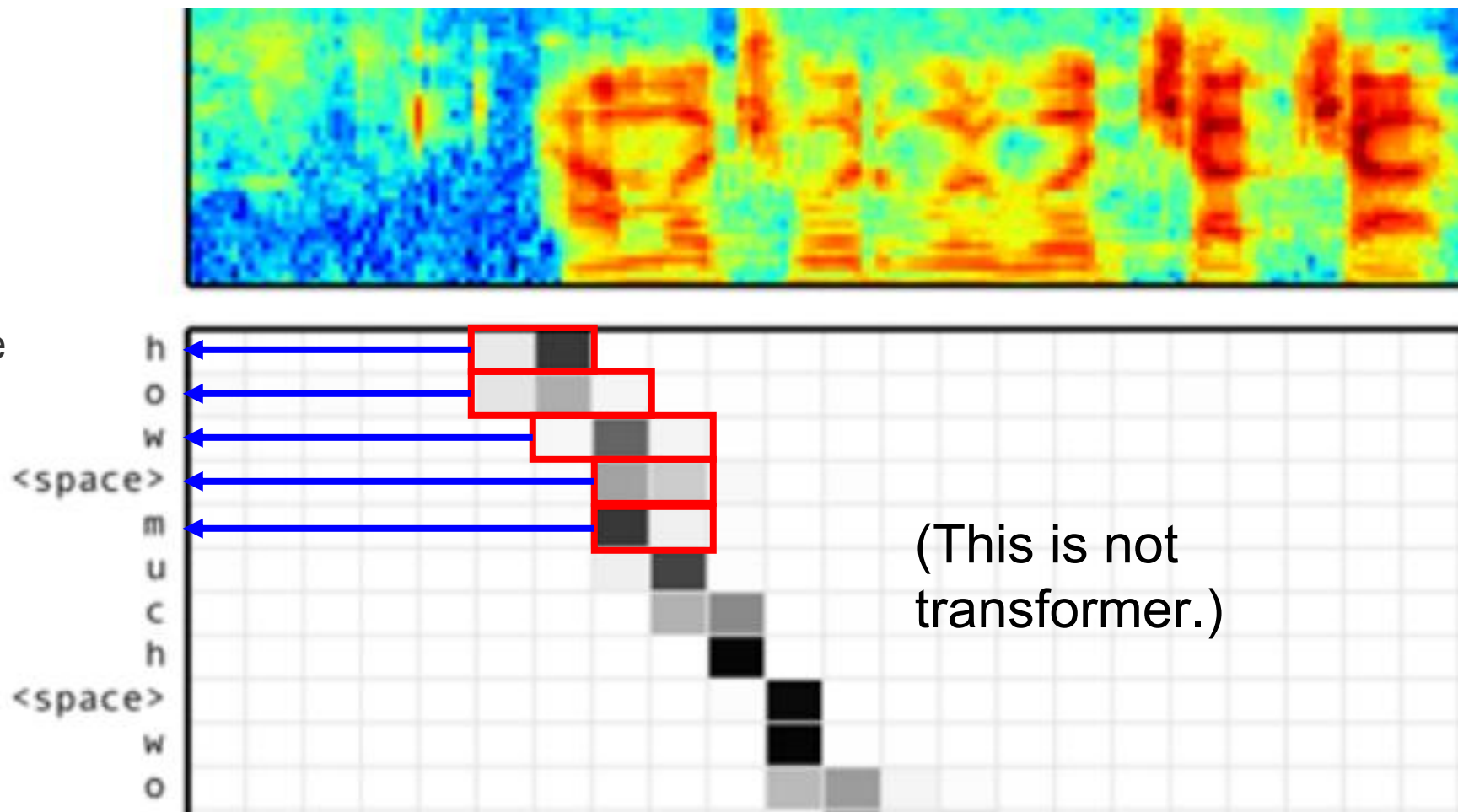




Cross Attention

Listen, attend and spell: A neural network for large vocabulary conversational speech recognition

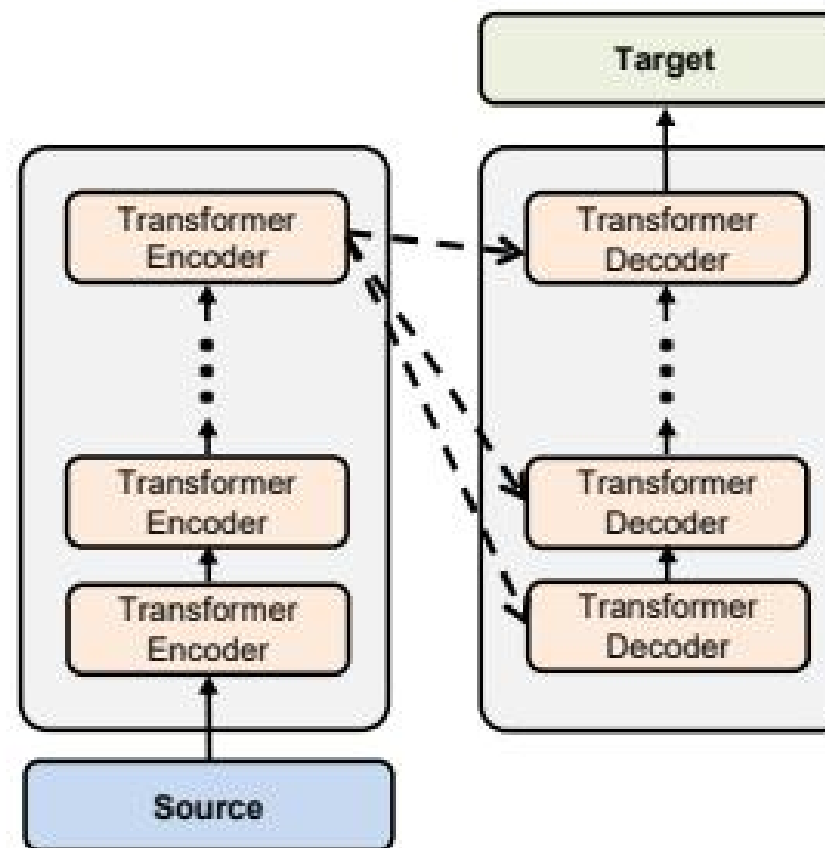
<https://ieeexplore.ieee.org/document/7472621>



Cross Attention

Source of image:

<https://arxiv.org/abs/2005.08081>

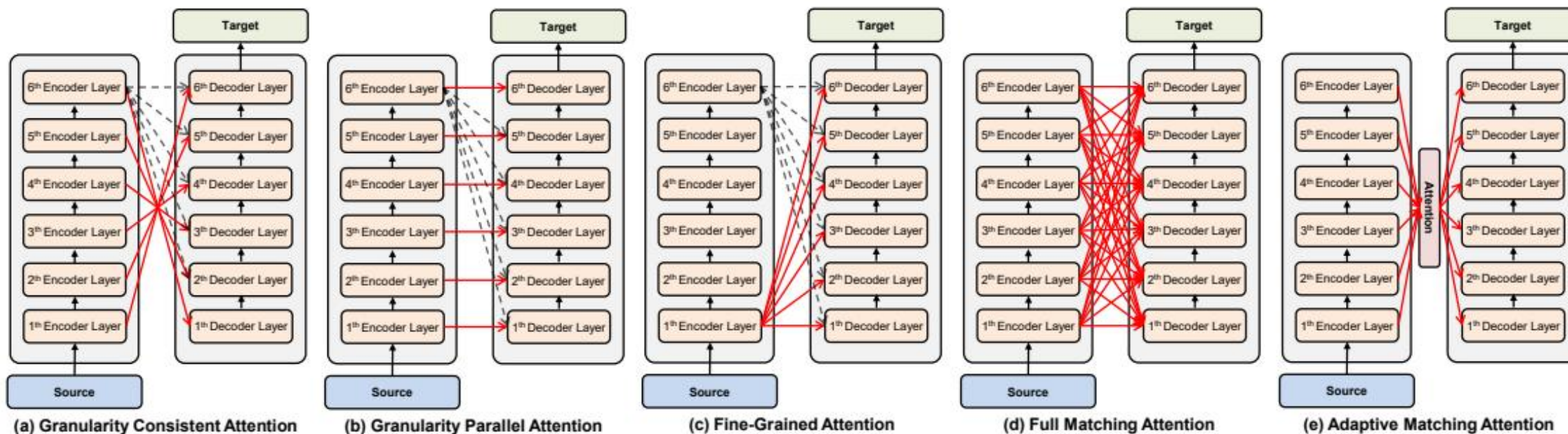


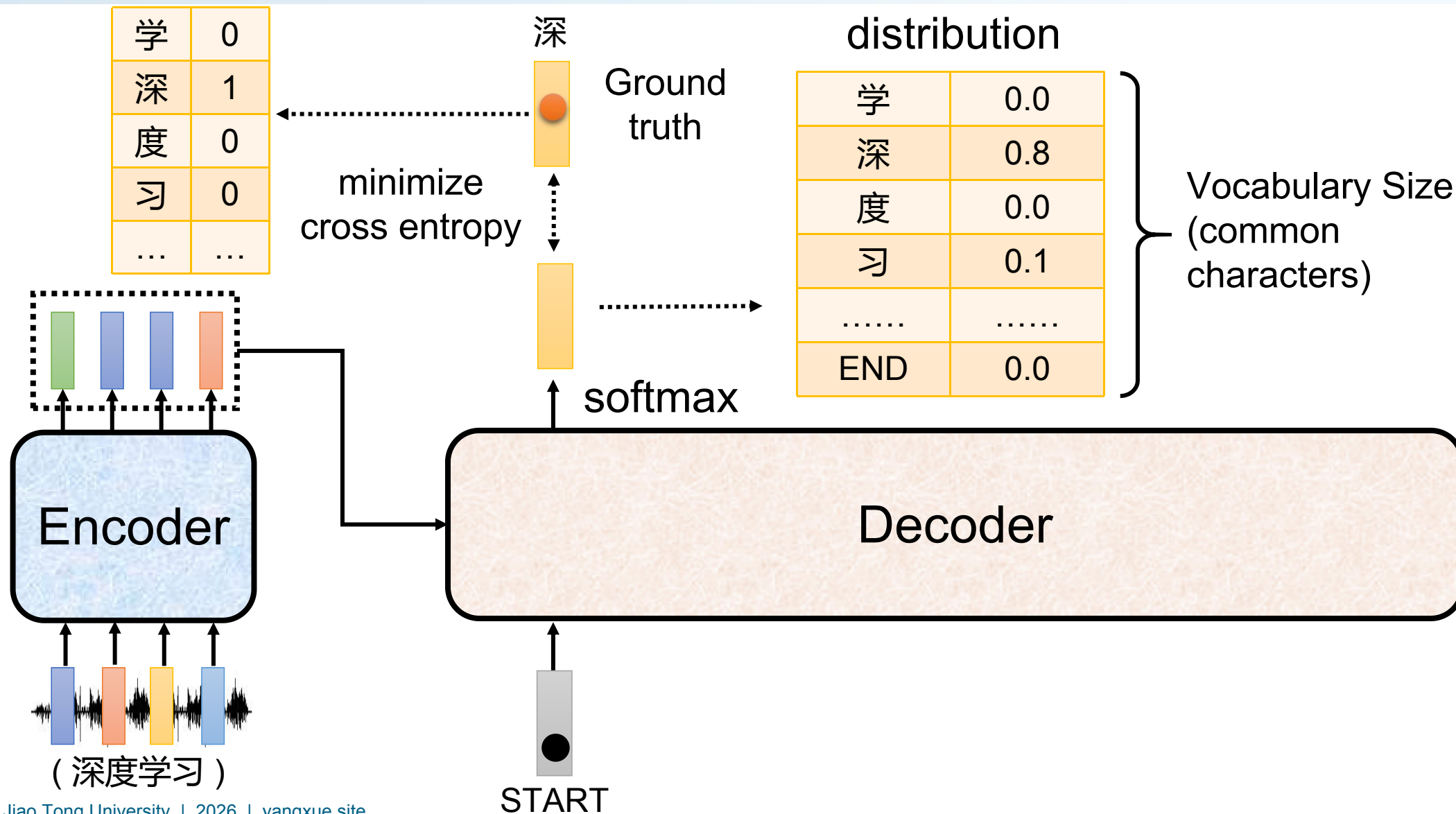
(a) Conventional Transformer

Cross Attention

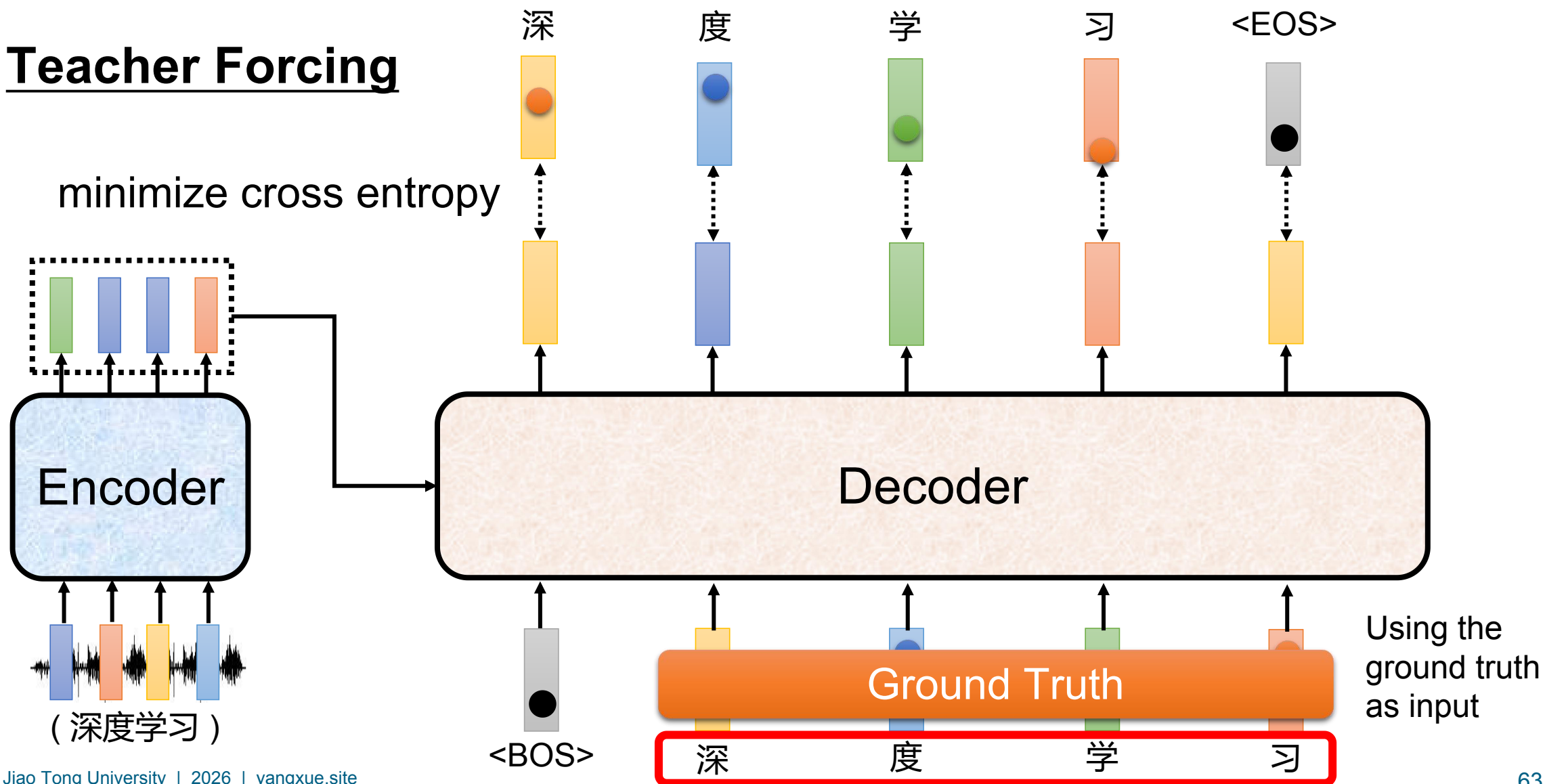
Source of image:

<https://arxiv.org/abs/2005.08081>





Teacher Forcing

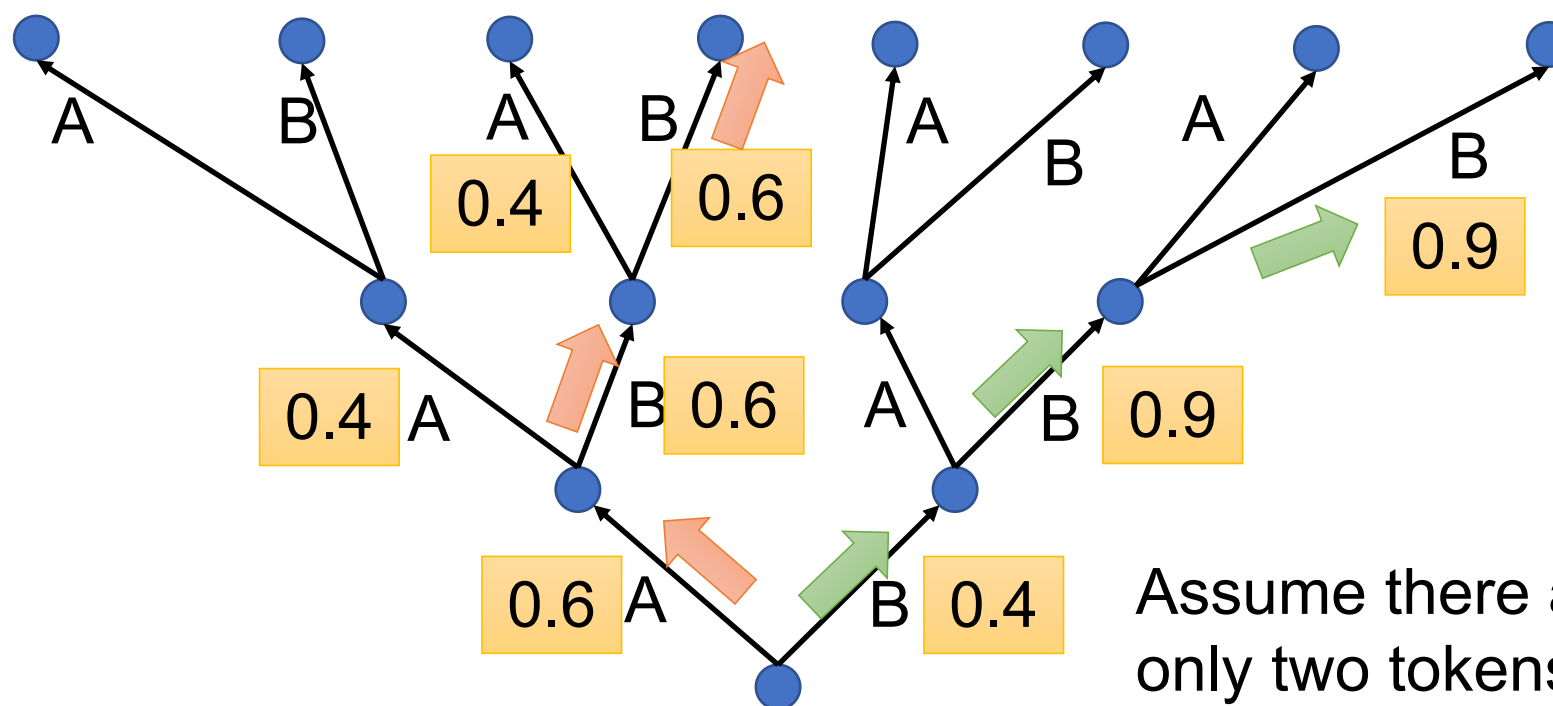


Beam Search

The **red** path is ***Greedy Decoding***.

The **green** path is the best one.

Not possible to check all the paths ... → Beam Search



Assume there are only two tokens ($V=2$).

Beam Search

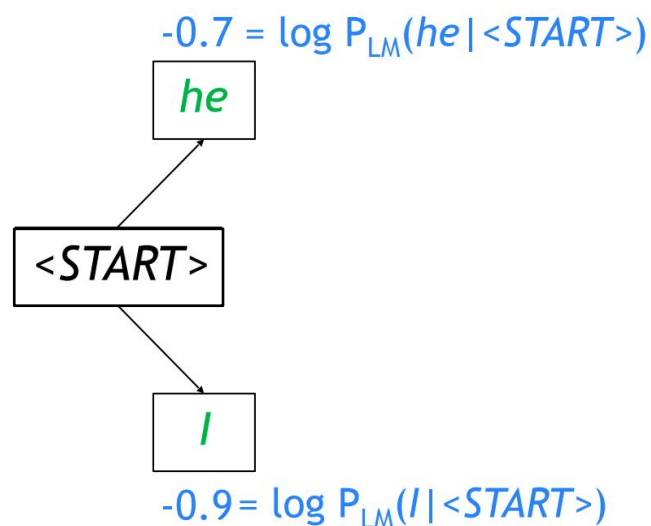
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$

<START>

Calculate prob
dist of next word

Beam Search

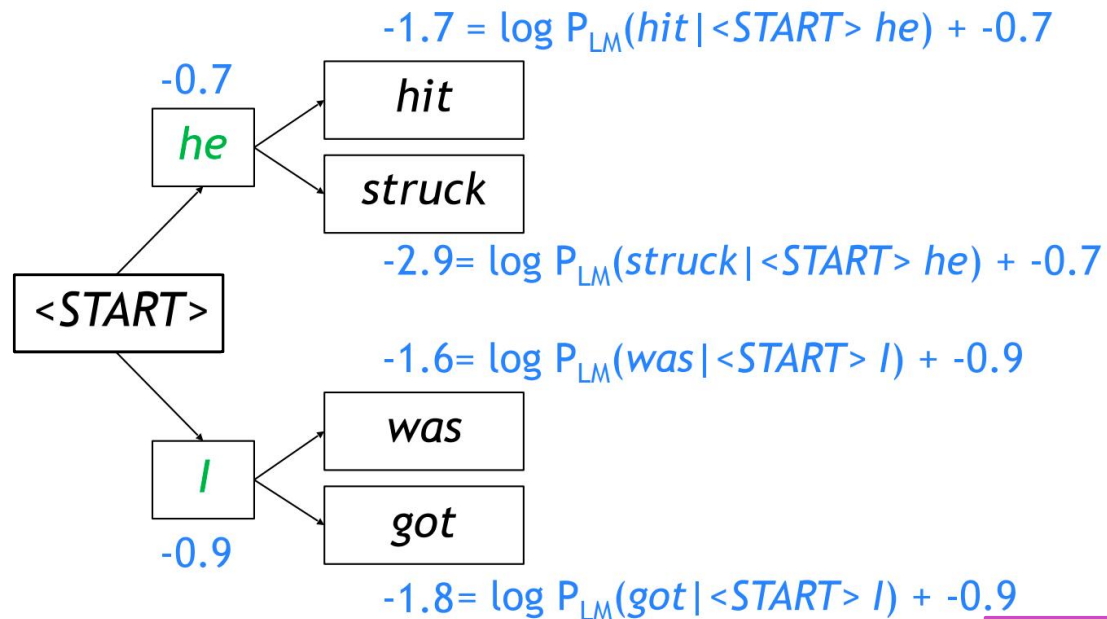
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



Take top k words
and compute scores

Beam Search

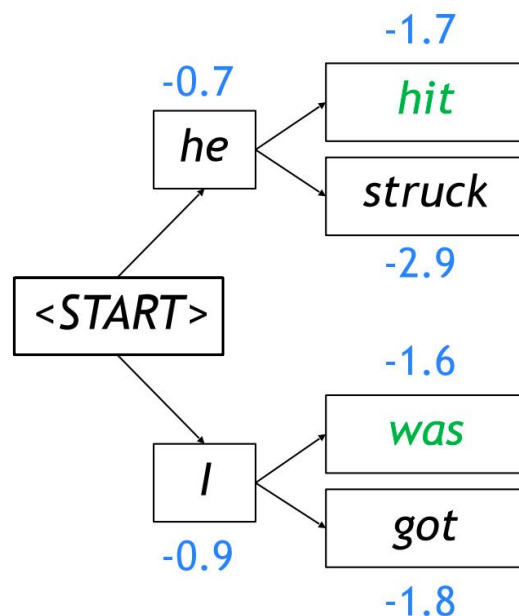
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search

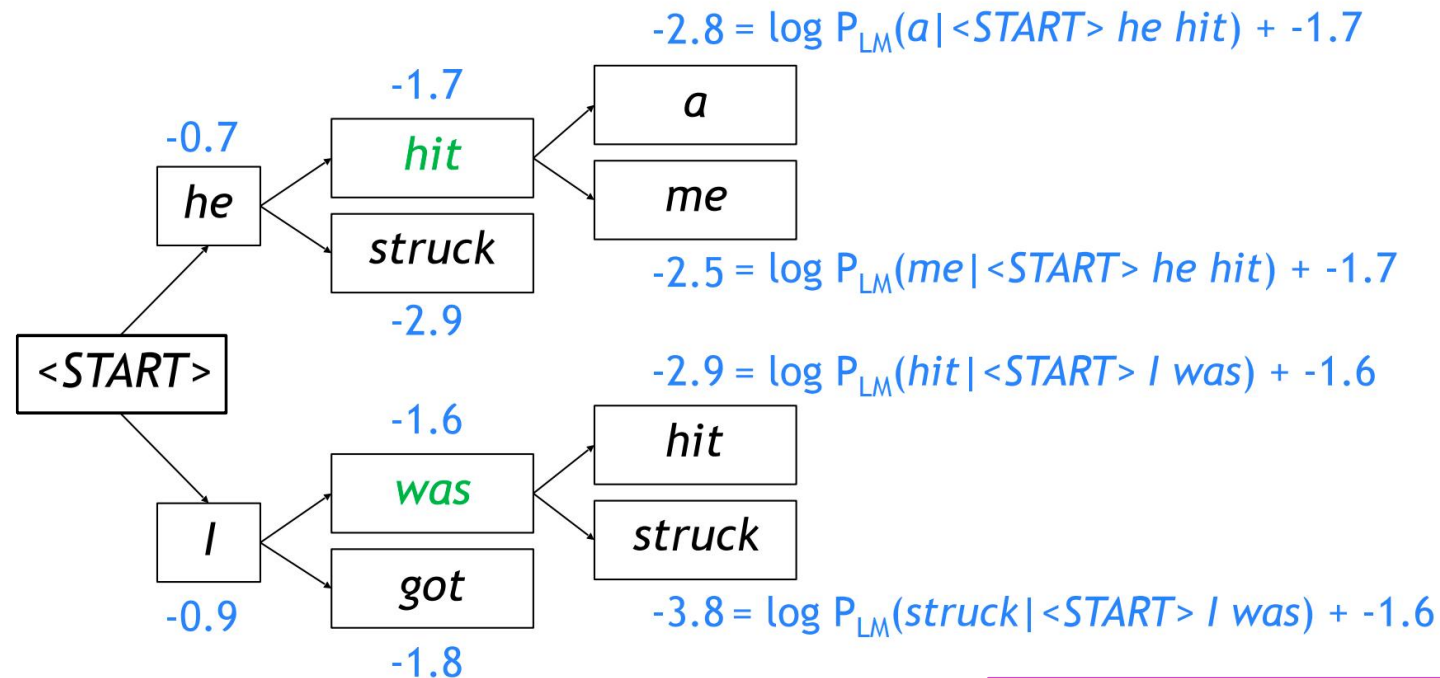
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses, just keep k with highest scores

Beam Search

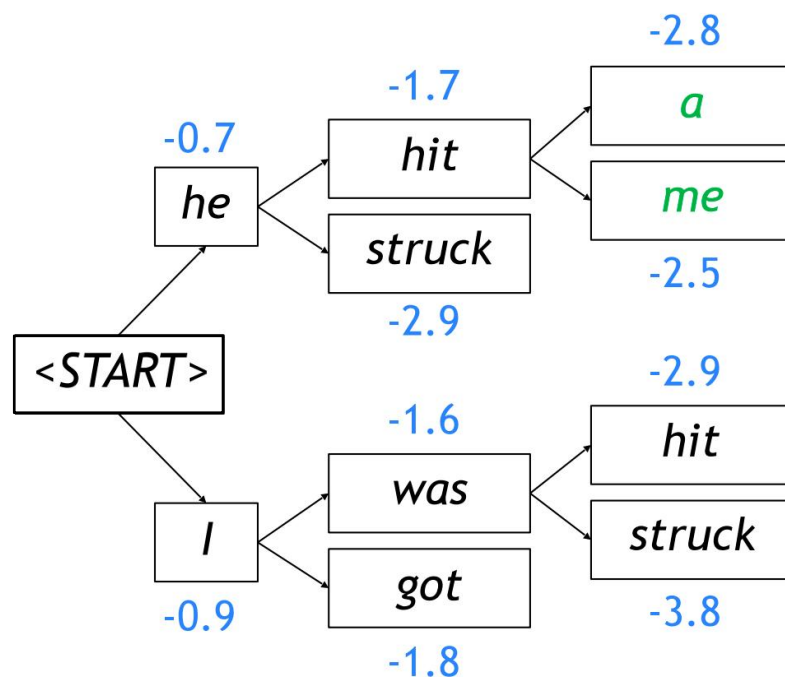
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search

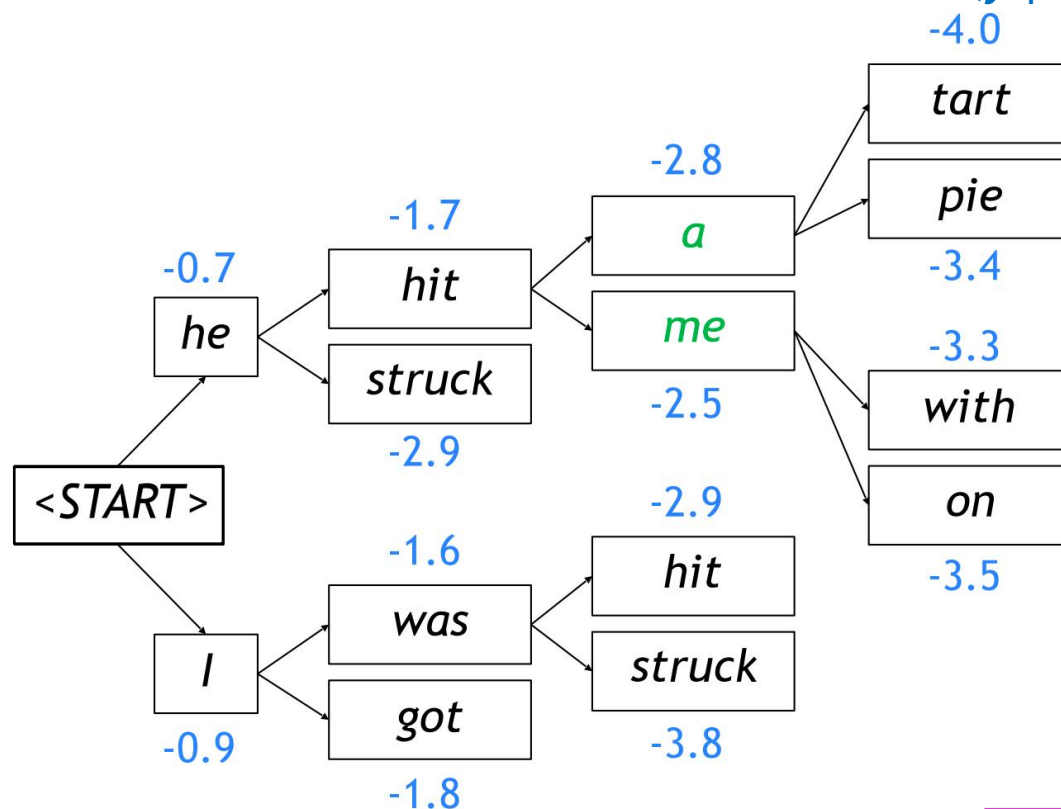
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses, just keep k with highest scores

Beam Search

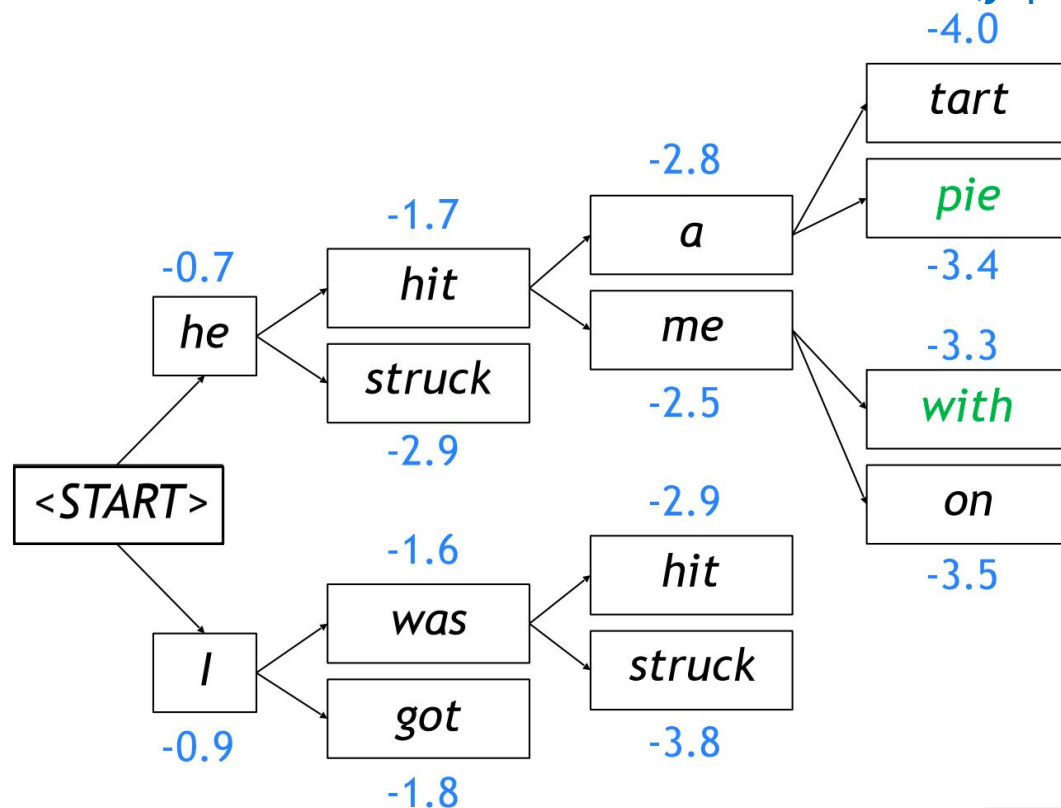
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search

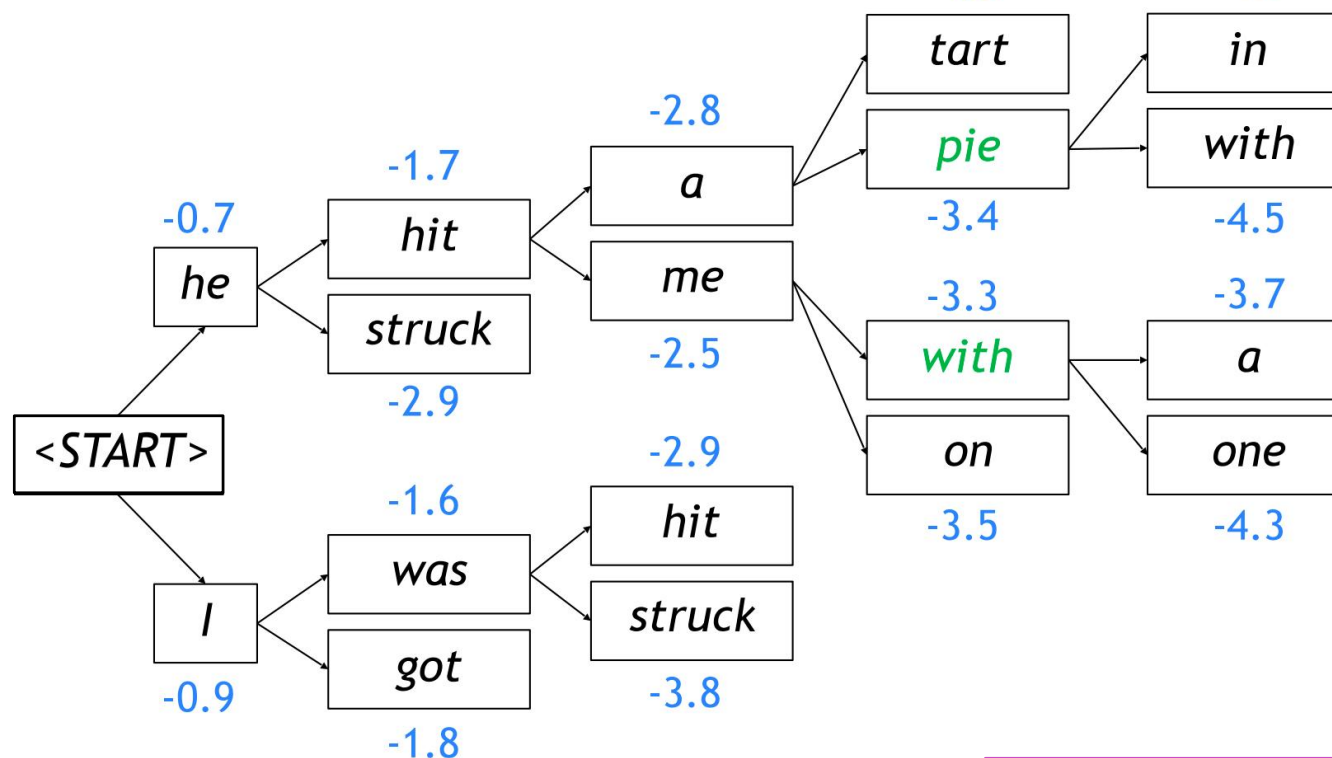
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses, just keep k with highest scores

Beam Search

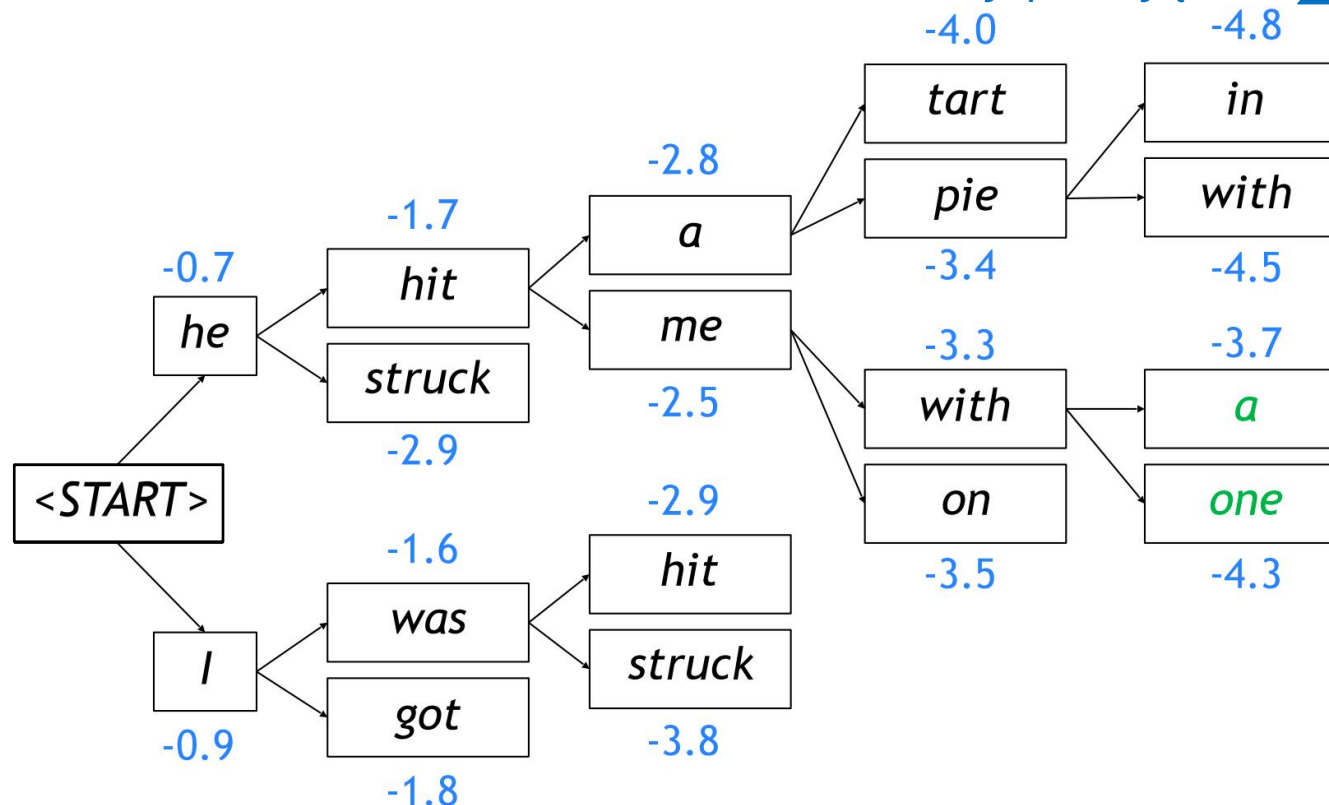
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search

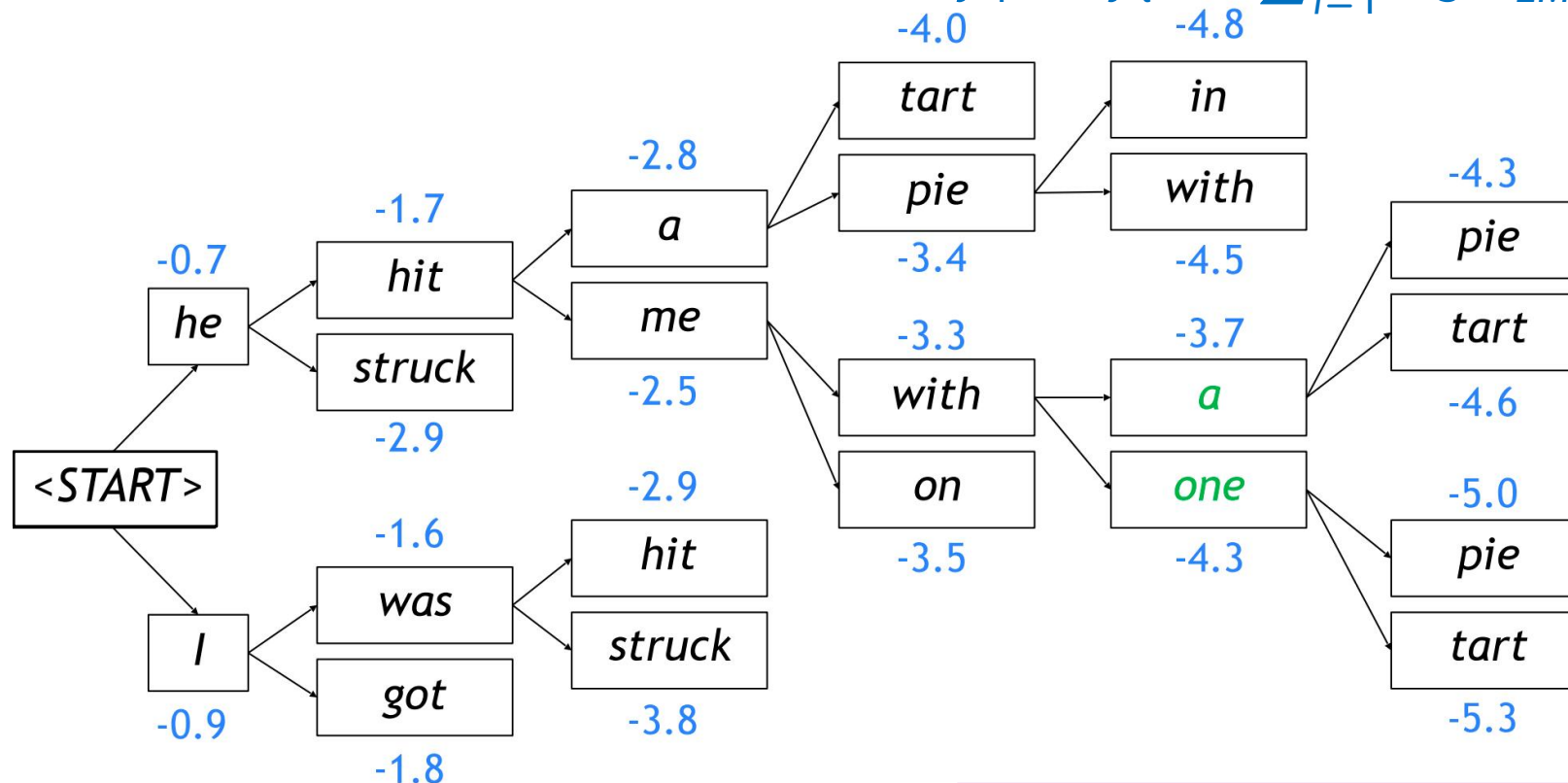
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



Of these k^2 hypotheses, just keep k with highest scores

Beam Search

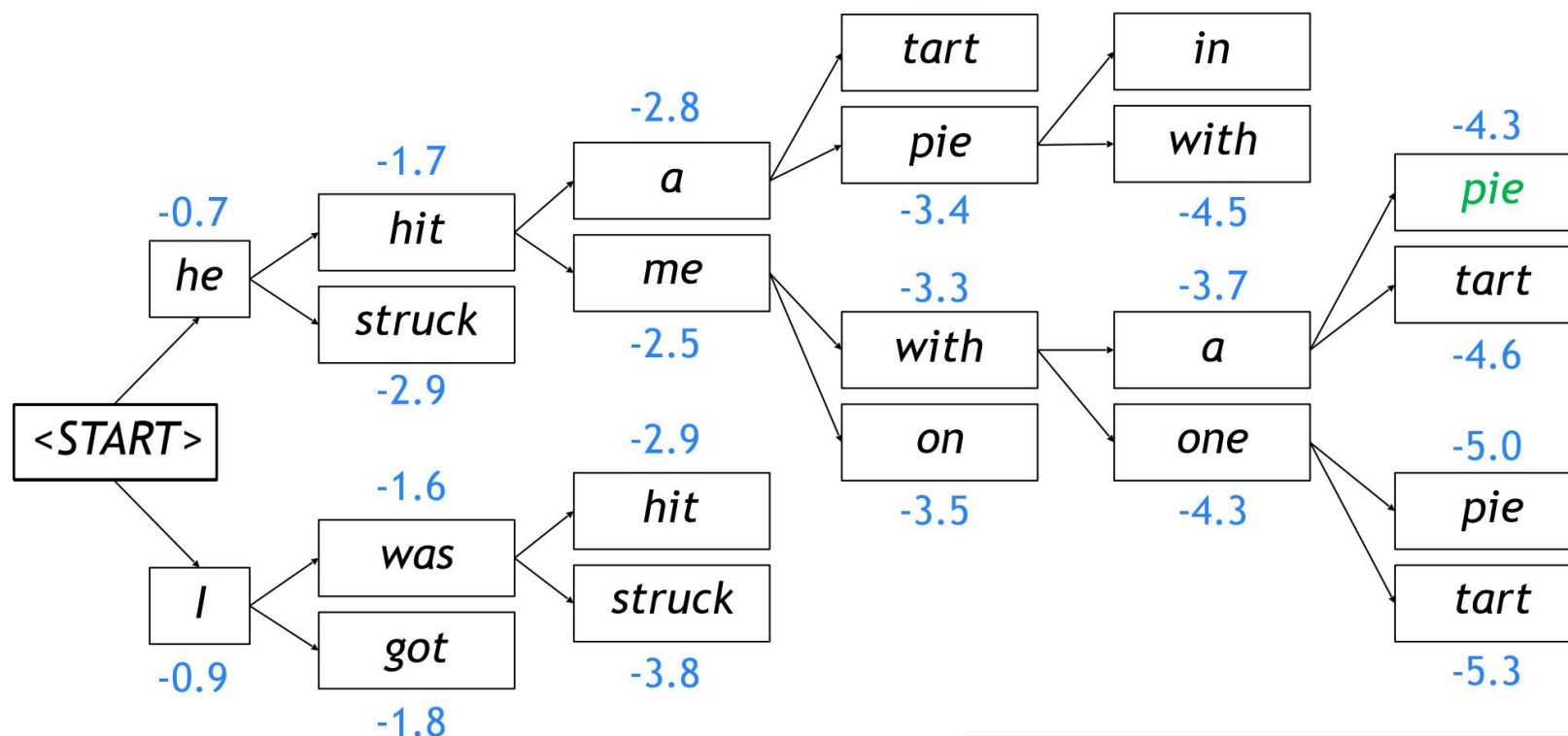
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



For each of the k hypotheses, find top k next words and calculate scores

Beam Search

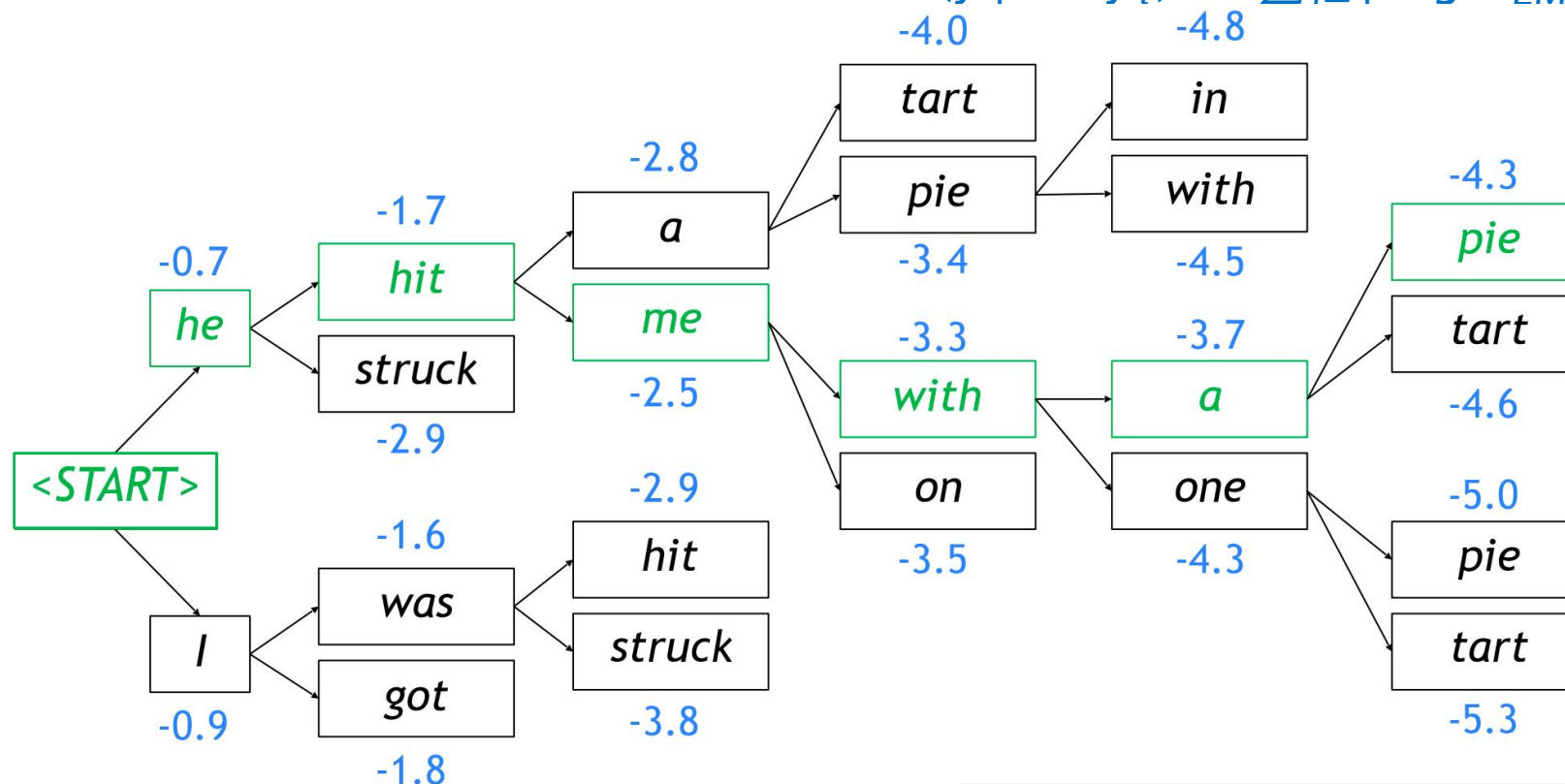
- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



This is the top-scoring hypothesis!

Beam Search

- Beam size = $k = 2$. Blue numbers $score(y_1, \dots, y_t) = \sum_{i=1}^t \log P_{LM}(y_i | y_1, \dots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis

The Curious Case of Neural Text Degeneration

Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

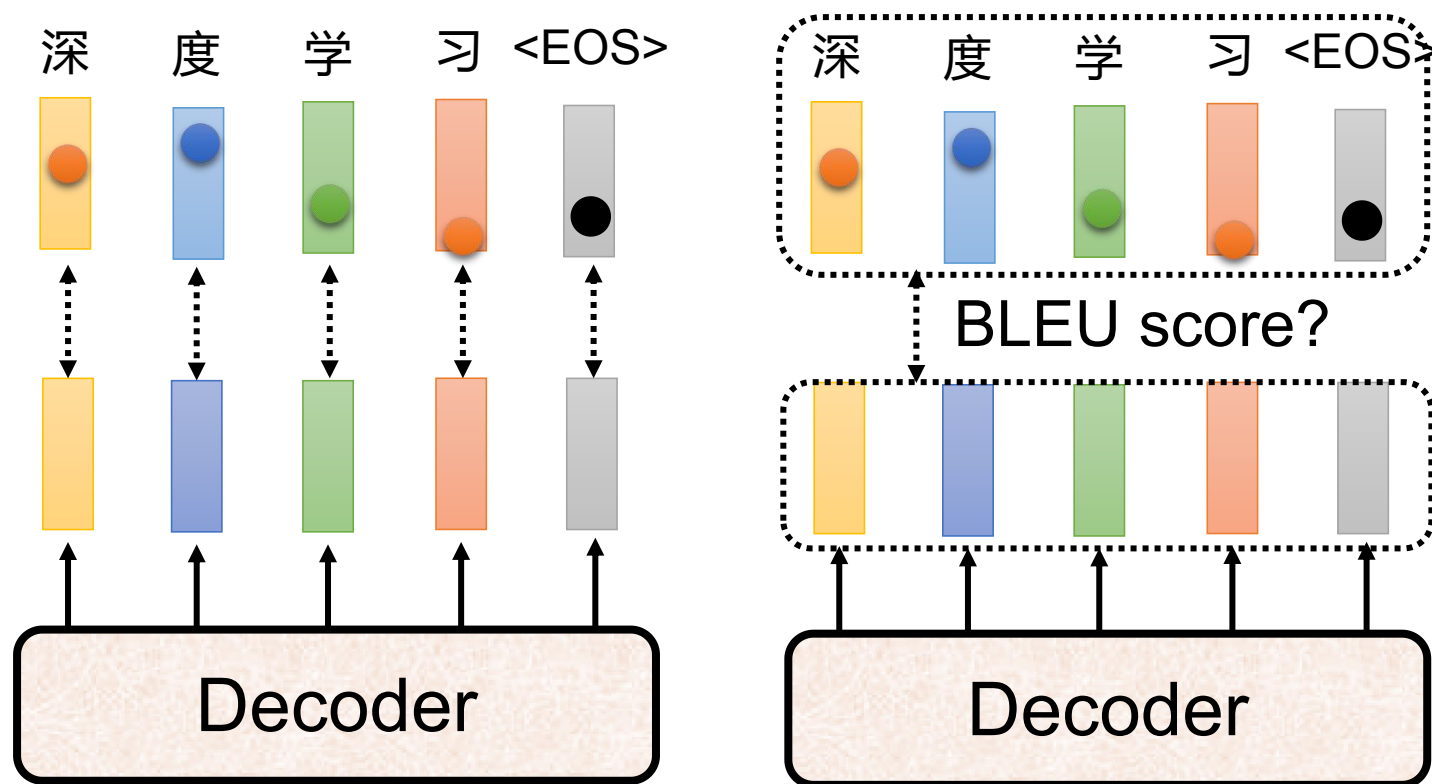
"The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the Universidad Nacional Autónoma de México (UNAM) and the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de México/Universidad Nacional Autónoma de ..."

They were cattle called Bolivian Cavalleros; they live in a remote desert uninterrupted by town, and they speak huge, beautiful, paradisiacal Bolivian linguistic thing. They say, 'Lunch, marge.' They don't tell what the lunch is," director Professor Chuperas Omwell told Sky News. "They've only been talking to scientists, like we're being interviewed by TV reporters. We don't even stick around to be interviewed by TV reporters. Maybe that's how they figured out that they're cosplaying as the Bolivian Cavalleros."

Randomness is needed for decoder when generating sequence in some tasks.

Accept that nothing is perfect. True beauty lies in the cracks of imperfection. 😊

Optimizing Evaluation Metrics?



1. 计算 n-gram 的精确匹配率:
计算候选翻译中每个 n-gram 在参考翻译中出现的次数与候选翻译中该 n-gram 总次数的比值

$$p_n = \frac{\sum \min(\text{count}_C(\text{n-gram}), \text{count}_R(\text{n-gram}))}{\sum \text{count}_C(\text{n-gram})}$$

2. 计算 Brevity Penalty (BP):
惩罚翻译结果过短

$$BP = \exp \left(1 - \frac{\text{length}_R}{\text{length}_C} \right)$$

3. 综合计算 BLEU

$$\text{BLEU} = BP \times \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

候选翻译 (C) : “the cat sat on the mat”

参考翻译 (R) : “the cat is on the mat”

1. 计算 n-gram 精度 (p_n):

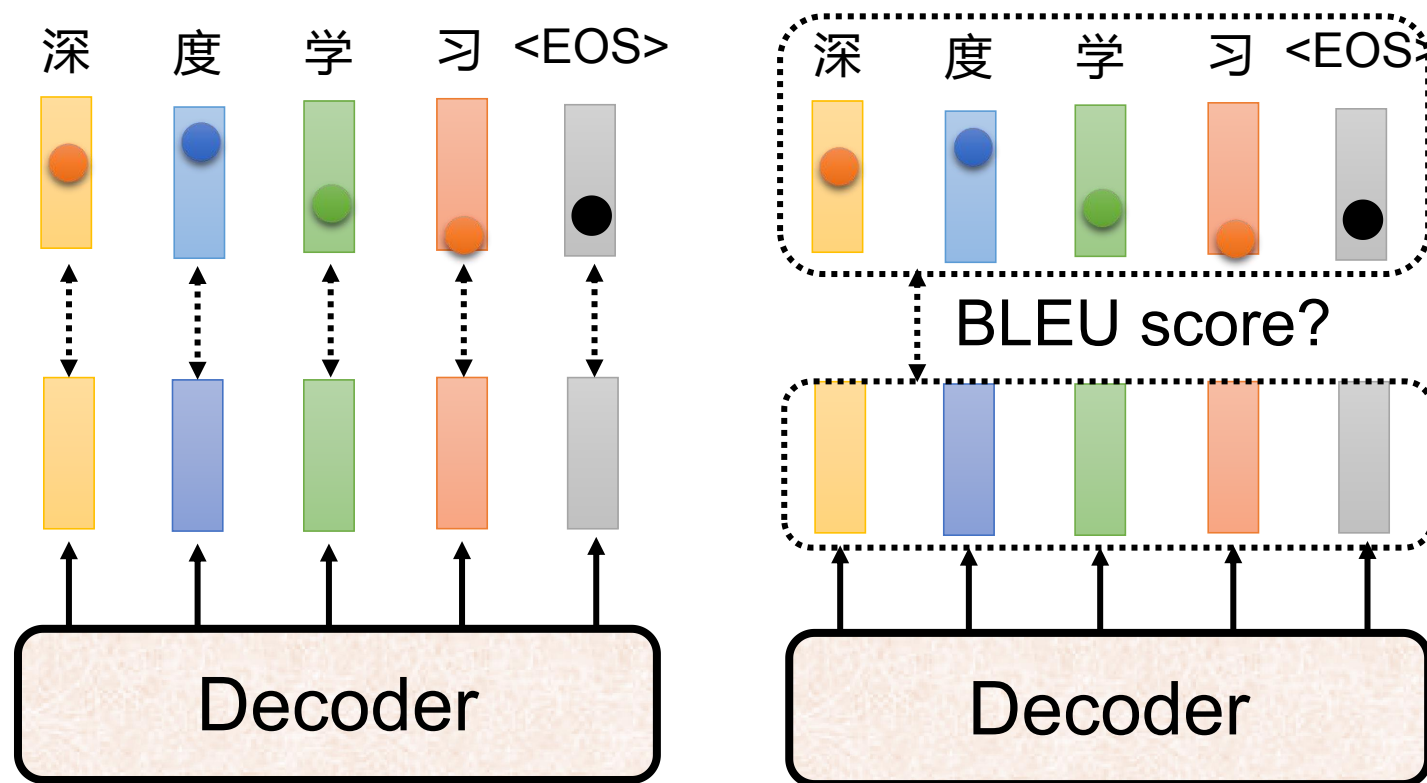
- 对于 $n=1$ (单词):
 - 候选翻译的 1-gram 列表: ["the", "cat", "sat", "on", "the", "mat"]
 - 参考翻译的 1-gram 列表: ["the", "cat", "is", "on", "the", "mat"]
 - 匹配的 1-gram: "the", "cat", "on", "the", "mat" (5 个)
 - $p_1 = \frac{5}{6} \approx 0.8333$
- 对于 $n=2$ (词组):
 - 候选翻译的 2-gram 列表: ["the cat", "cat sat", "sat on", "on the", "the mat"]
 - 参考翻译的 2-gram 列表: ["the cat", "cat is", "is on", "on the", "the mat"]
 - 匹配的 2-gram: "the cat", "on the", "the mat" (3 个)
 - $p_2 = \frac{3}{5} = 0.6$

2. 计算 Brevity Penalty (BP):

- 候选翻译长度: 6
- 参考翻译长度: 6
- 因为长度相等, 所以 $BP = 1$ 。

3. 计算 BLEU Score:

- 假设最大 $n=2$, 权重 $w_1 = w_2 = 0.5$
- $BLEU = BP \times \exp(w_1 \ln p_1 + w_2 \ln p_2)$
- 代入数值:
 - $BLEU = 1 \times \exp(0.5 \times \ln(0.8333) + 0.5 \times \ln(0.6)) \approx 0.549$



How to do the optimization?

When you don't know how to optimize, just use reinforcement learning (RL)!

<https://arxiv.org/abs/1511.06732>

1. 计算 **n-gram** 的精确匹配率:
计算候选翻译中每个 n-gram 在参考翻译中出现的次数与候选翻译中该 n-gram 总次数的比值

$$p_n = \frac{\sum \min(\text{count}_C(n\text{-gram}), \text{count}_R(n\text{-gram}))}{\sum \text{count}_C(n\text{-gram})}$$

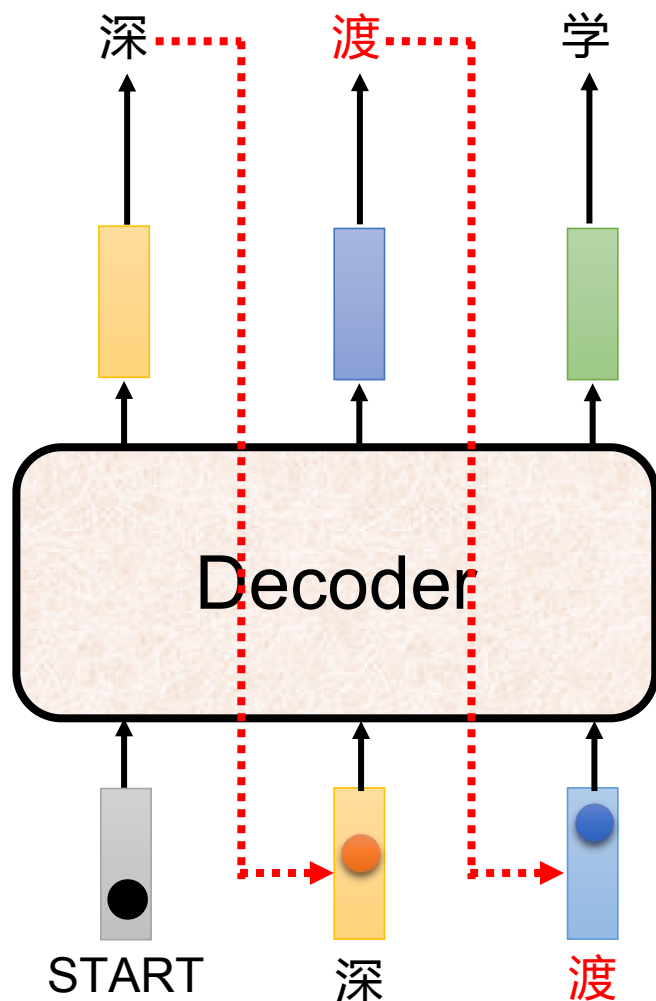
2. 计算 **Brevity Penalty (BP)**:
惩罚翻译结果过短

$$BP = \exp \left(1 - \frac{\text{length}_R}{\text{length}_C} \right)$$

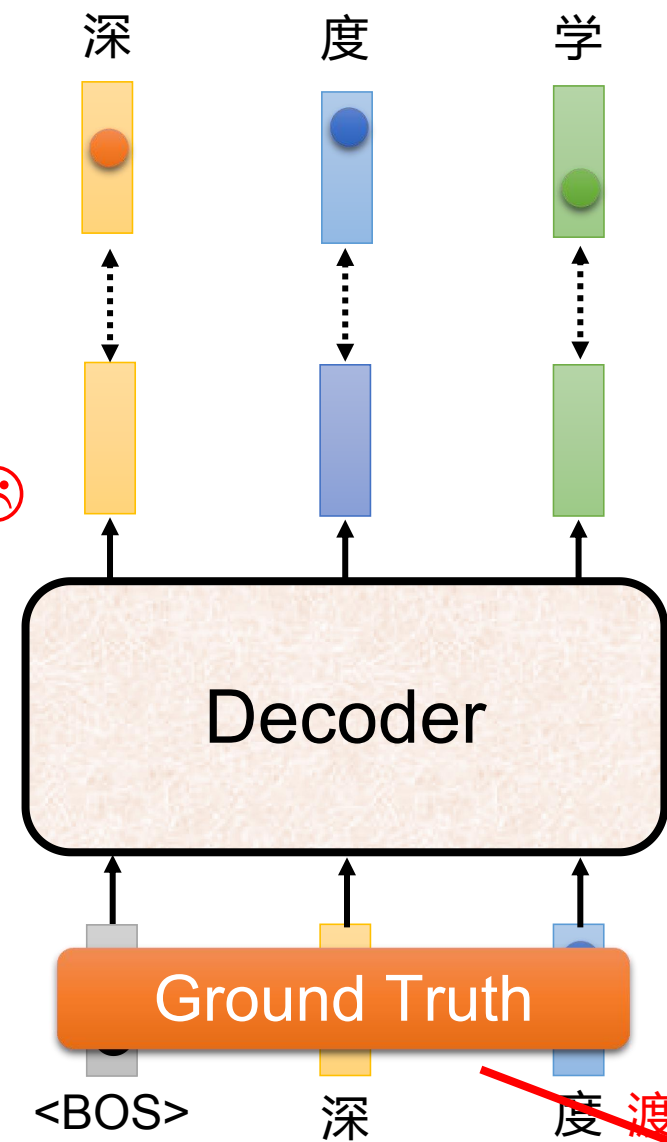
3. 综合计算 **BLEU**

$$\text{BLEU} = BP \times \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

Exposure bias?



There is a mismatch! ☹️



Scheduled Sampling

- Original Scheduled Sampling

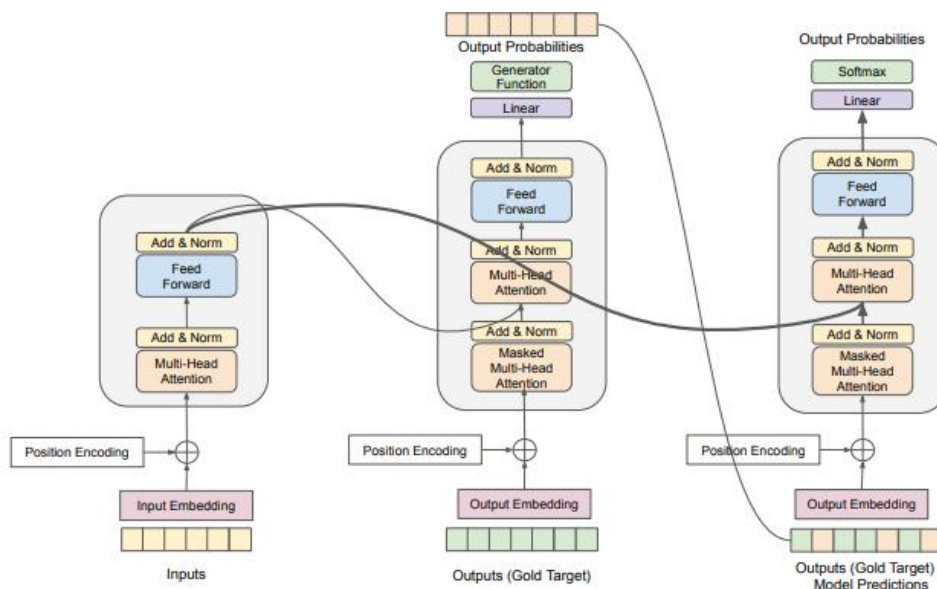
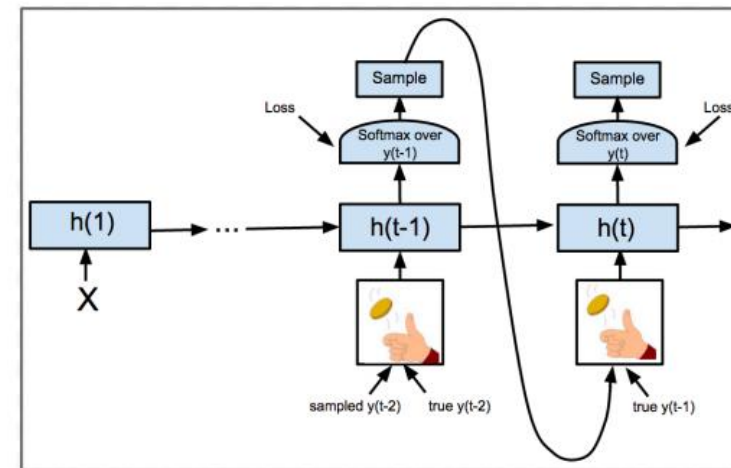
<https://arxiv.org/abs/1506.03099>

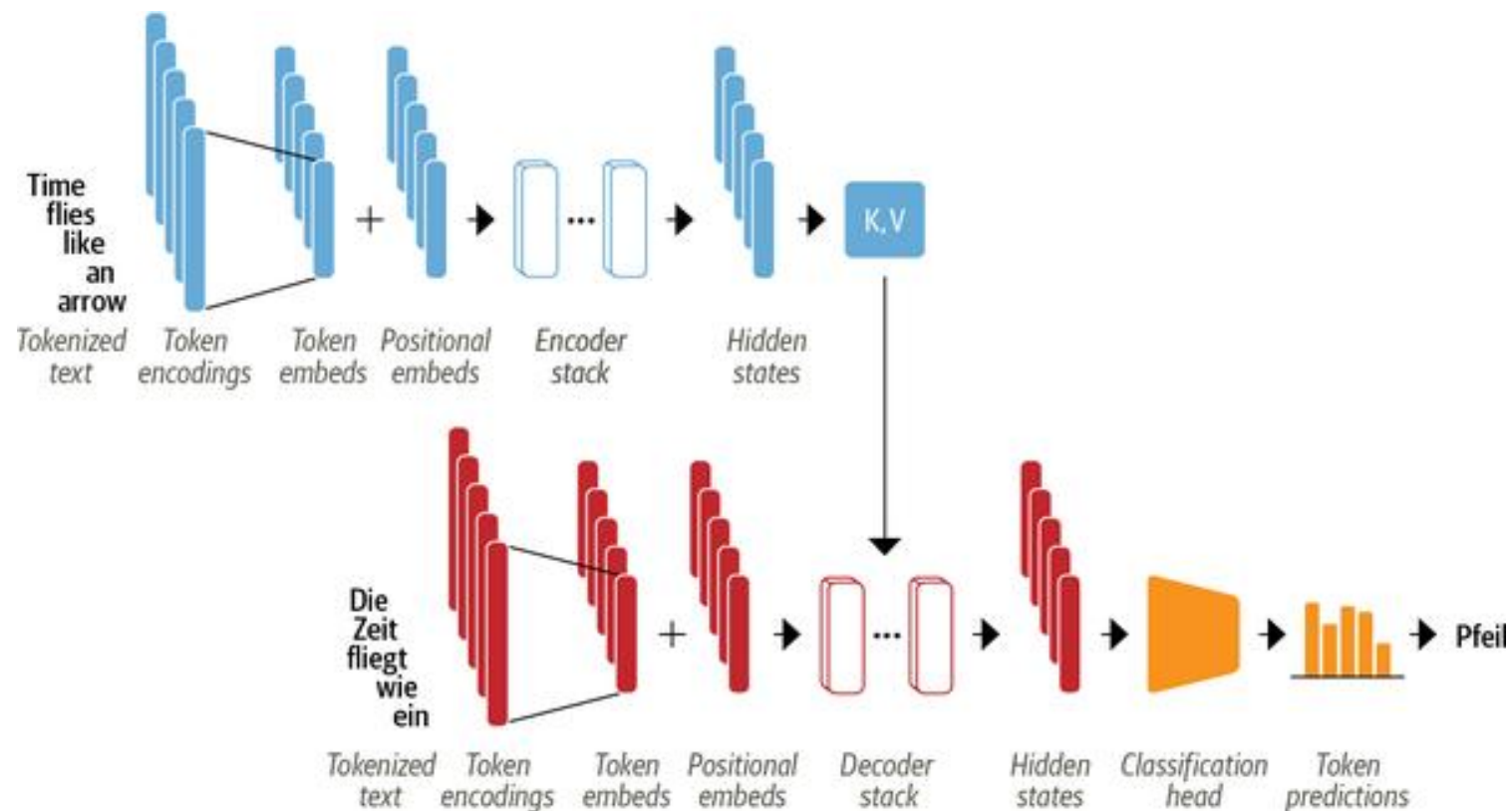
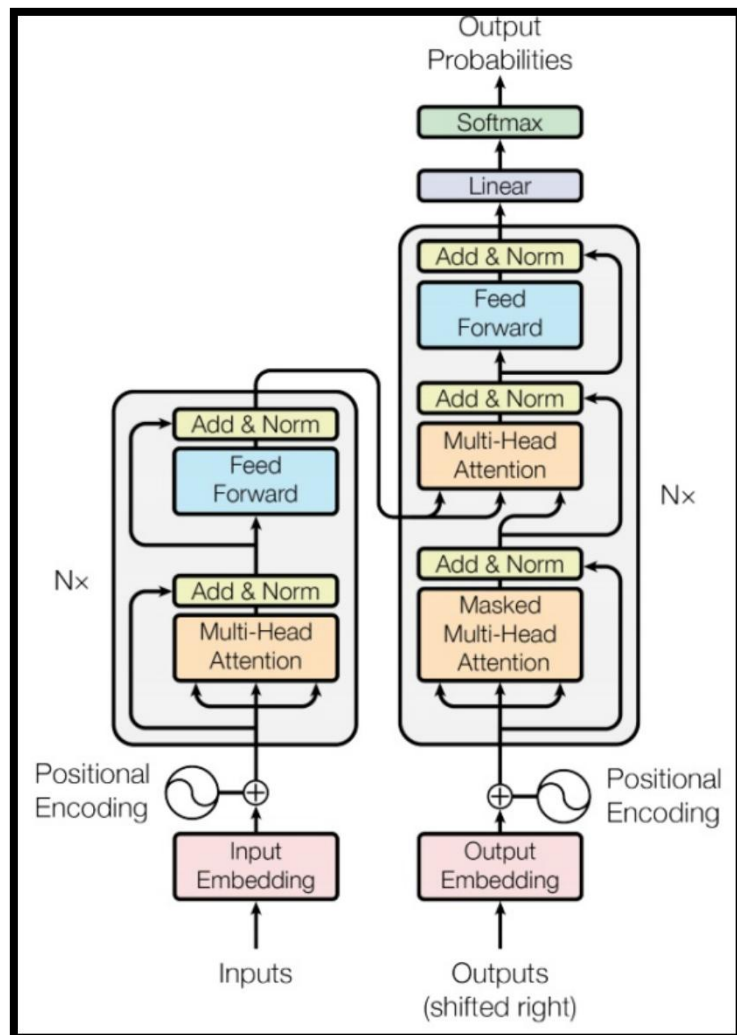
- Scheduled Sampling for Transformer

<https://arxiv.org/abs/1906.07651>

- Parallel Scheduled Sampling

<https://arxiv.org/abs/1906.04331>







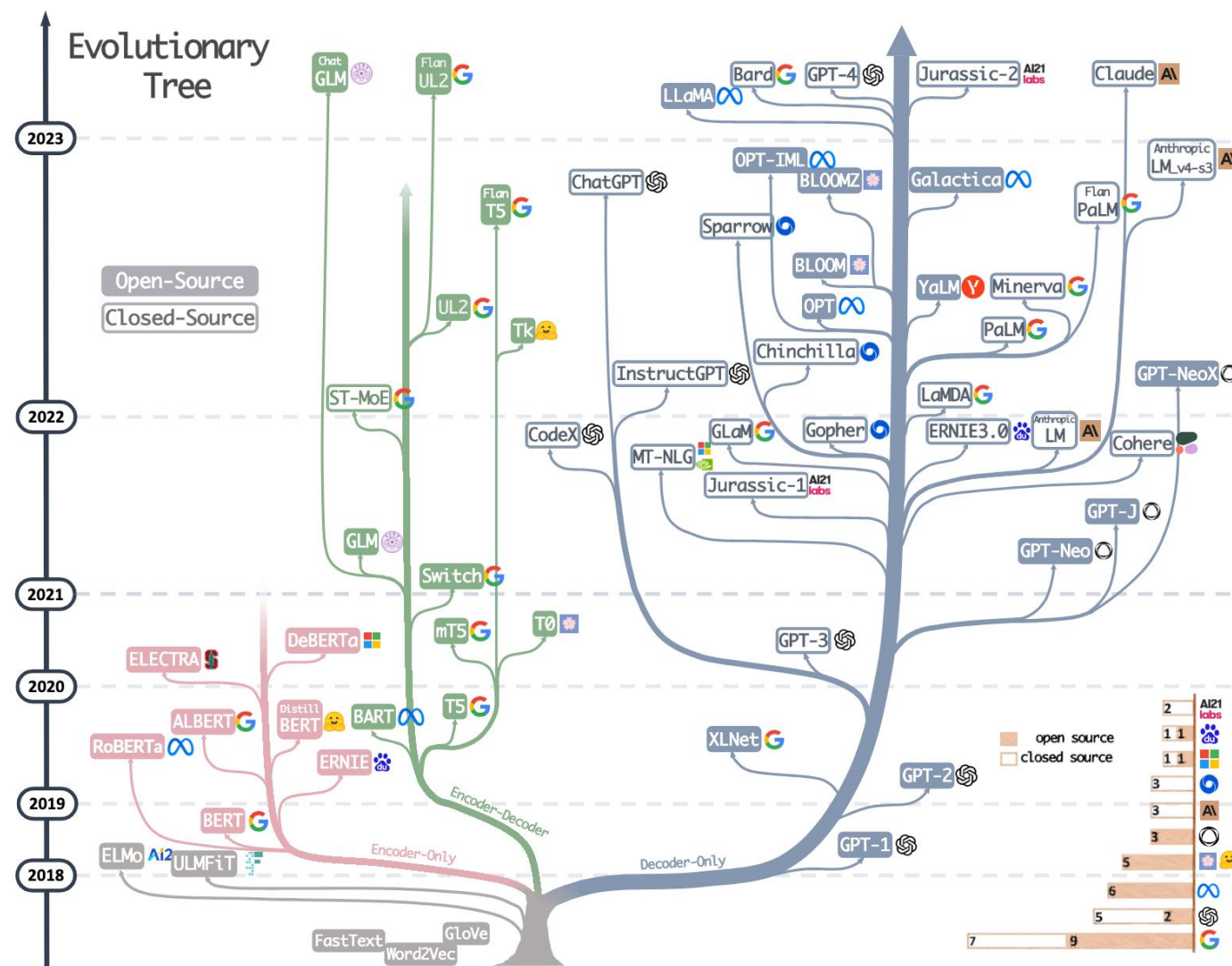
上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

Contents

- Self-attention
- Encoder
- Decoder
- Encoder-Decoder
- **NLP/Vision Transformer**
- Mixture of Experts

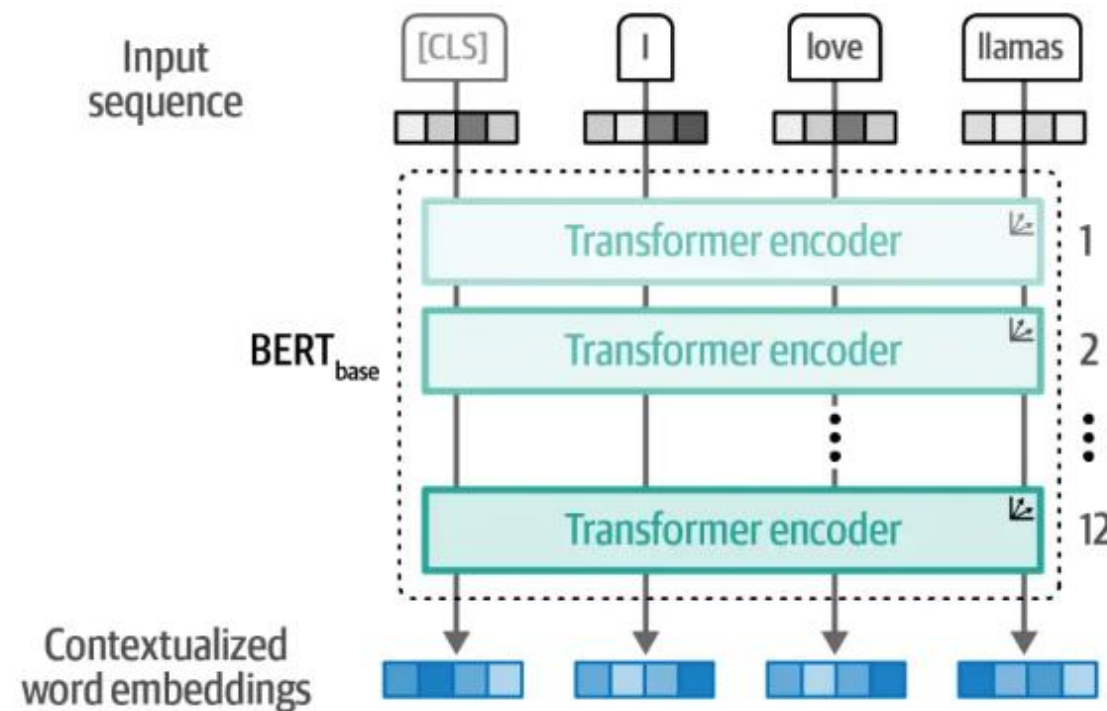


- Encoder-only (BERT)
- Encoder-Decoder (T5)
- Decoder-only (GPT)



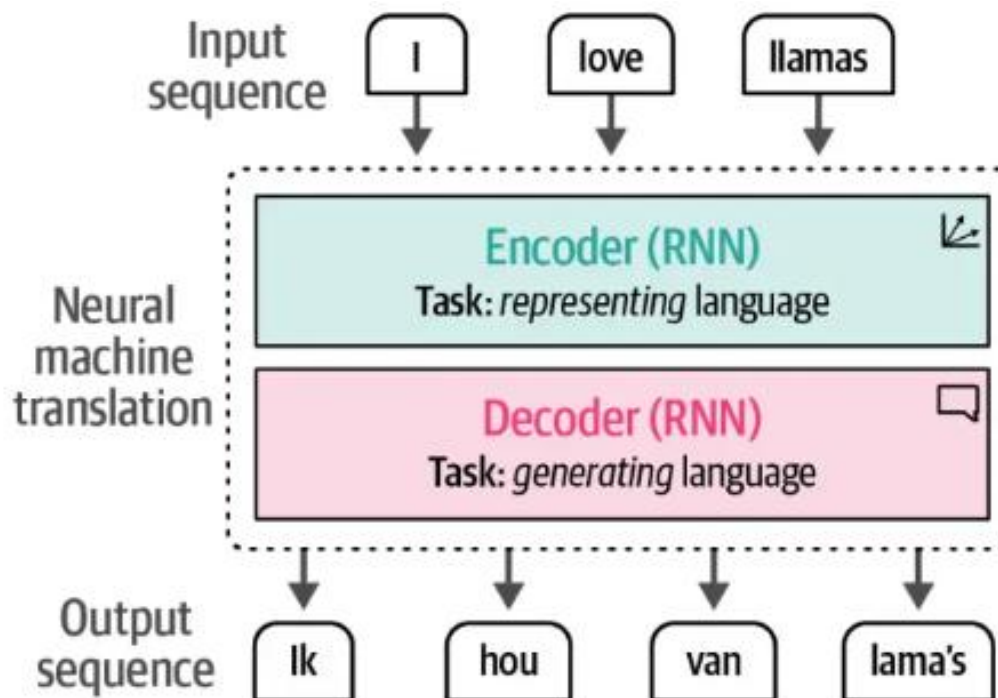
Encoder-only (BERT)

- **Focuses on understanding and analyzing input information rather than creating new content**
- **Good at understanding and classifying information, such as determining the sentiment tendency (positive or negative) of a text or topic classification**



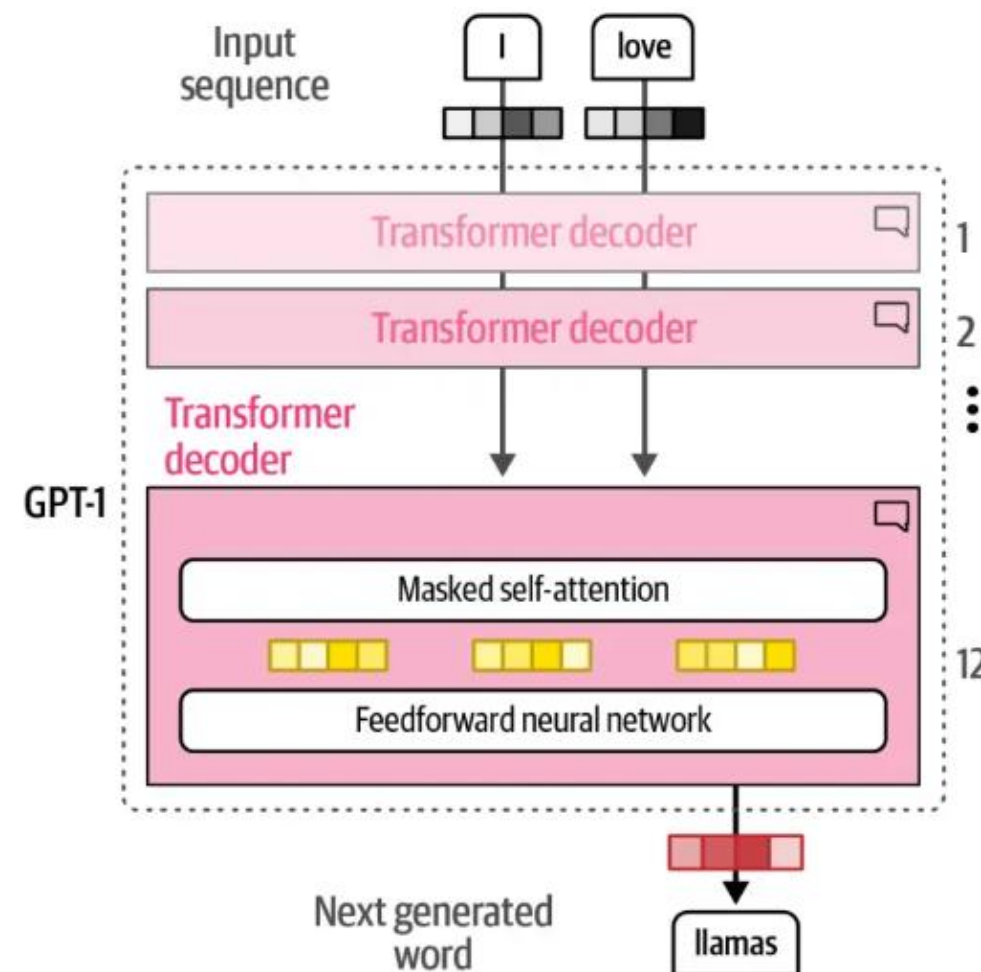
Encoder-Decoder (T5)

- They excel at tasks that require understanding input and producing relevant output
- Good at translation or question-answering systems



Decoder-only (GPT)

- OpenAI chose the Decoder-Only (e.g. GPT)
- Effective for natural language generation
- Better at understanding and predicting language patterns
- Particularly suitable for handling open-ended, generative tasks (e.g. Caption)



Why Decoder-only?

Efficiency first: dual optimization of training and reasoning (KV-Cache)

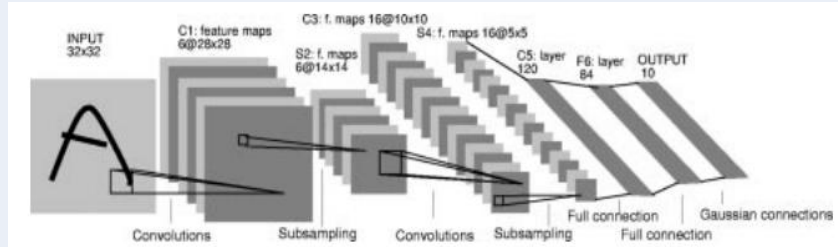
Parameter reuse: capacity maximization

Full-rank attention: stronger theoretical modeling power (Causal Mask)

Unified tasks: the king of zero-sample generalization

Ecological promotion: the victory of the open source community (LLaMA, DeepSeek, Megatron)

Structure

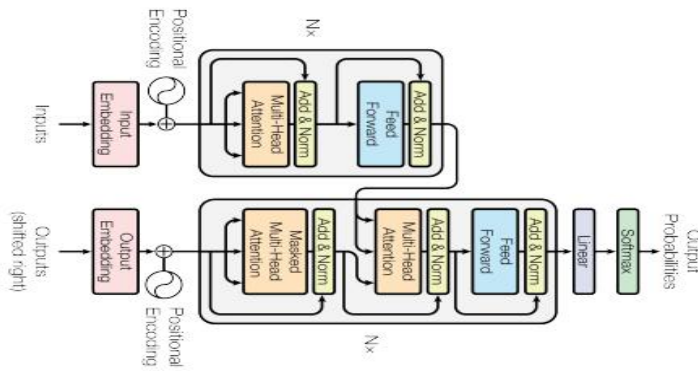


Advantage

Inductive bias:
translation
equivariance, locality

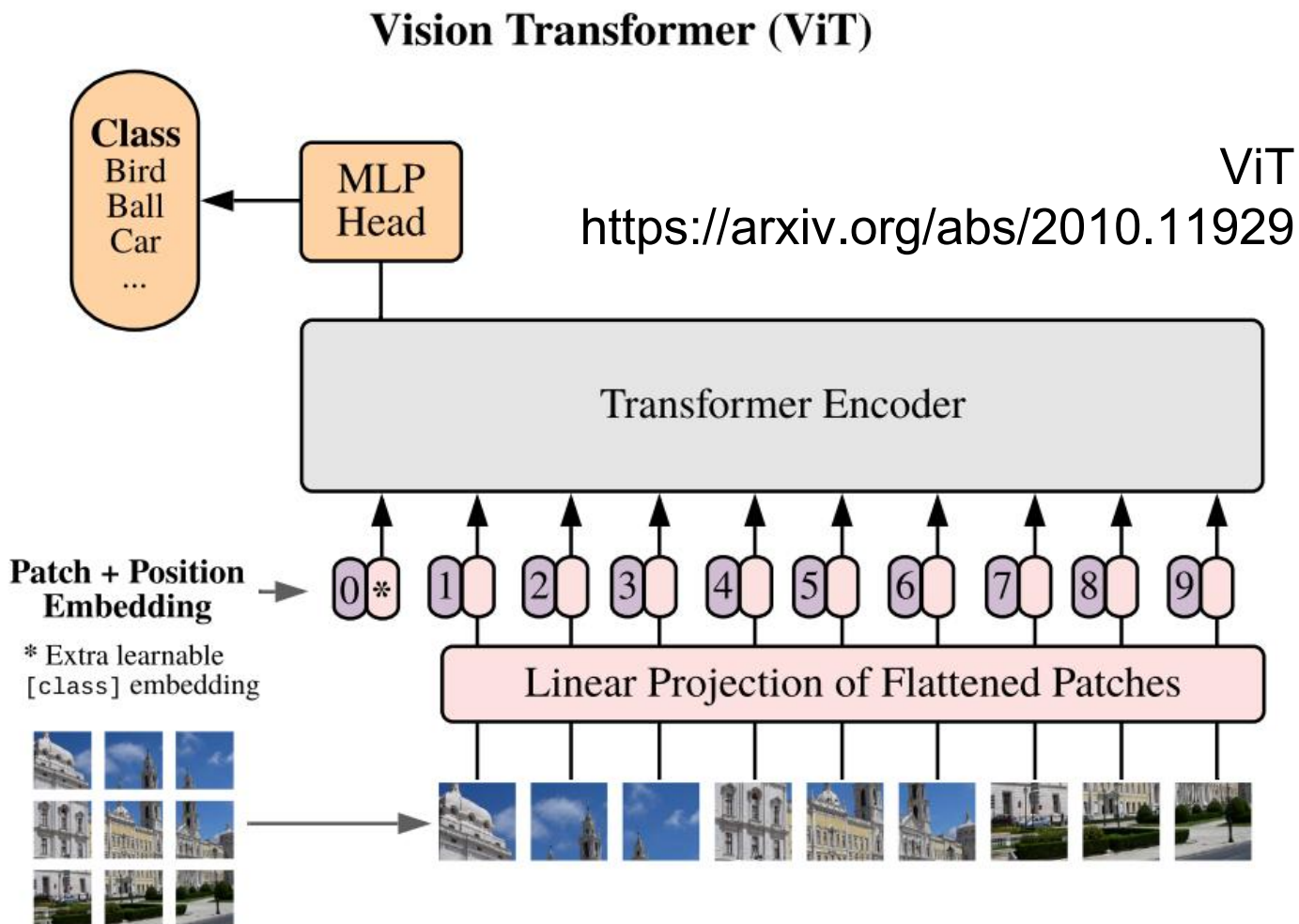
Disadvantage

Limited receptive field

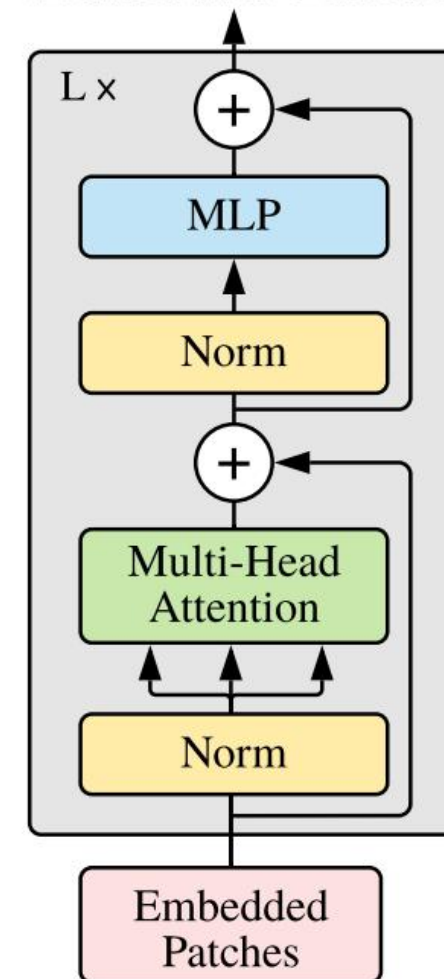


Long-range
receptive field

Training is difficult, more
data is needed.
Large amount of
calculation



Transformer Encoder



Patch Embedding

- **Input:** 2-D image, mark as $x \in \mathbb{R}^{H \times W \times C}$
- **Patch size:** $P \times P$, mark as $x_p \in \mathbb{R}^{P \times P \times C}$
- **Patch num:** $N = HW/P^2$



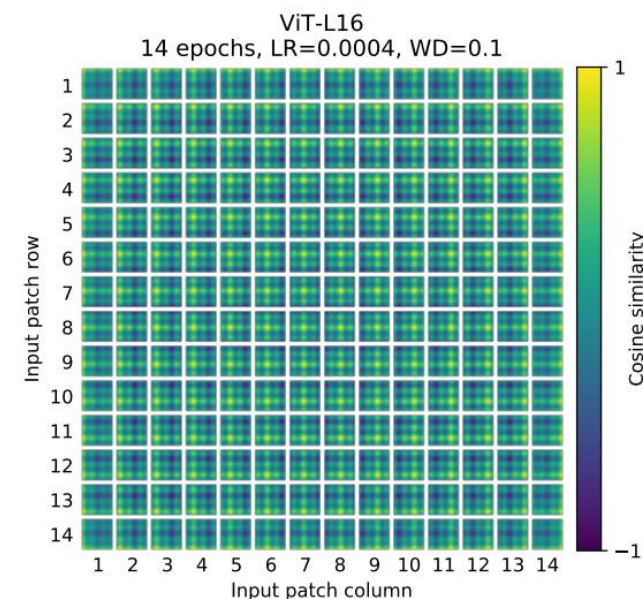
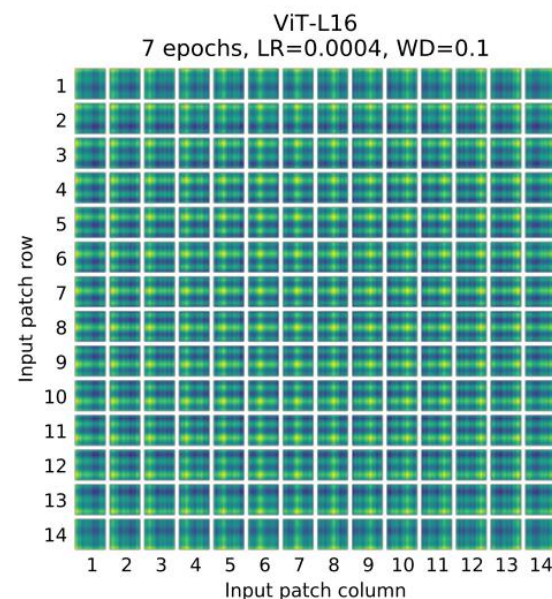
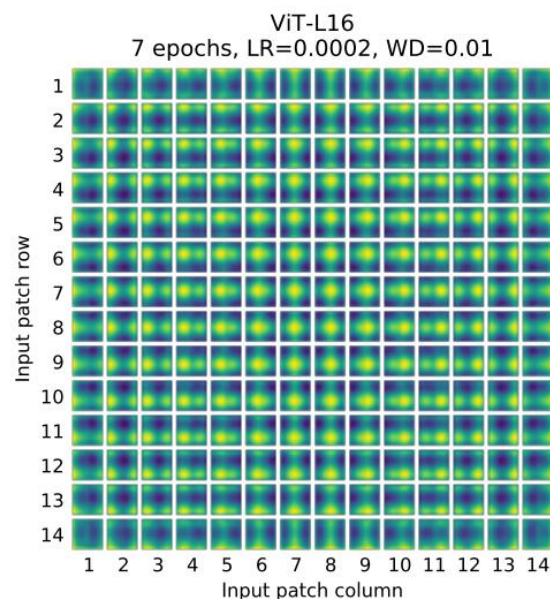
Position Embedding

- No positional embedding
- **1-D positional embedding: consider 2-D patches as 1-D sequences (used in ViT)**
- 2-D positional embedding: consider the 2-D position of patches (x, y)
- Relative positional embeddings



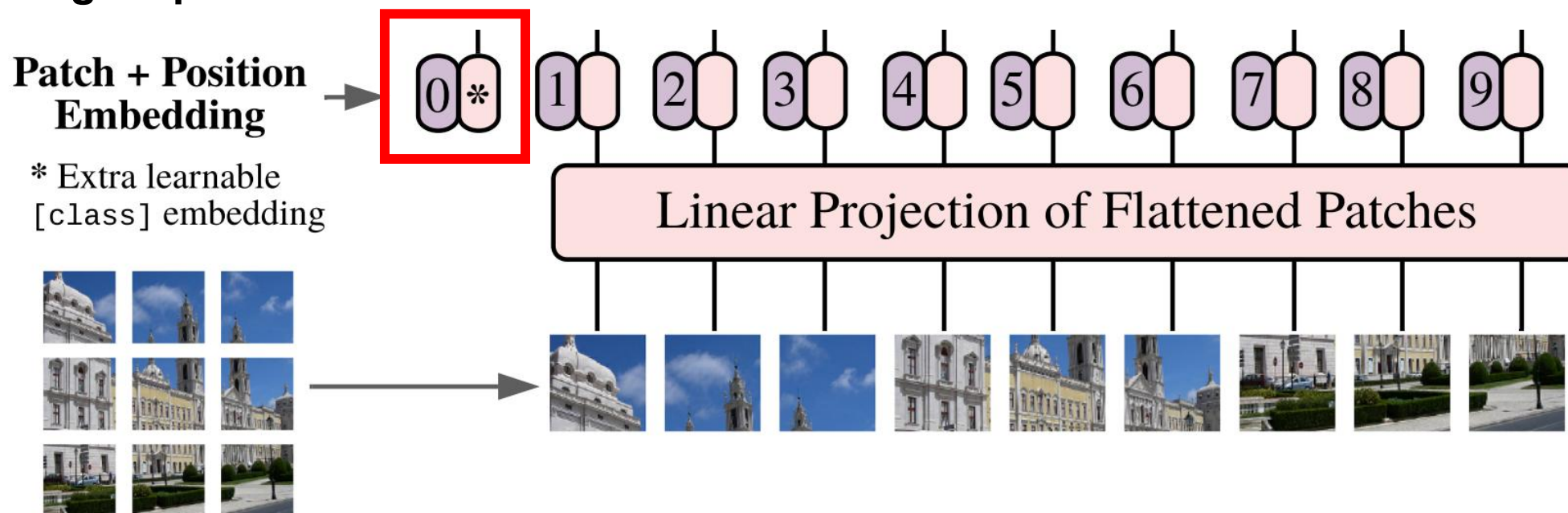
Position Embedding

- No positional embedding
- 1-D positional embedding: consider 2-D patches as 1-D sequences (used in ViT)
- 2-D positional embedding: consider the 2-D position of patches (x, y)
- Relative positional embeddings



Class Token

- A learnable embedding (class token) is prepended to the sequence of embedded patches, whose state at the output of the Transformer encoder serves as the image representation.



Transformer Encoder

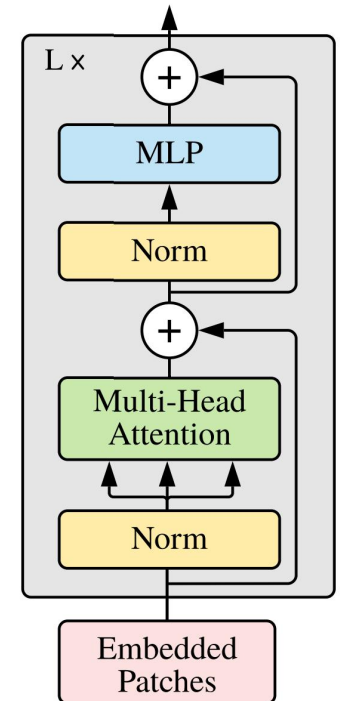
$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \cdots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \quad (1)$$

$$\mathbf{z}'_\ell = \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L \quad (2)$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \quad \ell = 1 \dots L \quad (3)$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \quad (4)$$

Transformer Encoder



Vision Transformer



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Parameters and Calculations

- **Layers:** the number of blocks;
- **Hidden size D :** hidden layer features, D is always the same in each block;
- **MLP size:** generally set to $4D$;
- **Heads:** the number of heads in MSA;
- **Patch size:** The patch size of the model input, this only affects the amount of calculation;

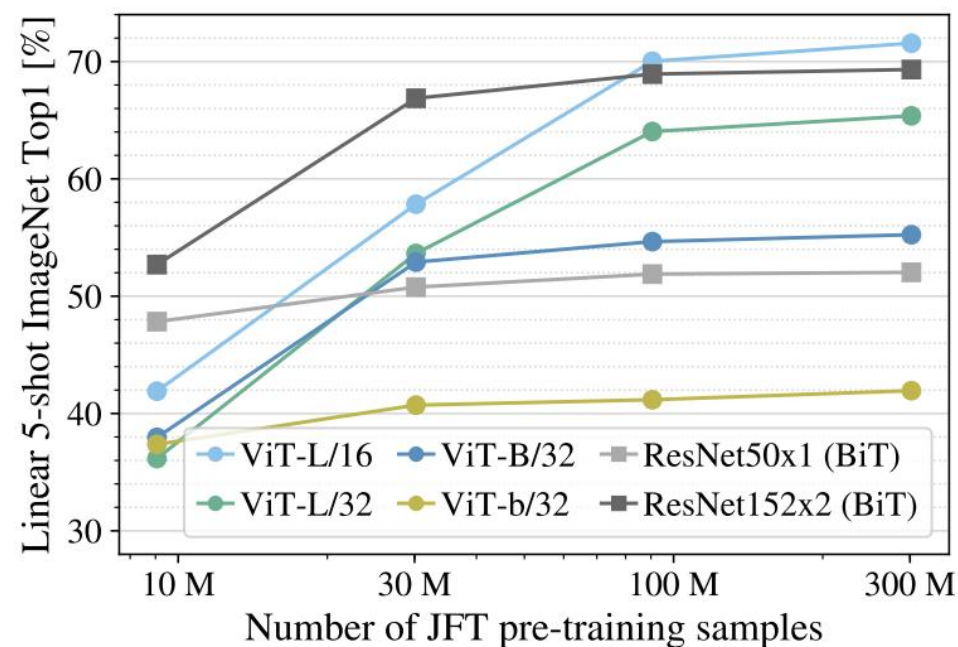
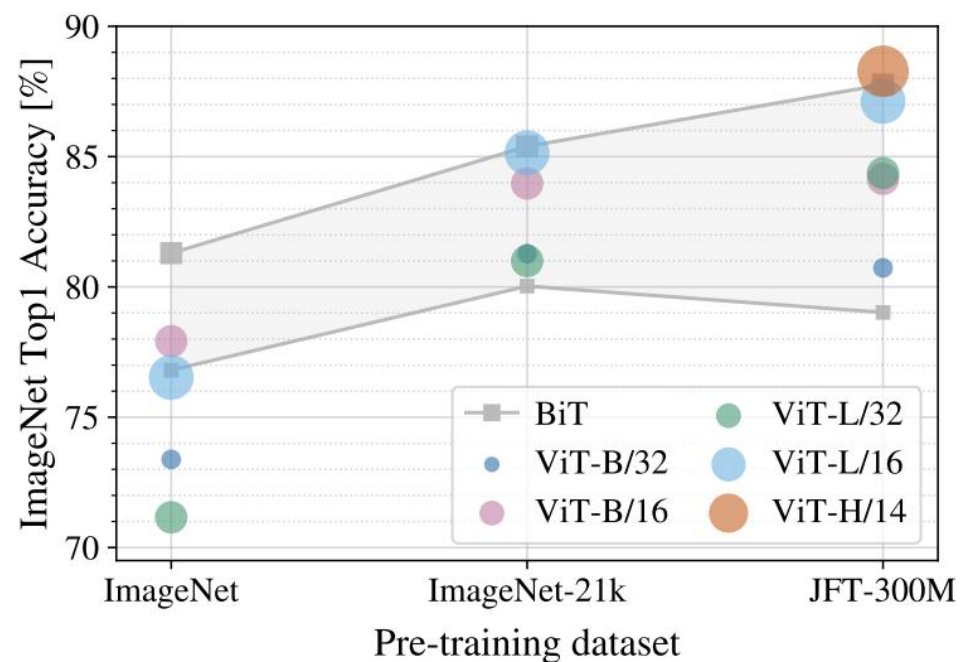
Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Performance

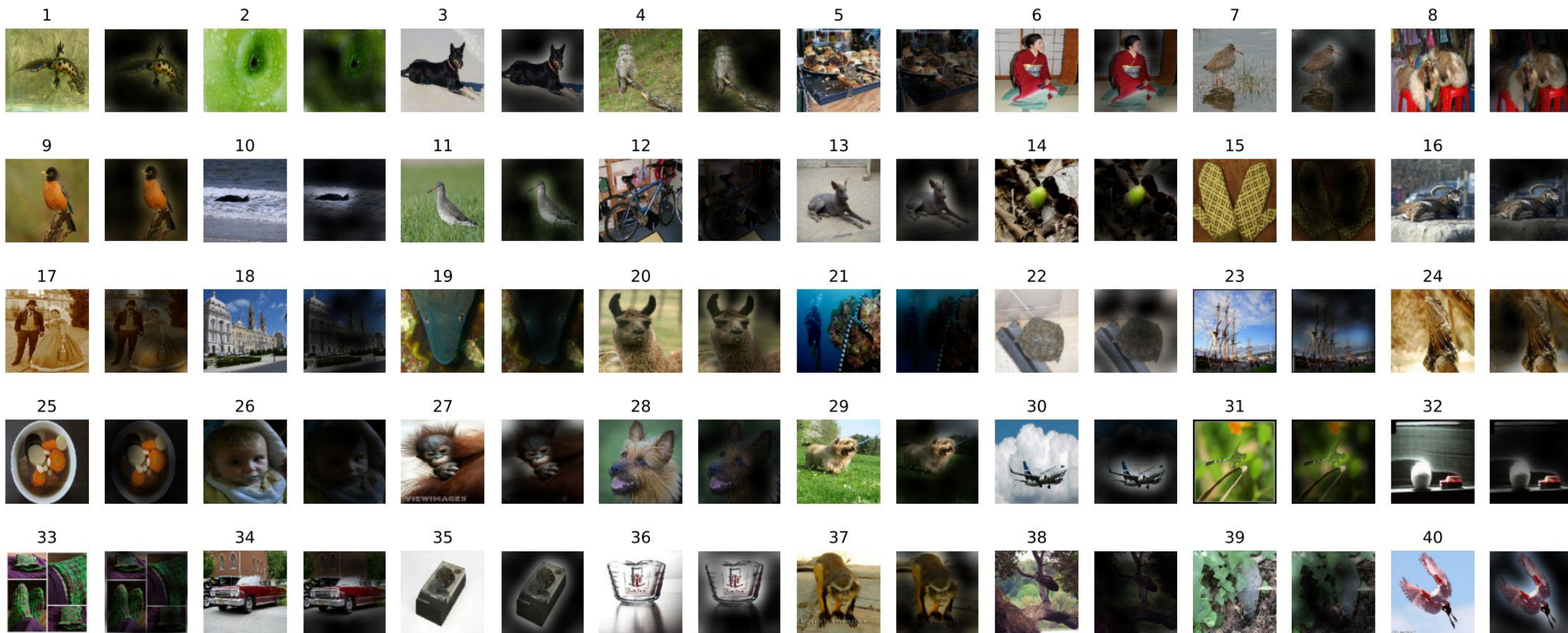
- Vision Transformer models pre-trained on the JFT-300M dataset outperform ResNet-based baselines on all datasets, while taking substantially less computational resources to pre-train.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Performance



Vision Transformer

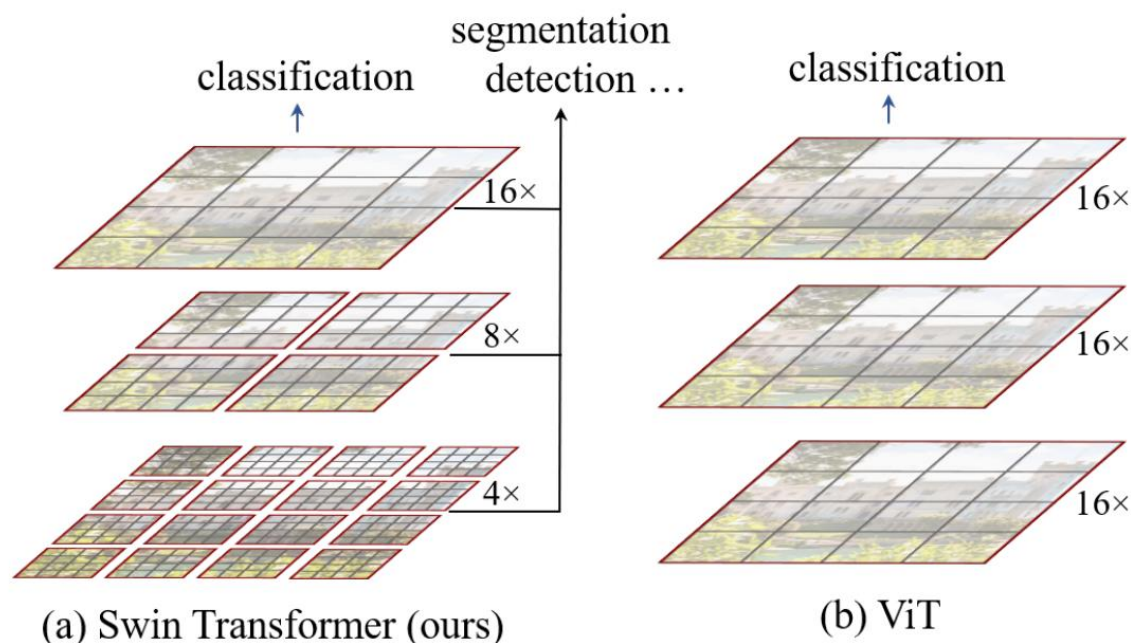


Limitations of ViT

- In existing Transformer-based models, tokens are all of a **fixed scale**, a property unsuitable for these vision applications.
- Computational complexity of its self-attention is **quadratic** to image size, causing problems in vision tasks with dense predictions at the pixel level.



- Hierarchical feature maps for convenient utilization of techniques like feature pyramid networks (FPN) or U-Net for dense predictions.
- Computing self-attention locally within non-overlapping windows equal number of patches to achieve linear complexity.

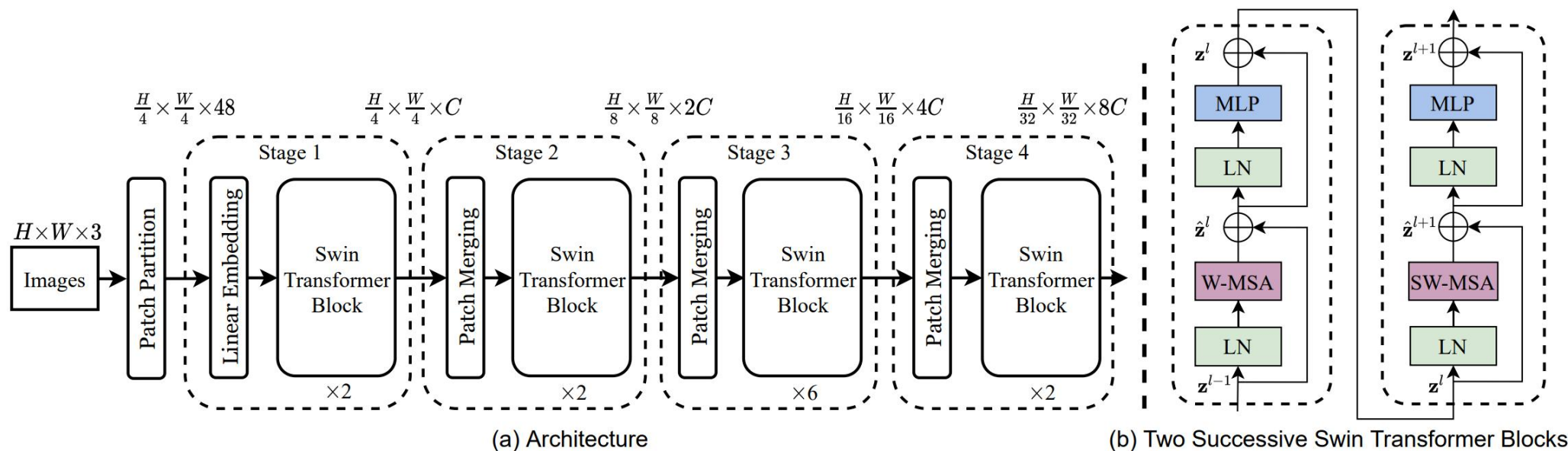
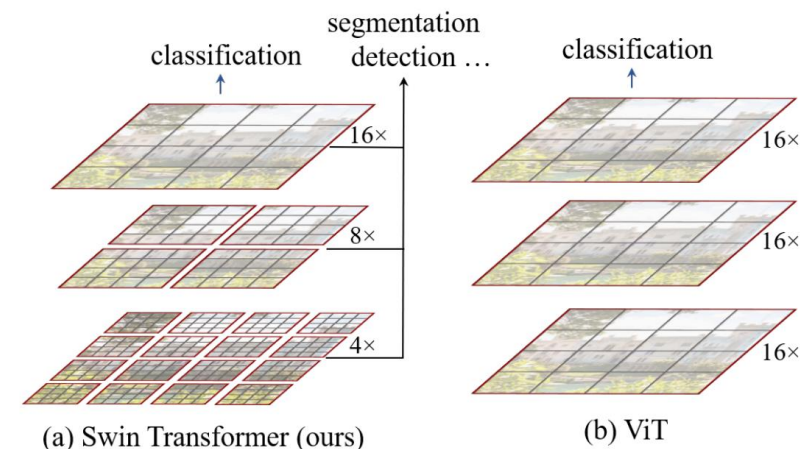


Swin
<https://arxiv.org/abs/2103.14030>

Architecture

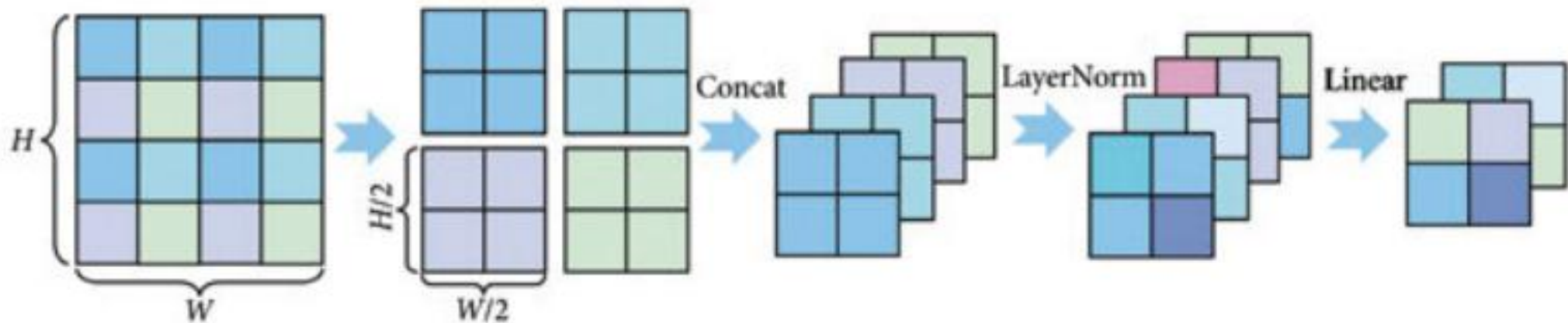
Novel Components

- Patch Merging
- Swin Transformer Block



Patch Merging

- To produce a hierarchical representation, the number of tokens is reduced by patch merging layers as the network gets deeper



Swin Transformer Block

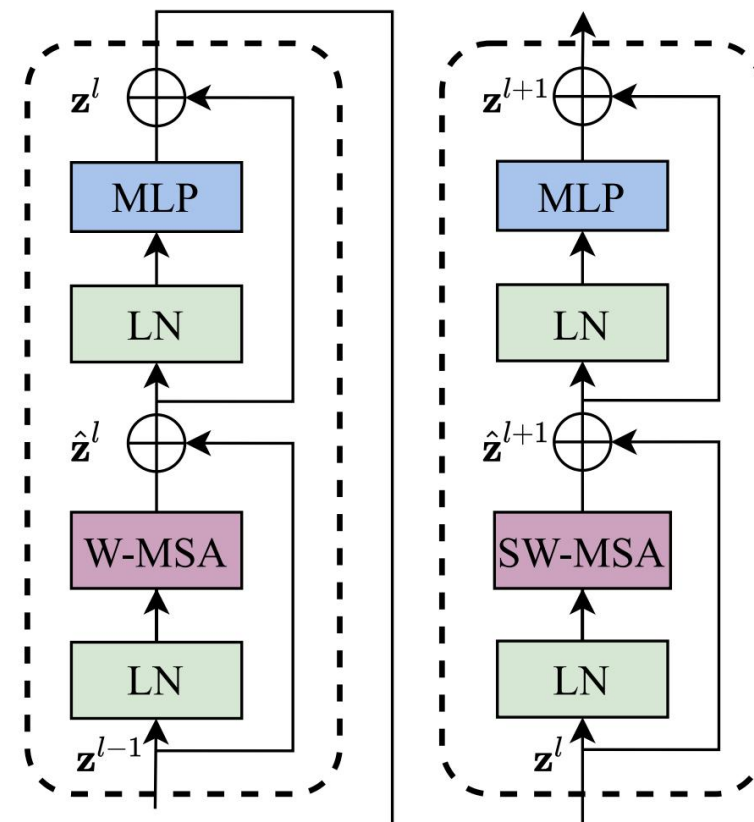
- W-MSA: Regular **W**indow based **M**ulti-head **S**elf **A**ttention
- SW-MSA: **S**hifting **W**indow based **M**ulti-head **S**elf **A**ttention

$$\hat{\mathbf{z}}^l = \text{W-MSA}(\text{LN}(\mathbf{z}^{l-1})) + \mathbf{z}^{l-1},$$

$$\mathbf{z}^l = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^l)) + \hat{\mathbf{z}}^l,$$

$$\hat{\mathbf{z}}^{l+1} = \text{SW-MSA}(\text{LN}(\mathbf{z}^l)) + \mathbf{z}^l,$$

$$\mathbf{z}^{l+1} = \text{MLP}(\text{LN}(\hat{\mathbf{z}}^{l+1})) + \hat{\mathbf{z}}^{l+1},$$



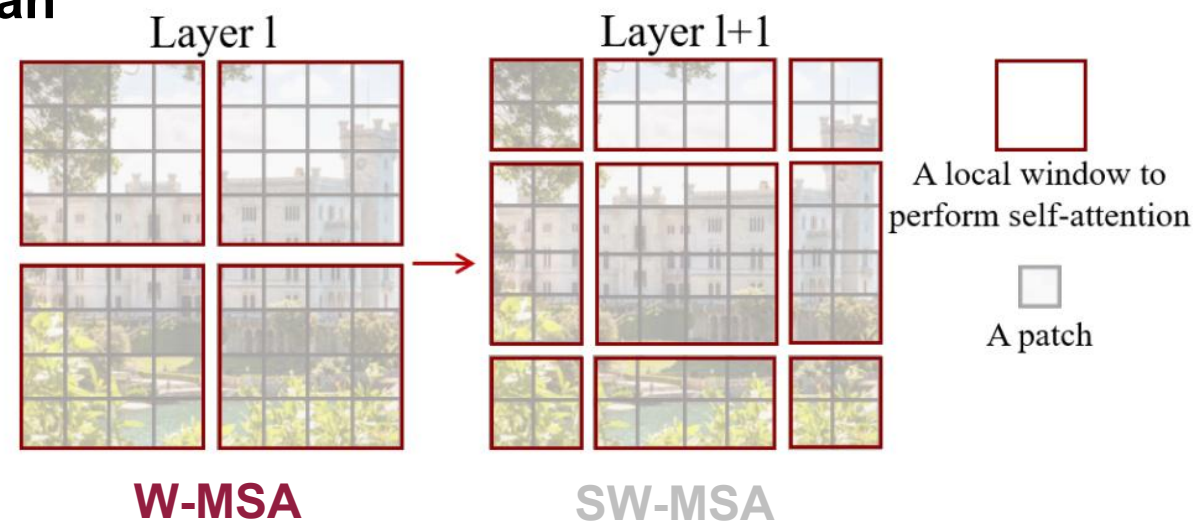
W-MSA

- The computational complexity of a global MSA module and a window based one on an image of $h \times w$ patches are:

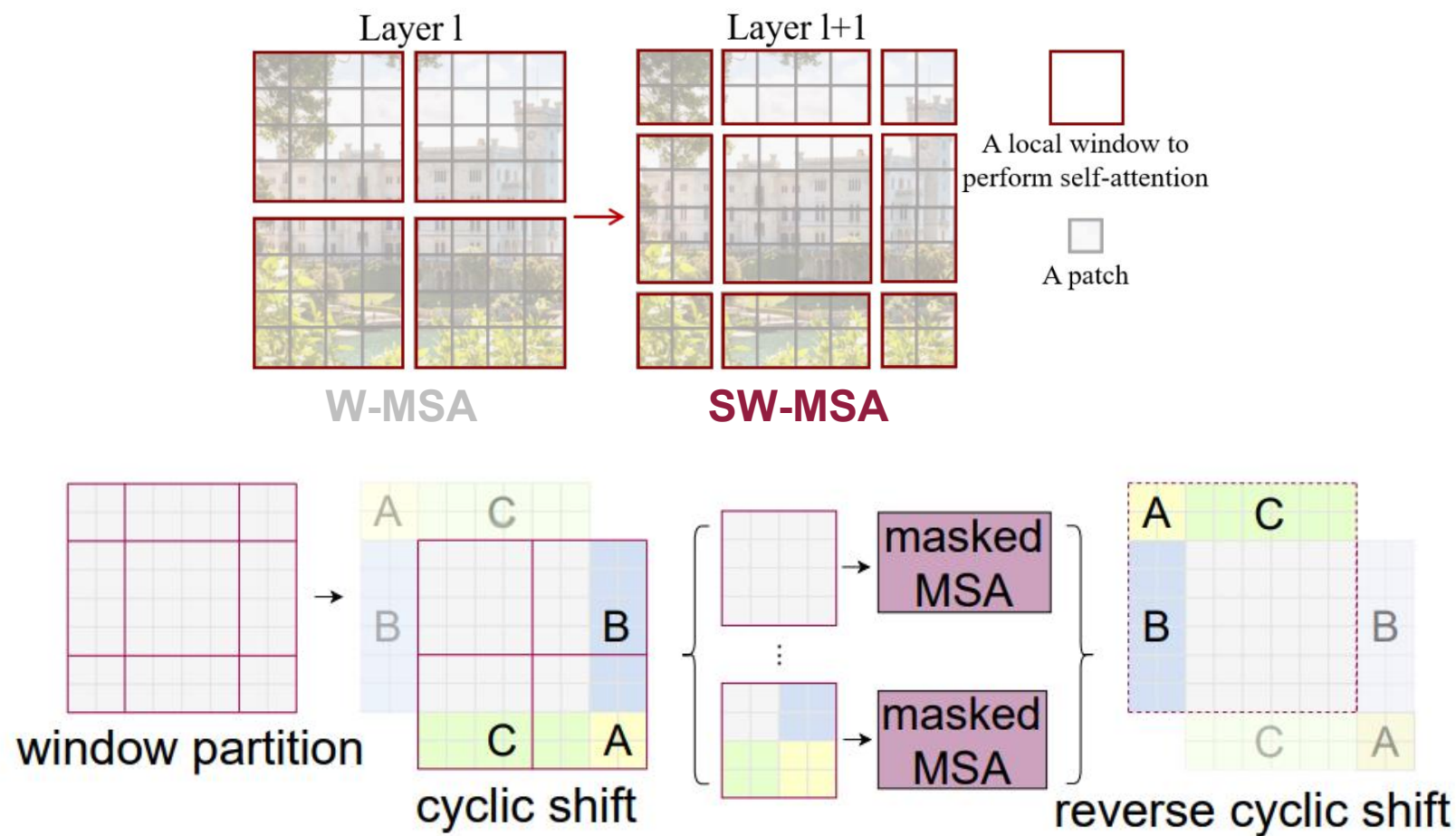
$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C,$$

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

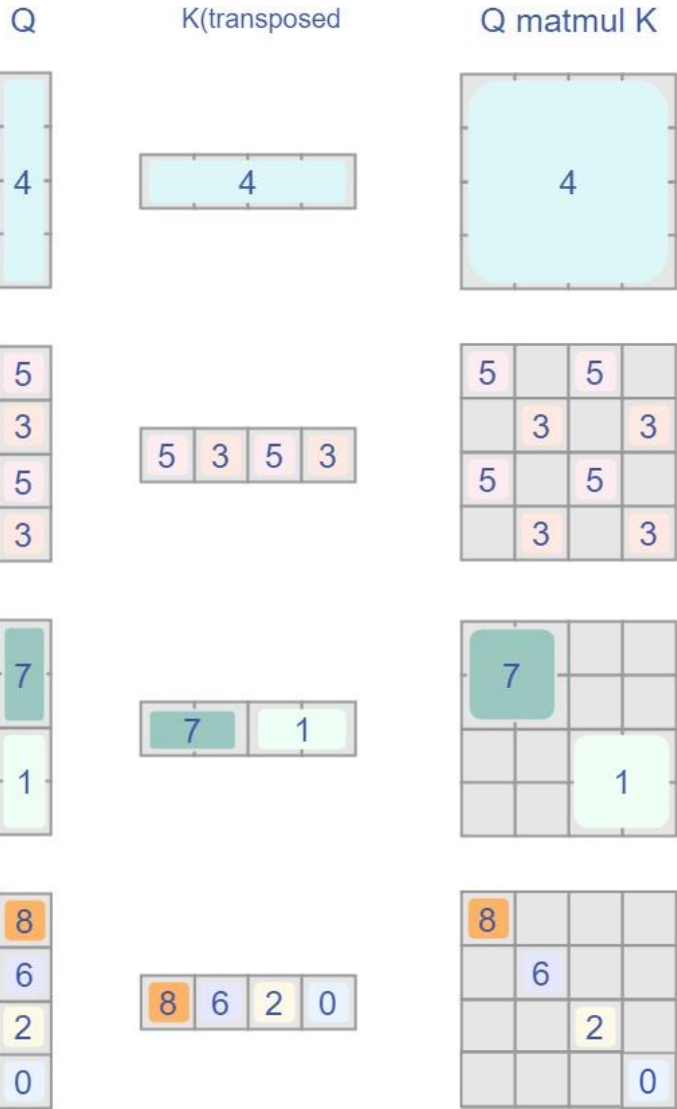
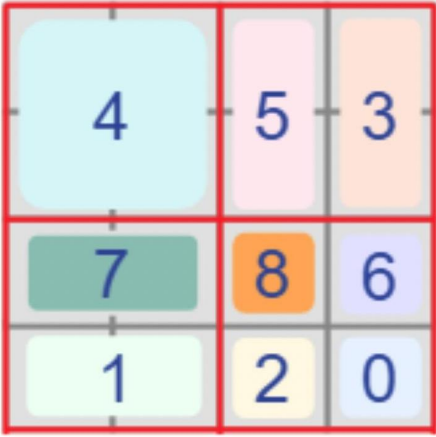
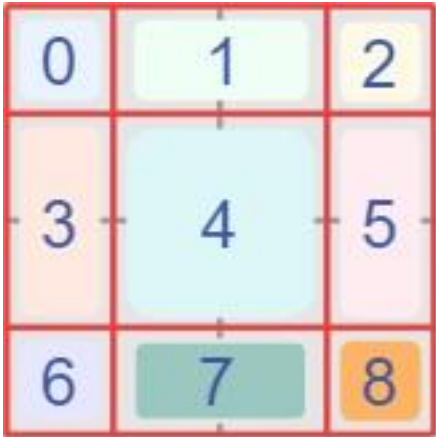
- Global self-attention computation is generally unaffordable for a large hw , while the window based self-attention is scalable.



SW-MSA



SW-MSA



Attention Mask

Position Embedding

- No positional embedding
- 1-D positional embedding: consider 2-D patches as 1-D sequences
- 2-D positional embedding: consider the 2-D position of patches (x, y)
- **Relative positional embeddings/bias**

$$\text{Attention}(Q, K, V) = \text{SoftMax}(QK^T / \sqrt{d} + B)V,$$

Detailed Architecture Specifications

	downsp. rate (output size)	Swin-T	Swin-S	Swin-B	Swin-L
stage 1	$4\times$ (56×56)	concat 4×4 , 96-d, LN	concat 4×4 , 96-d, LN	concat 4×4 , 128-d, LN	concat 4×4 , 192-d, LN
		$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 96, head 3} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 96, head 3} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 128, head 4} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 192, head 6} \end{bmatrix} \times 2$
stage 2	$8\times$ (28×28)	concat 2×2 , 192-d, LN	concat 2×2 , 192-d, LN	concat 2×2 , 256-d, LN	concat 2×2 , 384-d, LN
		$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 192, head 6} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 192, head 6} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 256, head 8} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 384, head 12} \end{bmatrix} \times 2$
stage 3	$16\times$ (14×14)	concat 2×2 , 384-d, LN	concat 2×2 , 384-d, LN	concat 2×2 , 512-d, LN	concat 2×2 , 768-d, LN
		$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 384, head 12} \end{bmatrix} \times 6$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 384, head 12} \end{bmatrix} \times 18$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 512, head 16} \end{bmatrix} \times 18$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 768, head 24} \end{bmatrix} \times 18$
stage 4	$32\times$ (7×7)	concat 2×2 , 768-d, LN	concat 2×2 , 768-d, LN	concat 2×2 , 1024-d, LN	concat 2×2 , 1536-d, LN
		$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 768, head 24} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 768, head 24} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 1024, head 32} \end{bmatrix} \times 2$	$\begin{bmatrix} \text{win. sz. } 7\times 7, \\ \text{dim 1536, head 48} \end{bmatrix} \times 2$

Table 7. Detailed architecture specifications.

Performance

(b) ImageNet-22K pre-trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
R-101x3 [38]	384 ²	388M	204.6G	-	84.4
R-152x4 [38]	480 ²	937M	840.5G	-	85.4
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	84.0
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	85.2
Swin-B	224 ²	88M	15.4G	278.1	85.2
Swin-B	384 ²	88M	47.0G	84.7	86.4
Swin-L	384 ²	197M	103.9G	42.1	87.3

(a) Regular ImageNet-1K trained models					
method	image size	#param.	FLOPs	throughput (image / s)	ImageNet top-1 acc.
RegNetY-4G [48]	224 ²	21M	4.0G	1156.7	80.0
RegNetY-8G [48]	224 ²	39M	8.0G	591.6	81.7
RegNetY-16G [48]	224 ²	84M	16.0G	334.7	82.9
EffNet-B3 [58]	300 ²	12M	1.8G	732.1	81.6
EffNet-B4 [58]	380 ²	19M	4.2G	349.4	82.9
EffNet-B5 [58]	456 ²	30M	9.9G	169.1	83.6
EffNet-B6 [58]	528 ²	43M	19.0G	96.9	84.0
EffNet-B7 [58]	600 ²	66M	37.0G	55.1	84.3
ViT-B/16 [20]	384 ²	86M	55.4G	85.9	77.9
ViT-L/16 [20]	384 ²	307M	190.7G	27.3	76.5
DeiT-S [63]	224 ²	22M	4.6G	940.4	79.8
DeiT-B [63]	224 ²	86M	17.5G	292.3	81.8
DeiT-B [63]	384 ²	86M	55.4G	85.9	83.1
Swin-T	224 ²	29M	4.5G	755.2	81.3
Swin-S	224 ²	50M	8.7G	436.9	83.0
Swin-B	224 ²	88M	15.4G	278.1	83.5
Swin-B	384 ²	88M	47.0G	84.7	84.5

Performance

(a) Various frameworks							
Method	Backbone	AP ^{box}	AP ₅₀ ^{box}	AP ₇₅ ^{box}	#param.	FLOPs	FPS
Cascade	R-50	46.3	64.3	50.5	82M	739G	18.0
Mask R-CNN	Swin-T	50.5	69.3	54.9	86M	745G	15.3
ATSS	R-50	43.5	61.9	47.0	32M	205G	28.3
	Swin-T	47.2	66.5	51.3	36M	215G	22.3
RepPointsV2	R-50	46.5	64.6	50.3	42M	274G	13.6
	Swin-T	50.0	68.5	54.2	45M	283G	12.0
Sparse R-CNN	R-50	44.5	63.4	48.2	106M	166G	21.0
	Swin-T	47.9	67.3	52.3	110M	172G	18.4

Table 2. Results on COCO object detection and instance segmentation. [†] denotes that additional deconvolution layers are used to produce hierarchical feature maps. * indicates multi-scale testing.

ADE20K		val	test	#param.	FLOPs	FPS
Method	Backbone	mIoU	score			
DANet [23]	ResNet-101	45.2	-	69M	1119G	15.2
DLab.v3+ [11]	ResNet-101	44.1	-	63M	1021G	16.0
ACNet [24]	ResNet-101	45.9	38.5	-	-	-
DNL [71]	ResNet-101	46.0	56.2	69M	1249G	14.8
OCRNet [73]	ResNet-101	45.3	56.0	56M	923G	19.3
UperNet [69]	ResNet-101	44.9	-	86M	1029G	20.1
OCRNet [73]	HRNet-w48	45.7	-	71M	664G	12.5
DLab.v3+ [11]	ResNeSt-101	46.9	55.1	66M	1051G	11.9
DLab.v3+ [11]	ResNeSt-200	48.4	-	88M	1381G	8.1
SETR [81]	T-Large [‡]	50.3	61.7	308M	-	-
UperNet	DeiT-S [†]	44.0	-	52M	1099G	16.2
UperNet	Swin-T	46.1	-	60M	945G	18.5
UperNet	Swin-S	49.3	-	81M	1038G	15.2
UperNet	Swin-B [‡]	51.6	-	121M	1841G	8.7
UperNet	Swin-L [‡]	53.5	62.8	234M	3230G	6.2

Table 3. Results of semantic segmentation on the ADE20K val and test set. [†] indicates additional deconvolution layers are used to produce hierarchical feature maps. [‡] indicates that the model is pre-trained on ImageNet-22K.



上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

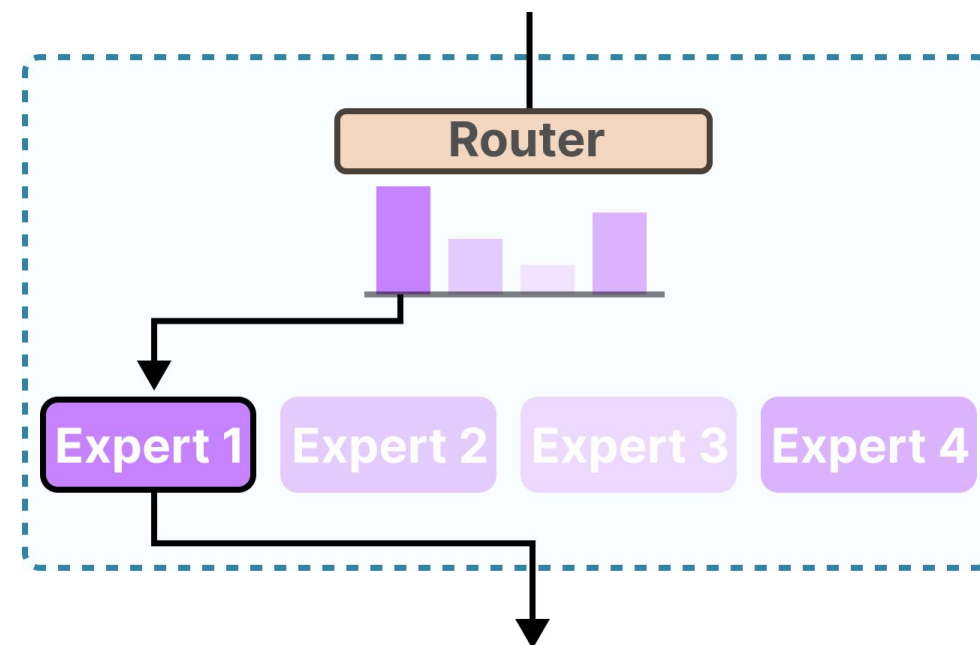
Contents

- Self-attention
- Encoder
- Decoder
- Encoder-Decoder
- NLP/Vision Transformer
- **Mixture of Experts**



What is Mixture of Experts?

- A technique that uses many different sub-models to improve the LLMs.
- Two main components define a MoE:
 - ✓ **Experts** - Each FFNN layer now has a set of “experts” of which a subset can be chosen. These “experts” are typically FFNNs themselves.
 - ✓ **Router or gate network** - Determines which tokens are sent to which experts.



What is Mixture of Experts?

- In each layer of an LLM with an MoE, we find (somewhat **specialized**) experts:



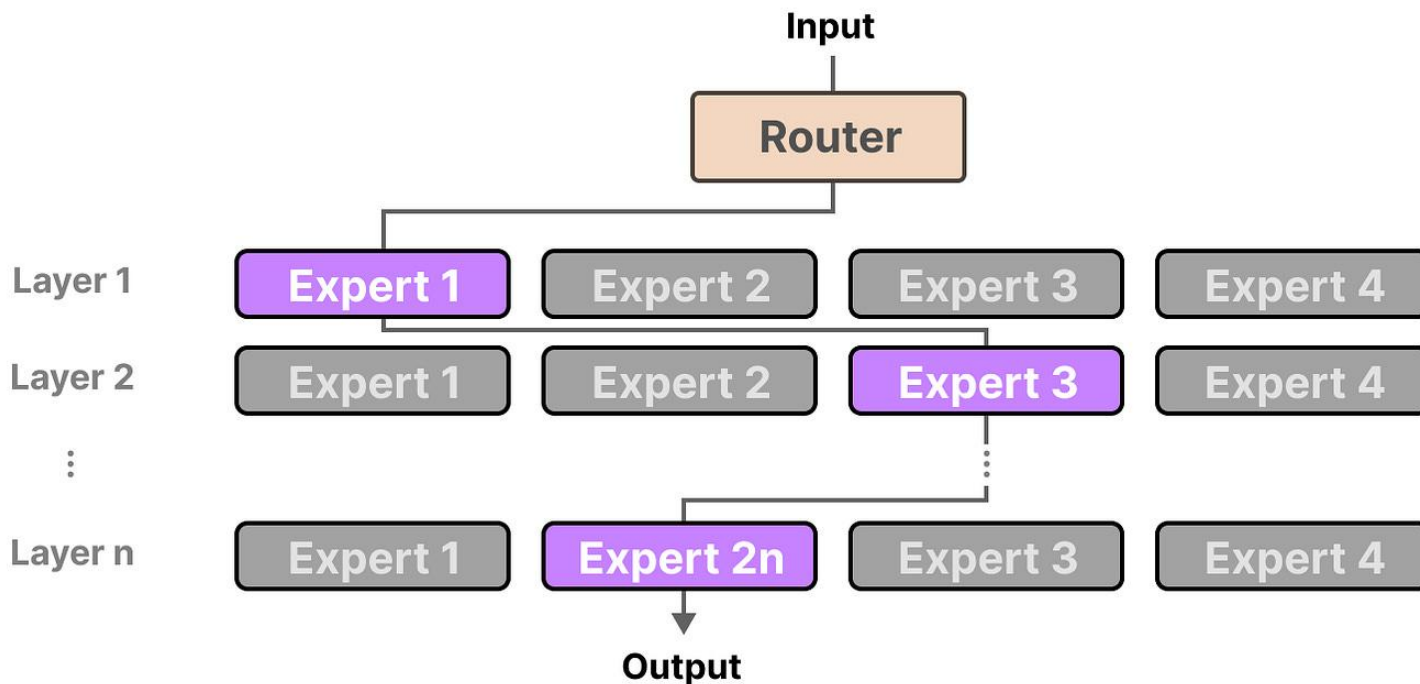
What is Mixture of Experts?

- Know that an “expert” is not specialized in a specific domain like “Psychology” or “Biology”. At most, it learns **syntactic information** on a word level instead:
- More specifically, their expertise is in handling specific tokens in specific contexts.



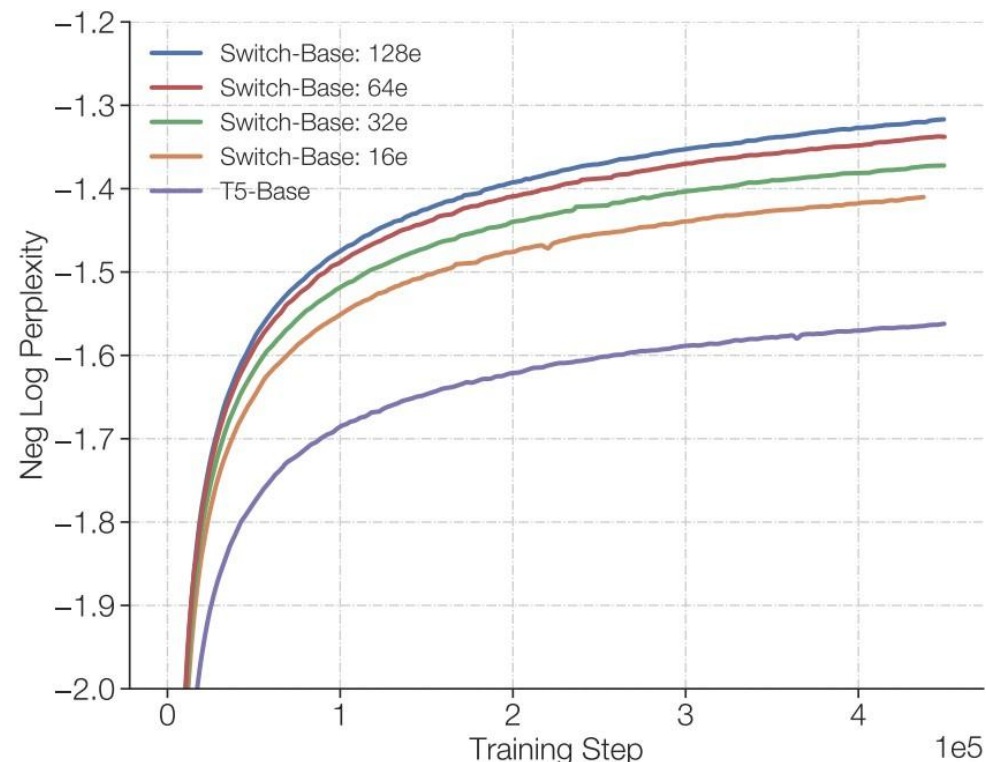
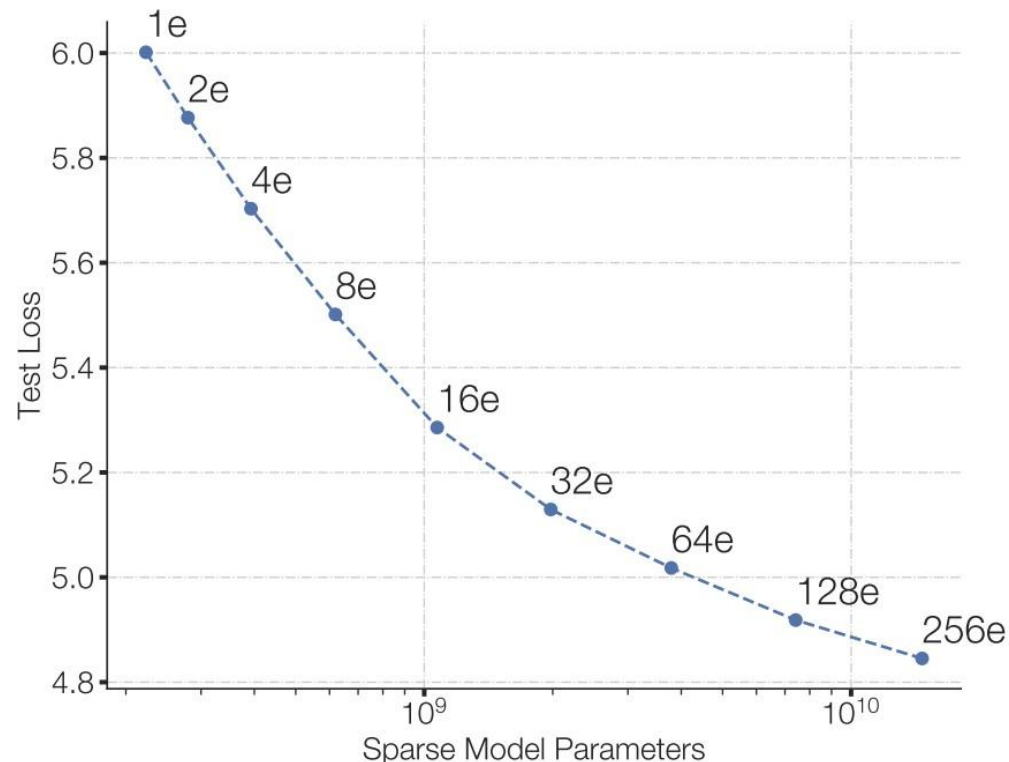
What is Mixture of Experts?

- The router (gate network) selects the expert(s) best suited for a given input:
- Each expert is not an entire LLM but a **sub-model part of** an LLM's architecture.



Why are MoEs getting popular?

- Same FLOP, more param does better



Why are MoEs getting popular?

- Faster to train MoEs

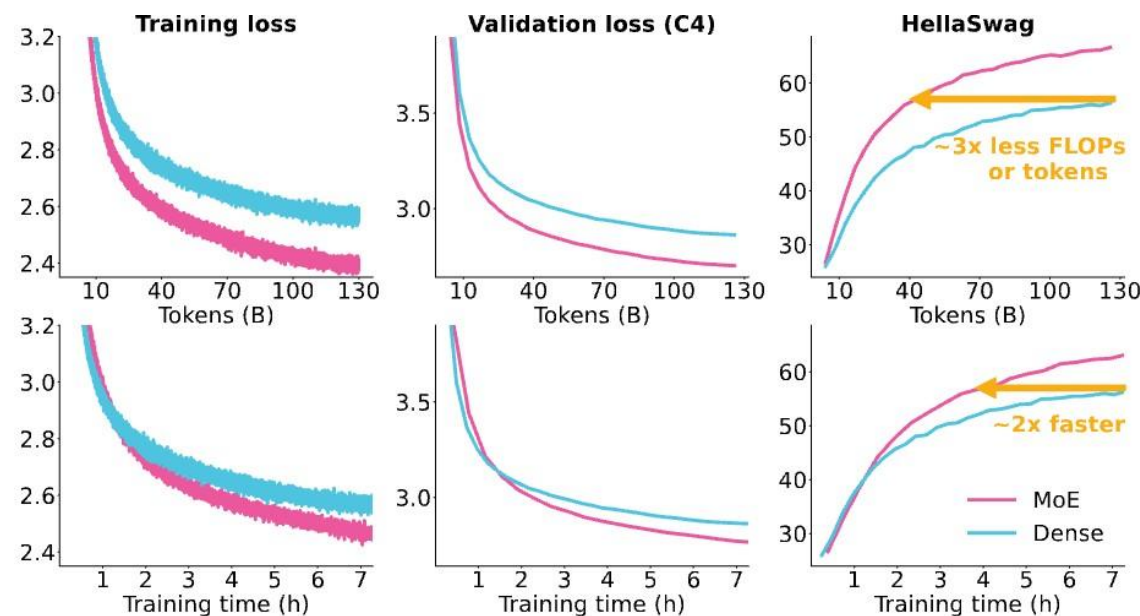
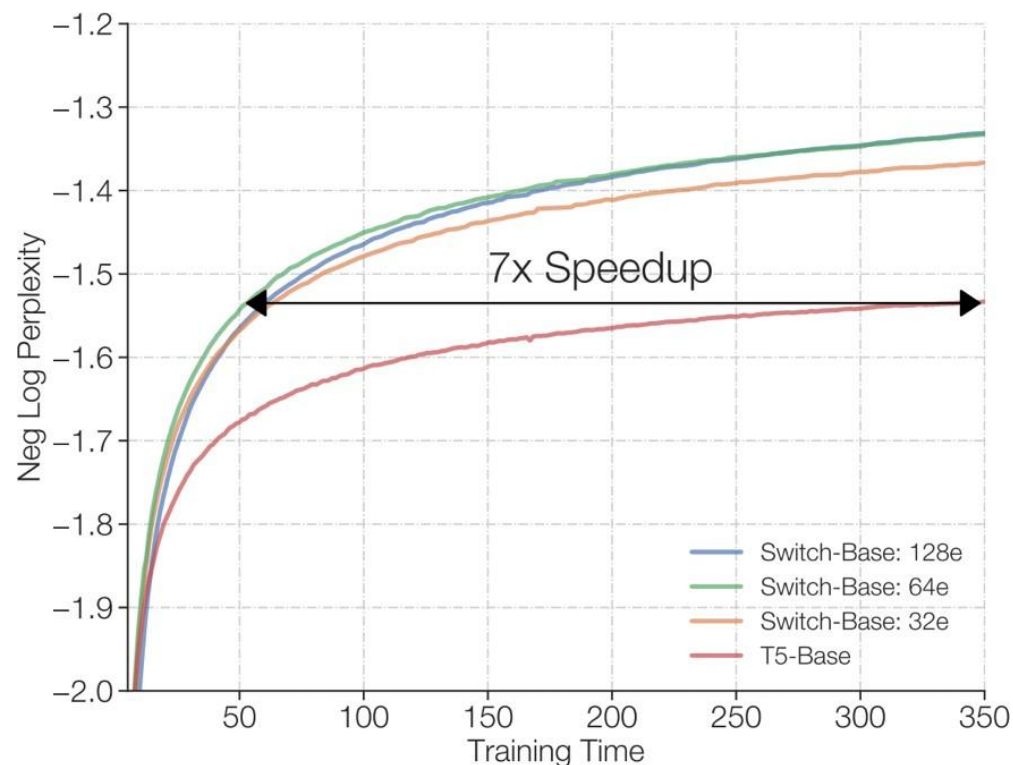
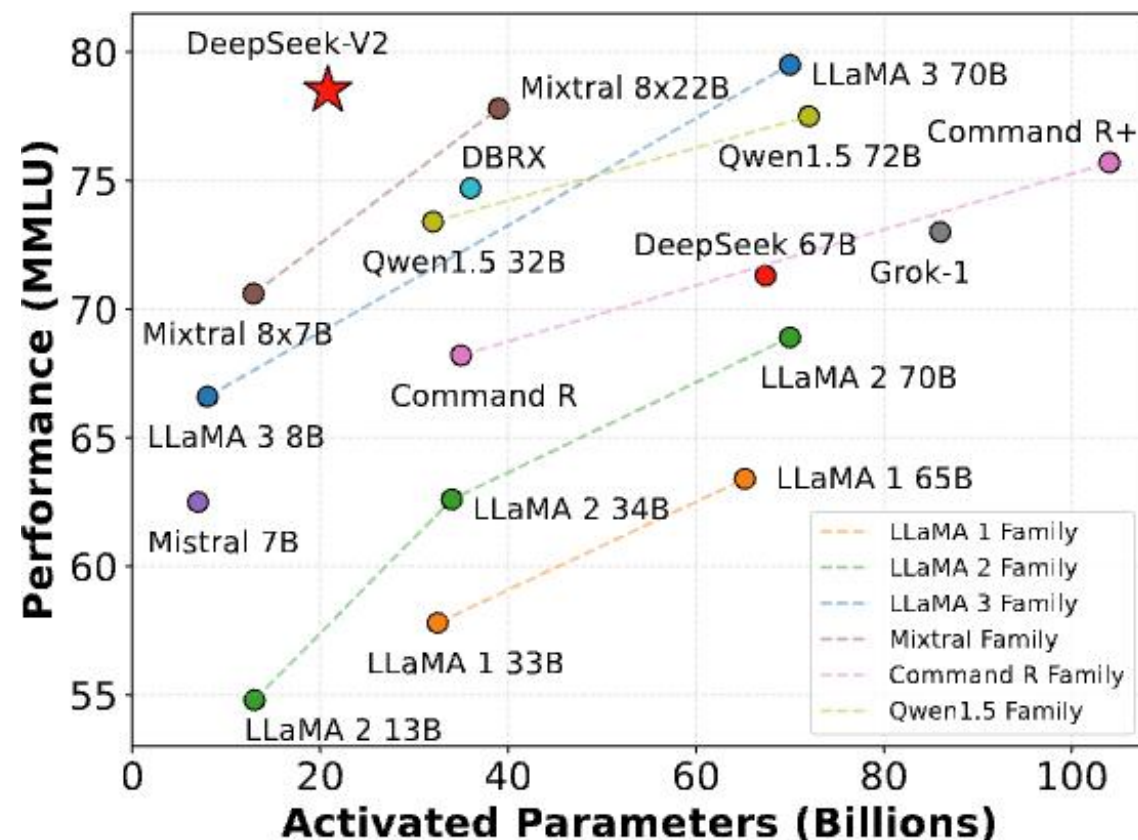


Figure 4: **MoE vs. Dense.** We train a 1.3B parameter dense model and a 1.3B active, 6.9B total parameter MoE model, each on 128 H100 GPUs. Apart from MoE-related changes, we train both

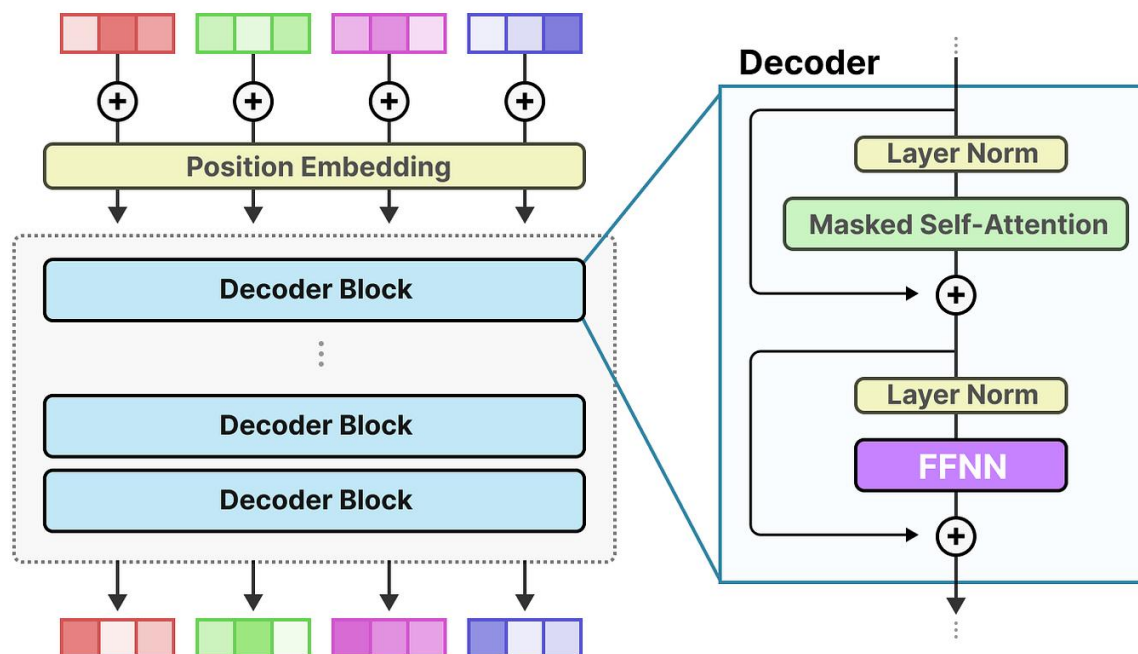
Why are MoEs getting popular?

- Highly competitive vs dense equivalents



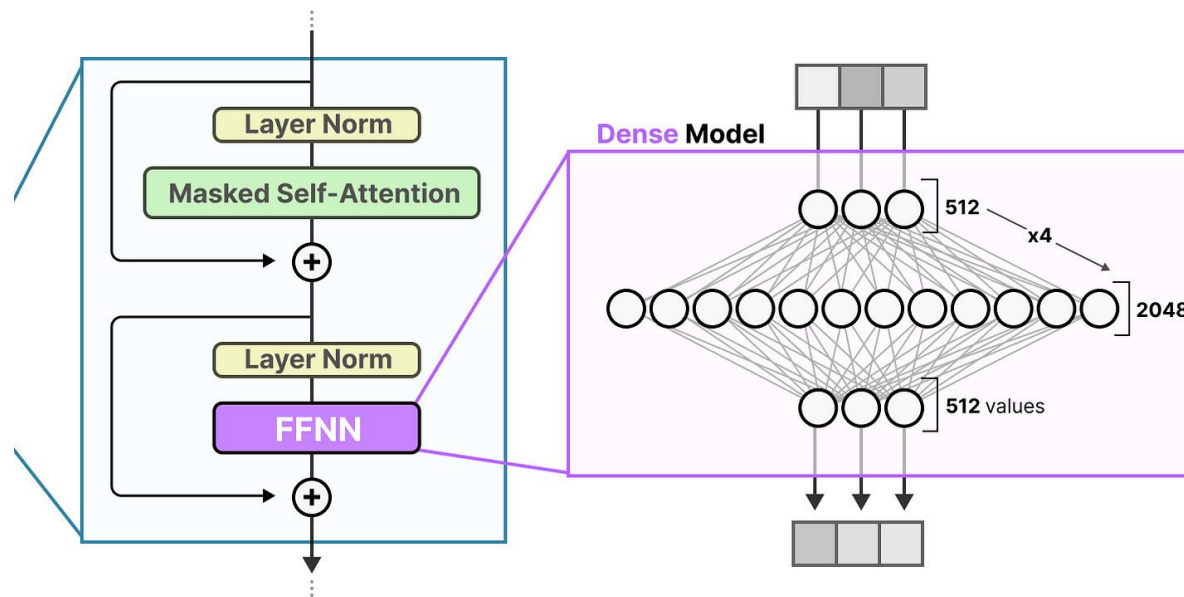
The Experts: Dense Layers

- Mixture of Experts (MoE) all start from a relatively basic functionality of LLMs, namely the Feedforward Neural Network (FFNN).



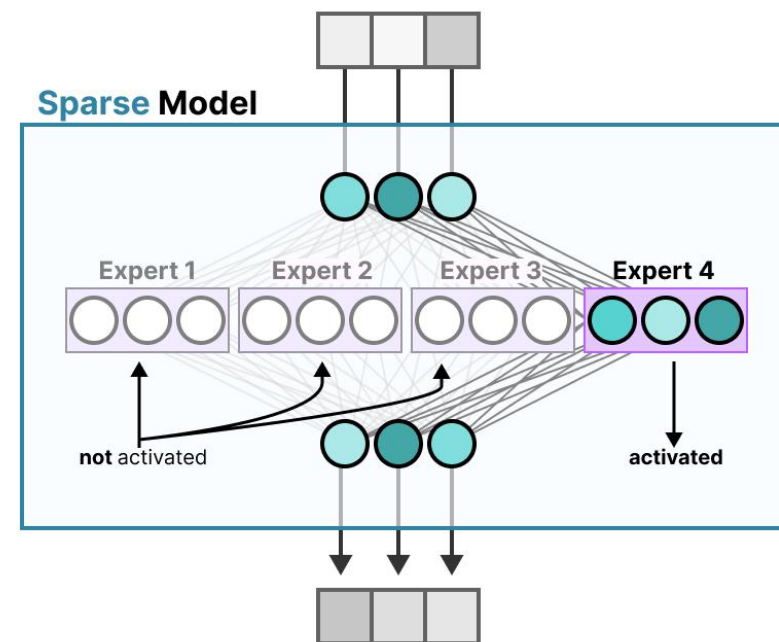
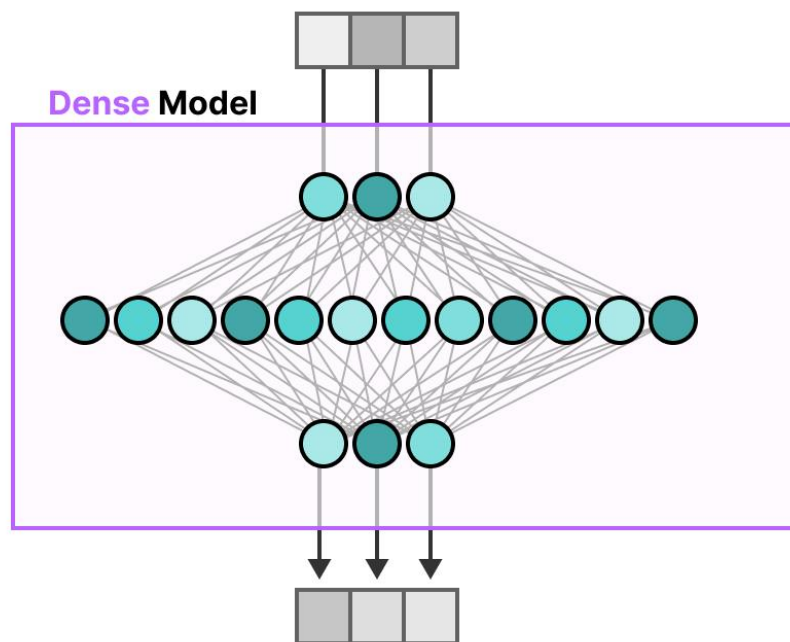
The Experts: Dense Layers

- An FFNN allows the model to use the contextual information created by the attention mechanism, transforming it further to **capture more complex relationships** in the data.



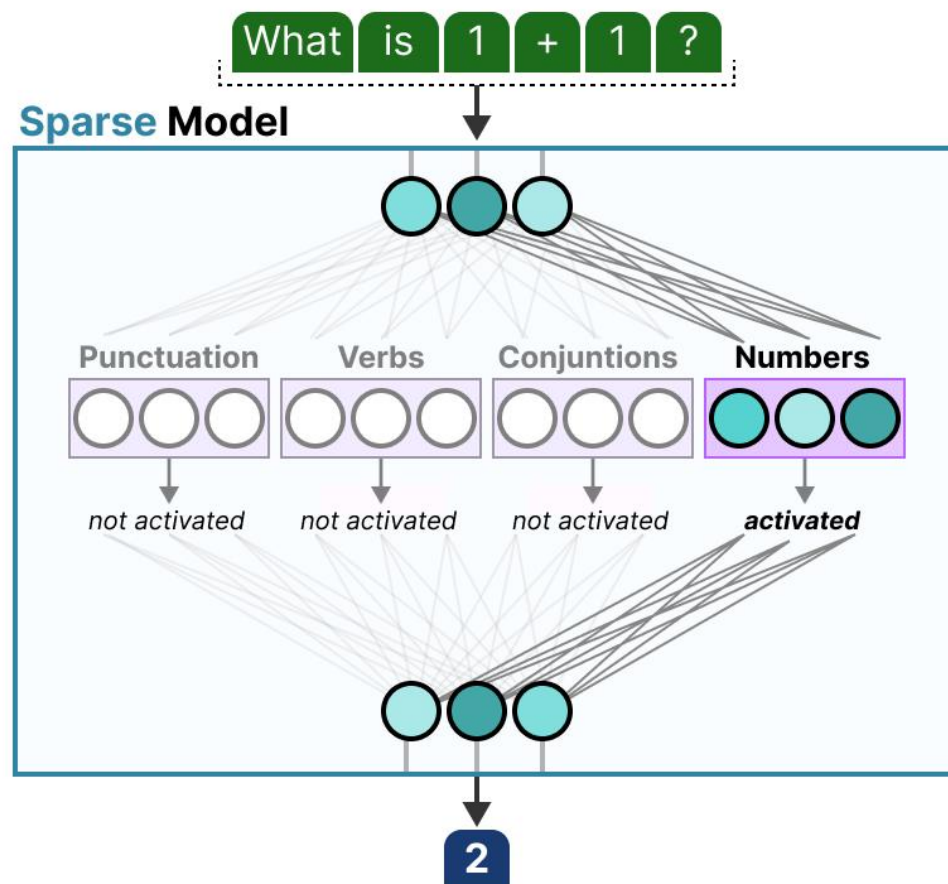
The Experts: Sparse Layers

- In contrast, sparse models only **activate a portion of** their total parameters and are closely related to Mixture of Experts.



The Experts: Sparse Layers

- The underlying idea is that each expert learns different information during training.
- When asked a question, we can select the expert best suited for a given task:



The Experts: What does an Expert Learn?

- As we have seen before, experts learn more fine-grained information than entire domains. As such, calling them “experts” has sometimes been seen as misleading.

Expert specialization	Expert position	Routed tokens
Punctuation	Layer 2	, , , , , , , , - , , , , , ,) .)
	Layer 6	, , , , , : : : , & , & & ? & - , ? , , , .
Conjunctions and articles	Layer 3	The the the the the the the the The...
	Layer 6	a and and and and and and and or and ...
Verbs	Layer 1	died falling identified fell closed left posted lost felt left said read miss place struggling falling signed died...
Visual descriptions <i>color, spatial position</i>	Layer 0	her over her know dark upper dark outer center upper blue inner yellow raw mama bright bright over open your dark blue
Counting and numbers <i>written and numerical forms</i>	Layer 1	after 37 19. 6. 27 Seven 25 4, 54 I two dead we Some 2012 who we few lower

The Experts: What does an Expert Learn?

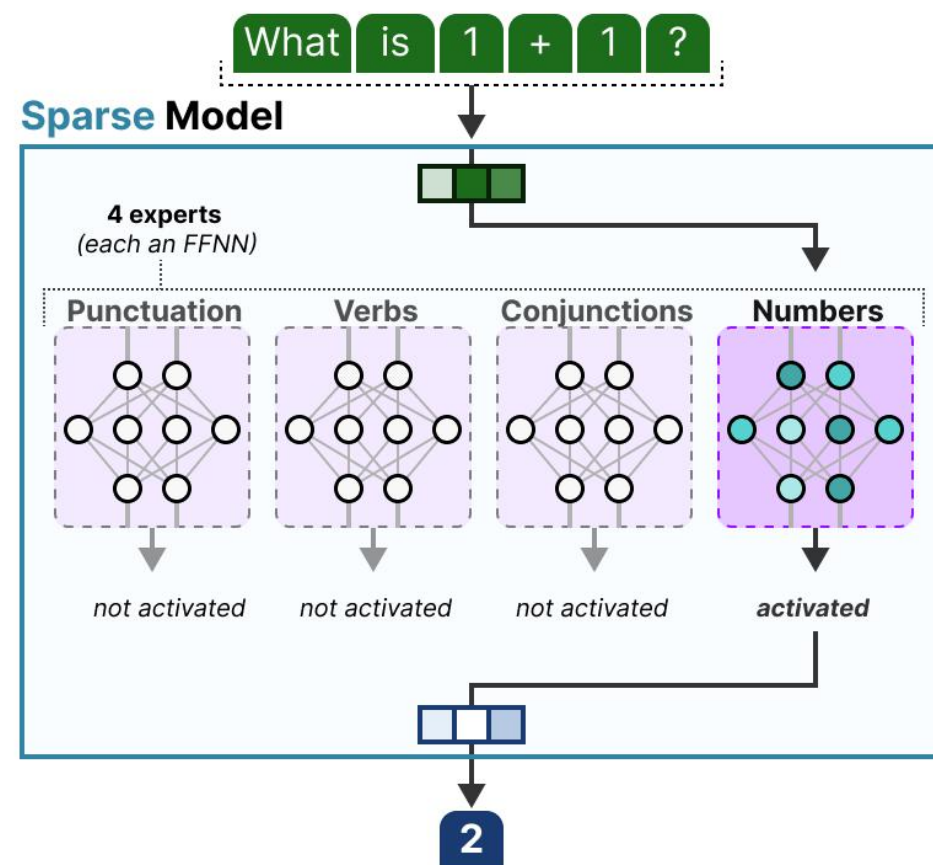
- Experts in decoder models, however, do not seem to have the same type of specialization. That does not mean though that all experts are equal.

```
class MoeLayer(nn.Module):  
    def __init__(self, experts: List[nn.Module],  
                 super().__init__()  
    assert len(experts) > 0  
    self.experts = nn.ModuleList(experts)  
    self.gate = gate  
    self.args = moe_args
```

An example in the Mixtral 8x7B paper, where each token is colored with the first expert choice.

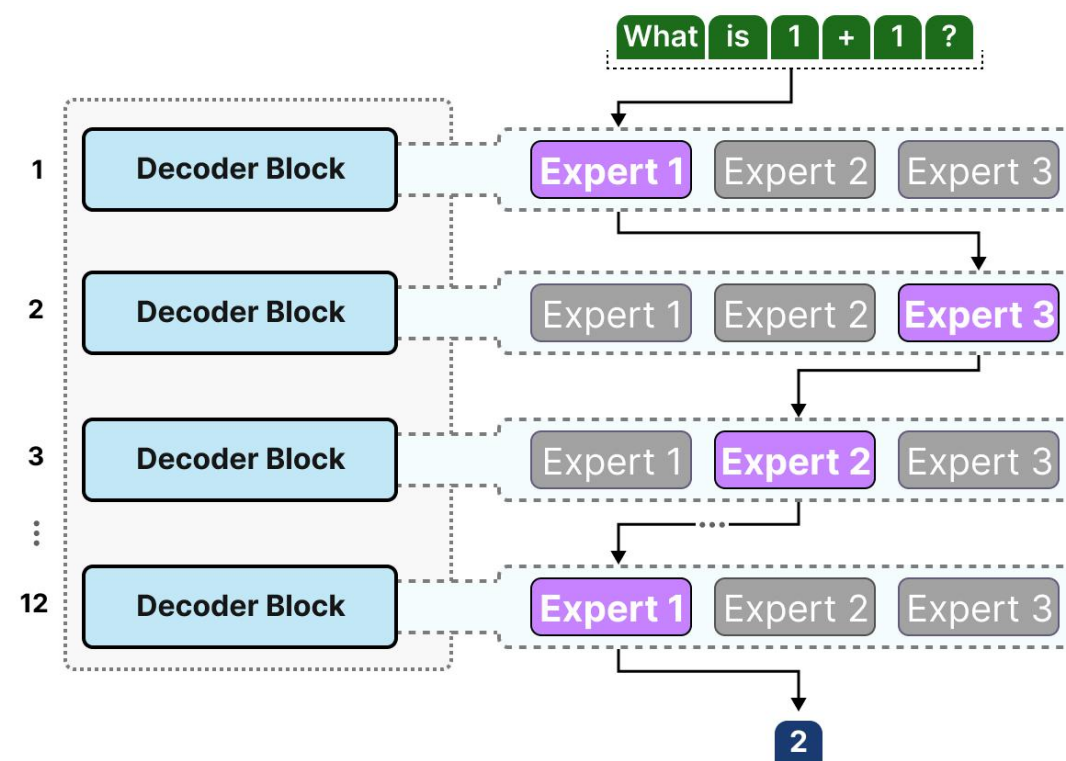
The Experts: The Architecture of Experts

- Although it's nice to visualize experts as a hidden layer of a dense model cut in pieces, they are often **whole FFNNs** themselves:



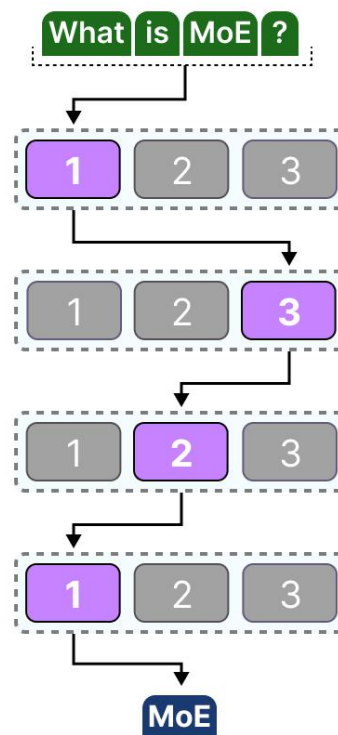
The Experts: The Architecture of Experts

- Since most LLMs have several decoder blocks, a given text will pass through multiple experts before the text is generated:

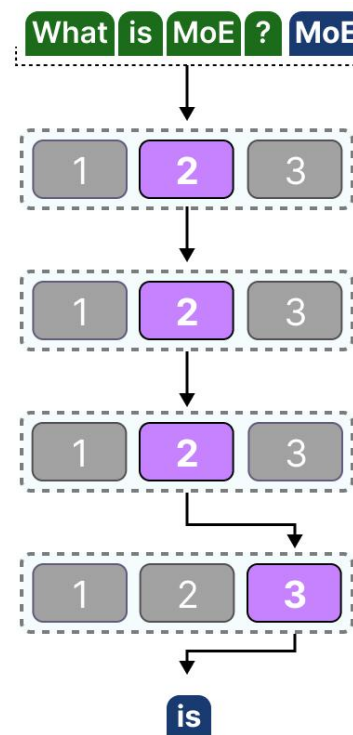


The Experts: The Architecture of Experts

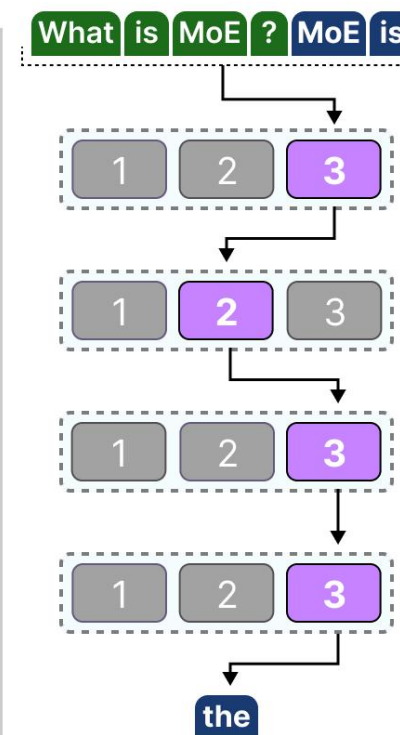
- The chosen experts likely differ between tokens which results in **different “paths”** being taken:



First pass



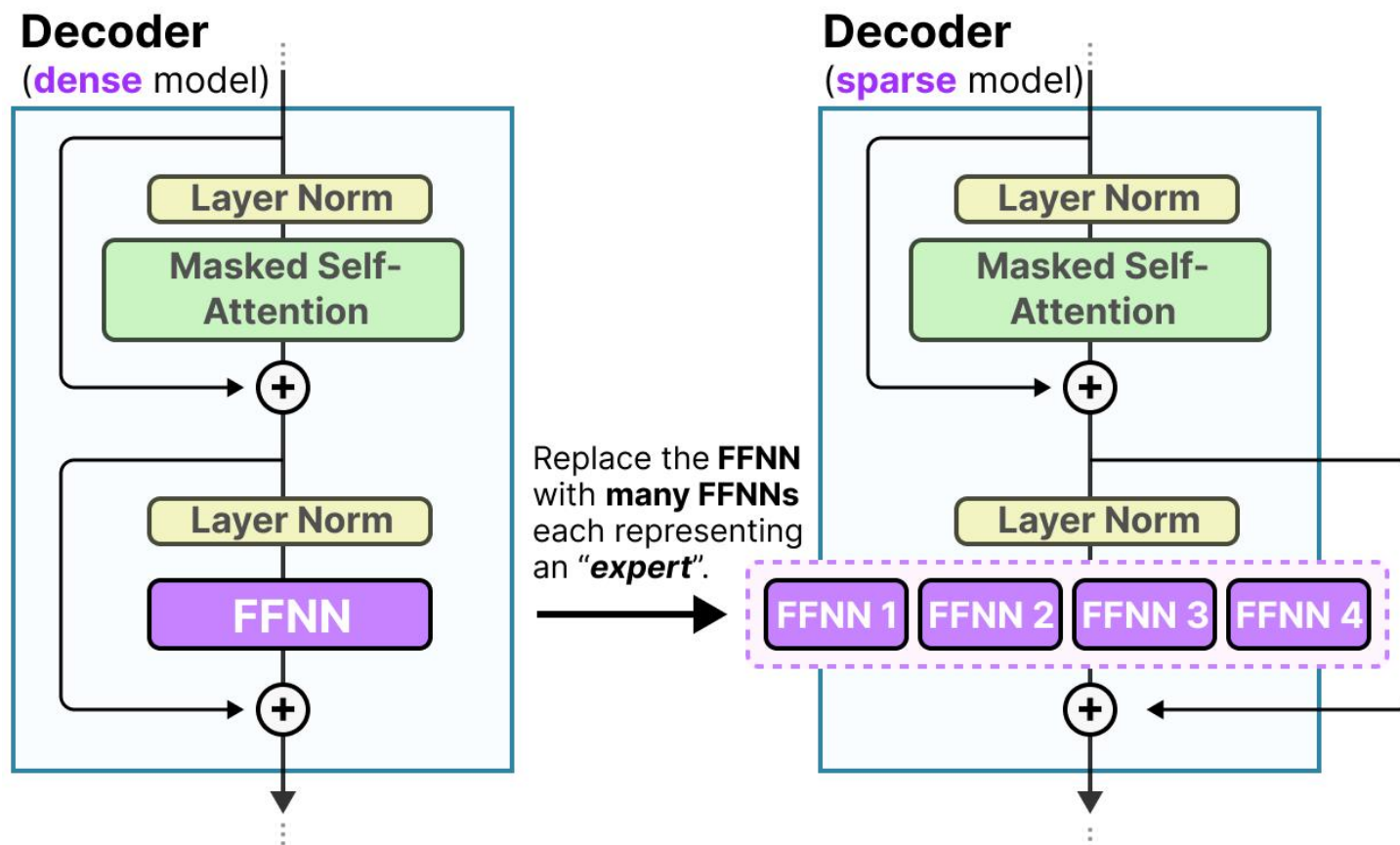
Second pass



Third pass

The Experts: The Architecture of Experts

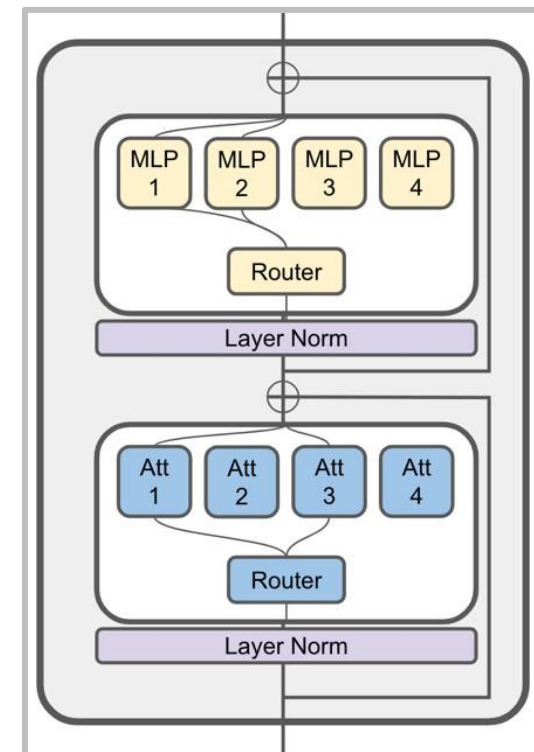
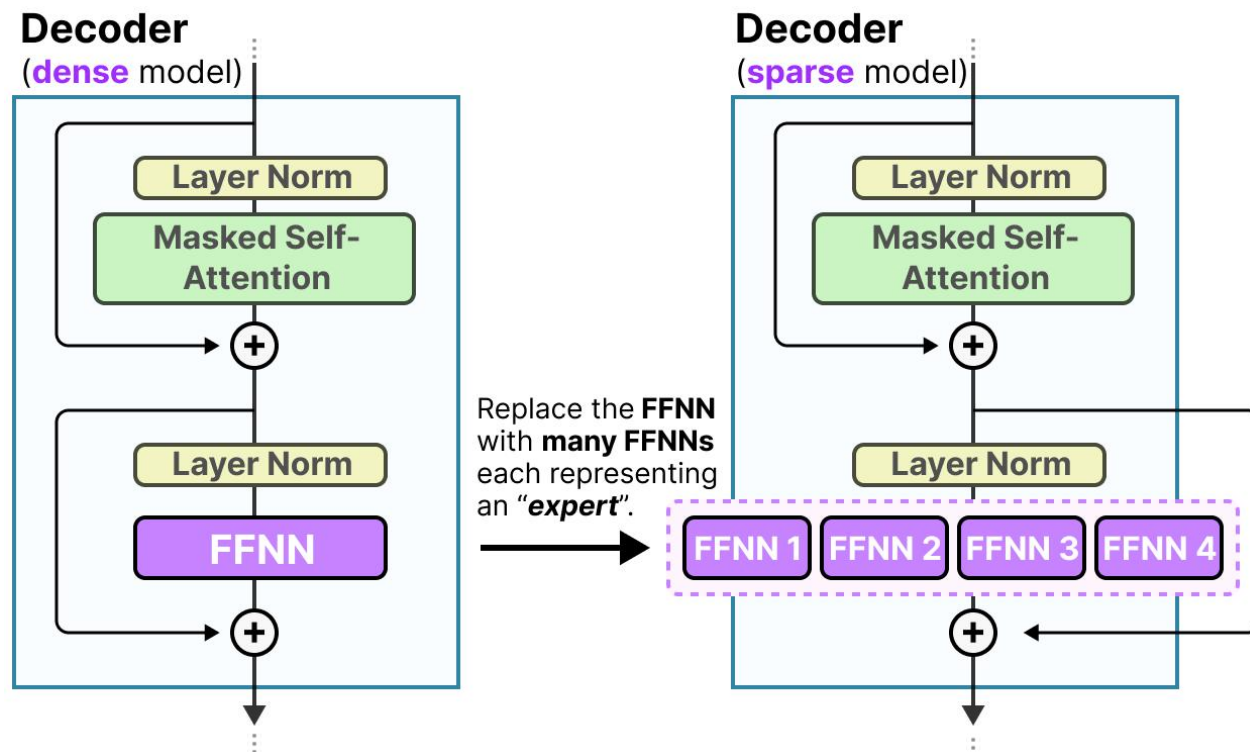
- If we update our visualization of the decoder block, it would now contain more FFNNs (one for each expert) instead:



The Experts: The Architecture of Experts

Typical: replace **MLP** with MoE layer

Less common: MoE for **attention** heads

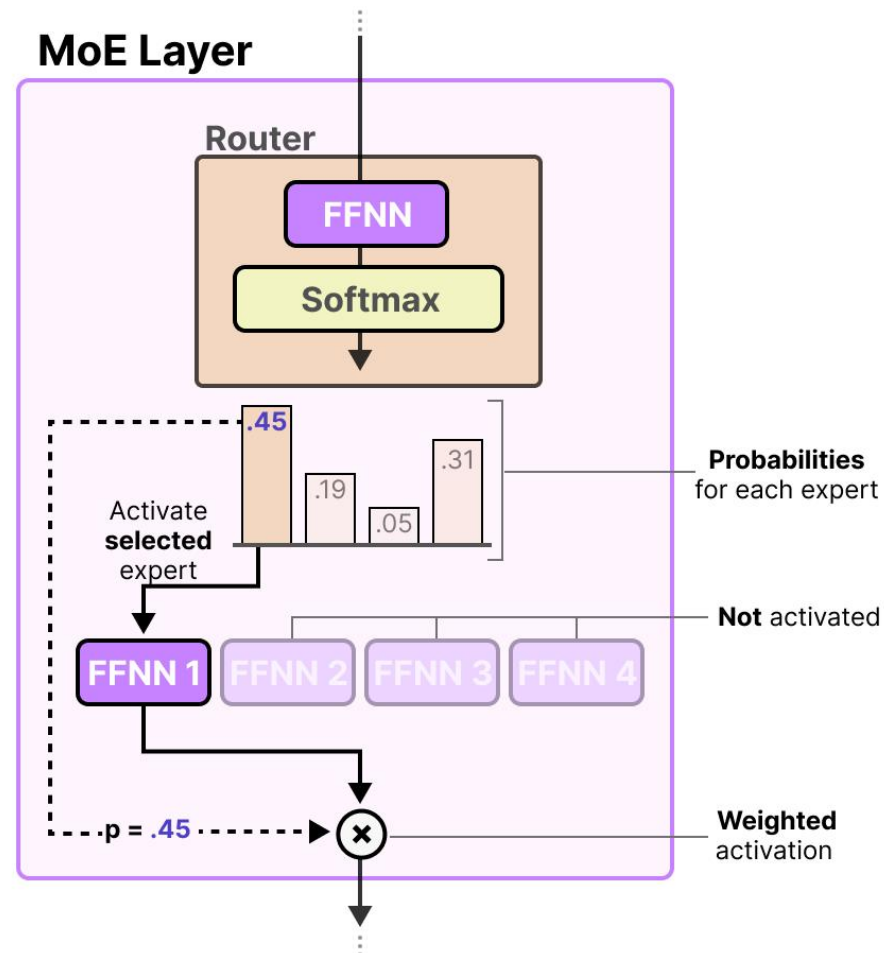


The Routing Mechanism

- Now that we have a set of experts, how does the model know which experts to use?
- Just before the experts, a **router** (also called a **gate network**) is added which is trained to choose which expert to choose for a given token.

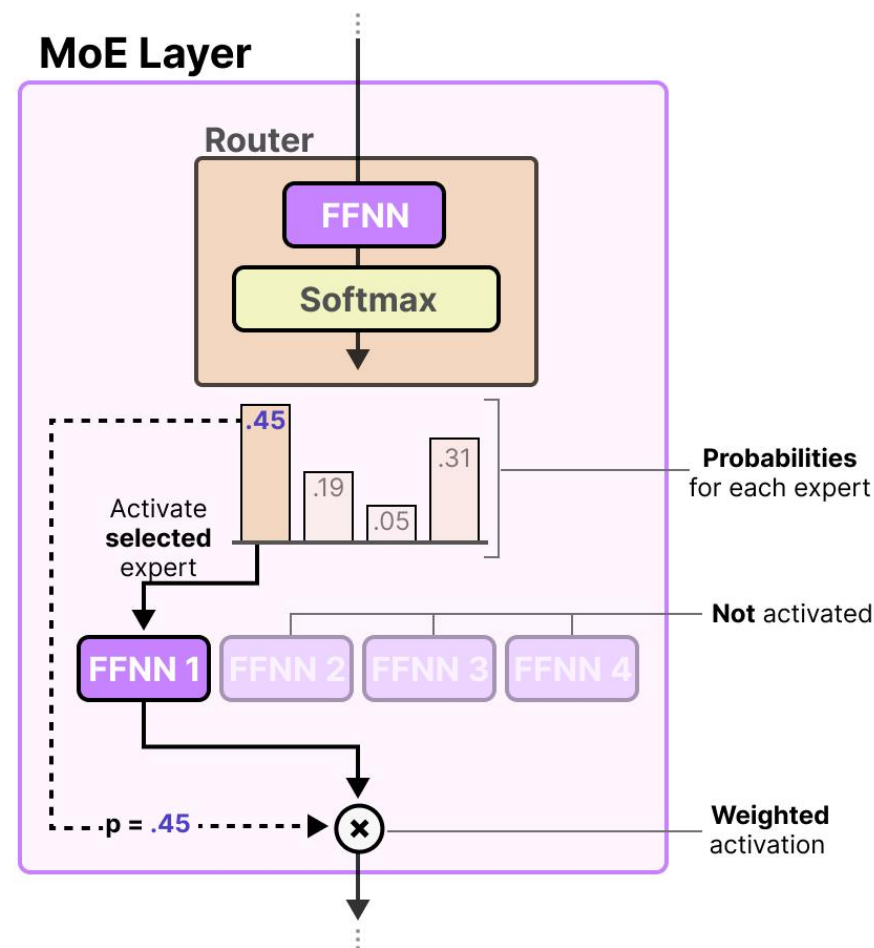
The Routing Mechanism: The Router

- **Step1:** The router (or gate network) is also an FFNN and is used to choose the expert based on a particular input. It outputs **probabilities** which it uses to select the best matching expert:



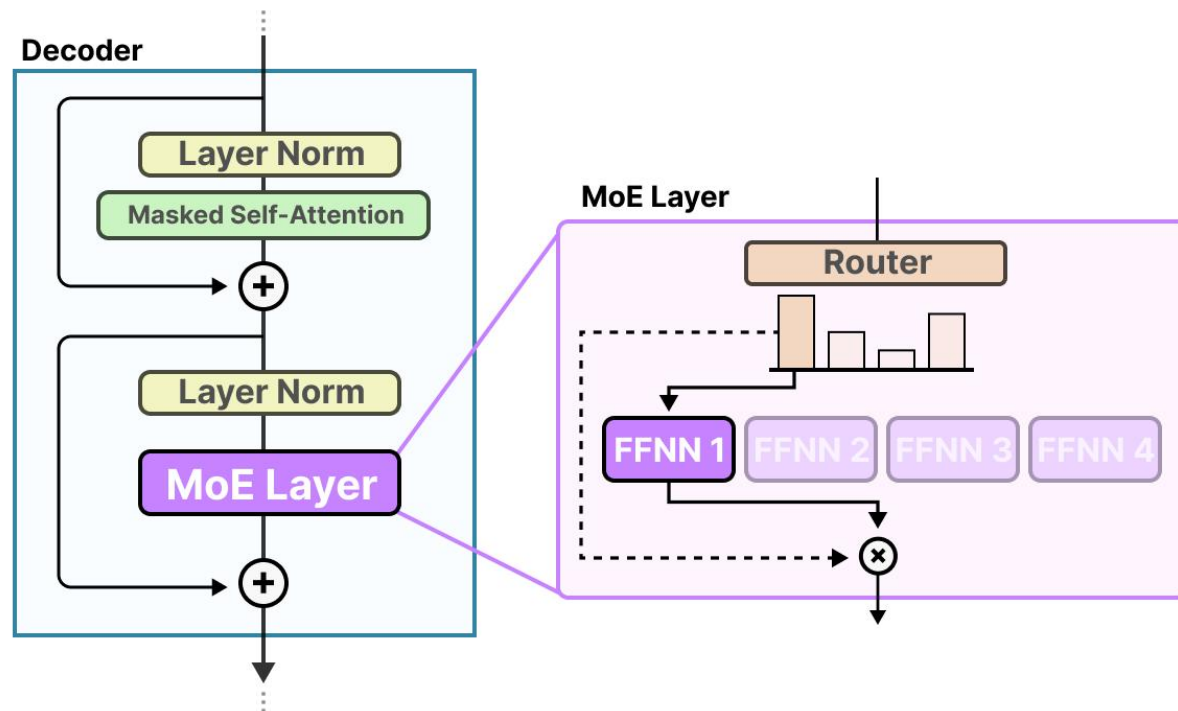
The Routing Mechanism: The Router

- **Step2:** The expert layer returns the output of the selected **expert multiplied by the gate value** (selection probabilities).



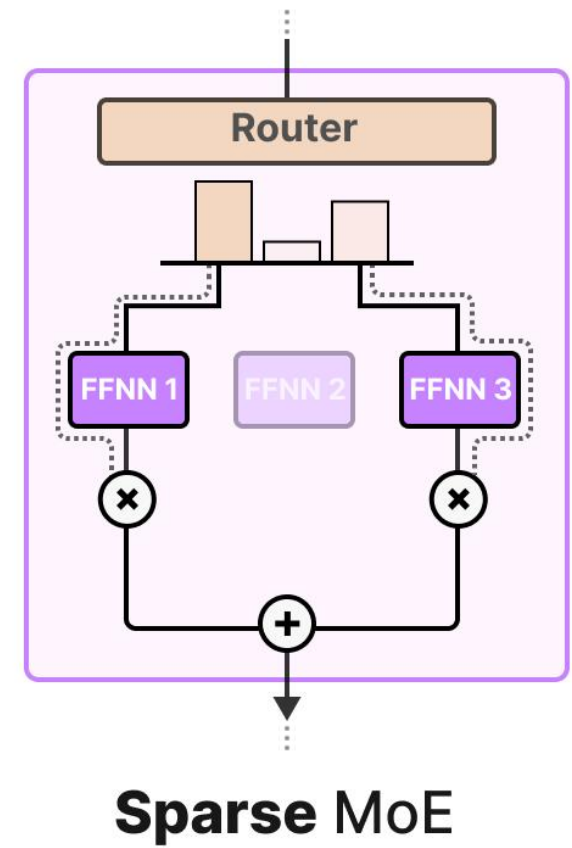
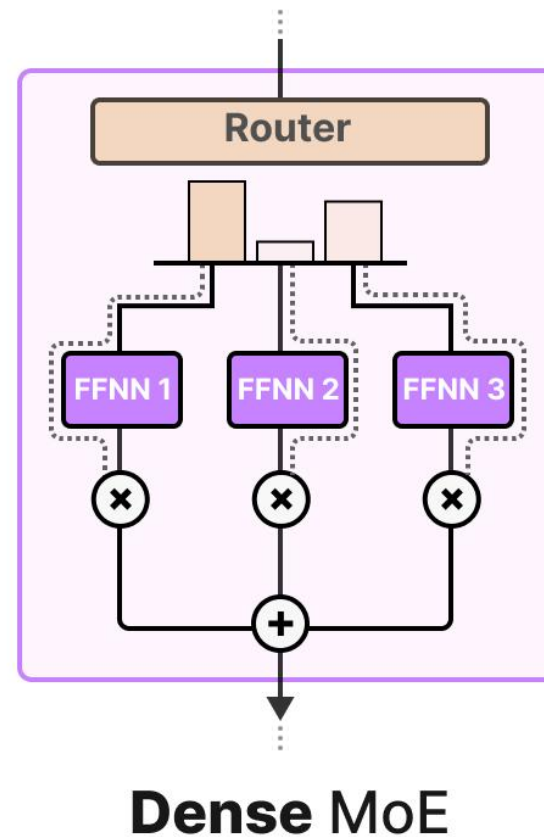
The Routing Mechanism: The Router

- The router together with the experts (of which only **a few are selected**) makes up the **MoE Layer**:



The Routing Mechanism: The Router

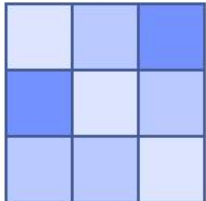
- A given MoE layer comes in two sizes, either a **sparse** or a **dense** mixture of experts.
- When you see a “MoE” it will typically be a Sparse MoE as it is **computationally cheaper**



The Routing Mechanism: Selection of Experts

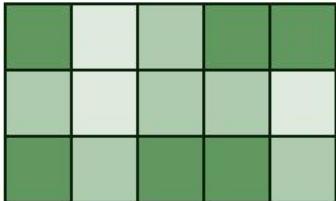
- The gating network is arguably the most important component of any MoE as it not only decides which experts to choose during inference but also training.
- In its most basic form, we multiply the input ($\mathbf{x}$) by the router weight ($\mathbf{W}$):

output



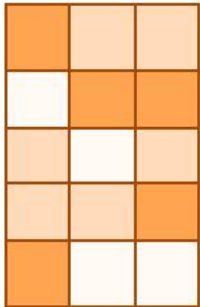
$\mathbf{H}(\mathbf{x})$

input



$\mathbf{x}$

router weights



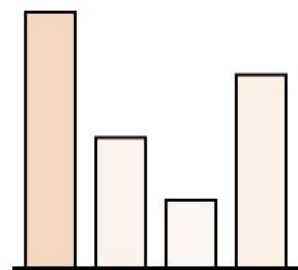
$\mathbf{W}$

$=$ $*$

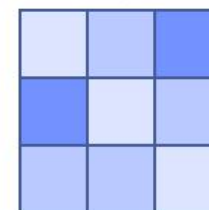
The Routing Mechanism: Selection of Experts

- Then, we apply a **SoftMax** on the output to create a probability distribution **G(x)** per expert:

*probability
distribution per expert*



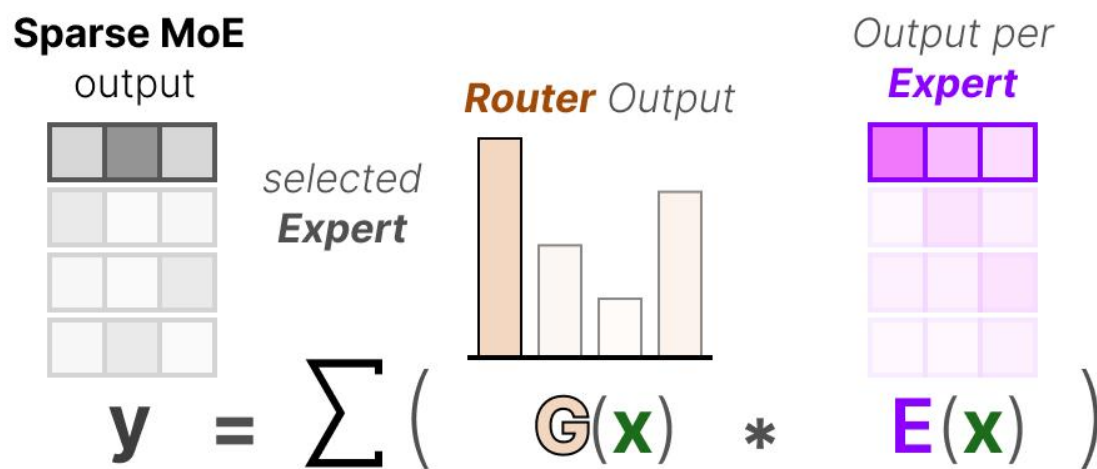
output



$$\mathbf{G}(\mathbf{x}) = \text{Softmax}(\mathbf{H}(\mathbf{x}))$$

The Routing Mechanism: Selection of Experts

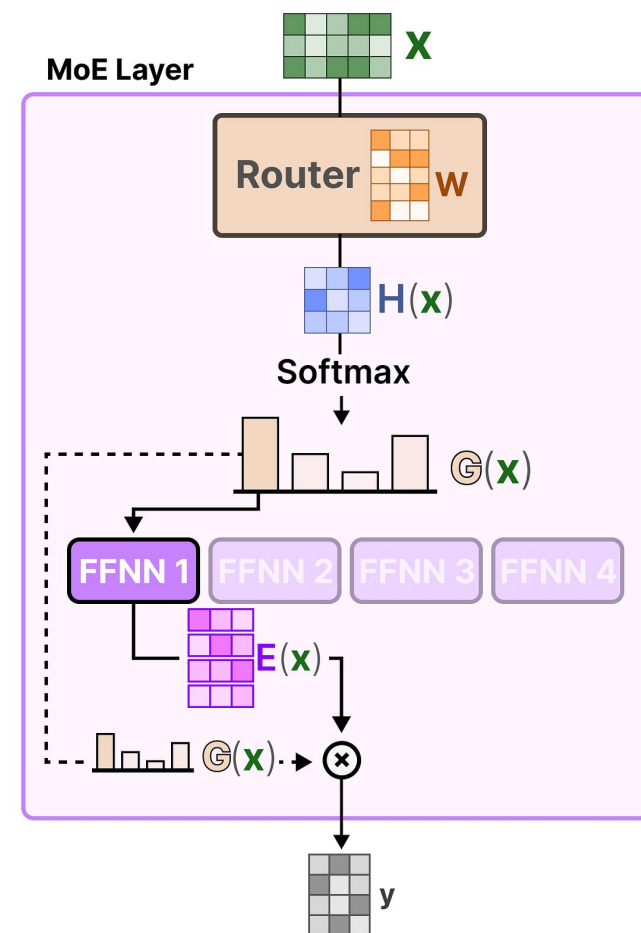
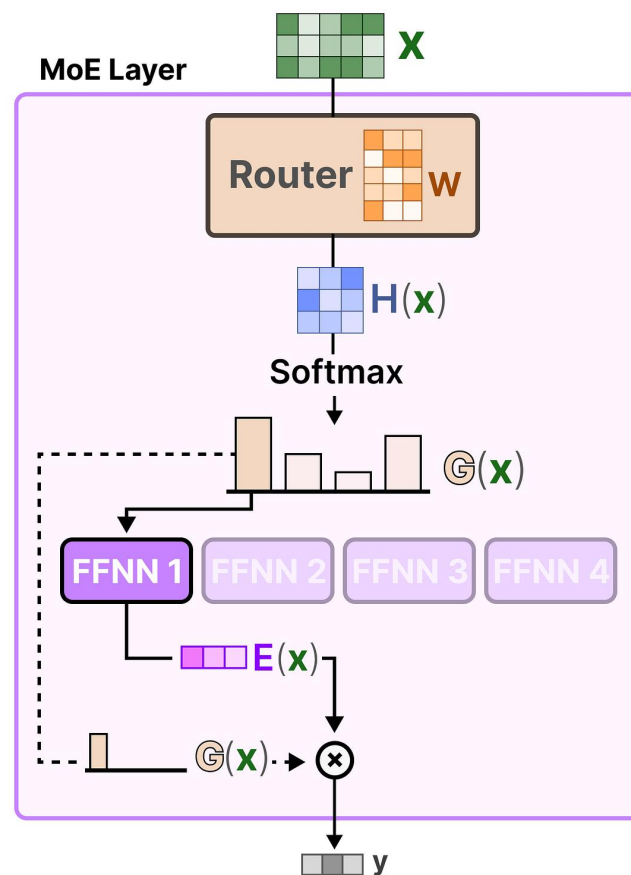
- The router uses probability distribution to choose the best matching expert.
- Finally, we multiply the output of each router with each selected expert and sum the results.



(only **1** expert is chosen in this example)

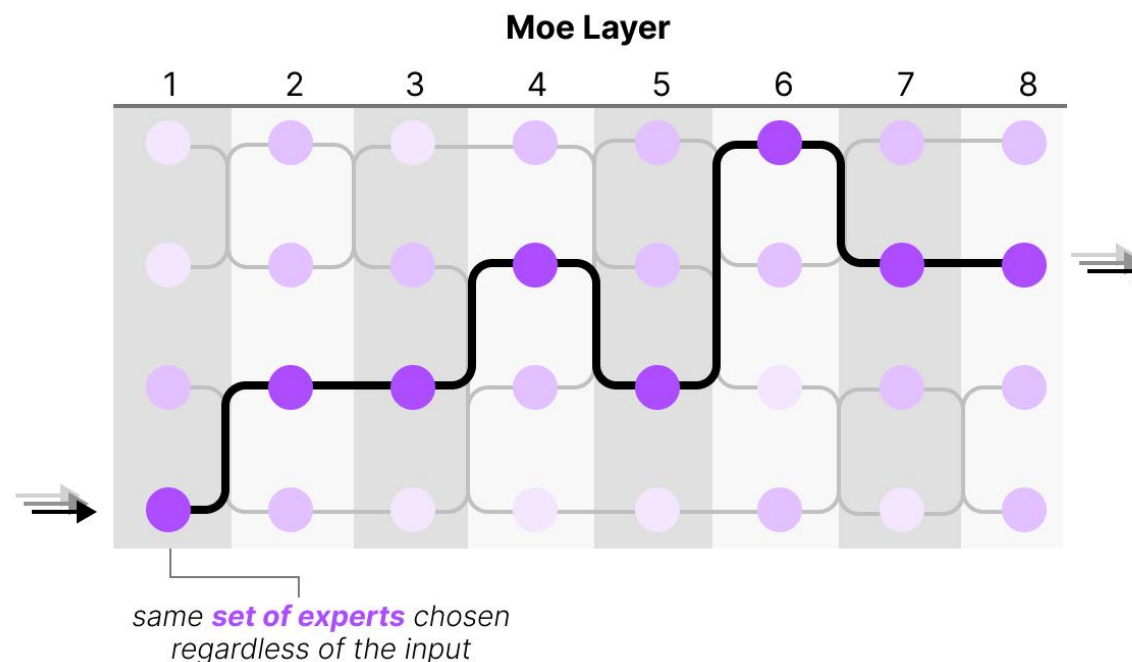
The Routing Mechanism: Selection of Experts

- Let's put everything together and explore how the input flows through the router and experts:



The Routing Mechanism: The Complexity of Routing

- However, this simple function often results in the router choosing the same expert since certain experts might learn faster than others:



The Routing Mechanism: The Complexity of Routing

- Not only will there be an uneven distribution of experts chosen, but **some experts will hardly be trained at all**. This results in issues during both training and inference.
- Instead, we want equal importance among experts during training and inference, which we call **load balancing**. In a way, it's to prevent overfitting on the same experts.

Load Balancing: KeepTopK

- One method of load balancing the router is through a straightforward extension called **KeepTopK**. By introducing trainable (gaussian) noise, we can prevent the same experts from always being picked:

(somewhat noisy)
output

input

router weights

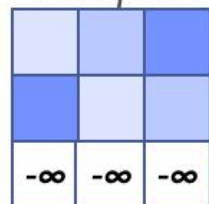
(small amount of)
gaussian
noise

$$\mathbf{H}(\mathbf{x}) = \mathbf{x} * \mathbf{W} + \mathbf{n}$$

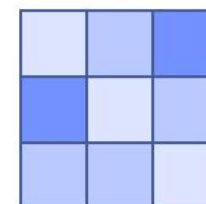
Load Balancing: KeepTopK

- Then, all but the top k experts that you want activating (for example 2) will have their weights set to $-\infty$:

(updated)
output



(somewhat noisy)
output

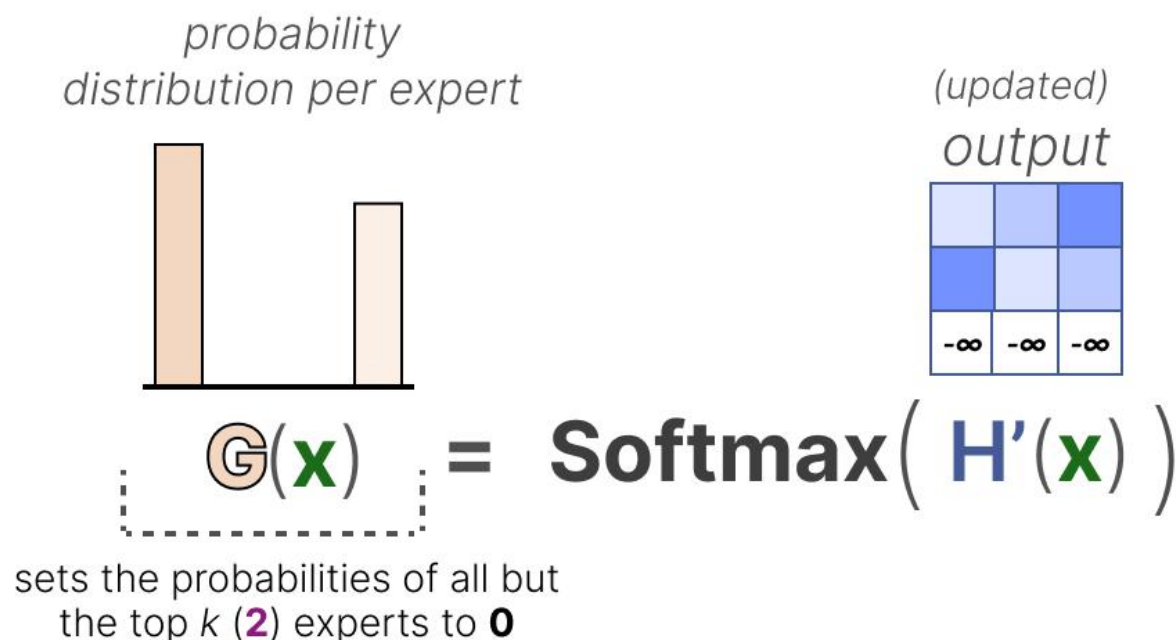


$$H'(\mathbf{x}) = \text{KeepTopK} \left(H(\mathbf{x}), 2 \right)$$

sets the weights of all but
the top k (2) experts to $-\infty$

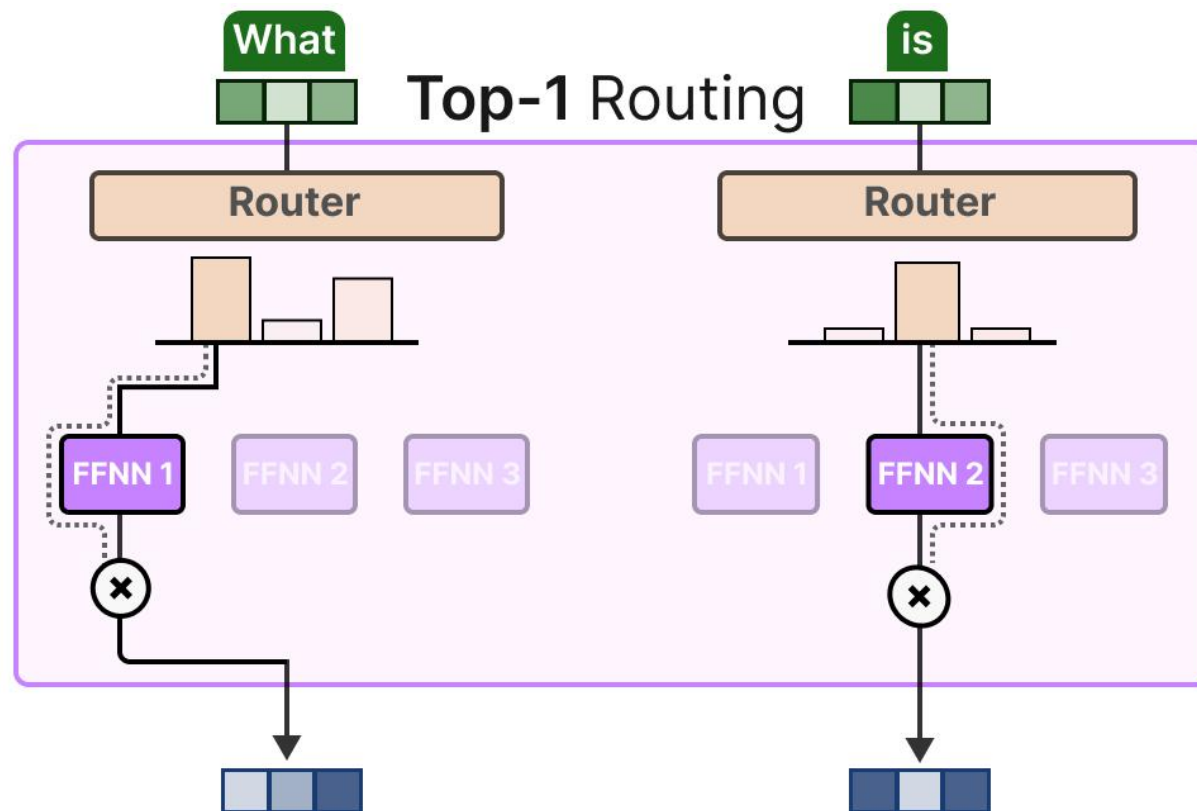
Load Balancing: KeepTopK

- By setting these weights to $-\infty$, the output of the SoftMax on these weights will result in a probability of 0:



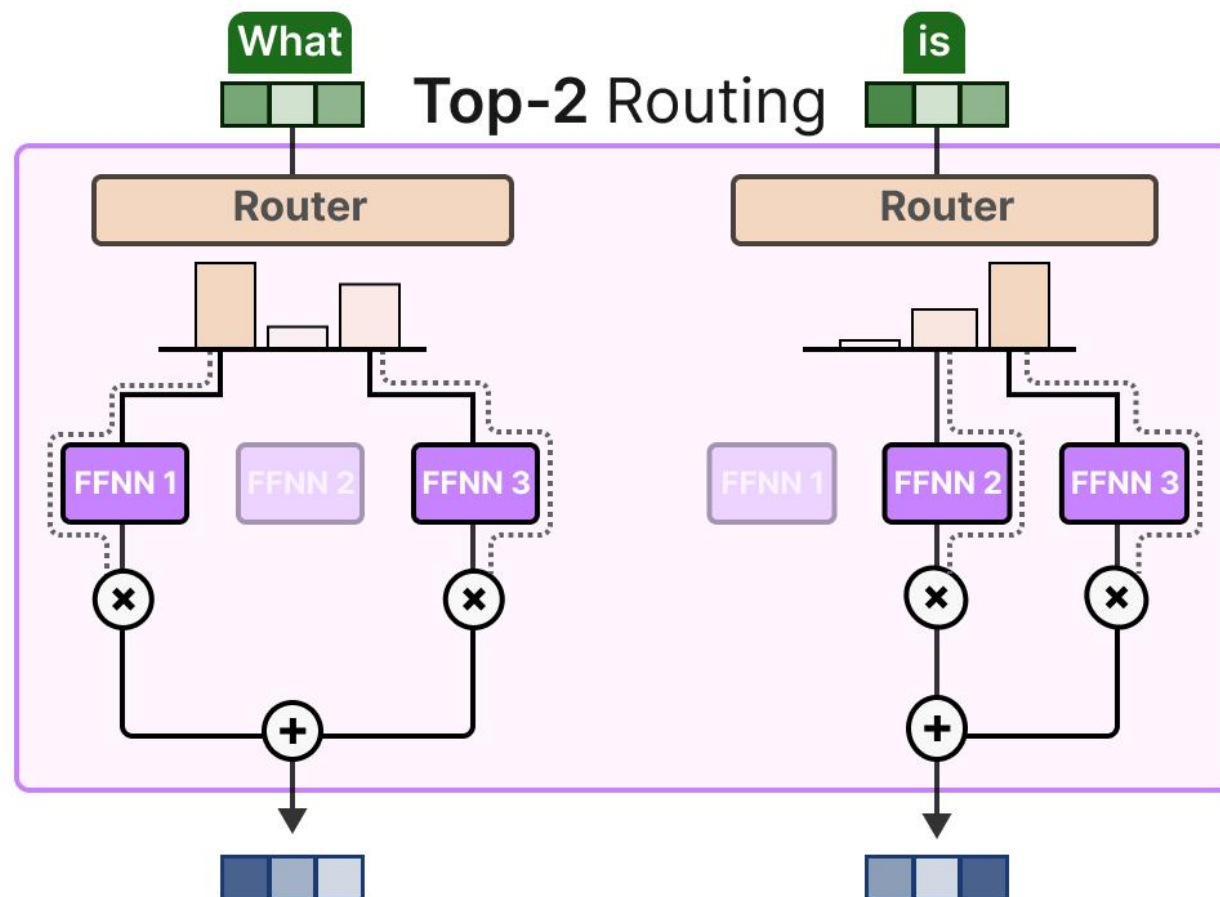
KeepTopK: Token Choice

- The KeepTopK strategy routes each token to a few selected experts. This method is called **Token Choice** and allows for a given token to be sent to one expert (top-1 routing):



KeepTopK: Token Choice

- Or to more than one expert (top-k routing):
- A major benefit is that it allows the experts' respective contributions to be weighed and integrated.



Other routing type

- Many of the routing algorithms boil down to ‘choose top k’

Experts

	T1	T2	T3
E1	3.13	0.14	0.74
E2	0.51	-0.25	1.58
E3	-1.32	1.97	0.1
E4	2.25	2.61	0.02
E5	-2.81	-0.68	-0.41

Token chooses expert

Experts

	T1	T2	T3
E1	Choose Top-K		
E2	0.51	-0.25	1.58
E3	-1.32	1.97	0.1
E4	2.25	2.61	0.02
E5	-2.81	-0.68	-0.41

Expert chooses token

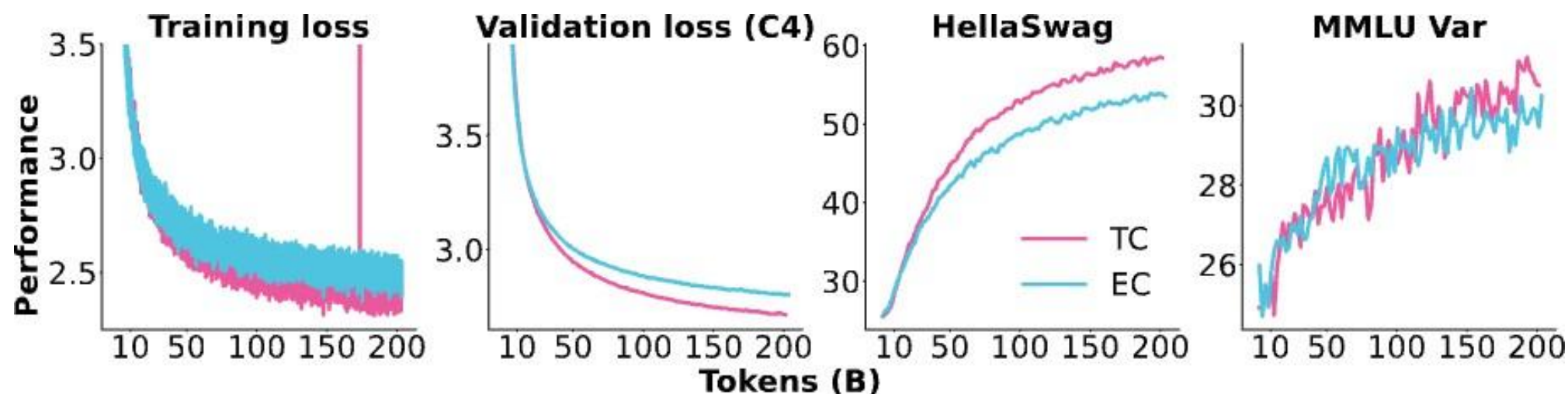
Experts

	T1	T2	T3
E1	3.13	0.14	0.74
E2	0.51	-0.25	1.58
E3	-1.32	1.97	0.1
E4	2.25	2.61	0.02
E5	-2.81	-0.68	-0.41

Global routing via optimization

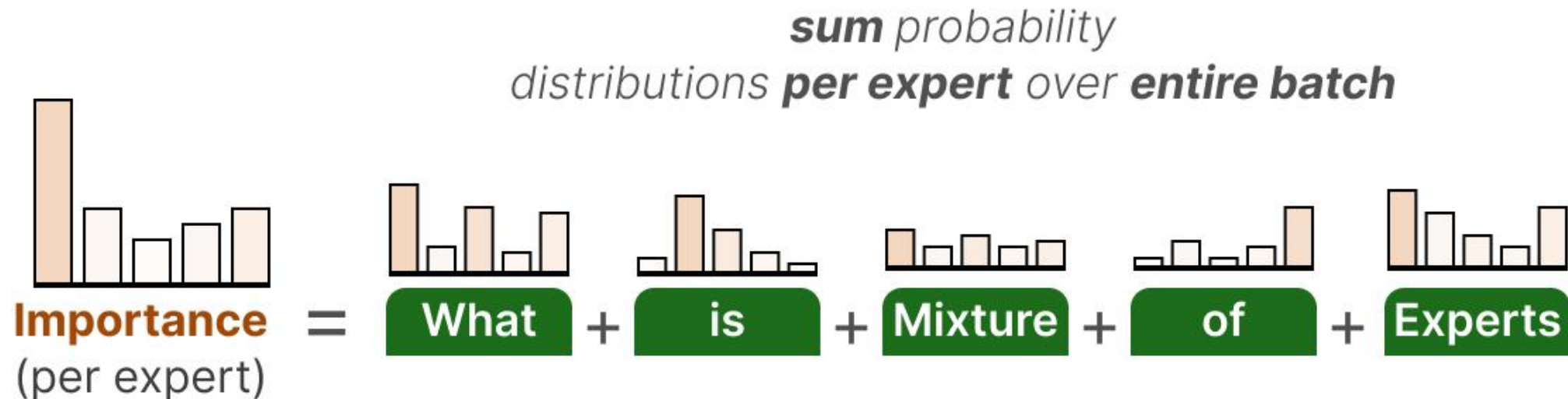
Other routing type

- Almost all the MoEs do a standard ‘token choice topk’ routing.



KeepTopK: Auxiliary Loss

- To get a more even distribution of experts during training, the **auxiliary loss** (also called **load balancing loss**) was added to the network's regular loss.
- It adds a constraint that forces experts to have **equal importance**.



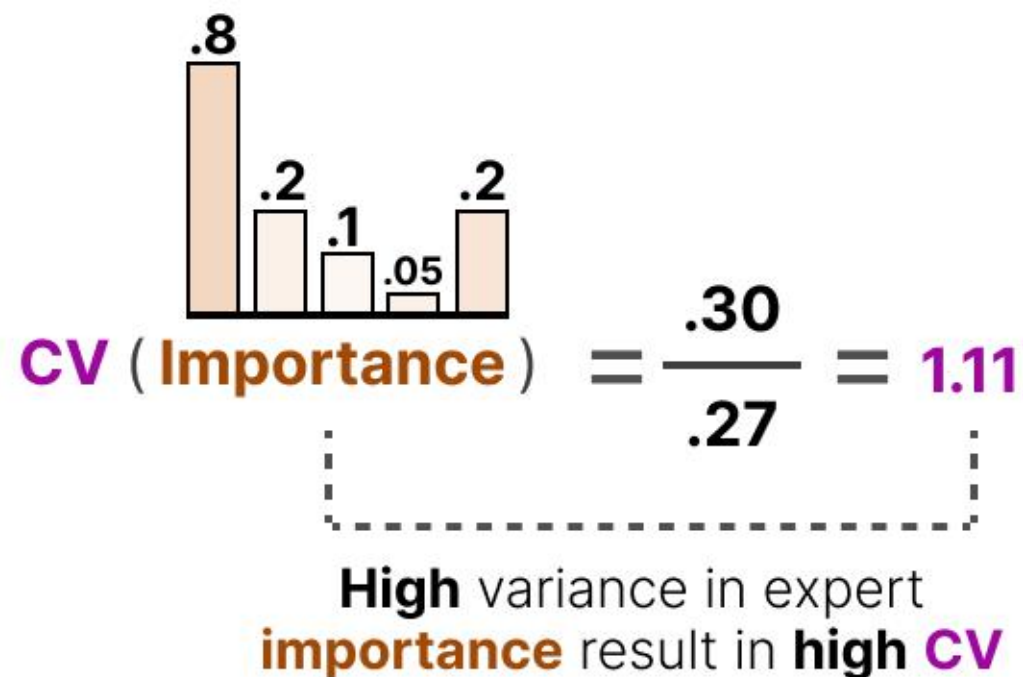
KeepTopK: Auxiliary Loss

- This gives us the importance scores per expert which represents how likely a given expert will be chosen regardless of the input.
- We can use this to calculate the **coefficient variation (CV)**, which tells us how different the importance scores are between experts.

$$\text{Coefficient Variation (CV)} = \frac{\text{standard deviation } (\sigma)}{\text{mean } (\mu)}$$

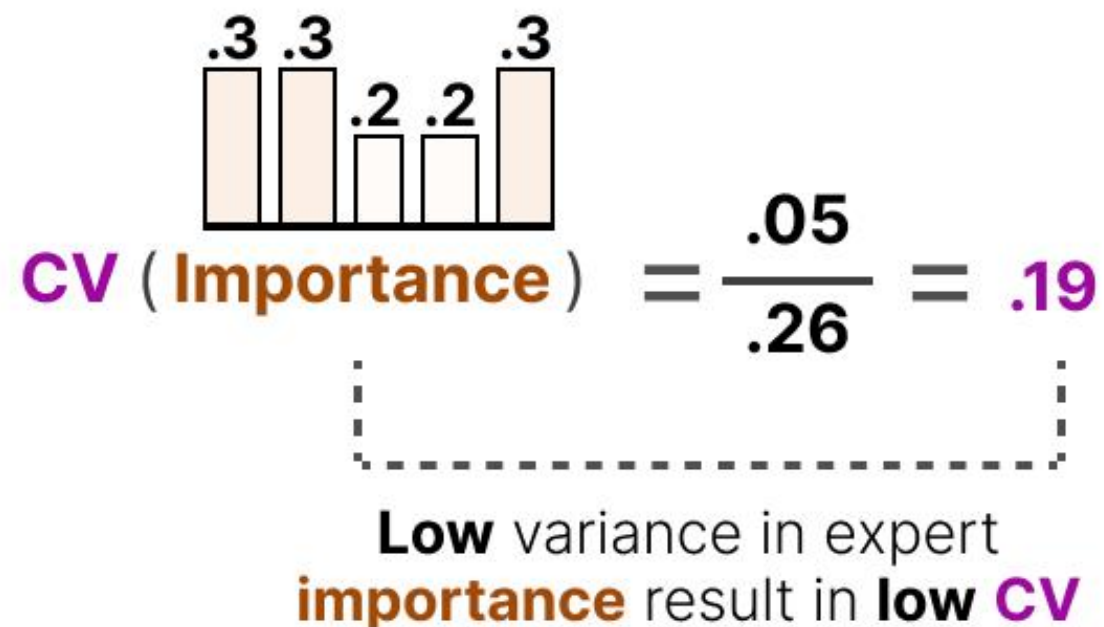
KeepTopK: Auxiliary Loss

- For instance, if there are a lot of differences in importance scores, the CV will be high:



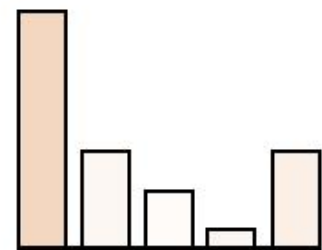
KeepTopK: Auxiliary Loss

- In contrast, if all experts have similar importance scores, the CV will be low (**which is what we aim for**):



KeepTopK: Auxiliary Loss

- Using this **CV** score, we can update the **auxiliary loss** during training such that it aims to lower the CV score as much as possible (*thereby giving equal importance to each expert*):

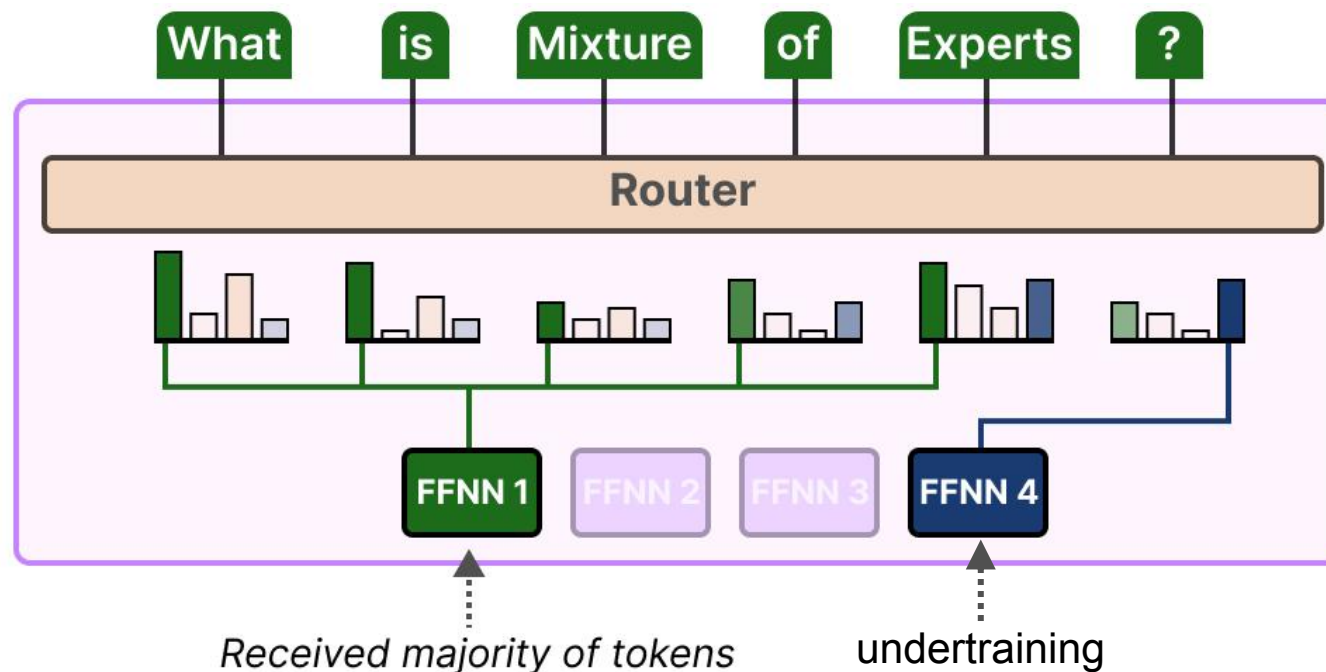
$$\text{Auxiliary Loss} = \overset{\substack{\text{(constant)} \\ \text{scaling factor}}}{W_{\text{importance}}} * \text{CV}(\text{Importance})^2$$


A high variance in expert importance (CV) results in high loss and vice versa

- Finally, the auxiliary loss is added as a separate loss to optimize during training.

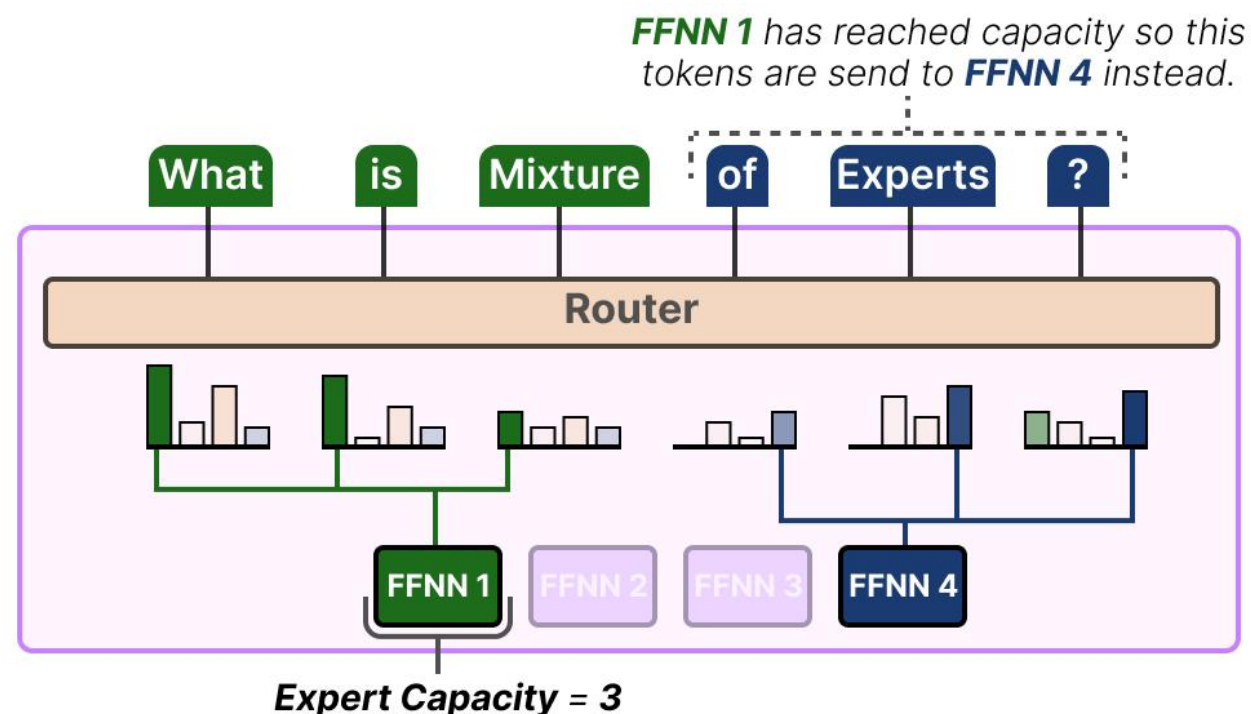
Load Balancing: Expert Capacity

- Imbalance is not just found in the experts that were chosen but also in the **distributions of tokens** that are sent to the expert.



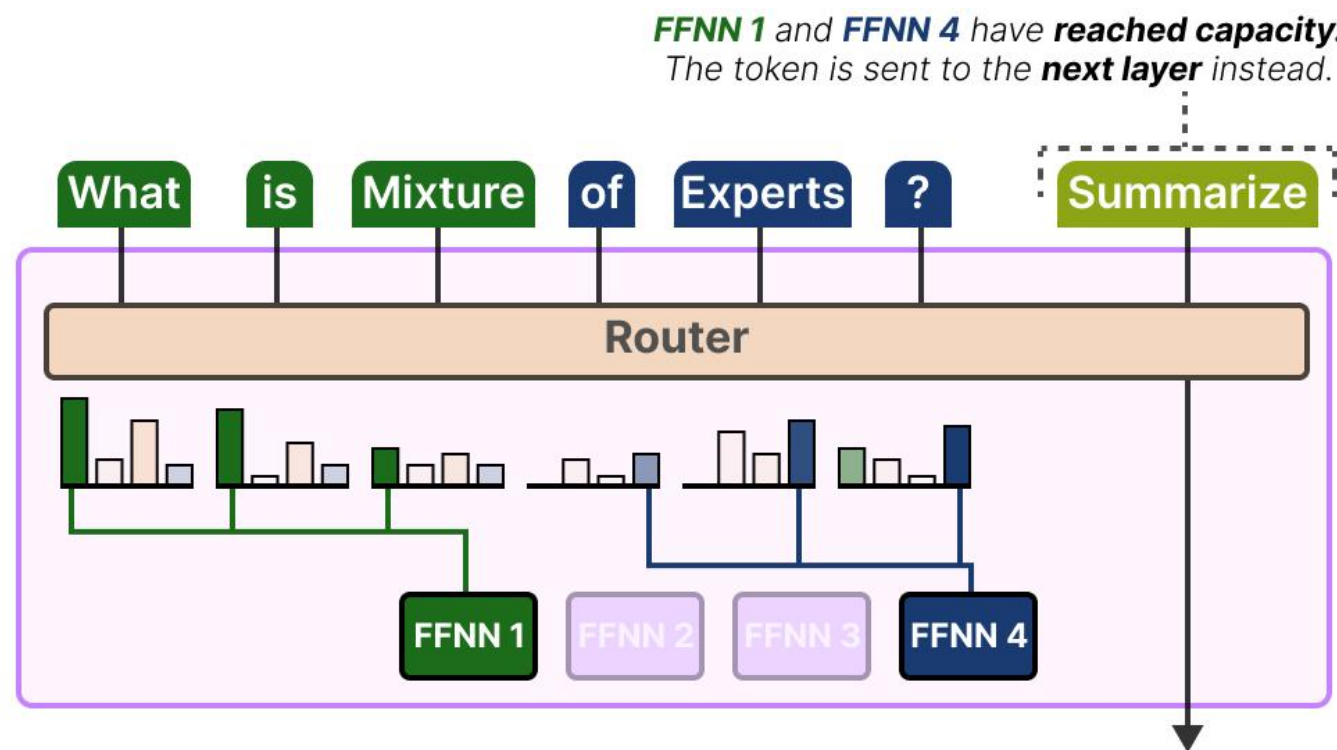
Load Balancing: Expert Capacity

- **Solution:** limit the amount of tokens a given expert can handle, namely **Expert Capacity**. By the time an expert has reached capacity, the resulting tokens will be sent to the next expert:



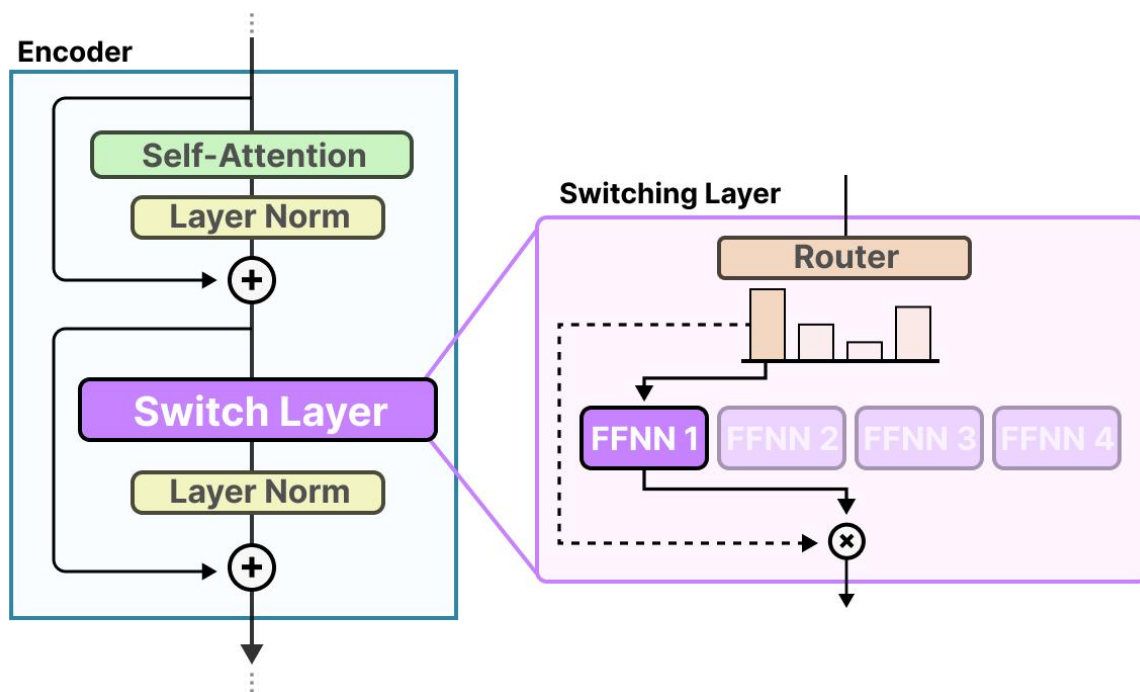
Load Balancing: Expert Capacity

- If both experts have reached their capacity, the token will not be processed by any expert but instead sent to the next layer. This is referred to as **token overflow**.

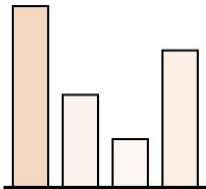


Load Balancing: Switch Transformer

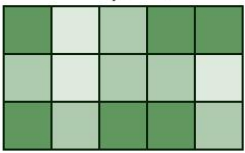
- The **Switching Layer** is a Sparse MoE layer that selects a single expert for each token (Top-1 routing).



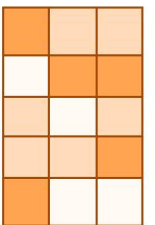
probability distribution per expert



input



router weights


$$\mathbf{G}(\mathbf{x}) = \text{Softmax}(\mathbf{x} * \mathbf{W})$$

Switch Transformer: Capacity Factor

- The capacity factor is an important value as it determines how many tokens an expert can process. The Switch Transformer extends upon this by introducing a **capacity factor** directly influencing the expert capacity.

$$\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) * \text{capacity factor}$$

Switch Transformer: Capacity Factor

- The components of expert capacity are straightforward:

tokens What is Mixture of Experts ?

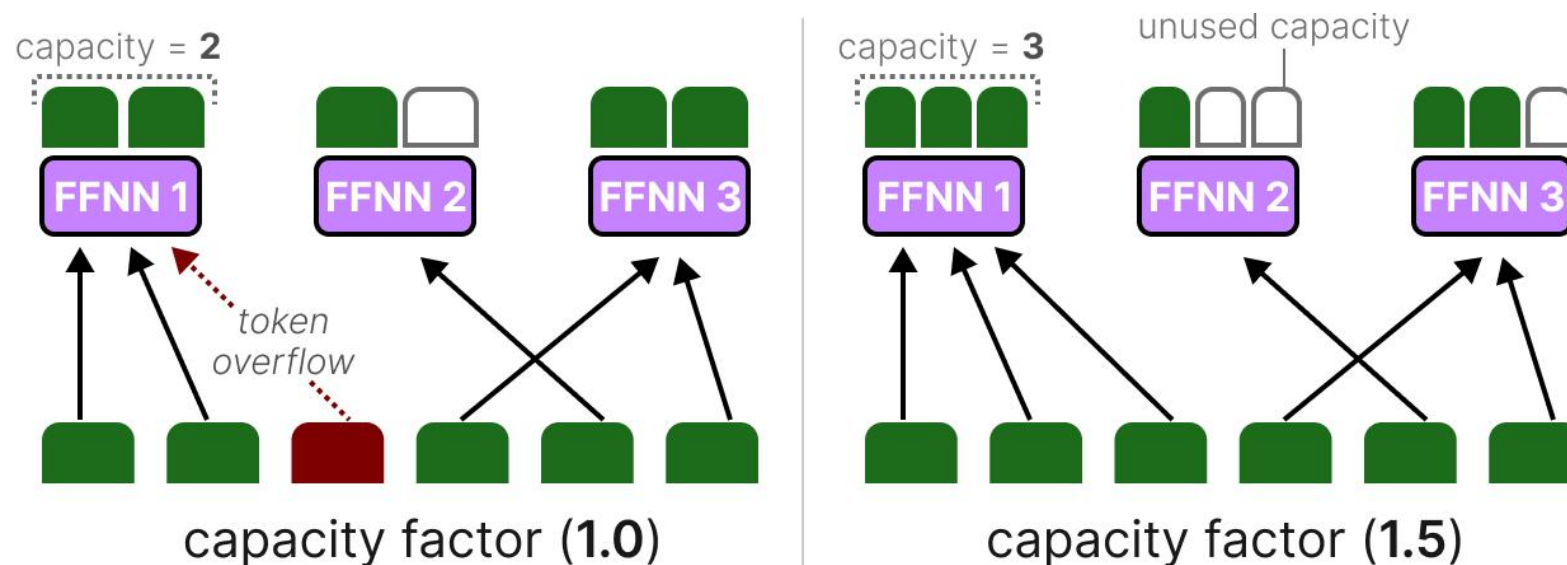
experts FFNN 1 FFNN 2 FFNN 3

capacity factor 1.0

$$\text{expert capacity} = \left(\frac{6}{3} \right) * 1.0 = 2$$

Switch Transformer: Capacity Factor

- A technique that uses many different sub-models to improve the LLMs.
 - ✓ if the capacity factor is too large, we waste computing resources.
 - ✓ if the capacity factor is too small, the performance will drop due to token overflow.



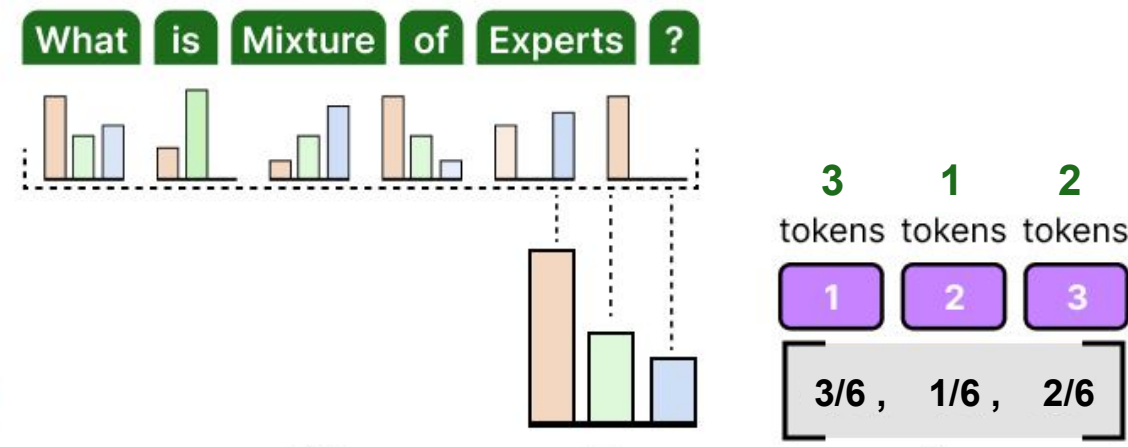
Switch Transformer: Auxiliary Loss

- Instead of calculating the coefficient variation, this simplified loss weighs *the fraction of tokens dispatched* against *the fraction of router probability per expert*:

$$L_{aux} = \alpha * N * \sum f_i p_i$$

Auxiliary Loss = α * N * P * f

α : tune-able parameter
 N : number of experts
 P : fraction of router probability per expert
 f : fraction of tokens dispatched per expert



Example from deepseek (v1-2)

- **Per-expert balancing** – same as the switch transformer

$$\begin{aligned}\mathcal{L}_{\text{ExpBal}} &= \alpha_1 \sum_{i=1}^{N'} f_i P_i, \\ f_i &= \frac{N'}{K'T} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ selects Expert } i), \\ P_i &= \frac{1}{T} \sum_{t=1}^T s_{i,t},\end{aligned}$$

- **Per-device balancing** – the objective above, but aggregated by device.

$$\begin{aligned}\mathcal{L}_{\text{DevBal}} &= \alpha_2 \sum_{i=1}^D f'_i P'_i, \\ f'_i &= \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j, \\ P'_i &= \sum_{j \in \mathcal{E}_i} P_j,\end{aligned}$$

DeepSeek v3 variation – per-expert biases

- Set up a per-expert bias (making it more likely to get tokens) and use online learning

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

- They call this ‘**auxiliary loss free balancing**’

Complementary Sequence-Wise Auxiliary Loss. Although DeepSeek-V3 mainly relies on the auxiliary-loss-free strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

$$\mathcal{L}_{\text{Bal}} = \alpha \sum_{i=1}^{N_r} f_i P_i, \quad (17)$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^T \mathbb{1}(s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r)), \quad (18)$$

$$s'_{i,t} = \frac{s_{i,t}}{\sum_{j=1}^{N_r} s_{j,t}}, \quad (19)$$

(but the approach is not fully aux loss free)

$$P_i = \frac{1}{T} \sum_{t=1}^T s'_{i,t}, \quad (20)$$

What happens when removing load balancing losses?

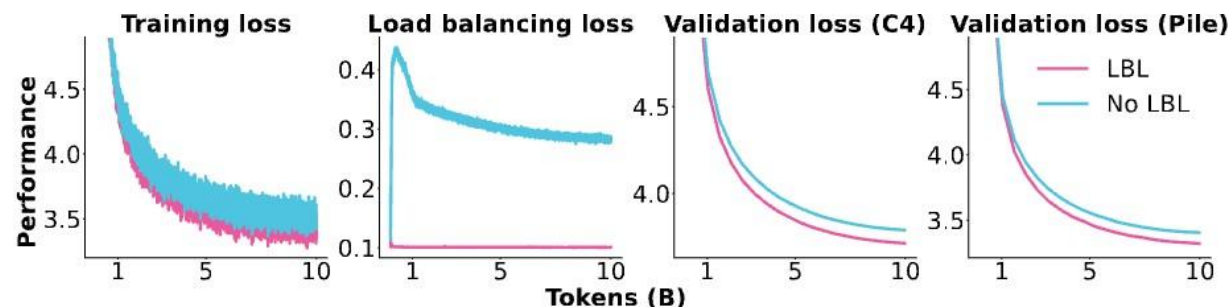


Figure 9: **Impact of applying a load balancing loss (LBL).** The training loss plot excludes the load balancing loss for both models. More results, logs, and configurations: <https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo40TkyNDg4>

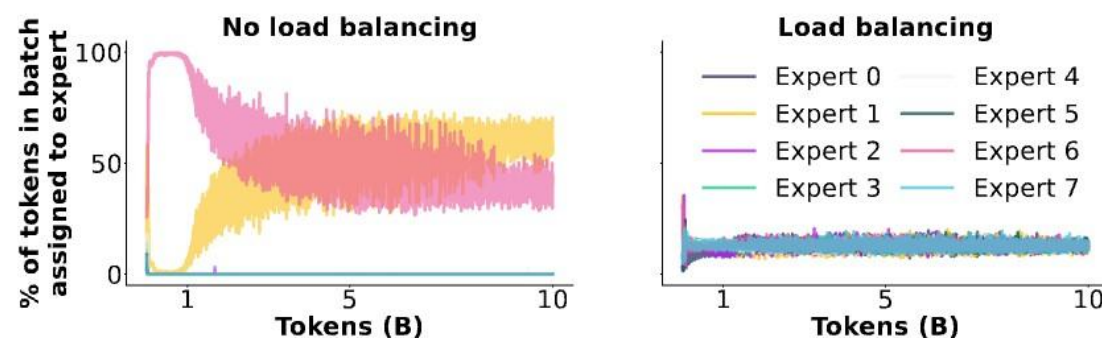
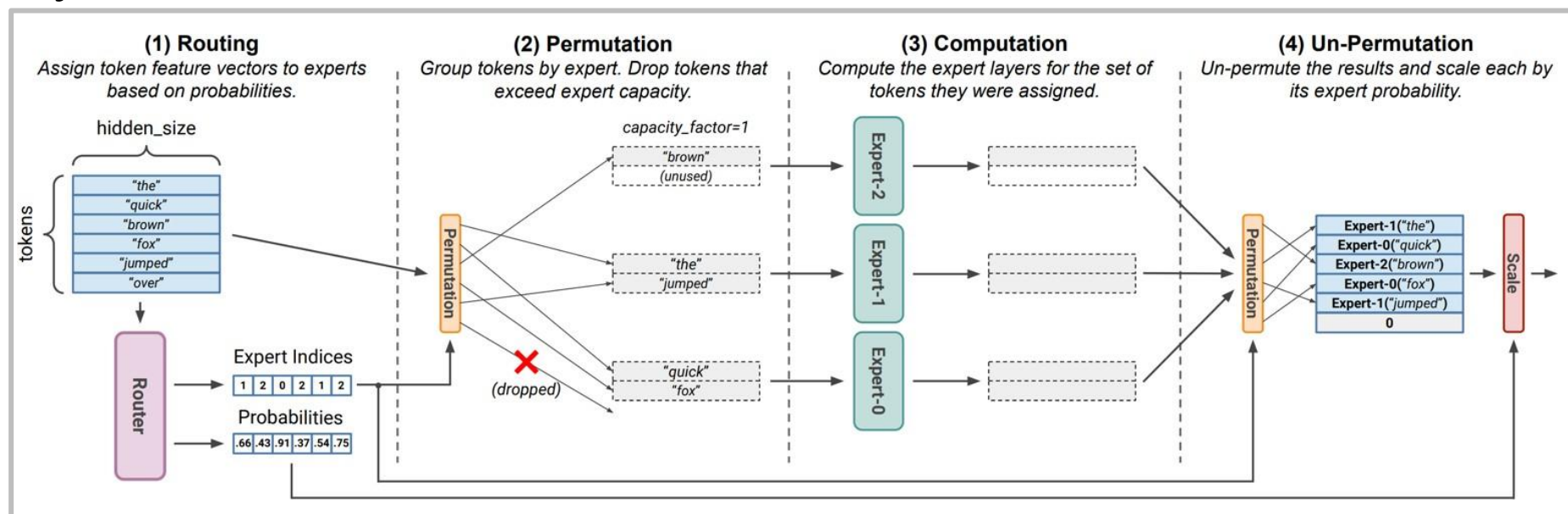


Figure 10: **Expert assignment during training when using or not using a load balancing loss for the first MoE layer.** More results, logs, and configurations: <https://wandb.ai/ai2-llm/olmoe/reports/Plot-LBL-vs-No-LBL--Vmlldzo40TkyNDg4>

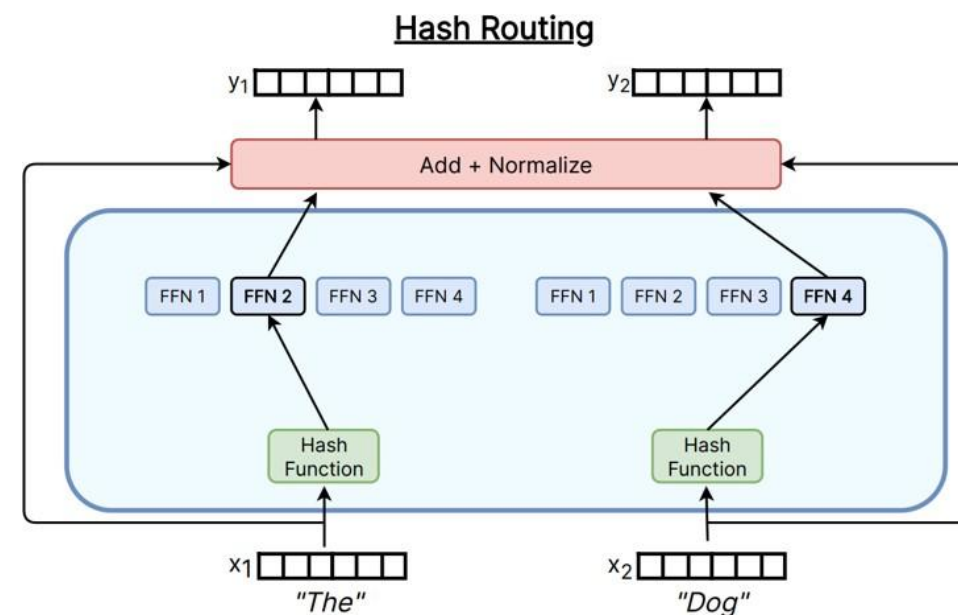
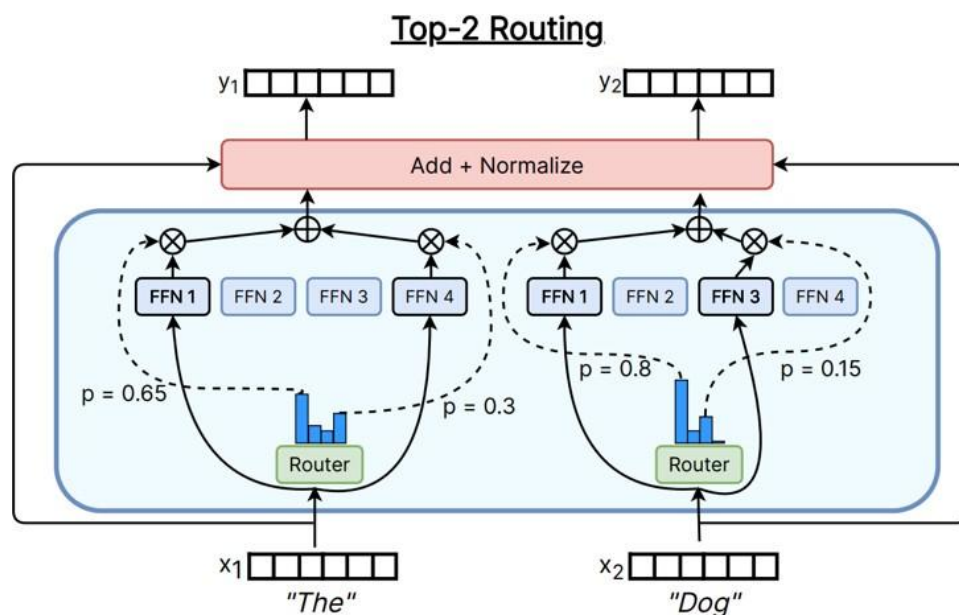
Fun side issue – stochasticity of MoE models

- There was speculation that GPT-4's stochasticity was due to MoE.
- Why would a MoE have additional randomness?



Token dropping from routing happens at a *batch* level – this means that other people's queries can drop your token!

More routing variants

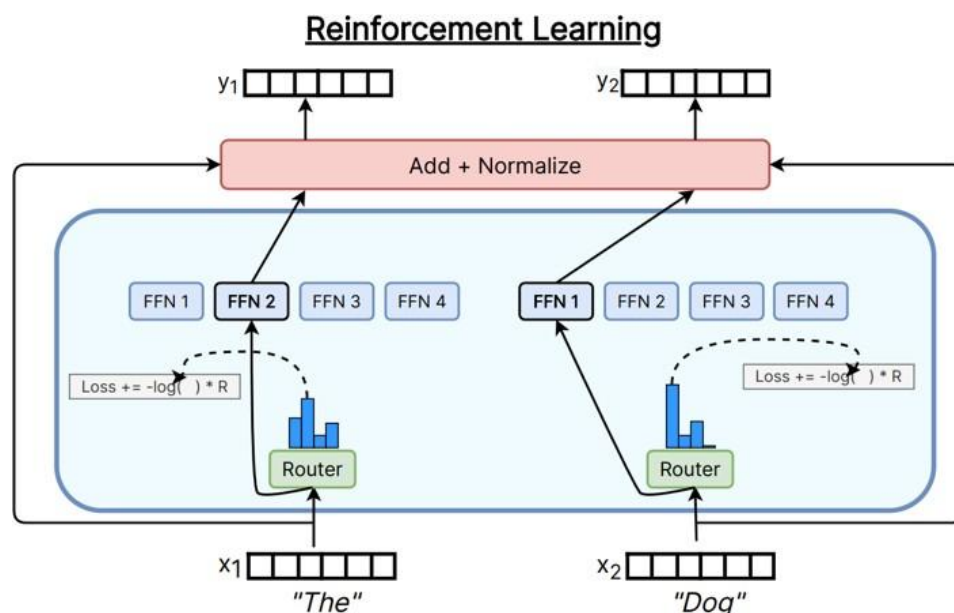


Top-K, used in most MoEs:

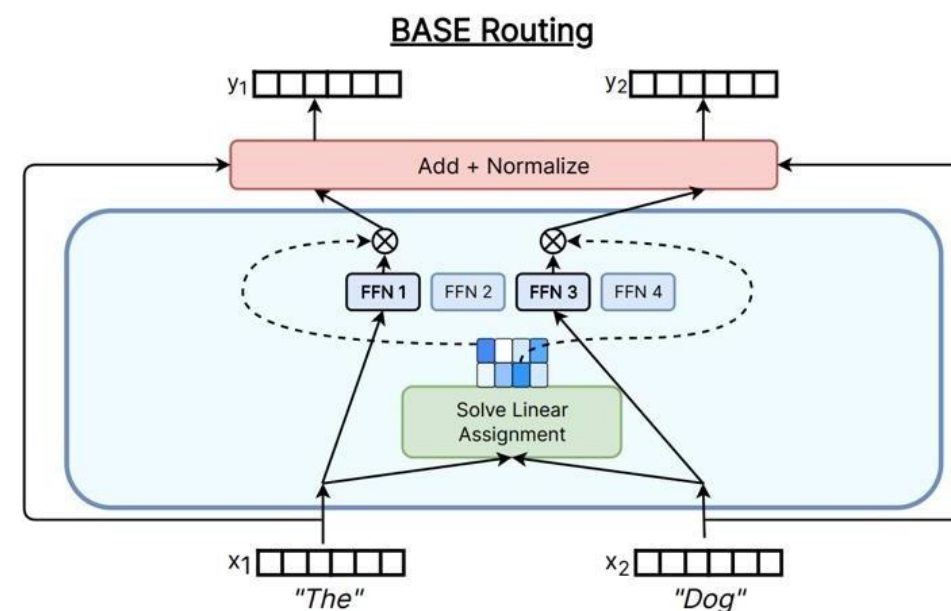
Switch Transformer ($k=1$) Gshard ($k=2$), Grok (2),
Mixtral (2), Qwen (4), DeepSeek (7), DBRX(4)

Hashing: common baseline

More routing variants



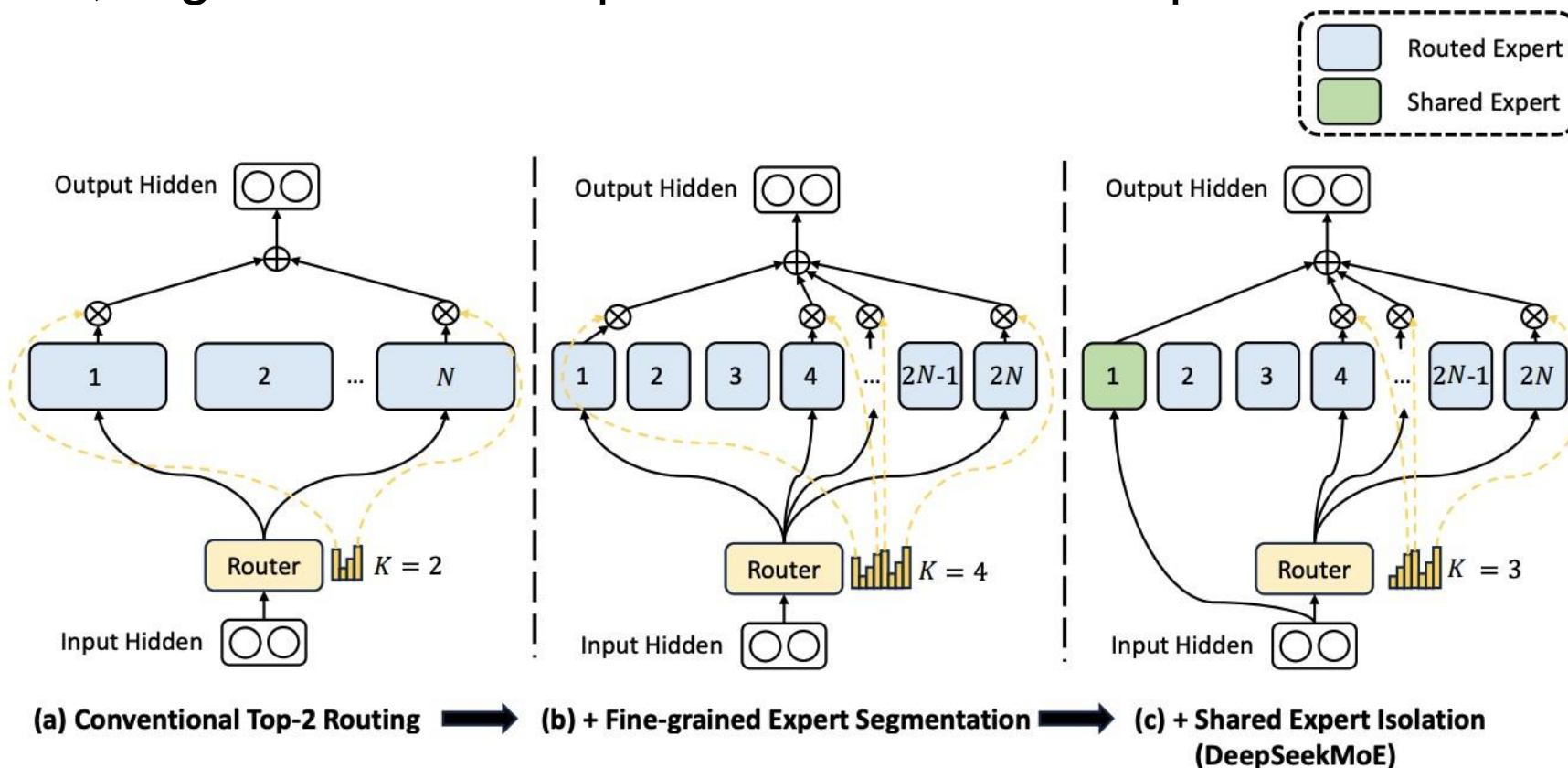
RL to learn routes:
used in some of the earliest work
Bengio 2013, not common now



Solve a matching problem:
linear assignment for routing, used in
various papers like Clark '22

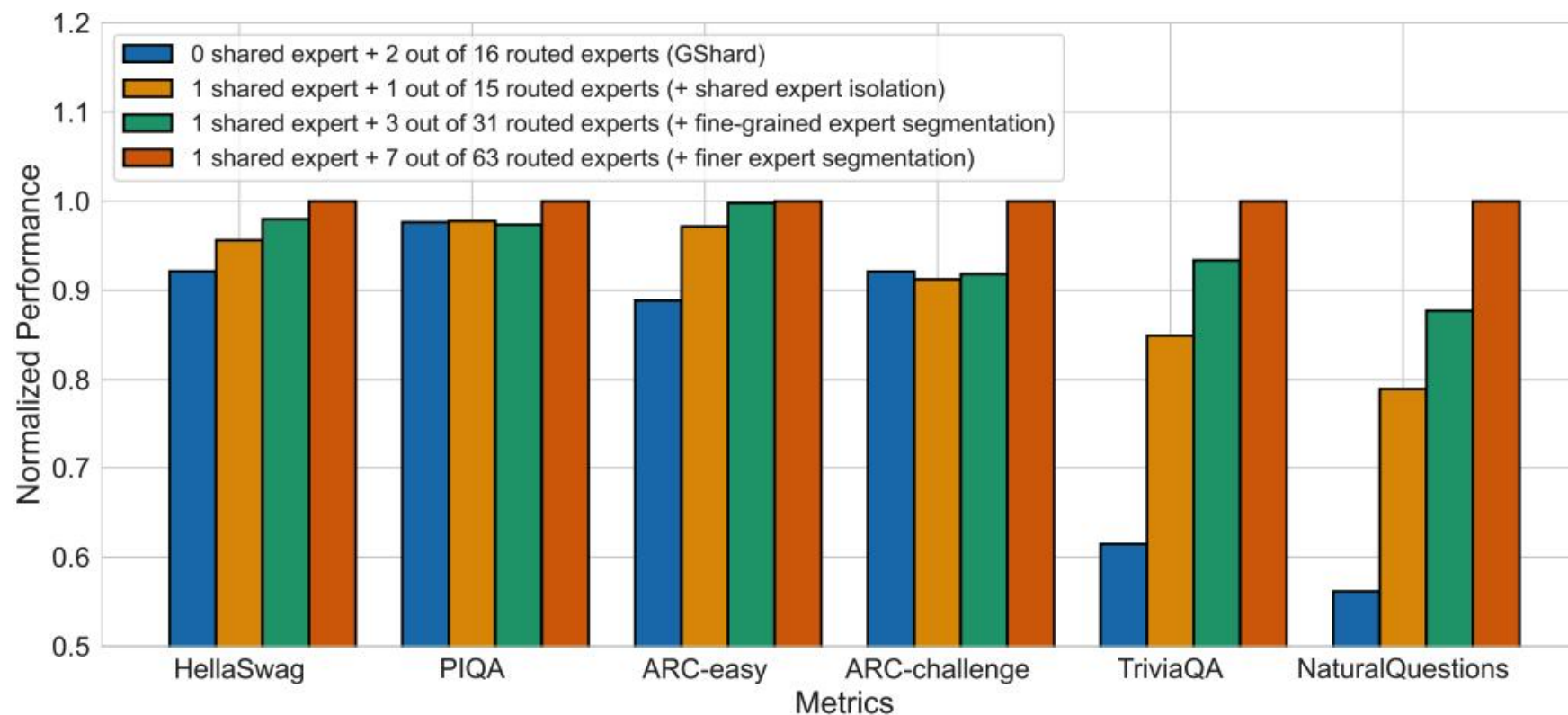
Recent variations from DeepSeek and other Chinese LMs

- Smaller, larger number of experts + a few shared experts that are always on



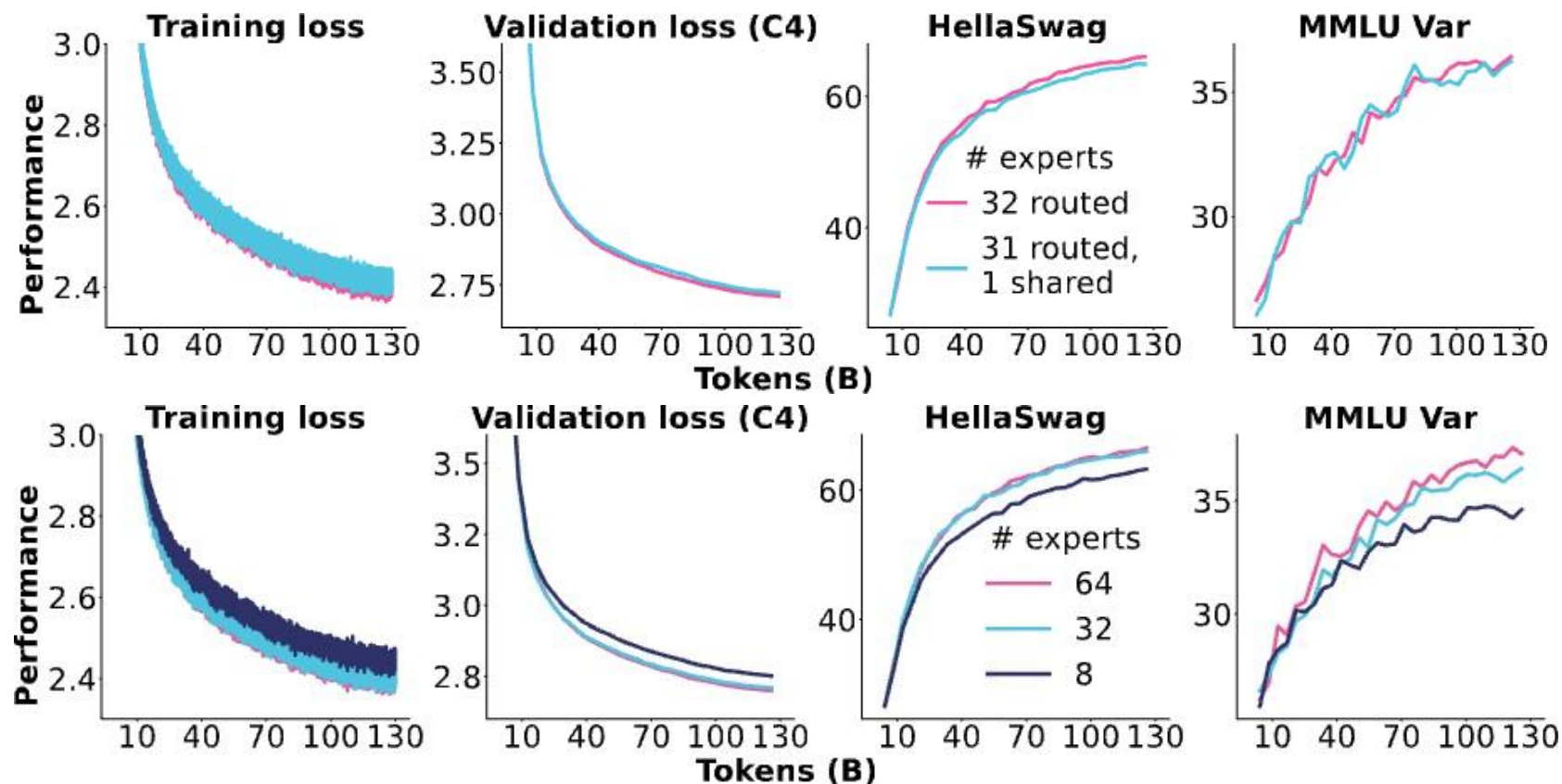
Various ablations from the DeepSeek paper

- More experts, shared experts all seem to generally help



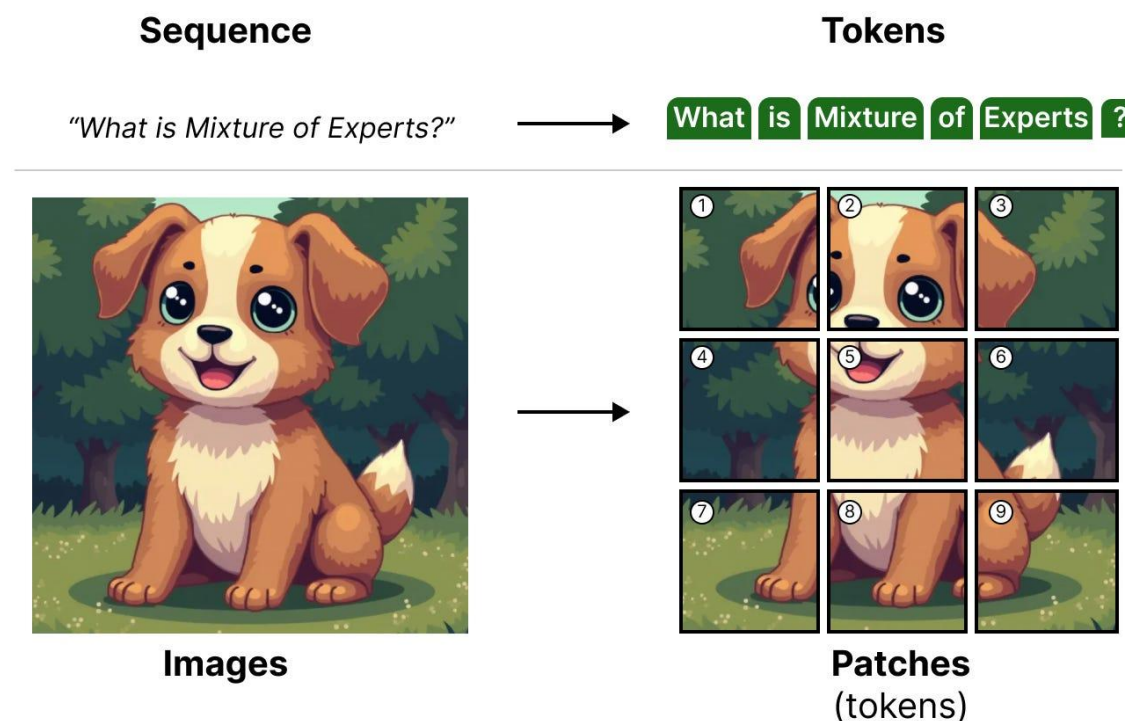
Ablations from OIMoE

- Gains from fine-grained experts, none from shared experts



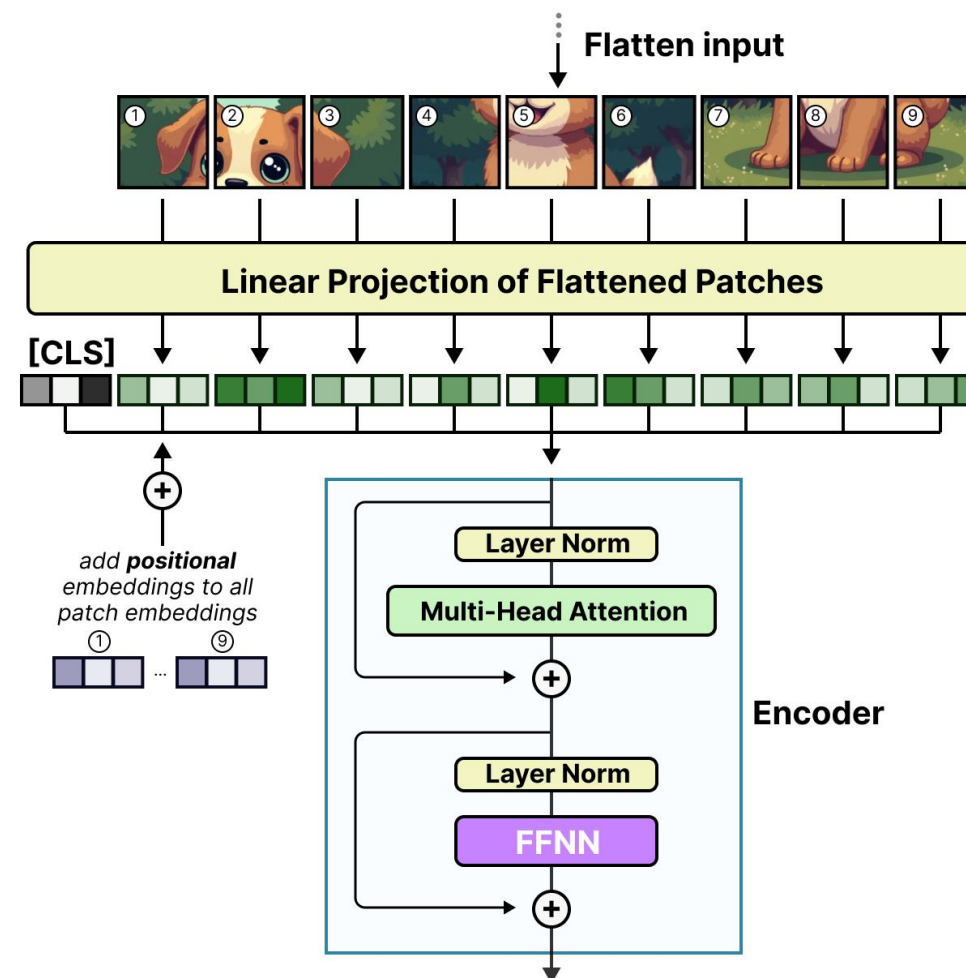
Mixture of Experts in Vision Models

- MoE is usable in both language and vision models (e.g. ViT).
- As a quick recap, ViT is an architecture that splits images into **patches** that are processed similarly to **tokens**.



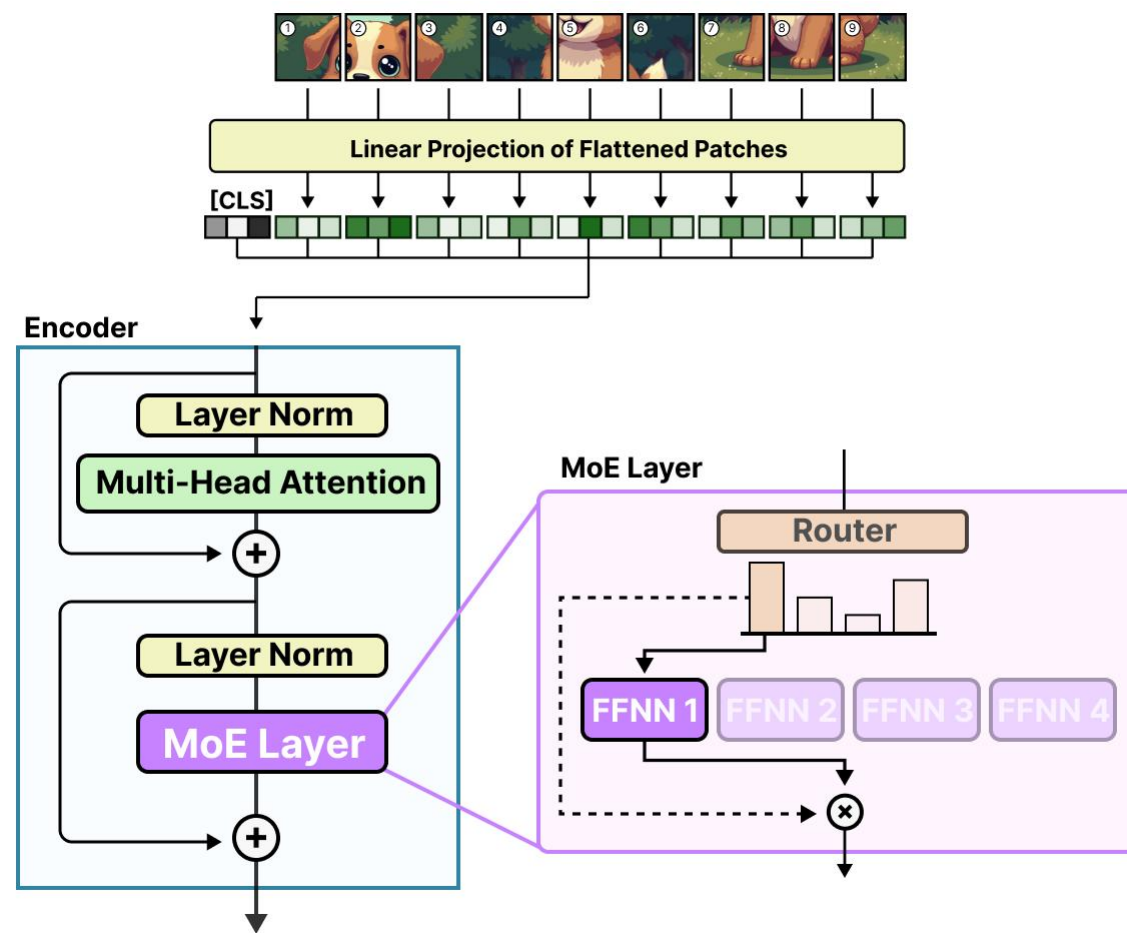
Mixture of Experts in Vision Models

- These patches (or tokens) are then projected into embeddings (with additional positional embeddings) before being fed into a regular encoder:



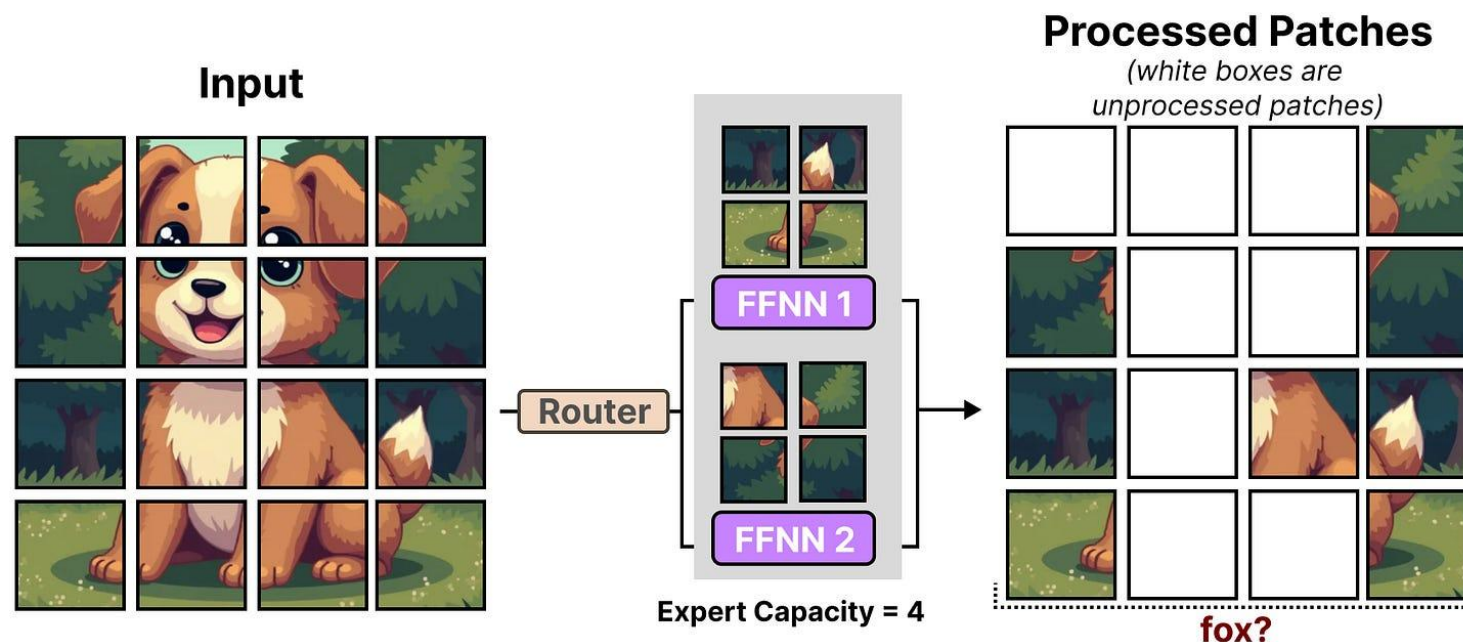
Vision-MoE

- Vision-MoE (V-MoE) is one of the first implementations of MoE in an image model. It takes the ViT as we saw before and replaces the dense FFNN in the encoder with a Sparse MoE.



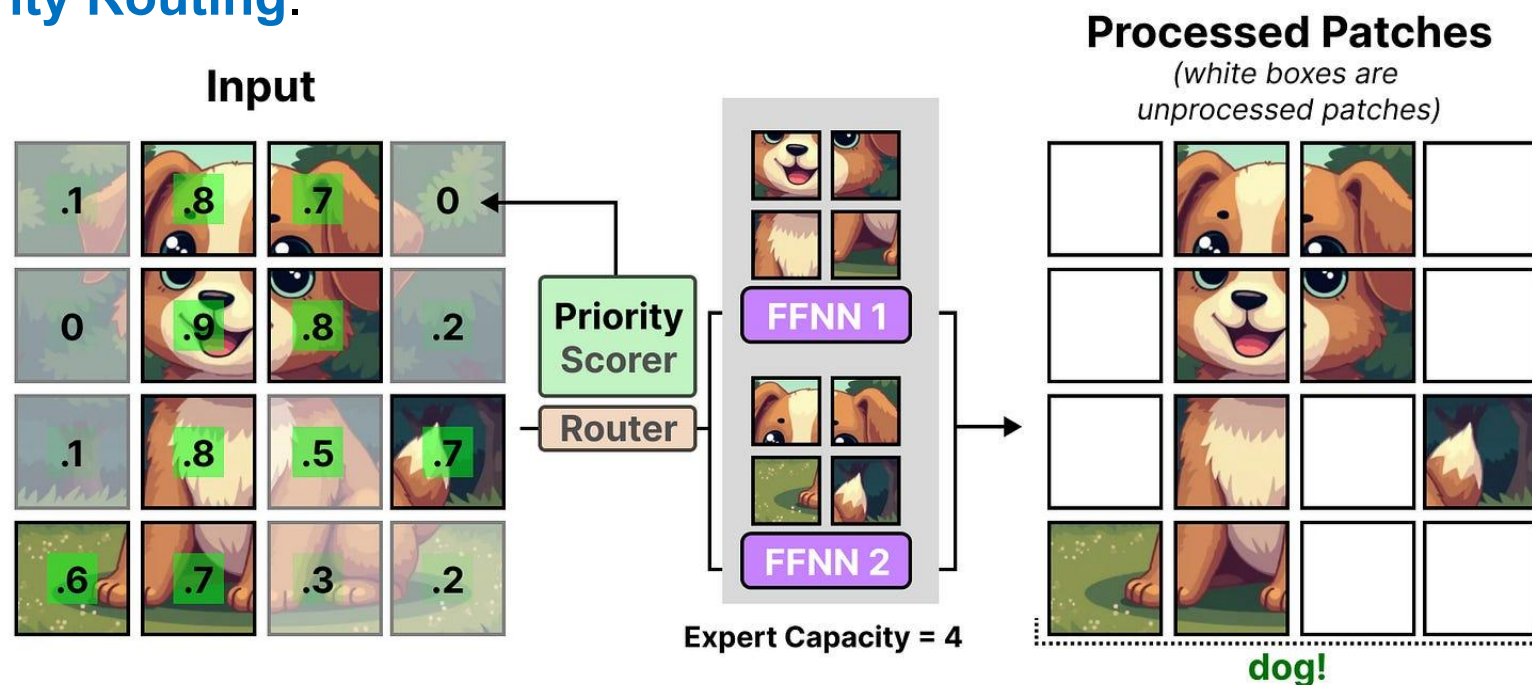
Vision-MoE

- A small pre-defined expert capacity was used for each expert to reduce hardware constraints since images generally have many patches.
- However, a low capacity tends to lead to patches being dropped (*akin to token overflow*).



Vision-MoE

- To keep the capacity low, the network assigns importance scores to patches and processes those first so that overflowed patches are generally less important. This is called **Batch Priority Routing**.



Vision-MoE

- We should still see important patches routed if the percentage of tokens decreases.
- The priority routing allows fewer patches to be processed by focusing on the most important ones.

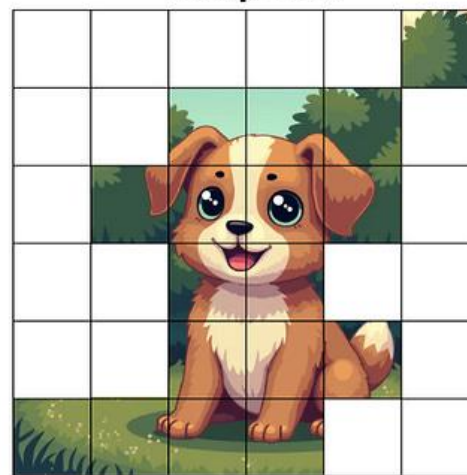
No dropped patches



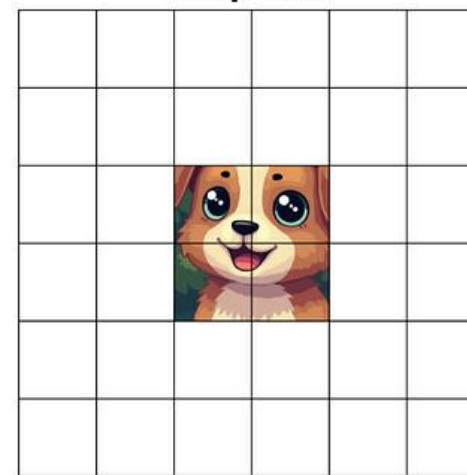
Keep 80%



Keep 50%

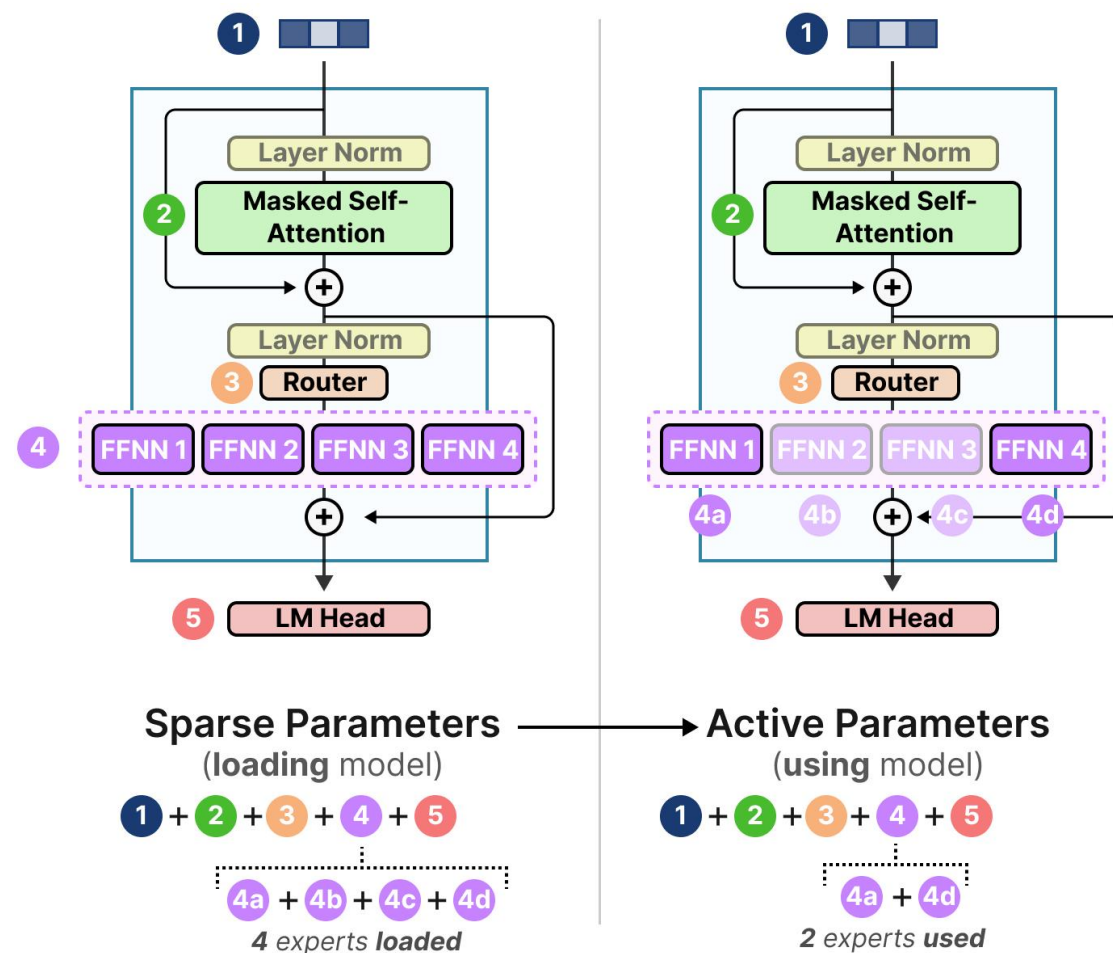


Keep 10%



Active vs. Sparse Parameters with Mixtral 8x7B

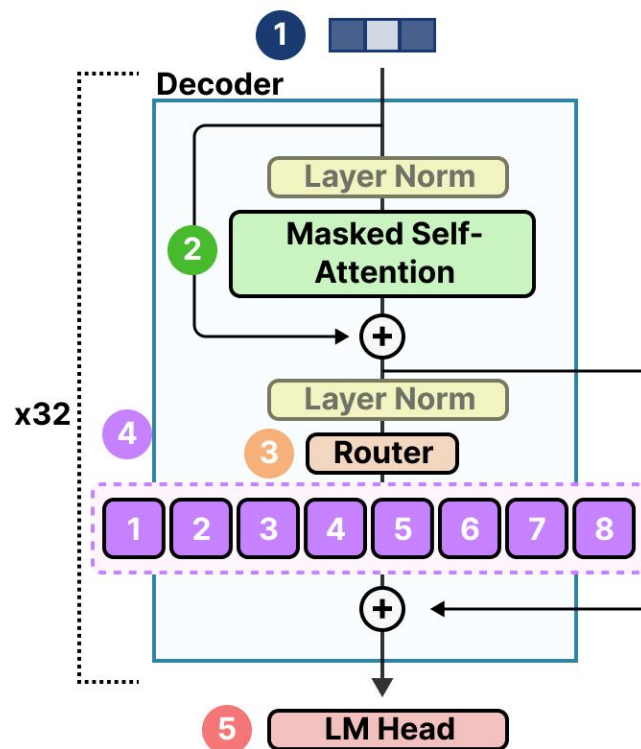
- Although a given MoE has more parameters to load (**sparse parameters**), fewer are activated since we only use some experts during inference (**active parameters**).



Active vs. Sparse Parameters with Mixtral 8x7B

- In other words, MoE models need **more VRAM** to load in all experts but **run faster** during inference.

Mixtral 8x7B



1 **Embeddings**
 $32000 \times \frac{4096}{\text{embedding size}} = \underline{131.072.000}$
shared parameters

2 **Attention**
 $32 \times \frac{41.943.040}{\text{repeated decoder blocks} \quad (q, k, v)} = \underline{1.342.177.280}$
shared parameters

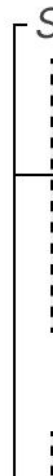
3 **Router**
 $8 \times 4096 = \underline{32.768}$
experts shared parameters

4 **Experts**
 $8 \times 5.637.144.576 = \underline{45.097.156.608}$
experts total parameters
 $2 \times 5.637.144.576 = \underline{11.274.289.152}$
experts expert size active parameters

5 **LM Head**
 $32000 \times 4096 = \underline{131.072.000}$
shared parameters

Active vs. Sparse Parameters with Mixtral 8x7B

- Here, we can see that each expert is 5.6B in size and not 7B (although there are 8 experts).

Shared parameters					
	1 Embeddings	131.072.000	1 Embeddings	131.072.000	
	2 Attention	1.342.177.280	2 Attention	1.342.177.280	
	3 Router	32.768	3 Router	32.768	
	4 Experts	45.097.156.608	4 Experts	11.274.289.152	
	5 LM Head	131.072.000	5 LM Head	131.072.000	
Sparse Parameters		46.7B	Active Parameters		12.8B
(all parameters)			(activated parameters)		

MoE summary

- MoEs take advantage of sparsity – not all inputs need the full model
- Discrete routing is hard, but top-k heuristics seem to work
- Lots of empirical evidence now that MoEs work, and are cost-effective

Thanks

Welcome to join VisionXLab

yangxue.site

